

```

In [1]: import os
import glob
import copy
import datetime
import matplotlib.pyplot as plt
from matplotlib import colors
from matplotlib import colormaps
import matplotlib.patches as patches
import numpy as np
import pandas as pd
import scipy.stats as stats

from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

import gsf_ims_fitness as fitness

import pickle

import seaborn as sns
import cmocean
import palettable
sns.set()

from Bio.Seq import Seq

%load_ext autoreload
%autoreload 2

%matplotlib inline

%autosave 0

sns.set_style("white")
sns.set_style("ticks", {'xtick.direction':'in', 'xtick.top':True, 'ytick.direction':'in', 'ytick.right':True})

plt.rcParams['axes.labelsize'] = 16
plt.rcParams['xtick.labelsize'] = 14
plt.rcParams['ytick.labelsize'] = 14

plt.rcParams['legend.fontsize'] = 12
plt.rcParams['legend.edgecolor'] = 'k'

Autosave disabled

```

In []:

```

In [2]: notebook_directory = os.getcwd()
notebook_directory

```

```

Out[2]: 'C:\\Users\\djross\\Documents\\Jcloud\\GSF-IMS\\E-Coli\\pLMSF-lacI\\2022-11-08_
two-lig_two-sel_DNA-5-plates\\BarSeq\\HiSeq'

```

```
In [3]: %%time
os.chdir(notebook_directory)
pickle_file = '2022-11-08_two-lig_two-sel_DNA-5-plates_single_amino_BarSeqFitness.pkl'
print(pickle_file)

barcode_frame = pickle.load(open(pickle_file, 'rb'))

hiseq_count_frame = barcode_frame.barcode_frame
```

```
2022-11-08_two-lig_two-sel_DNA-5-plates_single_amino_BarSeqFitnessFrame.pkl
CPU times: total: 484 ms
Wall time: 672 ms
```

```
In [4]: hiseq_count_frame.shape
```

```
Out[4]: (978, 597)
```

```
In [5]: len(np.unique(hiseq_count_frame.lacI_amino_seq))
```

```
Out[5]: 967
```

```
In [6]: np.unique(hiseq_count_frame.lacI_amino_mutations)
```

```
Out[6]: array([0, 1], dtype=int64)
```

```
In [7]: df = hiseq_count_frame
df = df[df.lacI_amino_mutations==1]
len(df), len(np.unique(df.lacI_amino_seq))
```

```
Out[7]: (966, 966)
```

```

In [8]: df_wt = hiseq_count_frame
df_wt = df_wt[df_wt.lacI_amino_mutations==0]
df_wt = df_wt[df_wt.total_counts>20000]
print(len(df_wt))
y1_wt_arr = np.array([x for x in df_wt.GP_log_g_IPTG])[:,2,:]
y1var_wt_arr = np.array([x for x in df_wt.GP_log_g_IPTG_var])

y2_wt_arr = np.array([x for x in df_wt.GP_log_g_ONPF])[:,2,:]
y2var_wt_arr = np.array([x for x in df_wt.GP_log_g_ONPF_var])

y1_wt = np.average(y1_wt_arr, axis=0, weights=1/y1var_wt_arr)
y1err_wt = np.std(y1_wt_arr, axis=0)

y1err_wt = fitness.log_plot_errorbars(y1_wt, y1err_wt)
y1_wt = 10**y1_wt

y2_wt = np.average(y2_wt_arr, axis=0, weights=1/y2var_wt_arr)
y2err_wt = np.std(y2_wt_arr, axis=0)

y2err_wt = fitness.log_plot_errorbars(y2_wt, y2err_wt)
y2_wt = 10**y2_wt

```

11

In []:

```

In [9]: def add_break(ax, x_0=0.13, w=0.04, d_x=0.01, d_y=0.025, zorder=2000):
    rect = [(x_0, -0.025), w, 1.05]
    ax.add_patch(patches.Rectangle(*rect, transform=ax.transAxes, clip_on=False,
    zorder += 1
    for y_0 in [0, 1]:
        ax.plot([x_0-d_x, x_0+d_x], [y_0-d_y, y_0+d_y], transform=ax.transAxes, c
        ax.plot([x_0-d_x+w, x_0+d_x+w], [y_0-d_y, y_0+d_y], transform=ax.transAxe

```

In []:

```

In [10]: def plot_dose_response_by_position(pos, include_stops=False, plot_ligands=['IPTG',
df = hiseq_count_frame#single_mutant_frame
df = df[df.position==pos]
if not include_stops:
    df = df[['*' not in x for x in df.mutation]]

if len(df)>0:
    plt.rcParams["figure.figsize"] = [4, 8/3]
    fig, ax = plt.subplots()

    x1 = [0] + list(barcode_frame.inducer_conc_lists[0])
    x2 = [0] + list(barcode_frame.inducer_conc_lists[1])

    ax.errorbar(x1, y1_wt, y1err_wt, fmt=fmt1, color='k', label='WT', alpha=0.7, ms=9)
    ax.errorbar(x2, y2_wt, y2err_wt, fmt=fmt2, color='k', alpha=0.7, ms=9)

    for (ind, row), color in zip(df.iterrows(), sns.color_palette()):
        if 'IPTG' in plot_ligands:
            y1 = row.GP_log_g_IPTG[2]
            y1err = np.sqrt(row.GP_log_g_IPTG_var)
            y1err = fitness.log_plot_errorbars(y1, y1err)
            y1 = 10**y1
            ax.errorbar(x1, y1, y1err, fmt=fmt1, color=color, label=row.mutation)

        if 'ONPF' in plot_ligands:
            y2 = row.GP_log_g_ONPF[2]
            y2err = np.sqrt(row.GP_log_g_ONPF_var)
            y2err = fitness.log_plot_errorbars(y2, y2err)
            y2 = 10**y2
            ax.errorbar(x2, y2, y2err, fmt=fmt2, color=color, alpha=0.7, ms=9)
    ax.set_yscale('log')
    ax.set_xscale('symlog', linthresh=1)
    ax.legend(loc='upper left', bbox_to_anchor=(1.03, 0.97), ncol=1, border=0)

    ax.set_xlim(-0.3, 8000)
    ax.set_title(f'Position {df.iloc[0].mutation[:-1]}', size=16)
    ax.set_xlabel('[ligand] (μmol/L)')
    ax.set_ylabel('Output (MEF)')

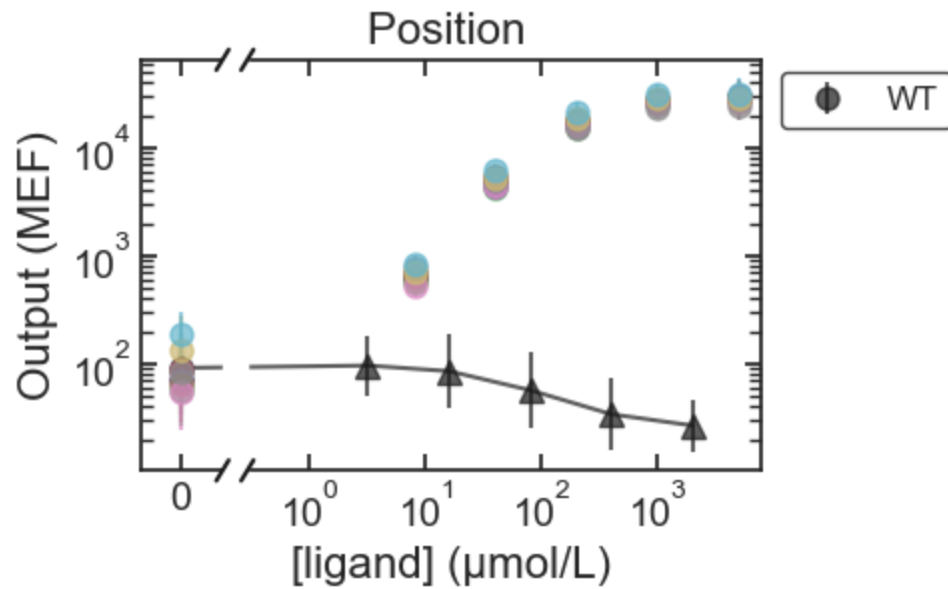
    add_break(ax)

    return fig, ax

```

In []:


```
In [11]: fig, ax = plot_dose_response_by_position(0, plot_ligands=['IPTG'], fmt1='o');
```



```
In [ ]:
```

```
In [12]: plot_params = ['log_k_a_1_mut', 'log_k_i_1_mut', 'log_k_a_2_mut', 'log_k_i_2_mut',
                        'log_k_ratio_1_mut', 'log_k_ratio_2_mut']
```

```
In [13]: save_params = plot_params + ['log_k_a_1_wt', 'log_k_i_1_wt', 'log_k_a_2_wt', 'log_k_i_2_wt',
                                       'delta_eps_AI_wt', 'delta_eps_RA_wt', 'log_y_max',
                                       'sigma']
save_params
```

```
Out[13]: ['log_k_a_1_mut',
          'log_k_i_1_mut',
          'log_k_a_2_mut',
          'log_k_i_2_mut',
          'delta_eps_AI_mut',
          'delta_eps_RA_mut',
          'log_k_ratio_1_mut',
          'log_k_ratio_2_mut',
          'log_k_a_1_wt',
          'log_k_i_1_wt',
          'log_k_a_2_wt',
          'log_k_i_2_wt',
          'delta_eps_AI_wt',
          'delta_eps_RA_wt',
          'log_y_max',
          'sigma']
```

```
In [ ]:
```

```
In [14]: os.chdir(notebook_directory)
os.chdir('../.../fit_groq_seq_w_biophysical_model/2022-11-08/')
mut_list = glob.glob('*.csv')
os.chdir(notebook_directory)
mut_list = [x.replace('.csv', '') for x in mut_list]
```

```
In [15]: [x for x in mut_list if int(x[1:-1])==1]
```

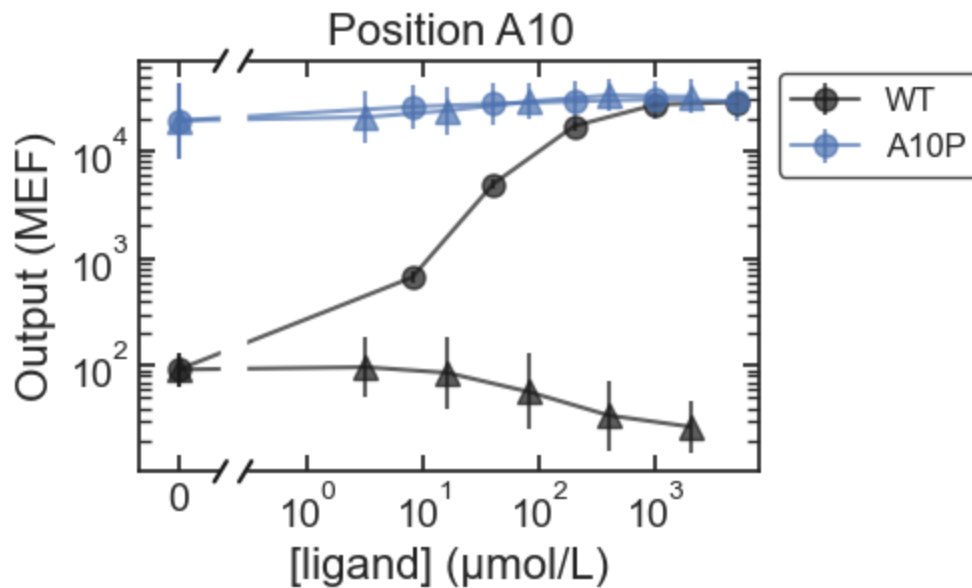
```
Out[15]: ['M1V', 'M1T', 'M1I']
```

```
In [ ]:
```

```
In [16]: os.chdir(notebook_directory)
os.chdir('../.../fit_groq_seq_w_biophysical_model/2022-11-08/')
mut_df_dict = {}
for mut in mut_list:
    file = f'{mut}.csv'
    mut = mut.replace('stop', '*')
    df = pd.read_csv(file)
    mut_df_dict[mut] = df
os.chdir(notebook_directory)
```

```
In [ ]:
```

```
In [17]: plot_dose_response_by_position(10);
```



```
In [ ]:
```

```

In [18]: plt.rcParams["figure.figsize"] = [25, 3]

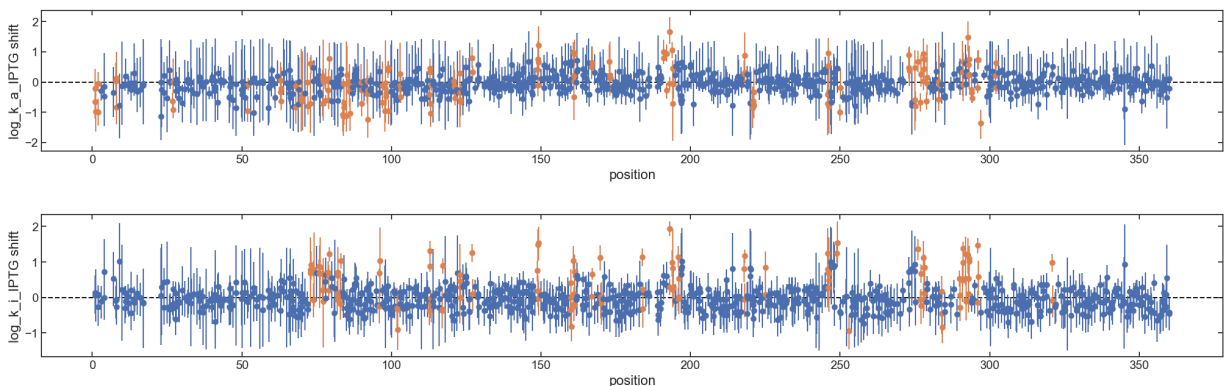
spacing = 0.1
min_fold_shift = 2
sig_positions_dict = {}
for p in save_params:
    min_shift = np.log10(min_fold_shift) if 'log_k_' in p else np.log(min_fold_shift)
    if p in save_params[:8]:
        sig_positions_dict[p] = []
    fig, ax = plt.subplots()
    y_label = p
    y_label = y_label.replace('_1_', '_IPTG ').replace('_2_', '_ONPF ').replace(
    for pos in [x for x in range(1, 361)]:
        x = []
        y = []
        yerr = []
        #pos_dict = {k:v for k, v in plot_fit_dict.items() if (int(k[1:-1])==pos)}
        pos_mut_list = [x for x in mut_list if ('stop' not in x) and (int(x[1:-1])
        span = spacing*len(pos_mut_list)
        for i, mut in enumerate(pos_mut_list):
            if '*' not in mut:
                df = mut_df_dict[mut]

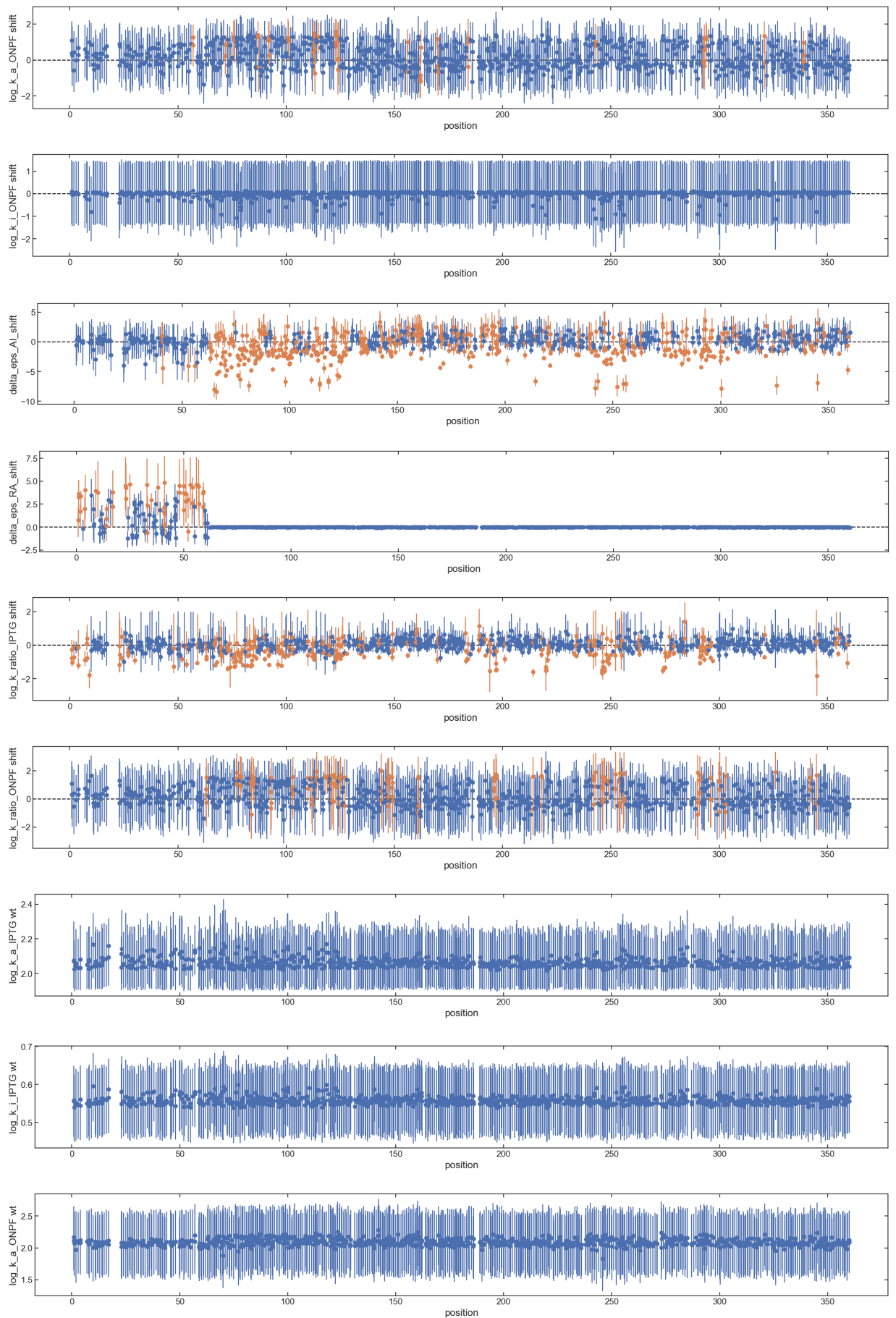
                x.append(int(mut[1:-1]) - span/2 + spacing*i + spacing/2)
                y_samp = df[p]
                y.append(np.mean(y_samp))
                yerr.append(np.quantile(y_samp, [0.05, 0.95]))

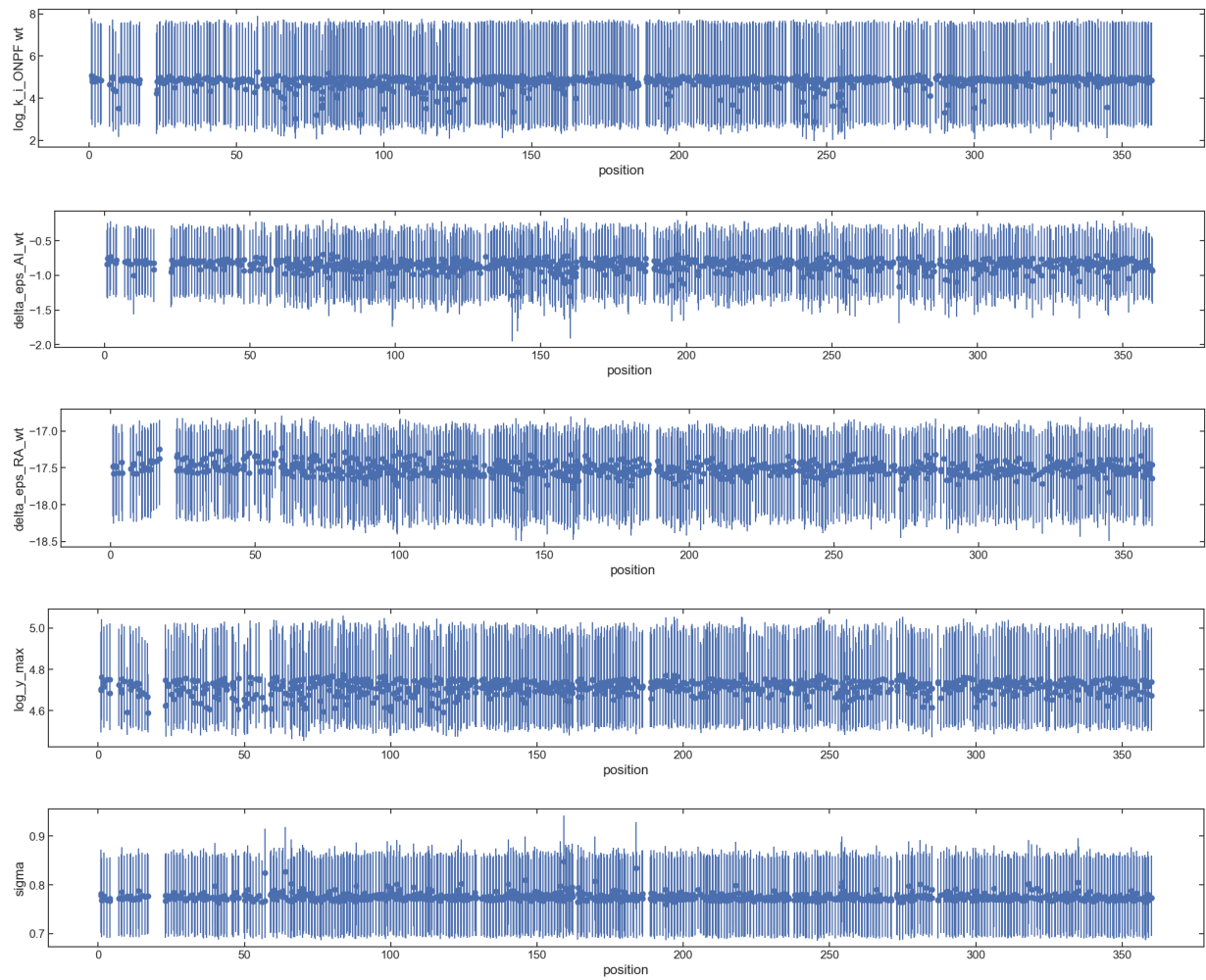
        if len(x)>0:
            color = sns.color_palette()[0]
            if np.any(np.array(yerr).transpose()[0] > min_shift) or np.any(np.array(
                if 'shift' in y_label:
                    color = sns.color_palette()[1]
                if p in save_params[:8]:
                    sig_positions_dict[p].append(pos)
            yerr = np.abs(y - np.array(yerr).transpose())
            ax.errorbar(x, y, yerr, fmt='o', color=color)

    ax.set_ylabel(y_label)
    ax.set_xlabel('position')
    if 'shift' in y_label:
        xlim = ax.get_xlim()
        ax.set_xlim(xlim)
        ax.plot(xlim, [0]*2, '--k', zorder=-10)
    #break

```







In []:

```
In [19]: pos_list = np.unique([int(x[1:-1]) for x in mut_list])
pos_list
```

```
Out[19]: array([ 1,  2,  3,  4,  7,  8,  9, 10, 11, 12, 13, 14, 15,
                16, 17, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
                34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 46, 47,
                48, 50, 51, 52, 53, 54, 55, 56, 57, 59, 60, 61, 62,
                63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75,
                76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88,
                89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101,
                102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114,
                115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127,
                128, 129, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141,
                142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154,
                155, 156, 157, 158, 159, 160, 161, 162, 164, 165, 166, 167, 168,
                169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181,
                182, 183, 184, 185, 186, 189, 190, 191, 192, 193, 194, 195, 196,
                197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209,
                210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222,
                223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235,
                236, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249,
                250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262,
                263, 264, 265, 266, 267, 268, 269, 270, 271, 273, 274, 275, 276,
                277, 278, 279, 280, 281, 282, 283, 284, 285, 287, 288, 289, 290,
                291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303,
                304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316,
                317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329,
                330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342,
                343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355,
                356, 357, 358, 359, 360])
```

```
In [20]: pos_count_dict = np.unique([int(x[1:-1]) for x in mut_list], return_counts=True)
pos_count_dict = {p:c for p, c in zip(pos_count_dict[0], pos_count_dict[1])}
#pos_count_dict
```

```
In [ ]:
```

```
In [21]: for k, v in sig_positions_dict.items():
          y_label = k
          y_label = y_label.replace('_1_', '_IPTG ').replace('_2_', '_ONPF ').replace(
              print(y_label)
              print(v)
              print()
```

log_k_a_IPTG shift

[1, 2, 8, 27, 52, 63, 68, 70, 72, 73, 77, 78, 79, 84, 85, 86, 90, 92, 97, 98, 99, 103, 113, 121, 123, 127, 149, 161, 167, 173, 191, 192, 193, 194, 218, 221, 246, 250, 273, 275, 276, 277, 278, 279, 283, 291, 293, 294, 296, 297, 302]

log_k_i_IPTG shift

[73, 74, 76, 77, 79, 82, 83, 96, 102, 113, 117, 123, 127, 149, 160, 161, 167, 170, 184, 193, 194, 196, 218, 225, 246, 249, 253, 276, 277, 278, 284, 290, 291, 292, 293, 296, 321]

log_k_a_ONPF shift

[57, 72, 76, 87, 92, 101, 113, 115, 123, 124, 156, 162, 170, 184, 243, 293, 294, 321, 339]

log_k_i_ONPF shift

[]

delta_eps_AI_shift

[41, 53, 56, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 90, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 103, 106, 108, 110, 111, 112, 113, 114, 115, 116, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 133, 134, 135, 136, 142, 143, 146, 147, 149, 151, 155, 156, 158, 159, 160, 161, 168, 170, 171, 174, 176, 178, 181, 183, 184, 185, 189, 191, 192, 194, 195, 196, 197, 201, 205, 209, 214, 218, 220, 221, 226, 233, 234, 239, 240, 242, 243, 246, 247, 249, 250, 251, 252, 254, 255, 256, 259, 260, 263, 273, 274, 275, 276, 277, 279, 281, 282, 283, 284, 285, 288, 291, 292, 293, 294, 295, 296, 297, 300, 301, 303, 306, 308, 318, 321, 324, 326, 329, 333, 339, 342, 343, 345, 348, 355, 359]

delta_eps_RA_shift

[1, 2, 4, 9, 10, 14, 17, 23, 25, 33, 35, 38, 41, 48, 50, 51, 52, 53, 54, 56, 57, 59]

log_k_ratio_IPTG shift

[1, 2, 4, 7, 8, 9, 23, 24, 27, 30, 48, 52, 54, 59, 63, 67, 68, 70, 73, 74, 76, 77, 78, 79, 80, 82, 83, 84, 85, 86, 90, 92, 96, 97, 98, 99, 101, 103, 110, 112, 113, 117, 120, 123, 124, 125, 126, 128, 134, 136, 143, 149, 156, 160, 170, 183, 184, 189, 192, 194, 197, 201, 214, 218, 220, 221, 233, 234, 240, 242, 243, 246, 247, 248, 249, 250, 252, 255, 263, 274, 275, 276, 277, 279, 284, 291, 292, 293, 294, 296, 297, 321, 345, 354, 355, 359]

log_k_ratio_ONPF shift

[63, 77, 79, 84, 85, 93, 103, 110, 112, 114, 118, 119, 121, 122, 123, 125, 127, 144, 147, 149, 161, 196, 197, 214, 218, 242, 243, 246, 247, 249, 252, 254, 256, 290, 291, 296, 300, 303, 326, 342, 345]

In []:

```

In [70]: row_list = []
for pos in [x for x in range(1, 361)]:
    row_d = {'position':pos, 'mutation_count':pos_count_dict.get(pos, 0)}
    for k, v in sig_positions_dict.items():
        k = k.replace('_mut', '_shift').replace('_1', '_IPTG').replace('_2', '_ONPF')
        row_d[k] = (pos in v)
    row = pd.Series(row_d)
    row_list.append(row)
    #if pos == 326:
    #    break

```

In []:

```

In [71]: out_frame = pd.DataFrame(row_list)
out_frame

```

Out[71]:

	position	mutation_count	log_k_a_IPTG_shift	log_k_i_IPTG_shift	log_k_a_ONPF_shift	log_k_i_
0	1	3	True	False	False	
1	2	2	True	False	False	
2	3	1	False	False	False	
3	4	2	False	False	False	
4	5	0	False	False	False	
...	...	...	...	...	...	...
355	356	1	False	False	False	
356	357	3	False	False	False	
357	358	5	False	False	False	
358	359	3	False	False	False	
359	360	2	False	False	False	

360 rows × 10 columns



In [72]: out_frame.columns

```

Out[72]: Index(['position', 'mutation_count', 'log_k_a_IPTG_shift',
               'log_k_i_IPTG_shift', 'log_k_a_ONPF_shift', 'log_k_i_ONPF_shift',
               'delta_eps_AI_shift', 'delta_eps_RA_shift', 'log_k_ratio_IPTG_shift',
               'log_k_ratio_ONPF_shift'],
              dtype='object')

```



```
In [73]: def apply_categories(df):
df['IPTG'] = df.log_k_a_IPTG_shift | df.log_k_i_IPTG_shift | df.log_k_ratio_IPTG
df['ONPF'] = df.log_k_a_ONPF_shift | df.log_k_i_ONPF_shift | df.log_k_ratio_ONPF
df['MWC'] = df.delta_eps_AI_shift

df['MWC_only'] = df['MWC'] & ~(df['IPTG'] | df['ONPF'])

df['MWC_IPTG'] = df['MWC'] & (df['IPTG'])
df['MWC_ONPF'] = df['MWC'] & (df['ONPF'])
df['MWC_IPTG_ONPF'] = df['MWC'] & (df['IPTG']) & (df['ONPF'])

df['IPTG_only'] = df['IPTG'] & ~(df['MWC'] | df['ONPF'])
df['ONPF_only'] = df['ONPF'] & ~(df['MWC'] | df['IPTG'])

df['Both_noMWC'] = (~df['MWC']) & (df['IPTG']) & (df['ONPF'])
#return df
```

```
In [74]: apply_categories(out_frame)
out_frame
```

Out[74]:

	position	mutation_count	log_k_a_IPTG_shift	log_k_i_IPTG_shift	log_k_a_ONPF_shift	log_k_i_
0	1	3	True	False	False	
1	2	2	True	False	False	
2	3	1	False	False	False	
3	4	2	False	False	False	
4	5	0	False	False	False	
...	...	...	...	...	...	...
355	356	1	False	False	False	
356	357	3	False	False	False	
357	358	5	False	False	False	
358	359	3	False	False	False	
359	360	2	False	False	False	

360 rows × 20 columns



In []:

```
In [75]: os.chdir(notebook_directory)
os.chdir('../fit_groq_seq_w_biophysical_model/')
out_frame.to_csv('IPTG_ONPF_positions.csv', index=False)
```

In []:

```
In [76]: df = out_frame  
df = df[df.ONPF_only]  
df
```

Out[76]:

	position	mutation_count	log_k_a_IPTG_shift	log_k_i_IPTG_shift	log_k_a_ONPF_shift	log_k_i_
56	57	2	False	False	True	
143	144	5	False	False	False	
161	162	4	False	False	True	



```
In [77]: mut_list[0]
```

Out[77]: 'V119I'

```
In [ ]:
```

```

In [83]: # Output 0.05, 0.0, and 0.95 quantiles for every mutation
mut = mut_list[0]
df = mut_df_dict[mut]
quant_columns = [x for x in df.columns if '_mut' in x]
min_fold_shift = 2

row_list = []
for mut in mut_list:
    df = mut_df_dict[mut]
    row_d = {'mutation':mut, 'position':int(mut[1:-1])}
    for c in quant_columns:
        c2 = c.replace('_1', '_IPTG').replace('_2', '_ONPF')
        for q in [5, 50, 95]:
            row_d[f'{c2}_{q:02}'] = np.quantile(df[c], q/100)

    for c in quant_columns:
        c1 = c.replace('_1', '_IPTG').replace('_2', '_ONPF')
        c2 = c1.replace('_mut', '_shift')
        min_shift = np.log10(min_fold_shift) if 'log_k_' in c else np.log(min_fold_shift)
        row_d[c2] = (row_d[f'{c1}_05'] > min_shift) or (row_d[f'{c1}_95'] < -min_shift)

    row_list.append(pd.Series(row_d))
    #break
out_mut_frame = pd.DataFrame(row_list)
apply_categories(out_mut_frame)
out_mut_frame = out_mut_frame.sort_values(by=['position', 'mutation'], ignore_index=True)
out_mut_frame

```

Out[83]:

	mutation	position	log_k_a_IPTG_mut_05	log_k_a_IPTG_mut_50	log_k_a_IPTG_mut_95	log_k_i
0	M1I	1	-1.649228	-0.967614	-0.305219	
1	M1T	1	-1.220760	-0.698248	0.128159	
2	M1V	1	-0.636930	-0.256099	0.442714	
3	K2E	2	-1.434717	-0.988525	-0.552492	
4	K2R	2	-0.313470	-0.089149	0.341082	
...	...	...	...	...	...	...
933	G359E	359	-0.284835	0.014223	0.829093	
934	G359R	359	-0.577390	-0.206972	0.201057	
935	G359W	359	-1.529151	-0.534439	0.590333	
936	Q360E	360	-0.257095	0.008067	0.881052	
937	Q360R	360	-0.412597	-0.202674	0.008972	

938 rows × 44 columns



In []:

```
In [84]: os.chdir(notebook_directory)
os.chdir('../.../fit_groq_seq_w_biophysical_model/')
out_mut_frame.to_csv('IPTG_ONPF_mutations.csv', index=False)
```

```
In [ ]:
```

```
In [85]: df = out_mut_frame
df = df[df.ONPF_only]
df[['mutation', 'position'] + [x for x in df.columns if 'ONPF' in x]]
```

Out[85]:

	mutation	position	log_k_a_ONPF_mut_05	log_k_a_ONPF_mut_50	log_k_a_ONPF_mut_95	log_
100	A57S	57	0.464597	1.312875	1.979213	
268	L114I	114	0.039727	1.207665	2.237159	
360	P144S	144	0.093874	1.102575	2.150112	
419	S162Y	162	-1.994588	-1.187975	-0.332062	
749	K290I	290	-0.020619	0.956071	2.072317	



```
In [ ]:
```

```
In [ ]:
```

```
In [87]: plot_categories = [x for x in out_frame.columns if '_shift' not in x]
plot_categories = [x for x in plot_categories if x not in ['position', 'mutation']]
plot_categories
```

Out[87]:

```
['IPTG',
 'ONPF',
 'MWC',
 'MWC_only',
 'MWC_IPTG',
 'MWC_ONPF',
 'MWC_IPTG_ONPF',
 'IPTG_only',
 'ONPF_only',
 'Both_noMWC']
```

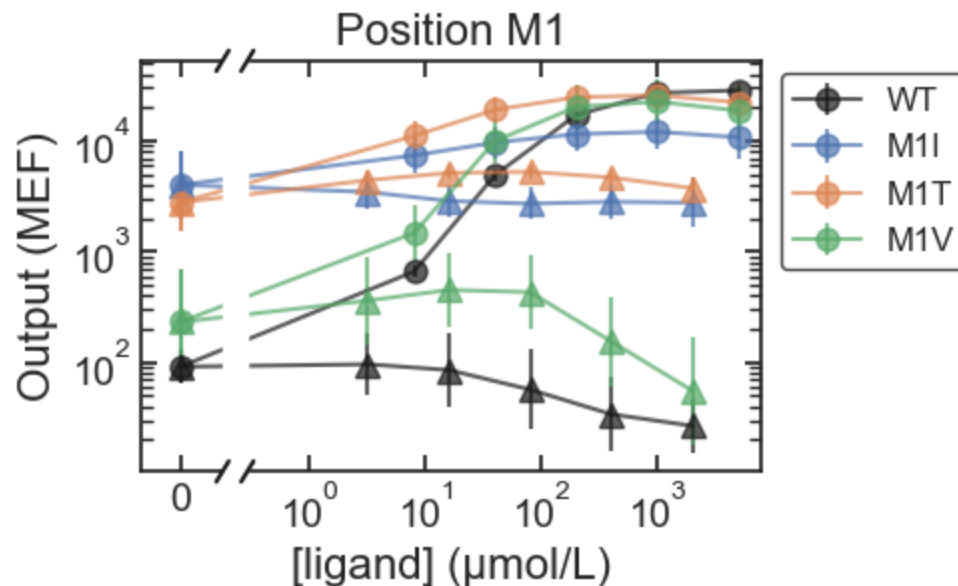
```
In [ ]:
```

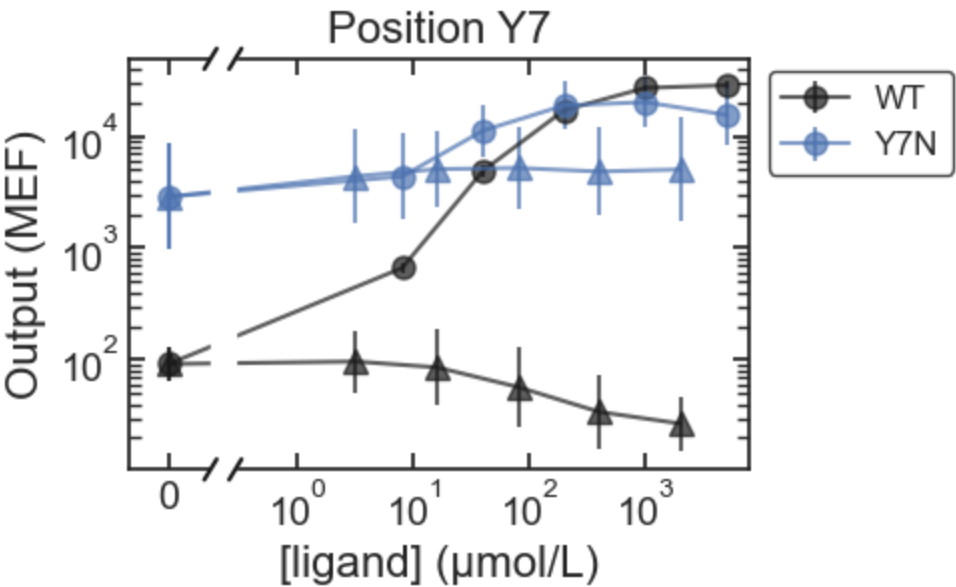
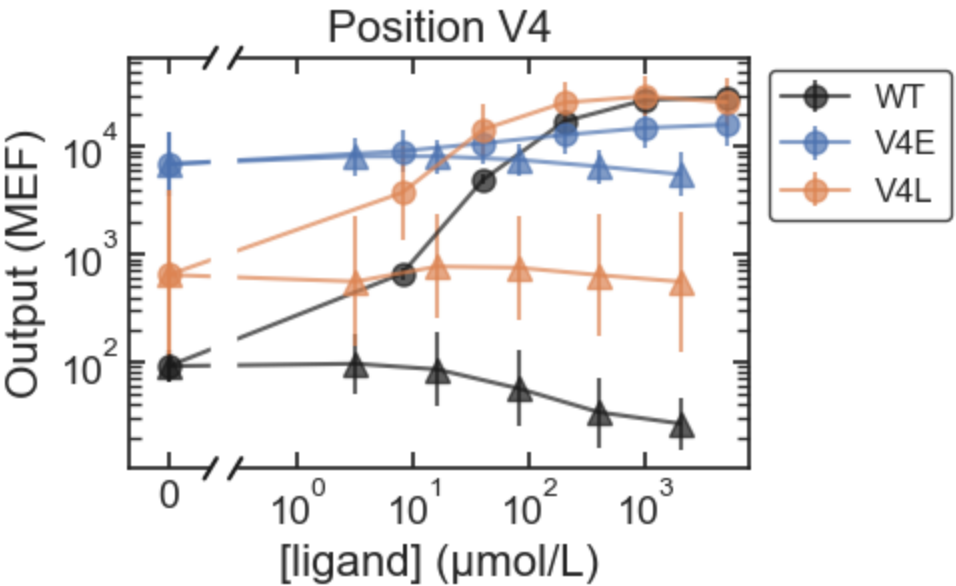
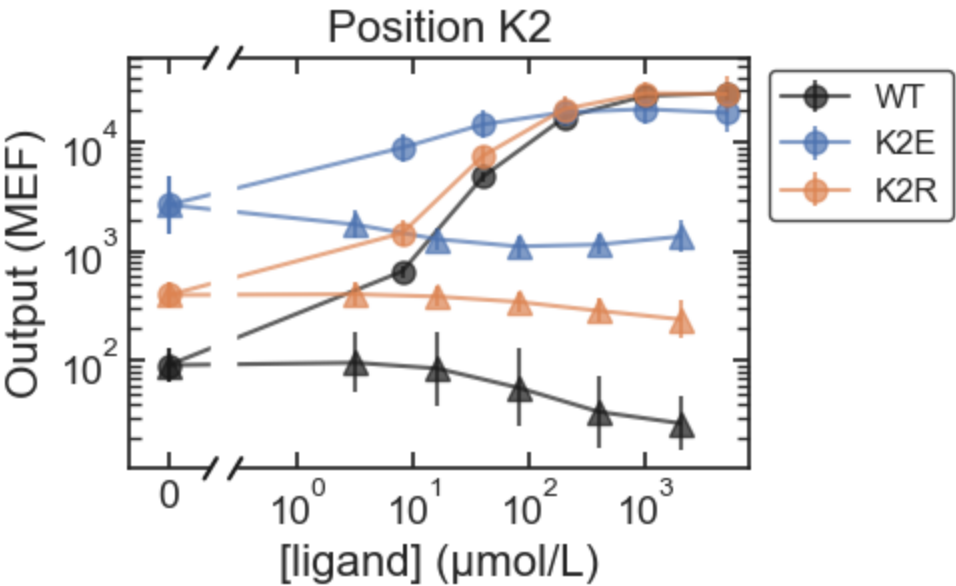
```
In [93]: for c in plot_categories:
df = out_frame
df = df[df[c]]
plt.rcParams["figure.figsize"] = [6, 0.2]
fig, ax = plt.subplots()
fig.suptitle(c, size=24)
ax.set_visible(False)
for pos in df.position:
    plot_dose_response_by_position(pos)
    #break
#break
```

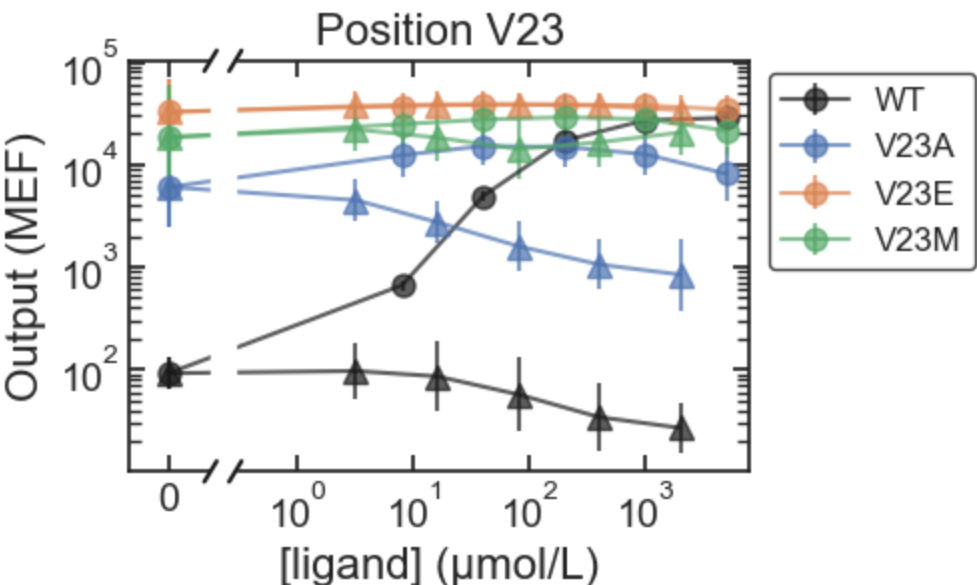
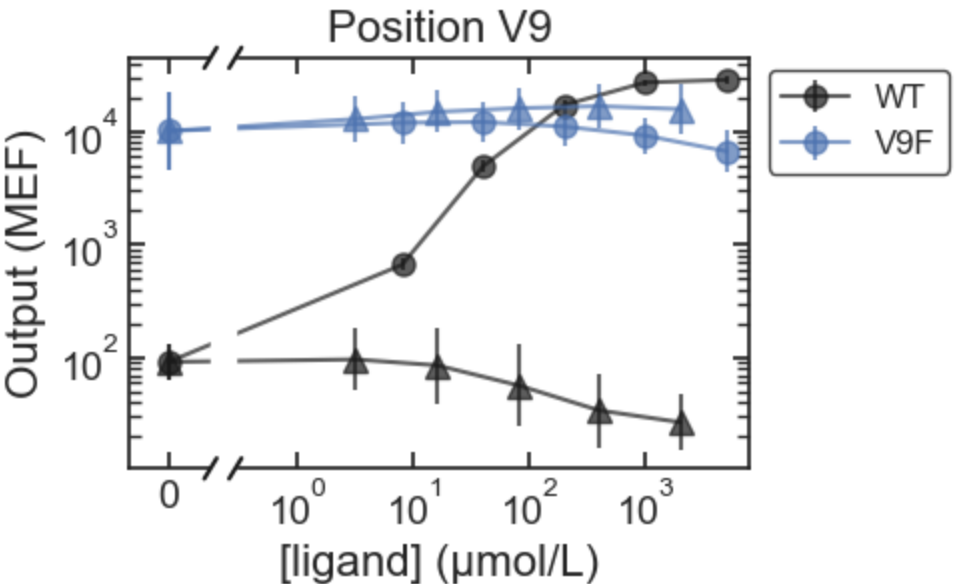
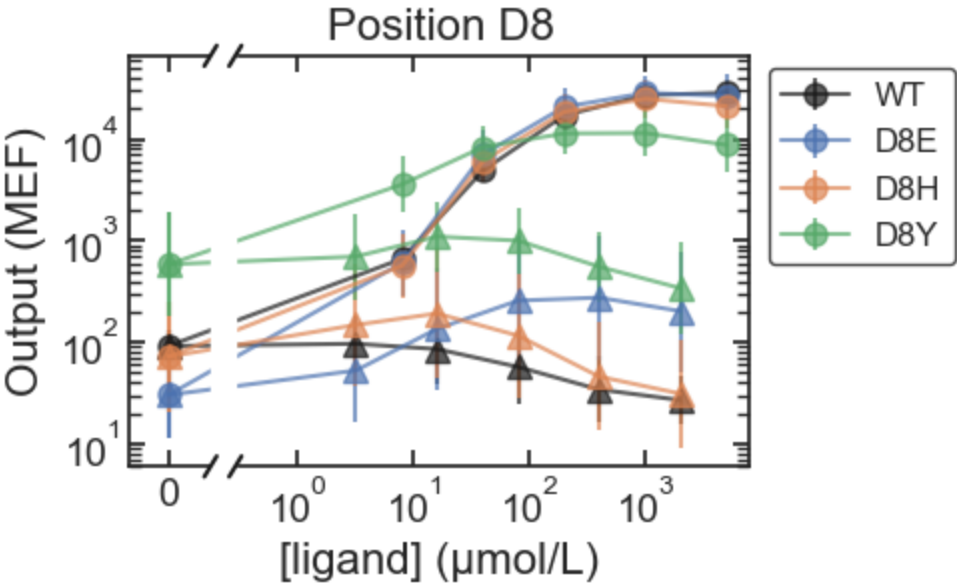
C:\Users\djross\AppData\Local\Temp\2\ipykernel_3888\2999772484.py:9: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may consume too much memory. (To control this warning, see the rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.

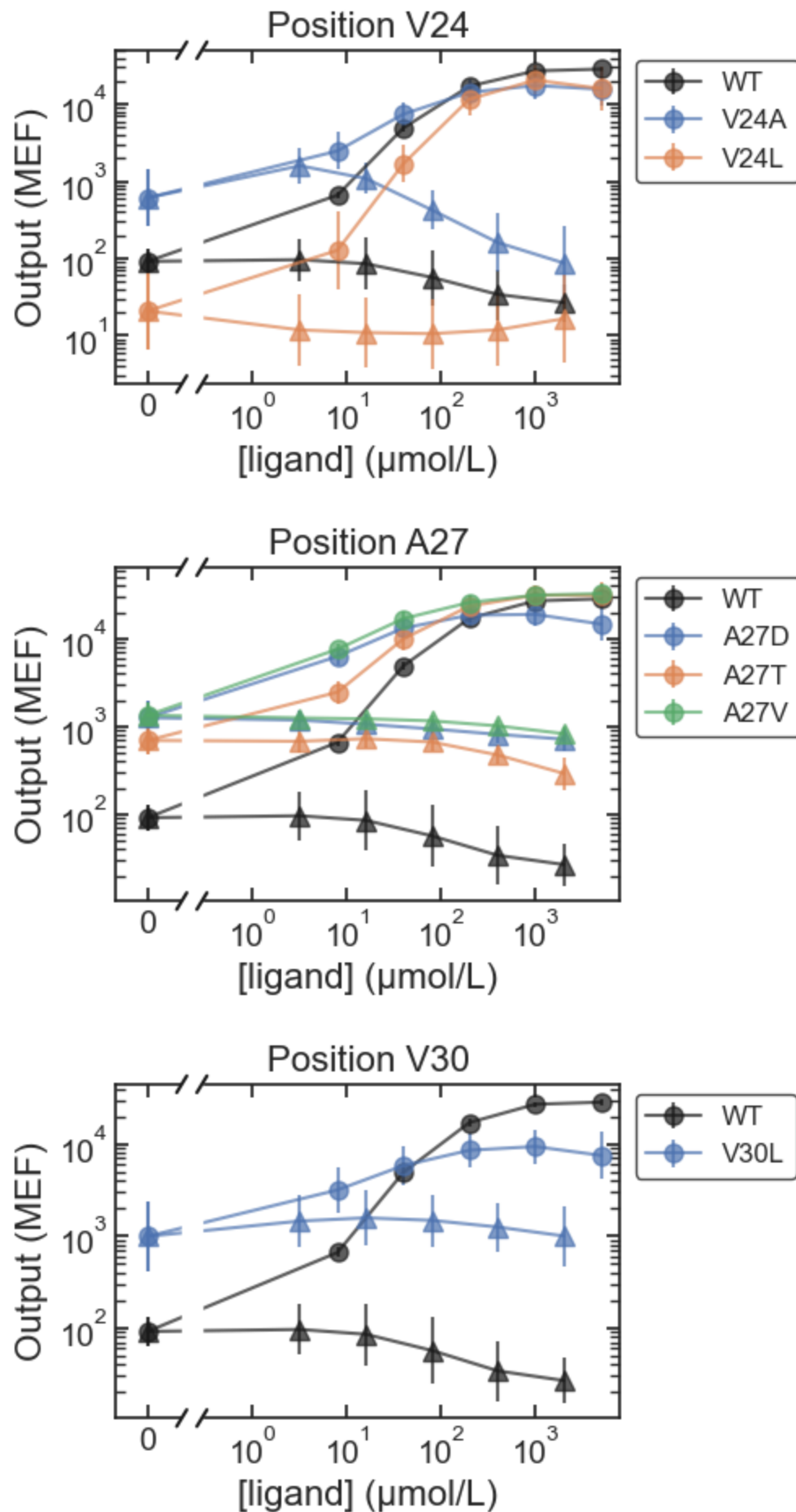
```
fig, ax = plt.subplots()
```

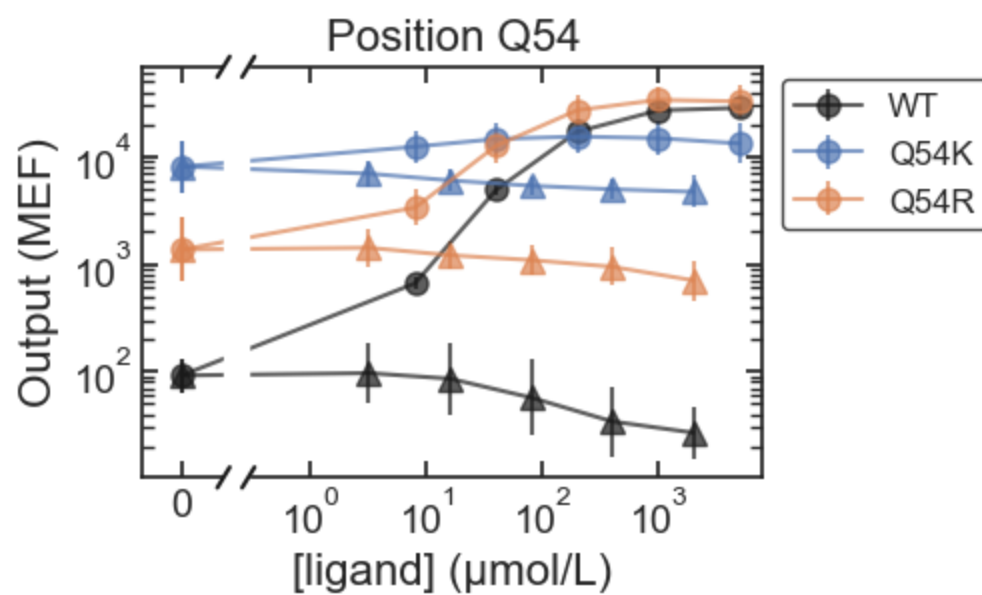
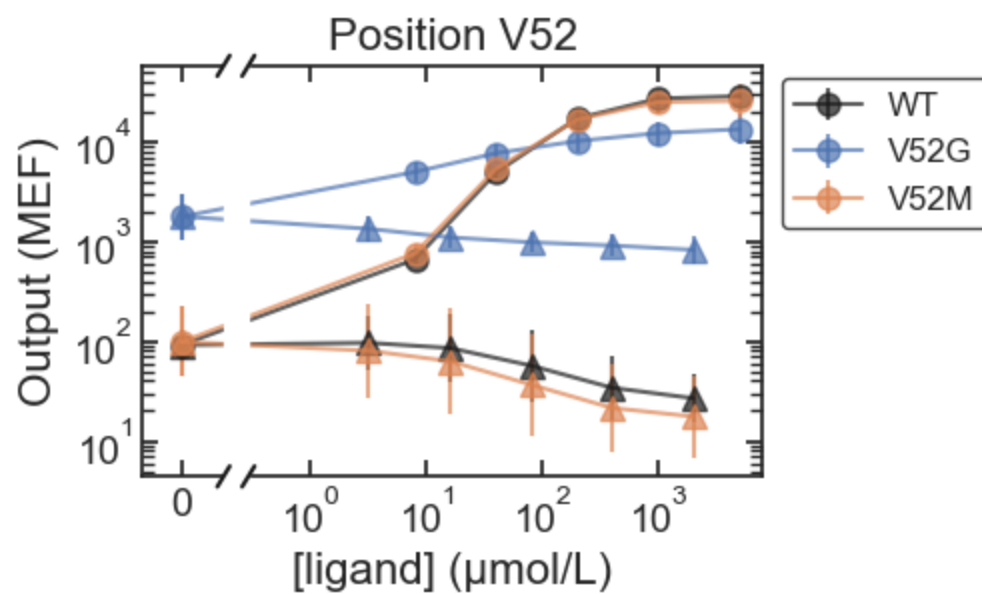
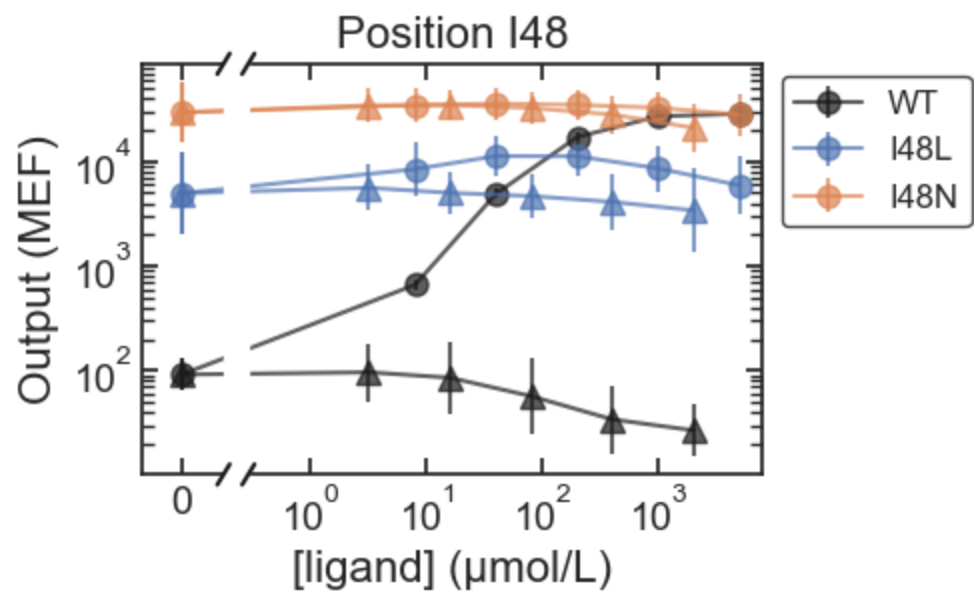
IPTG

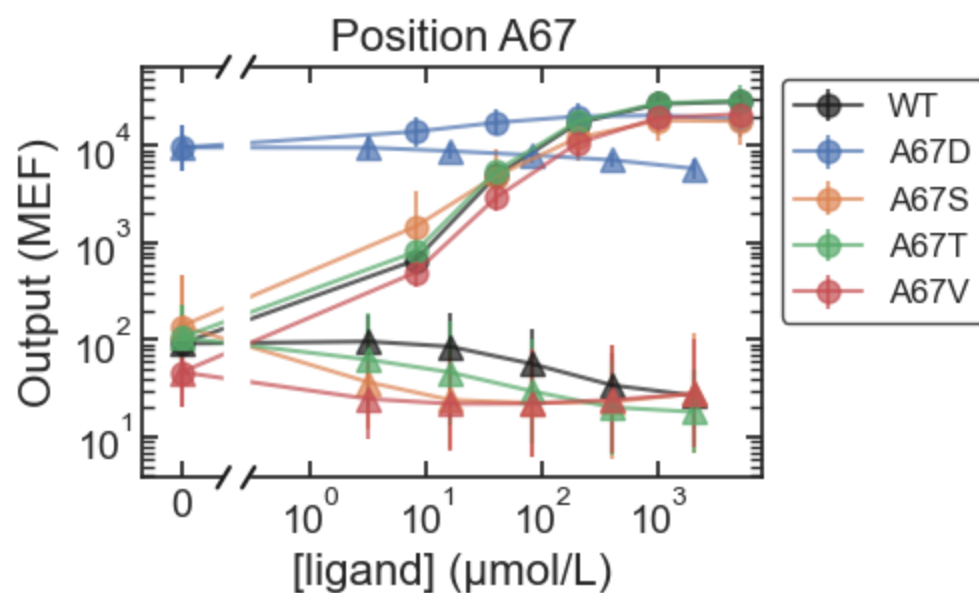
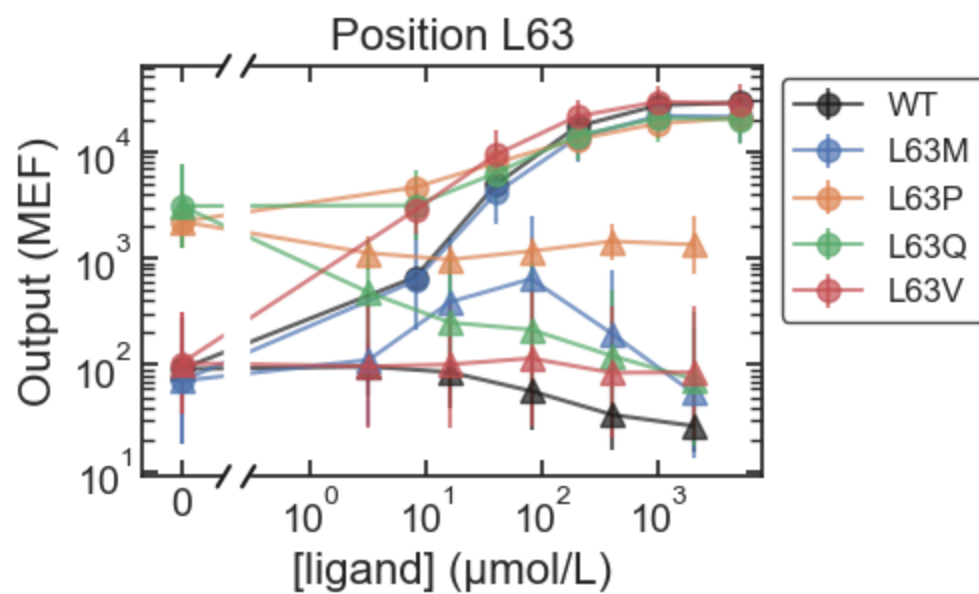
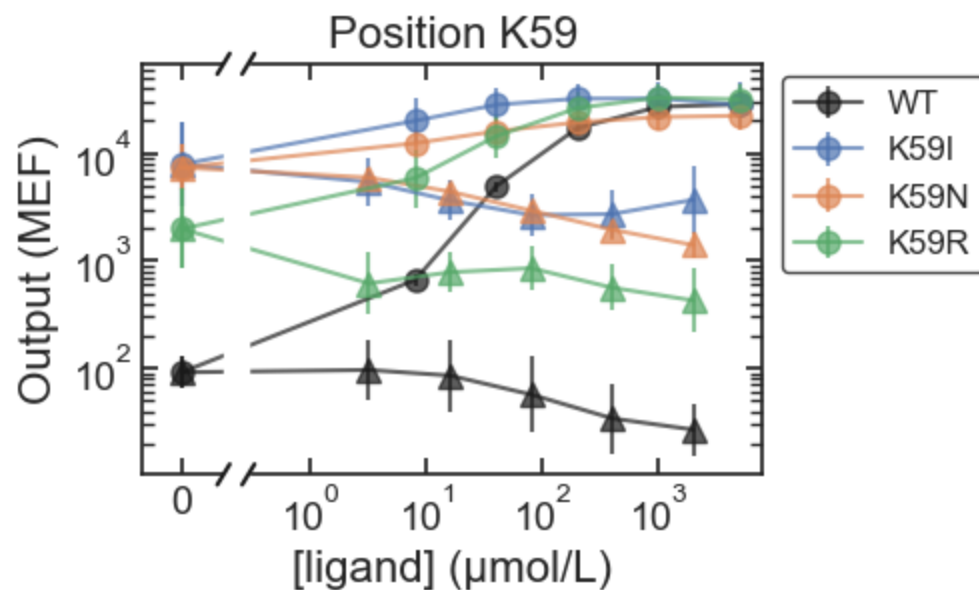


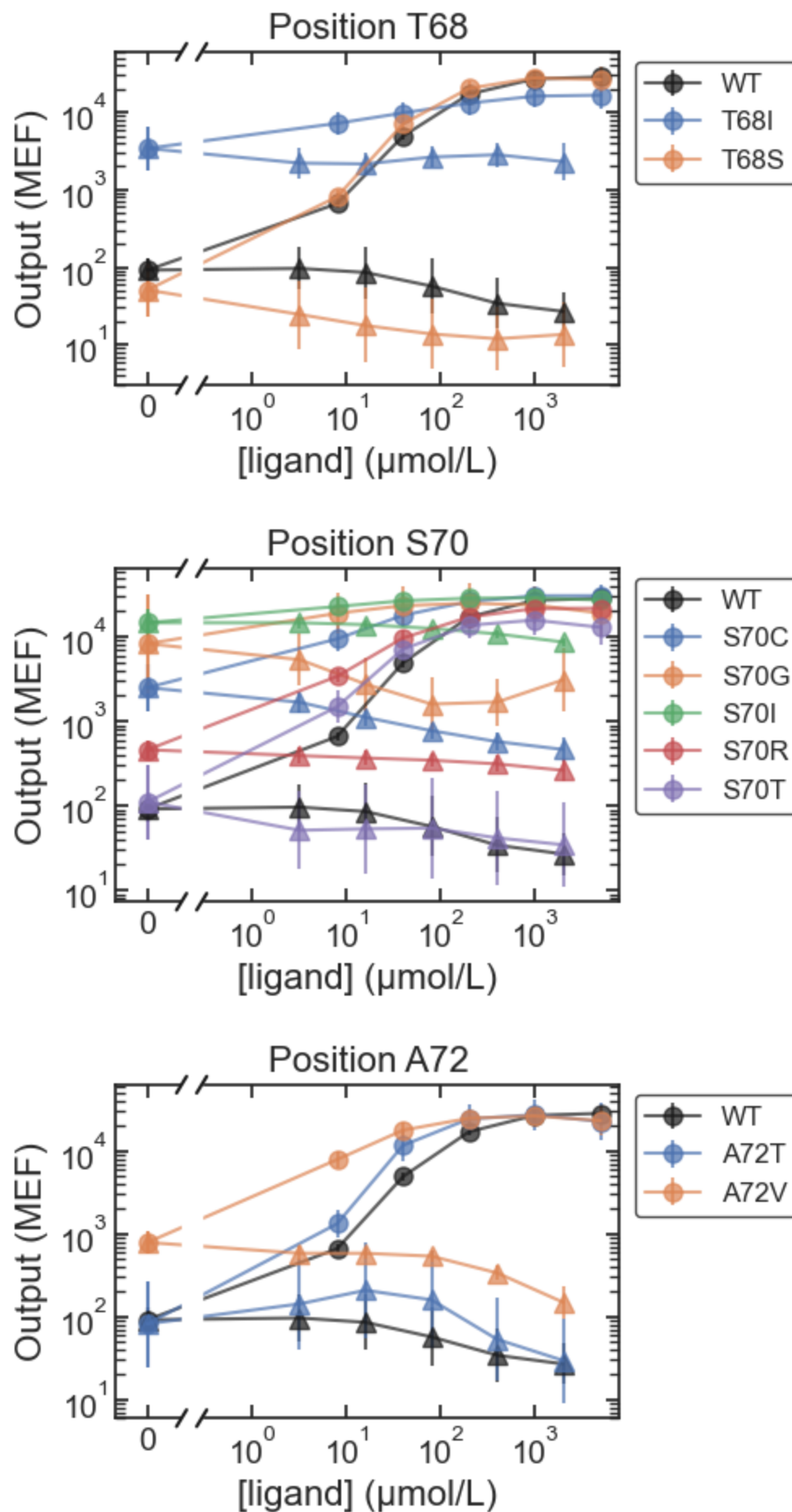


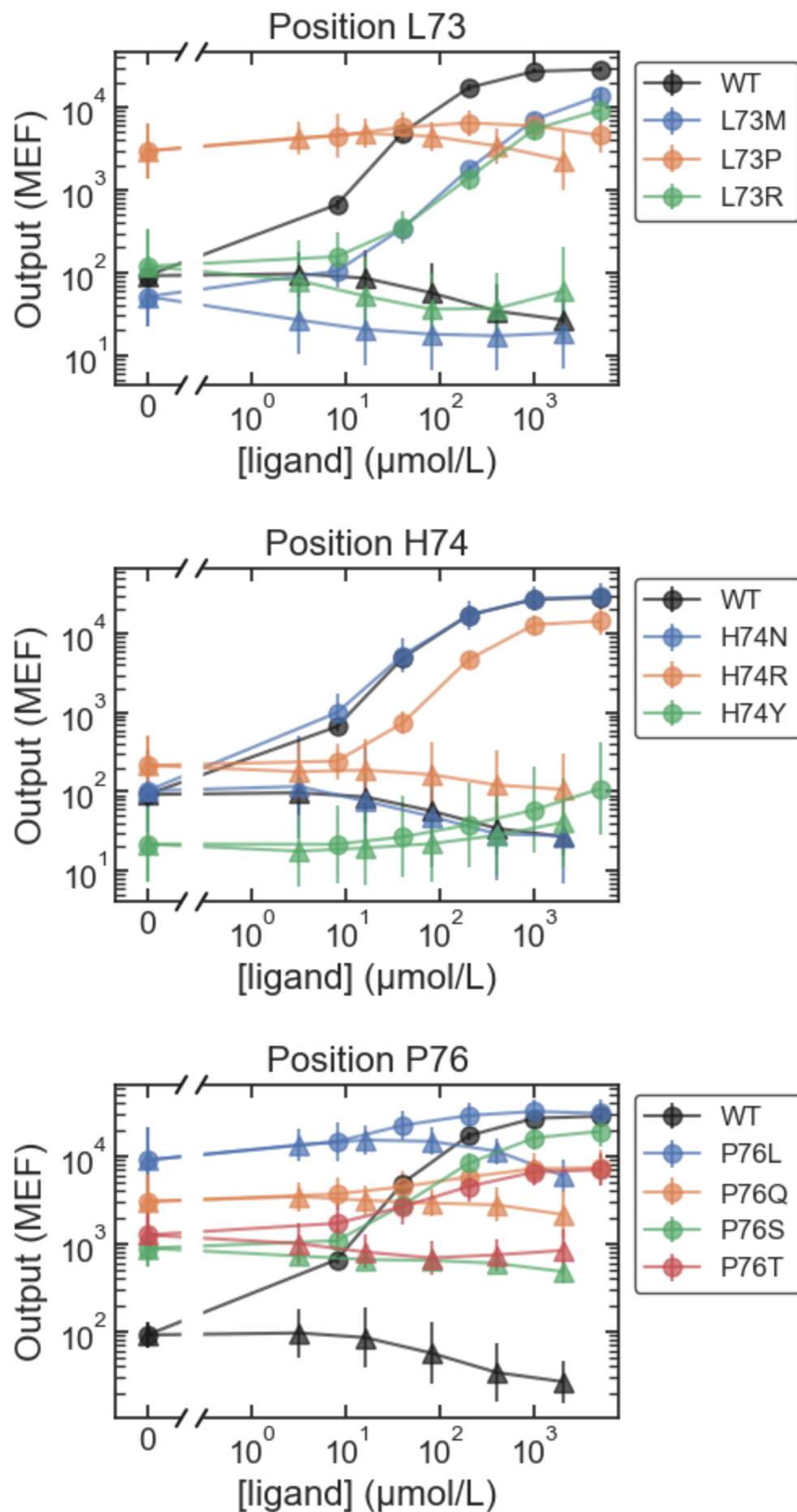


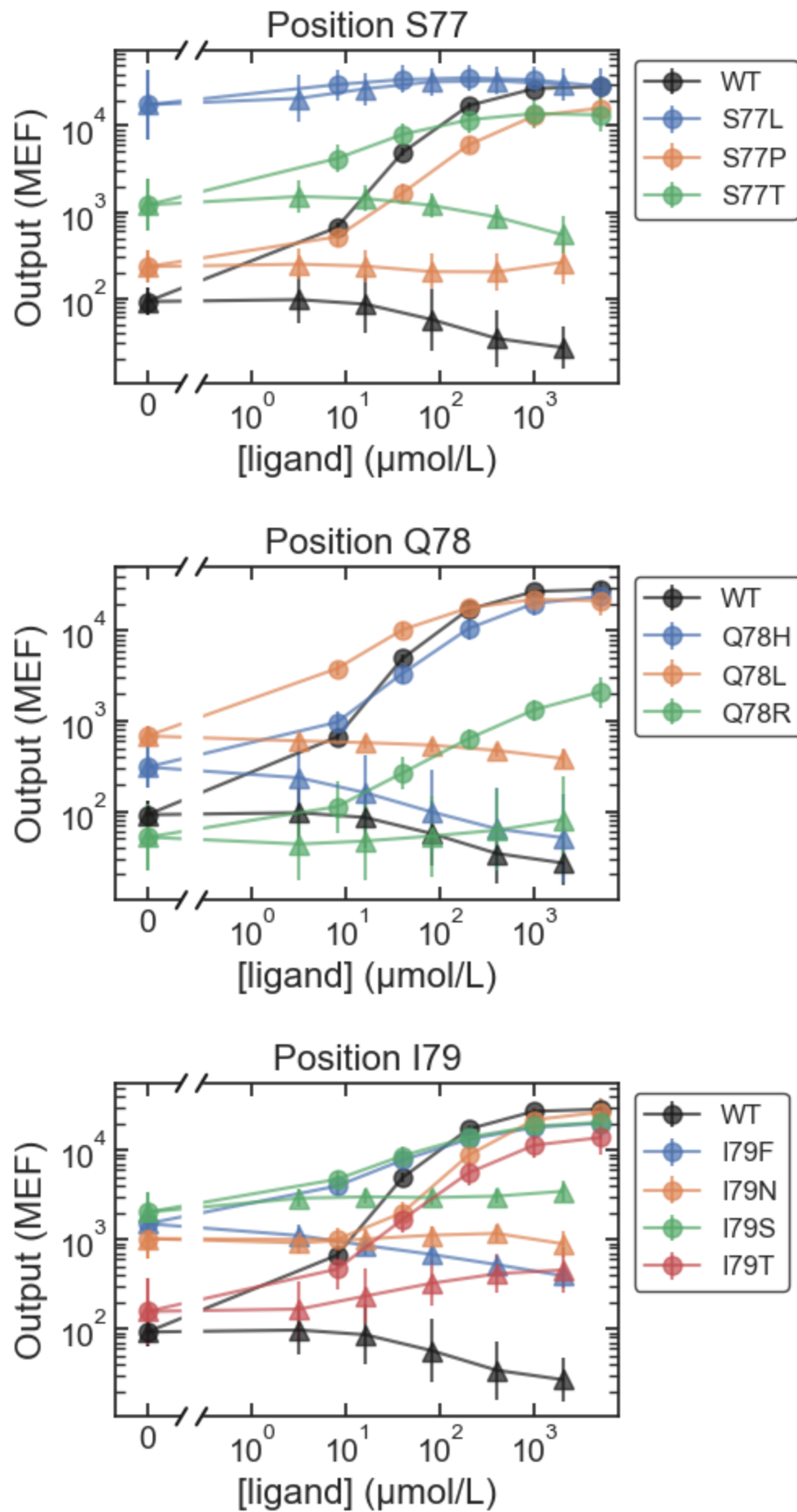


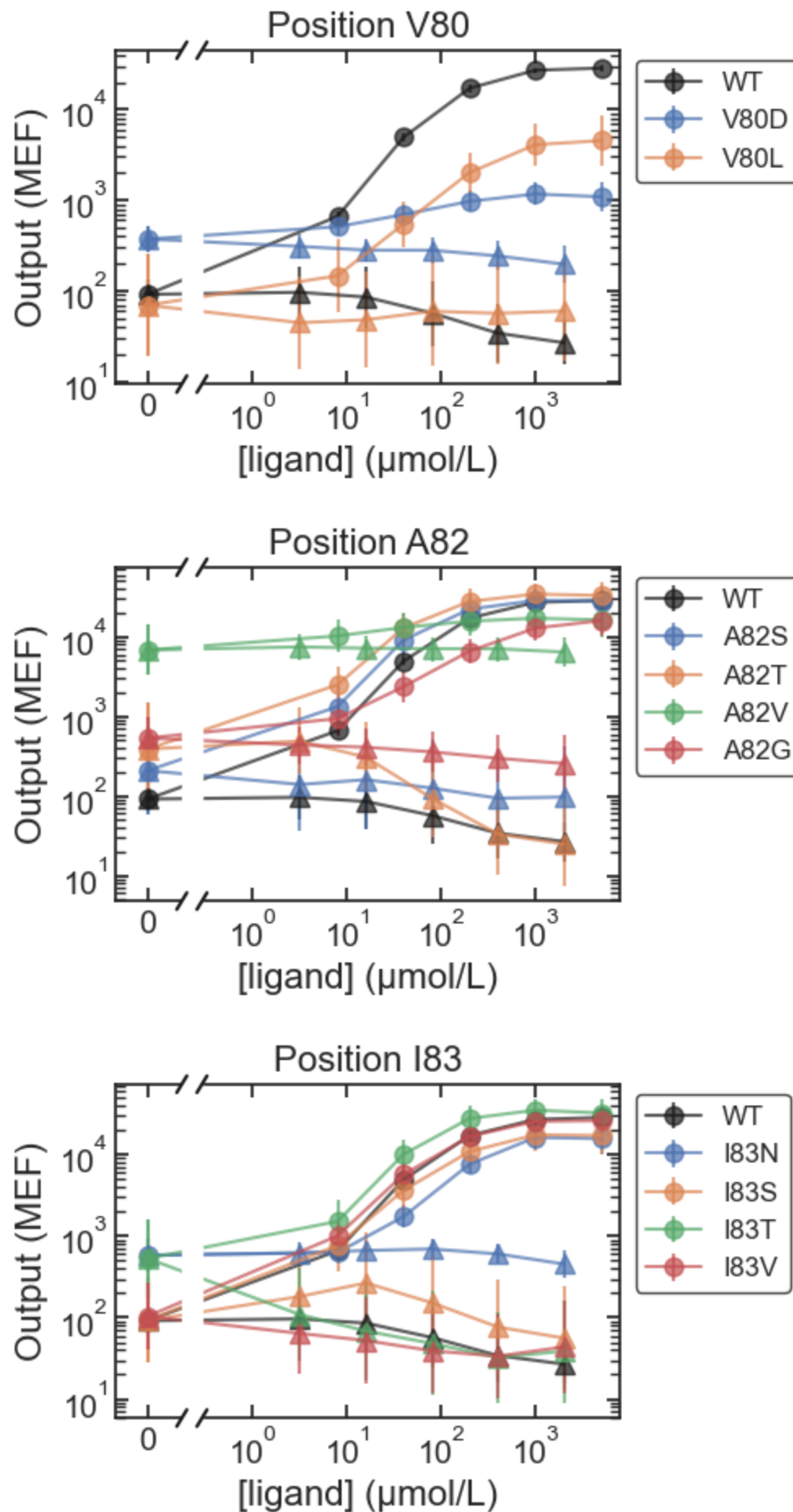


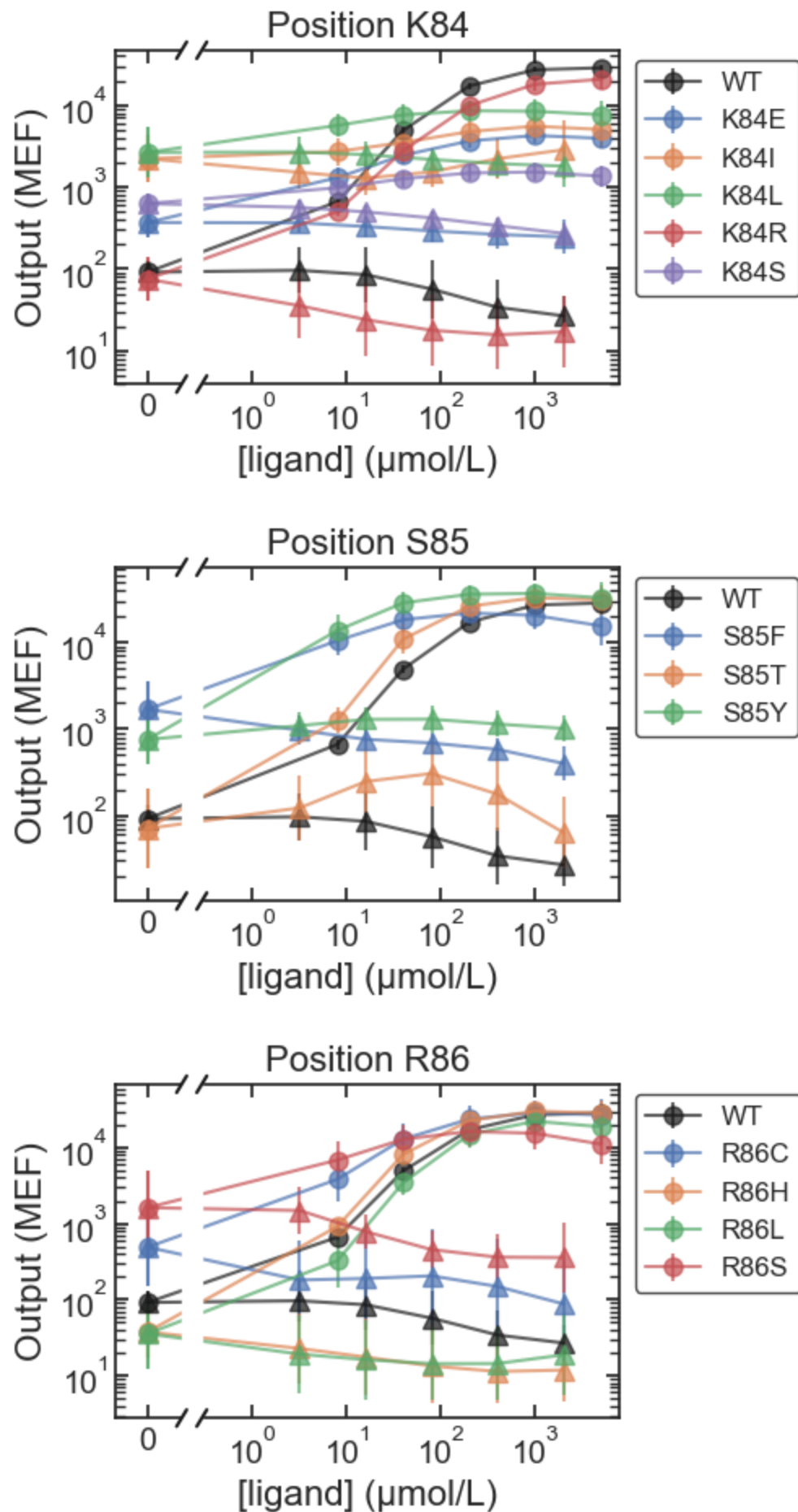


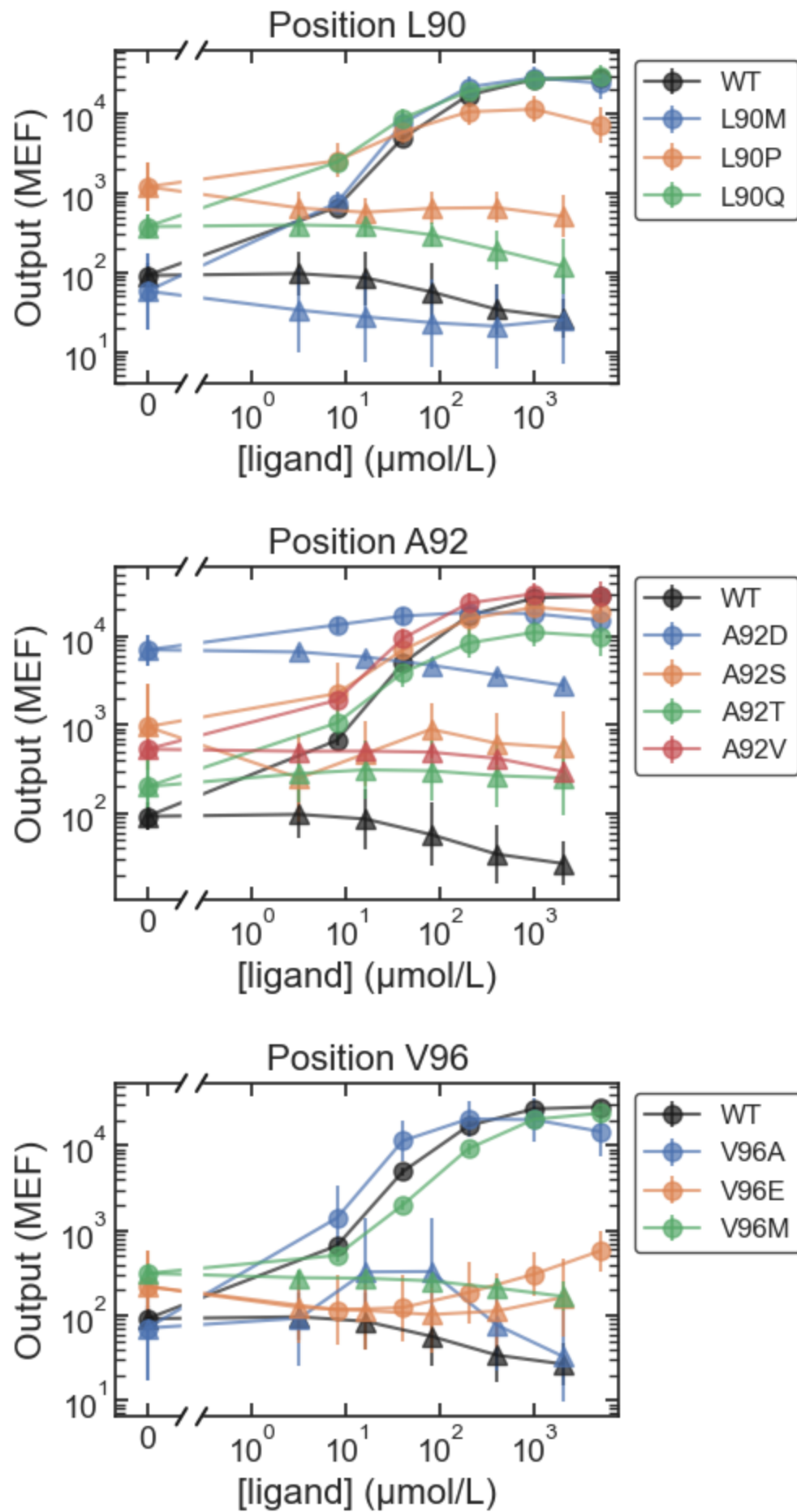


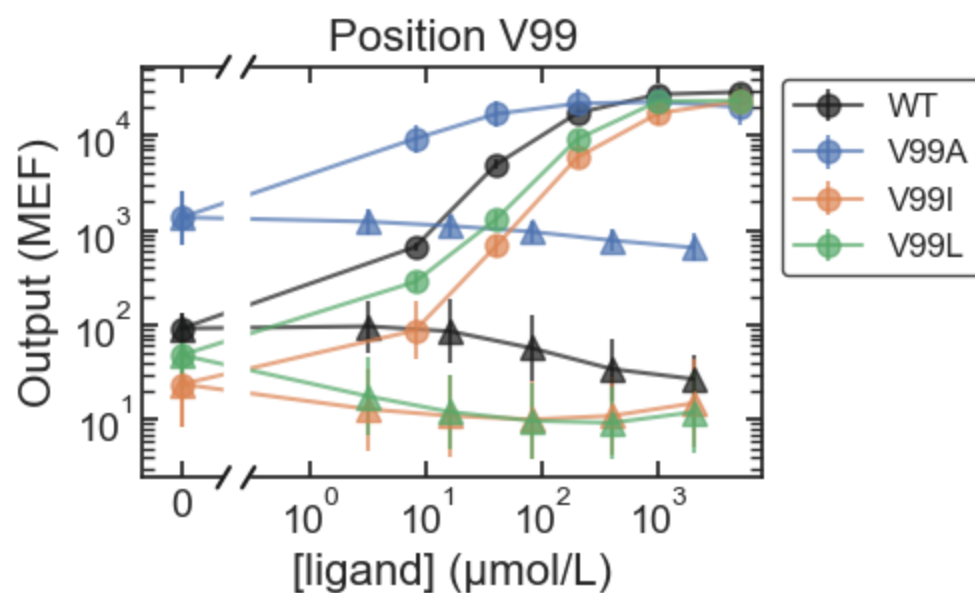
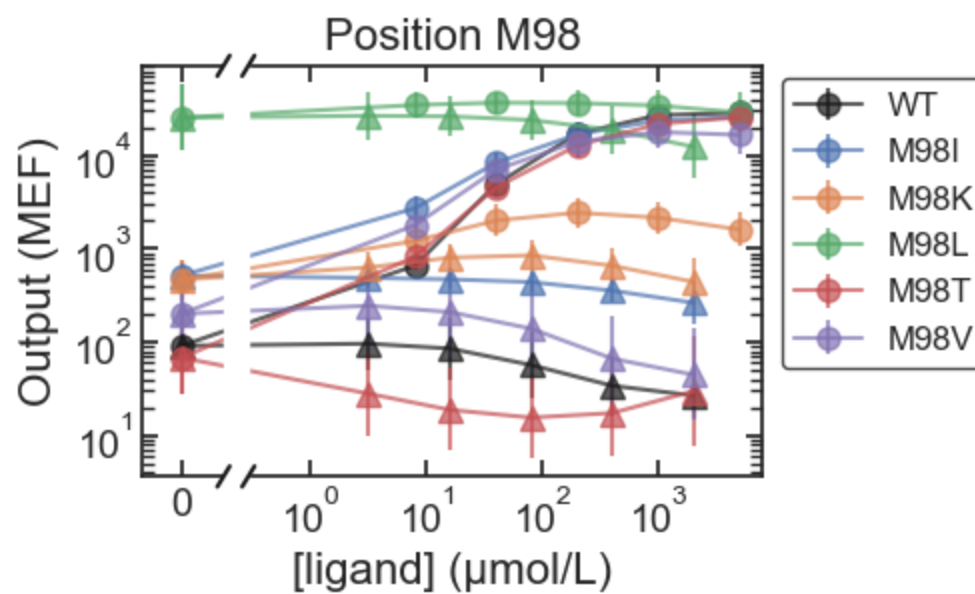
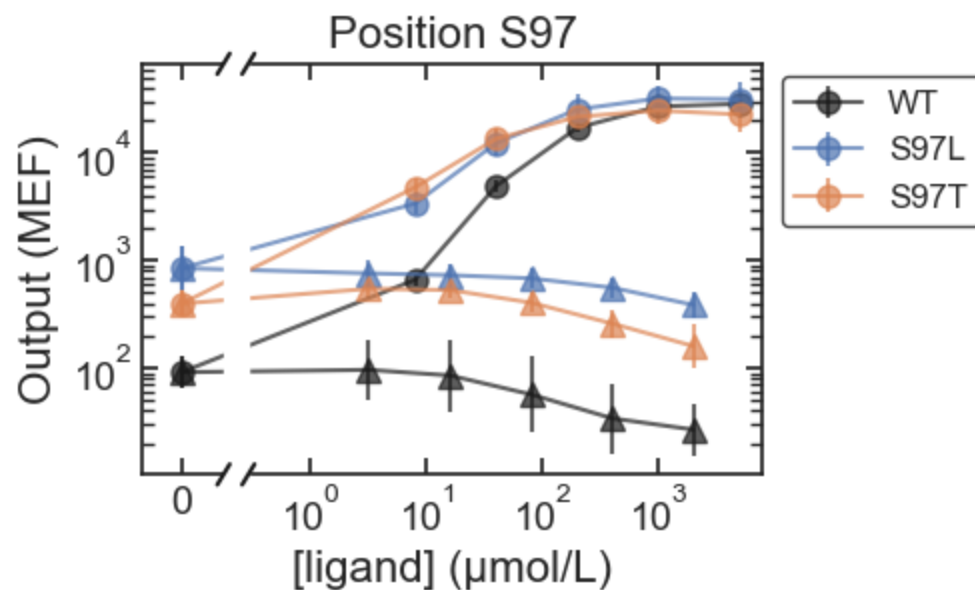


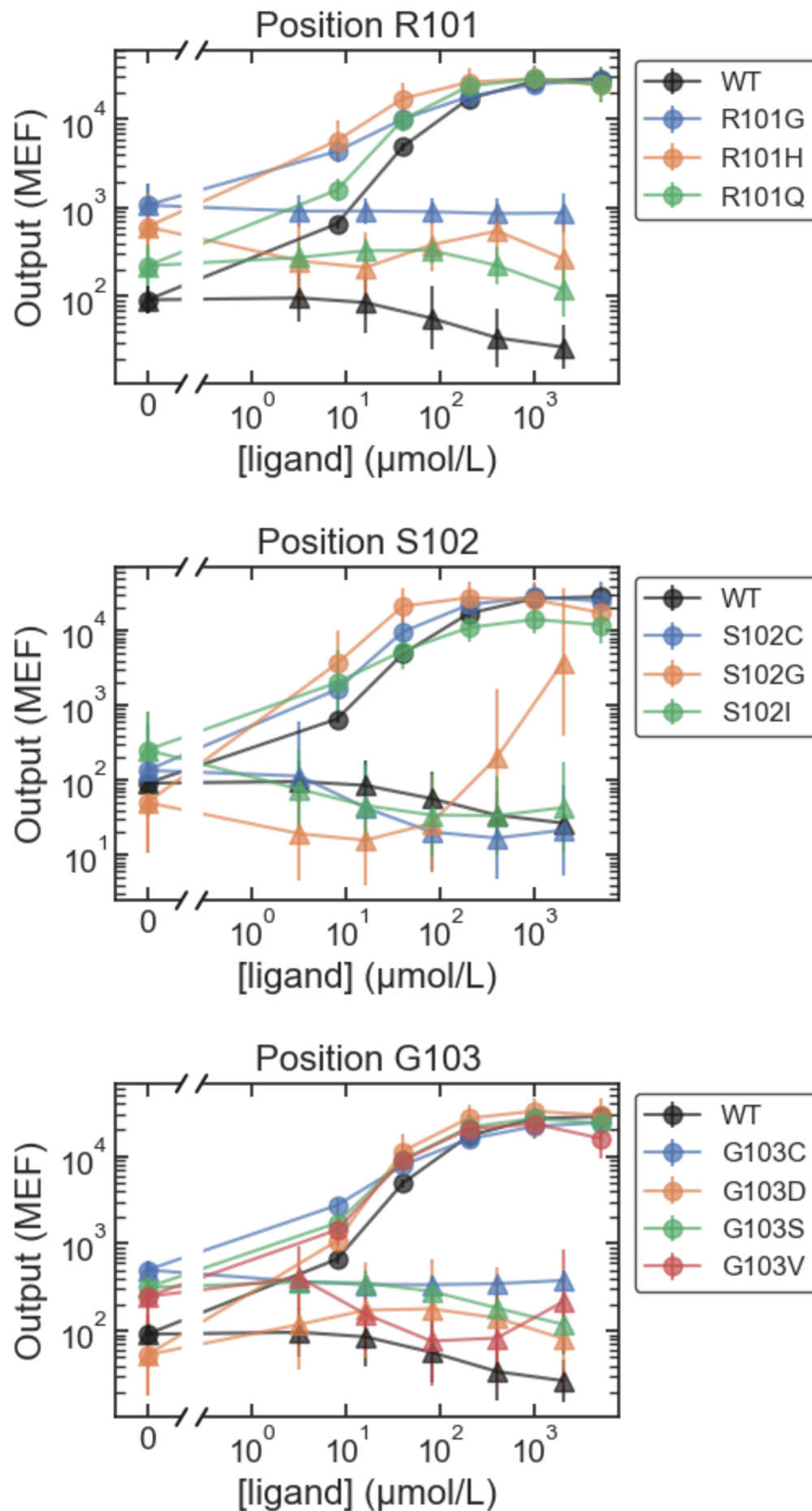


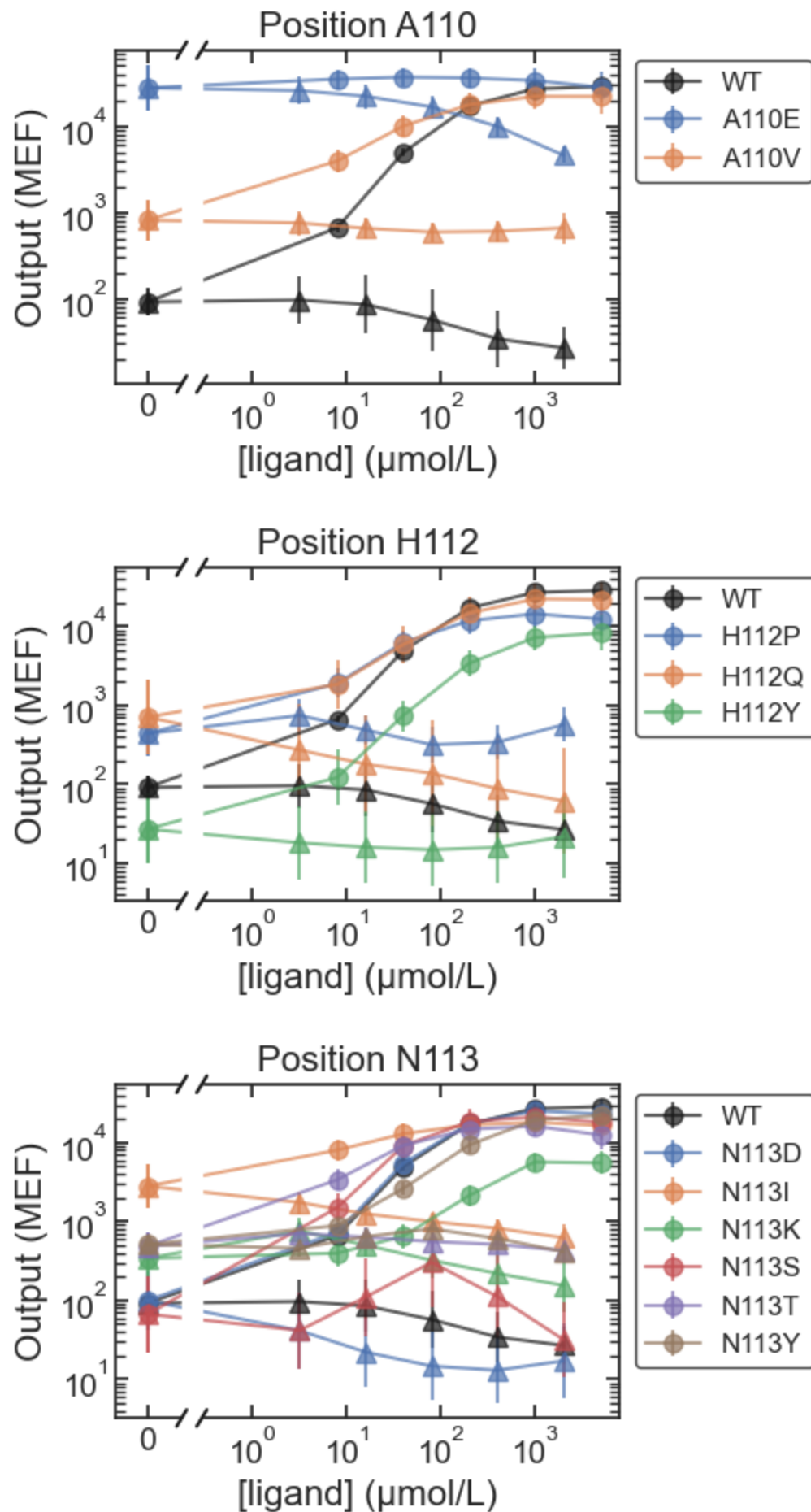


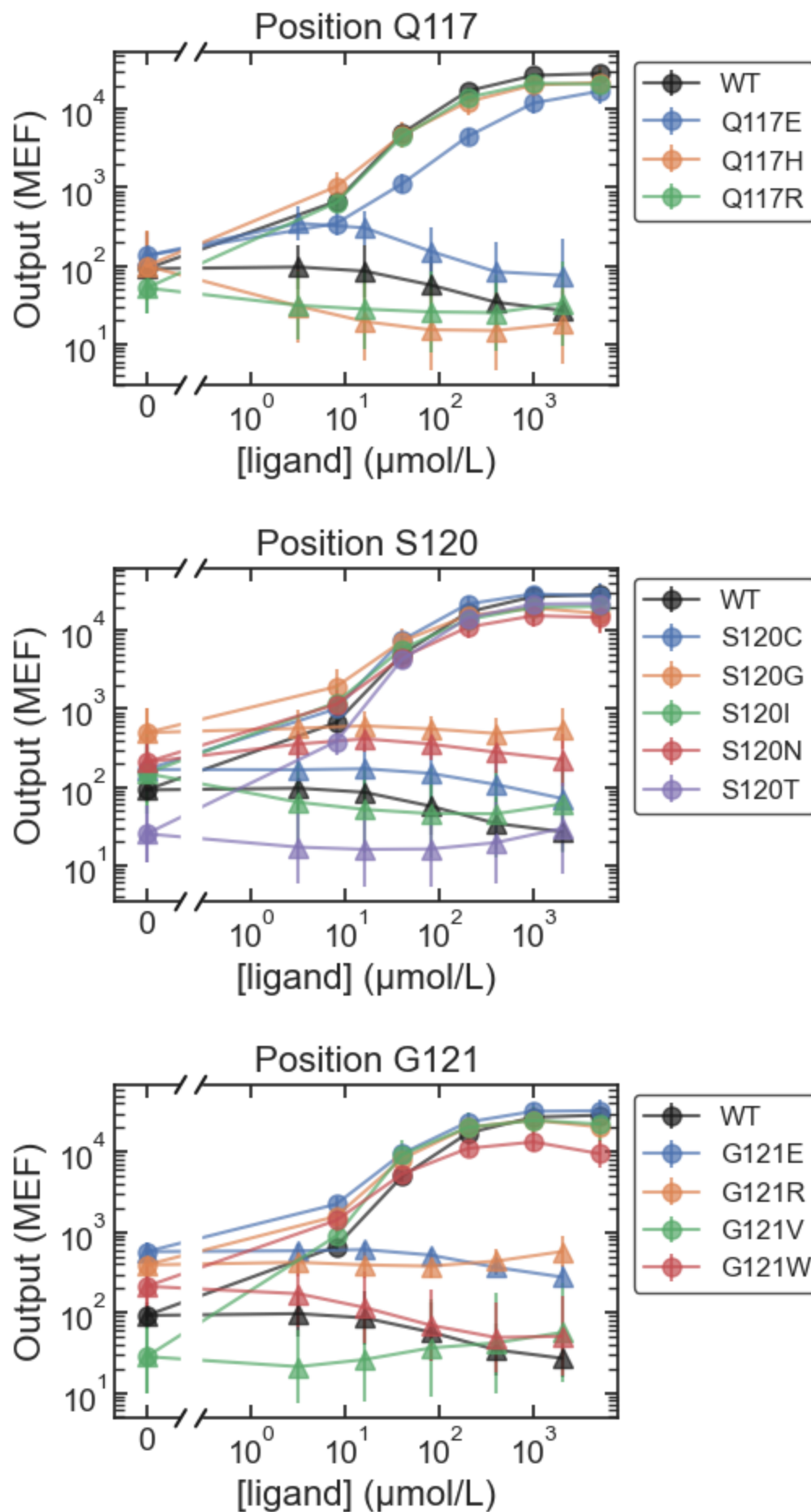


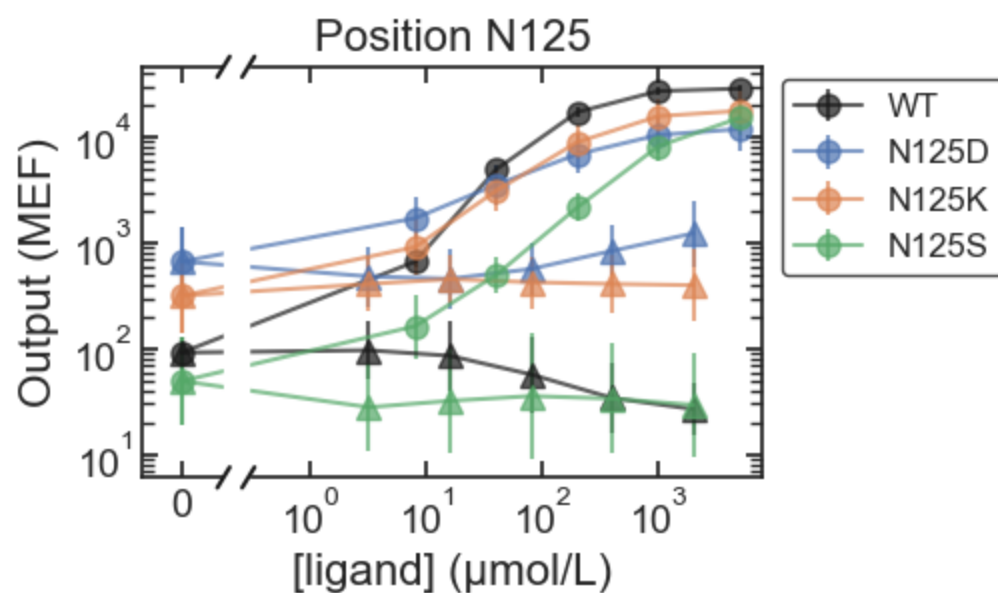
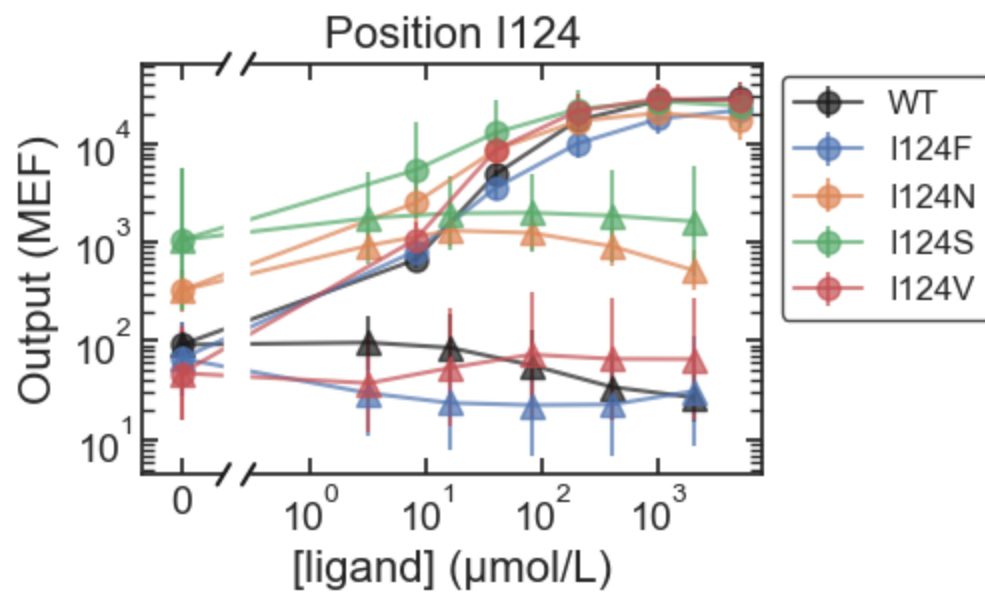
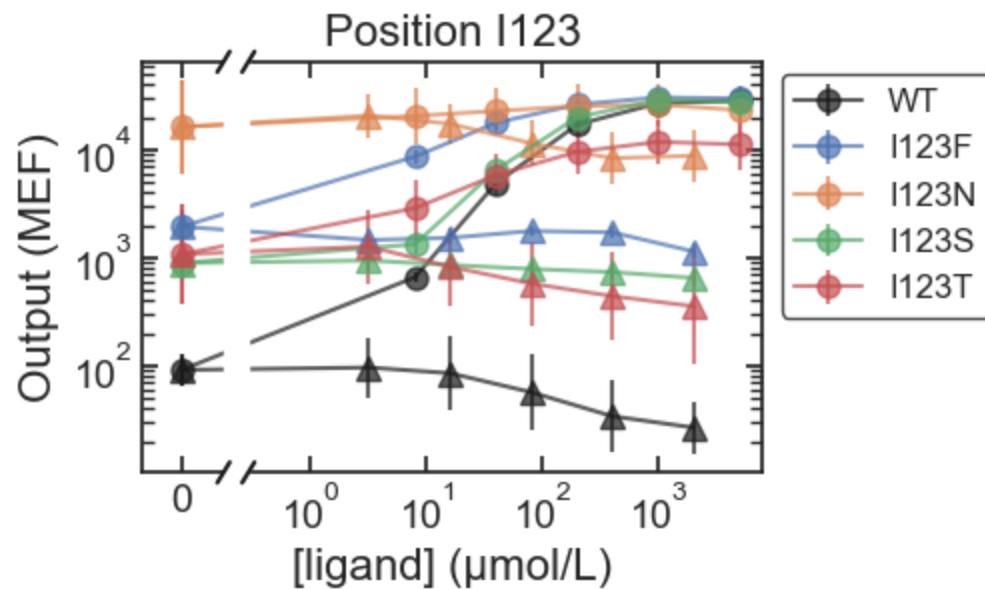


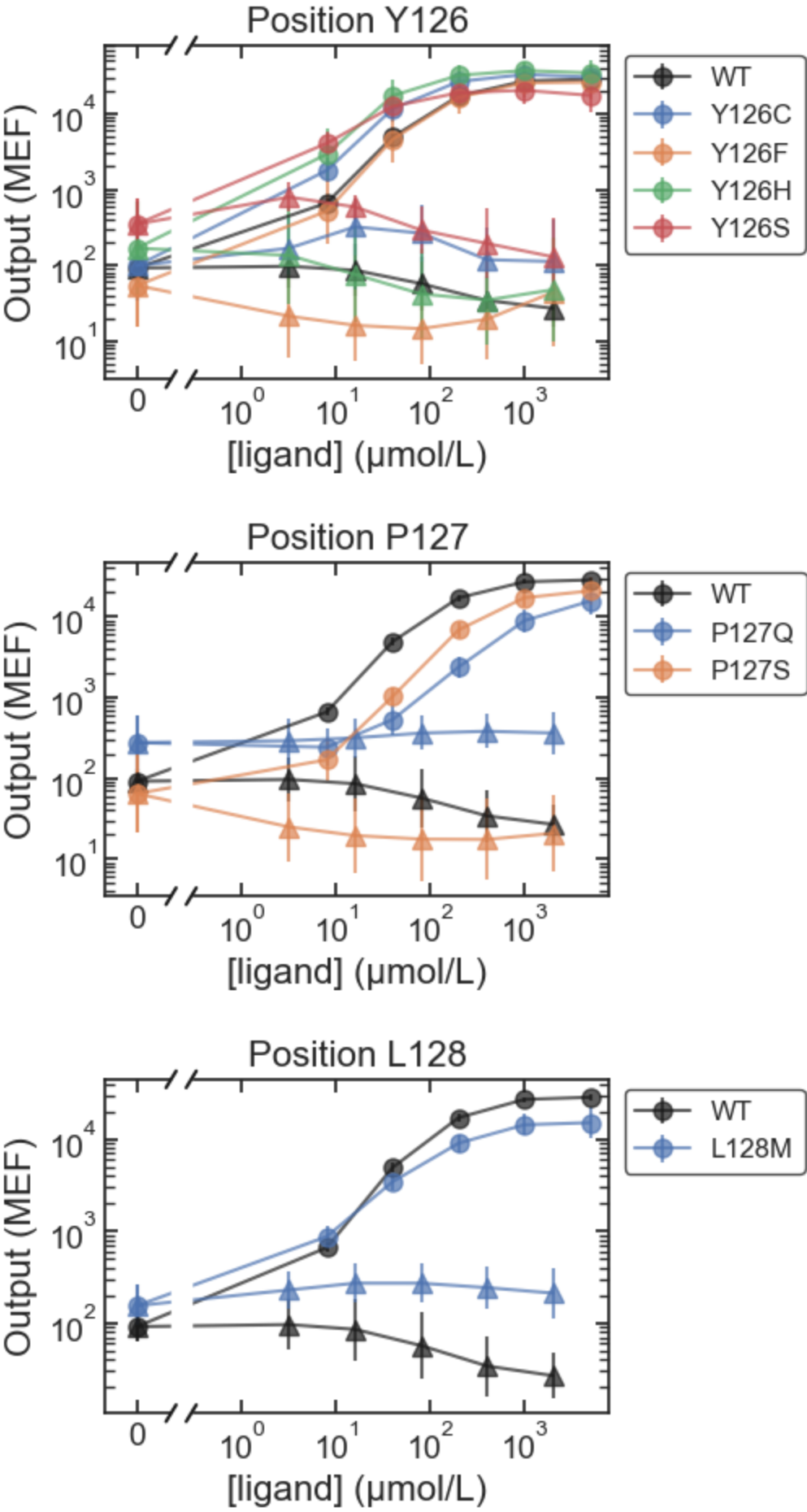


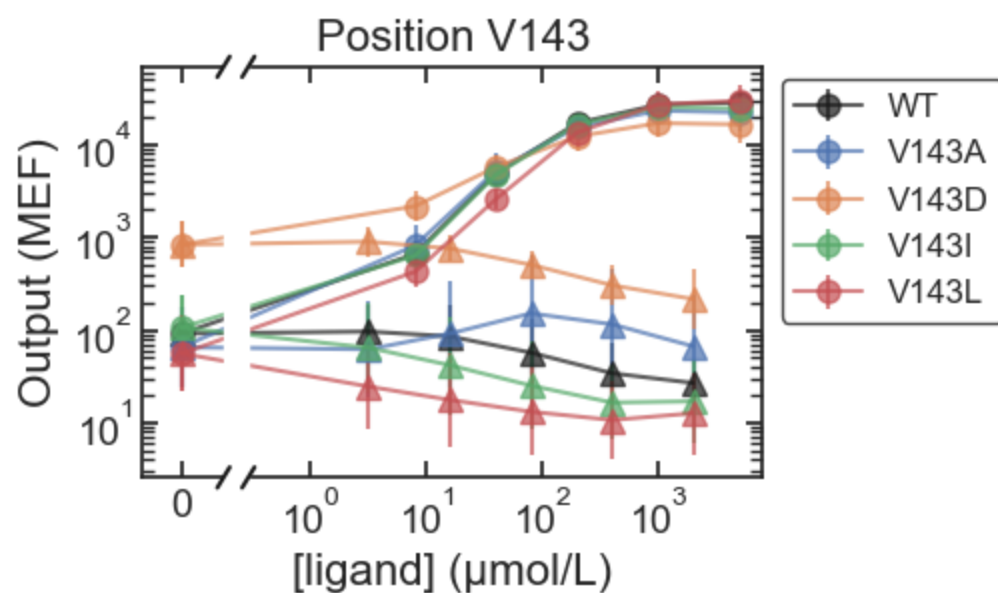
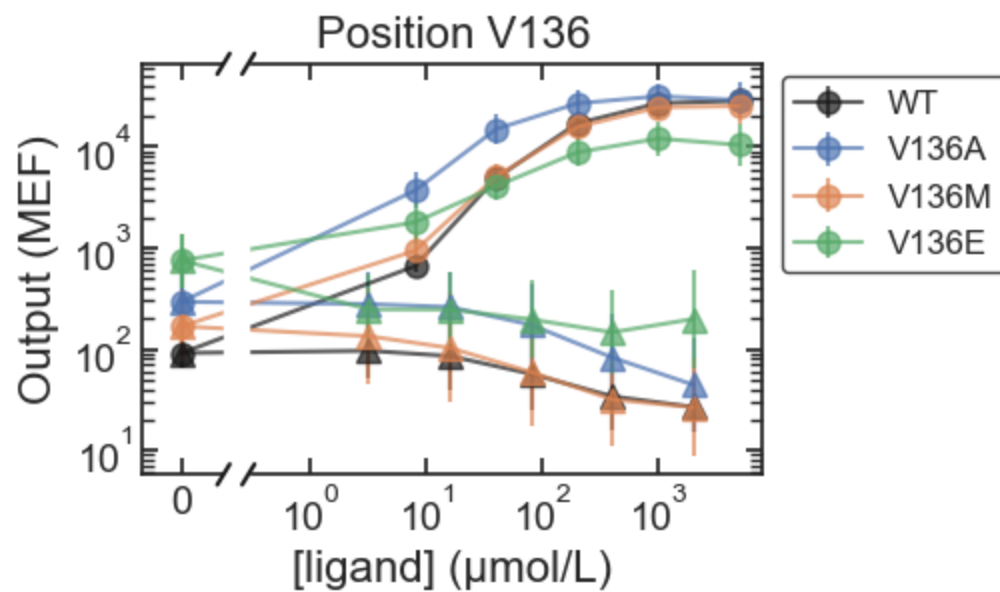
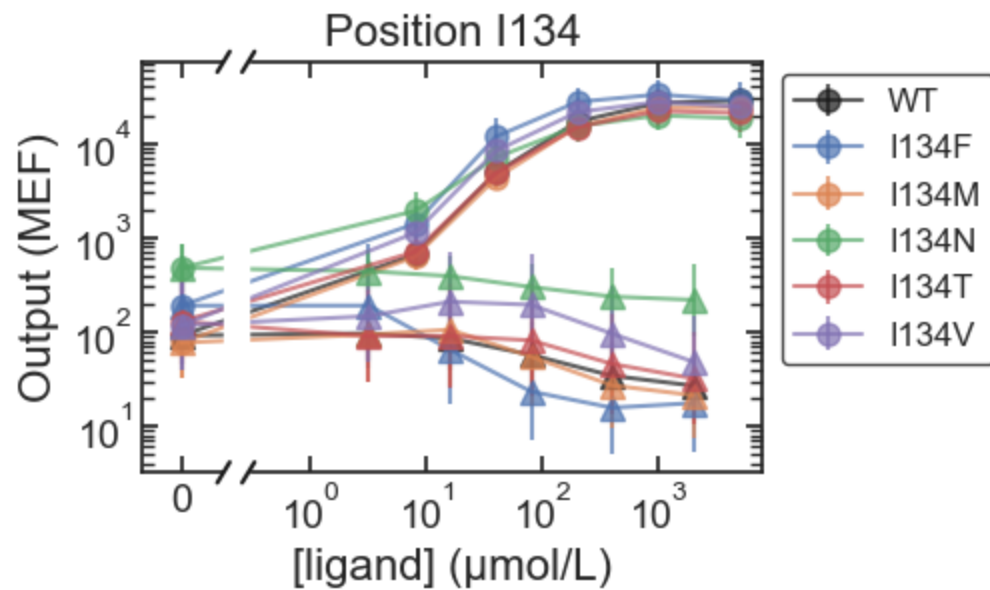


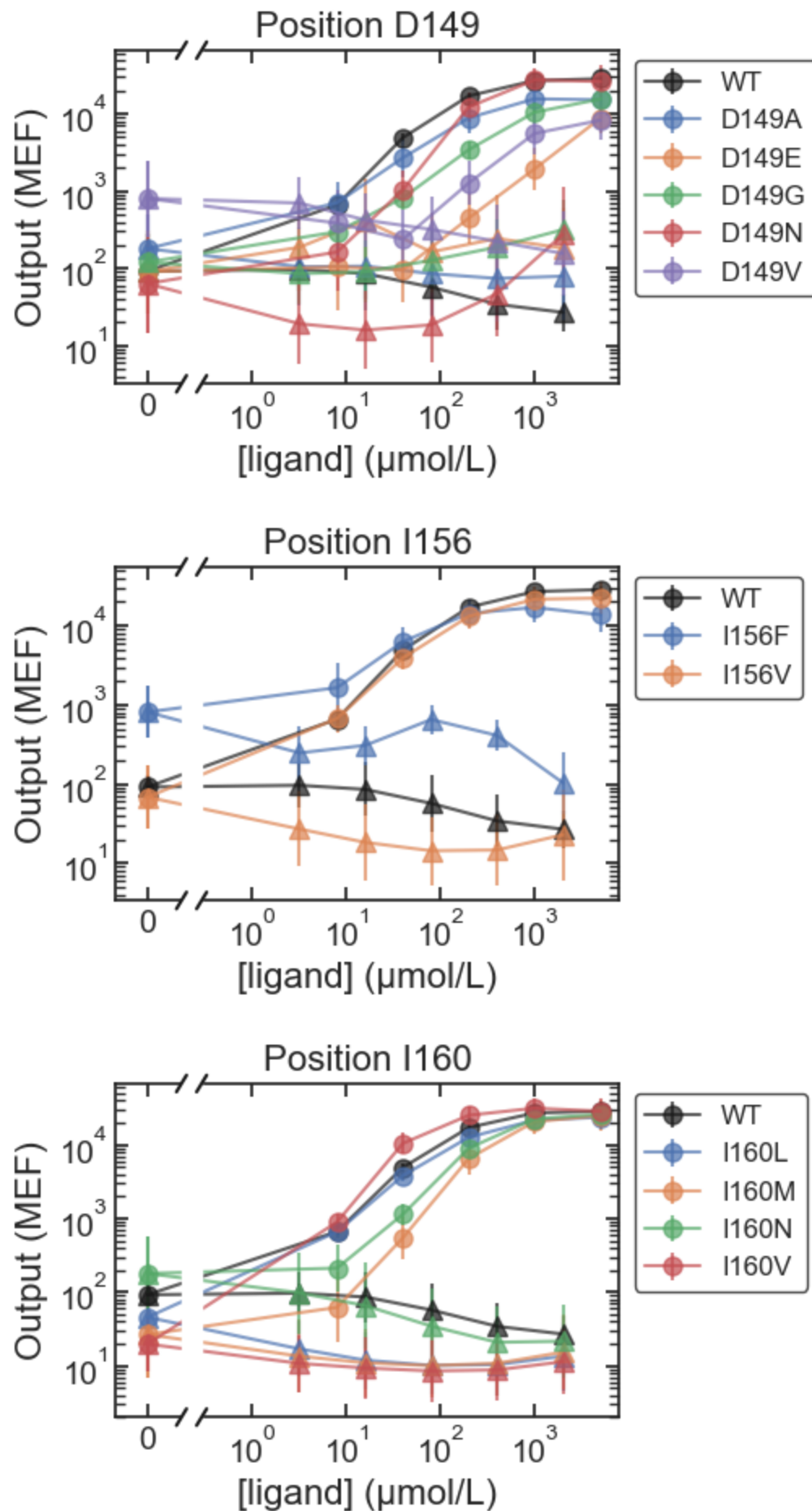


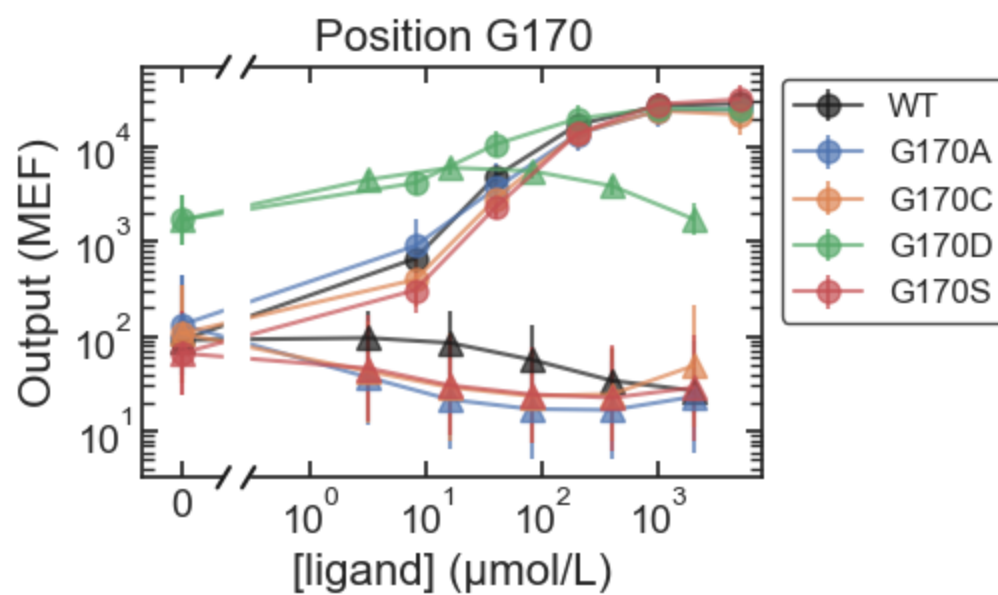
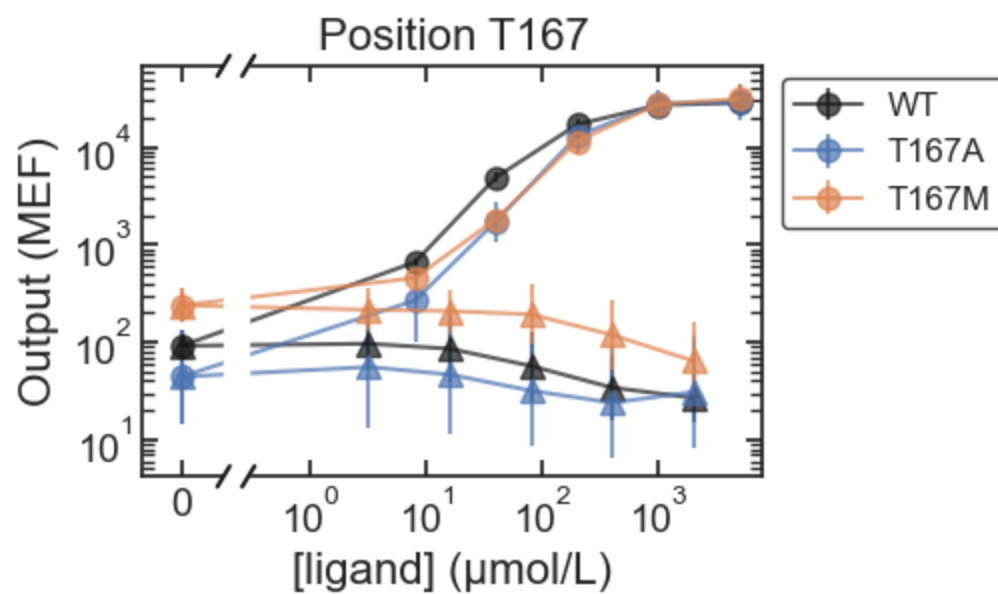
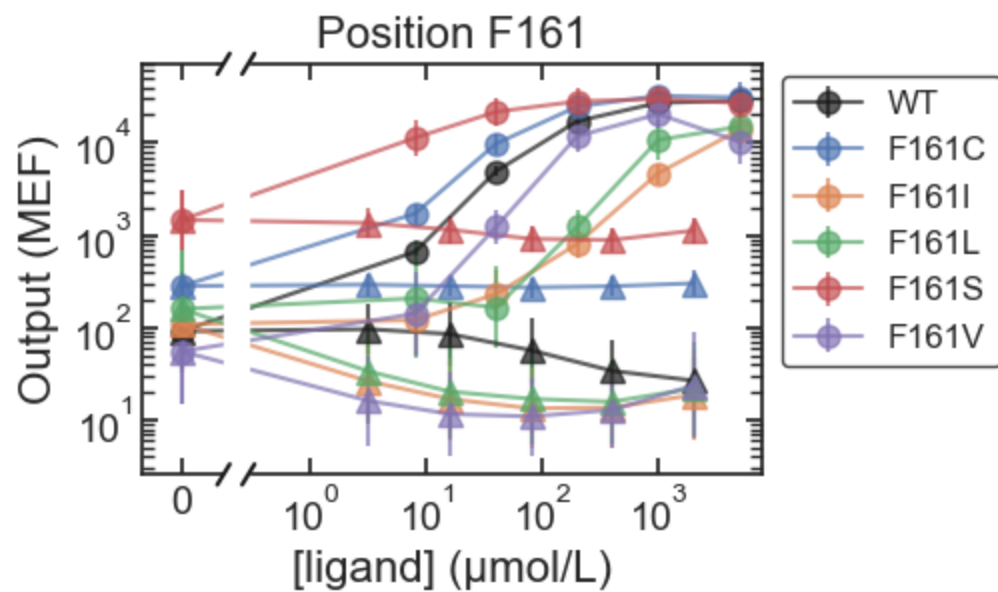


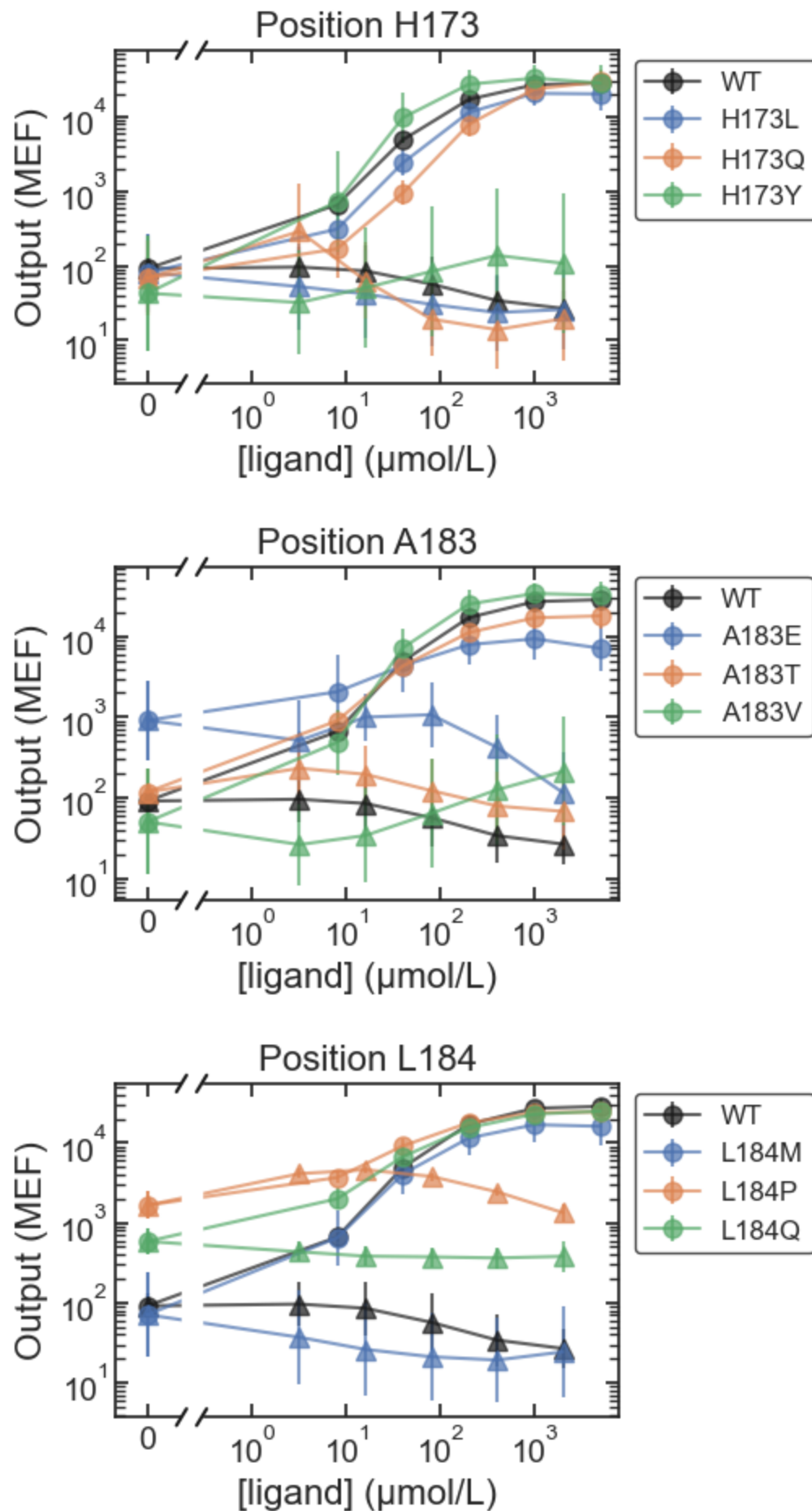


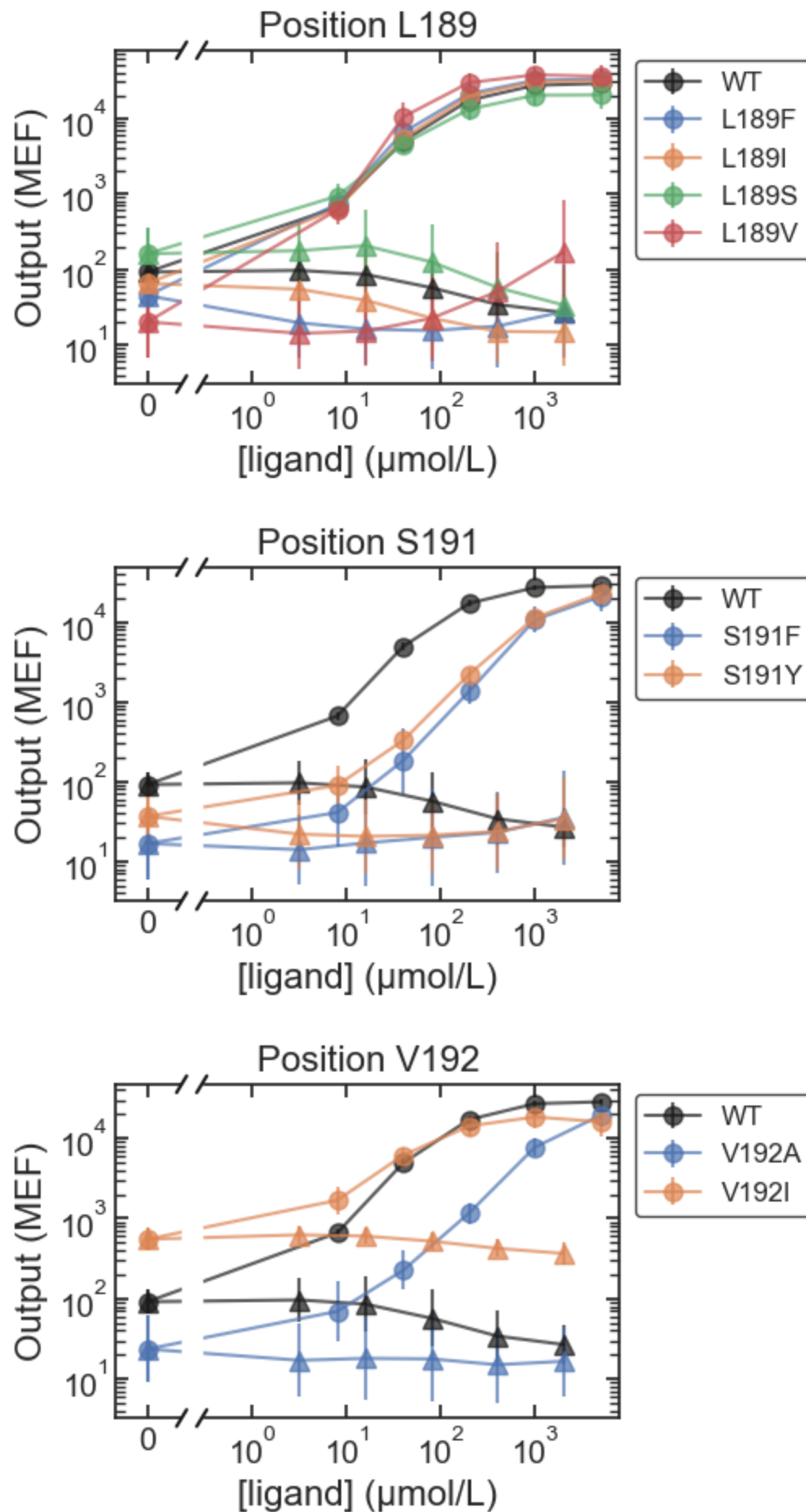


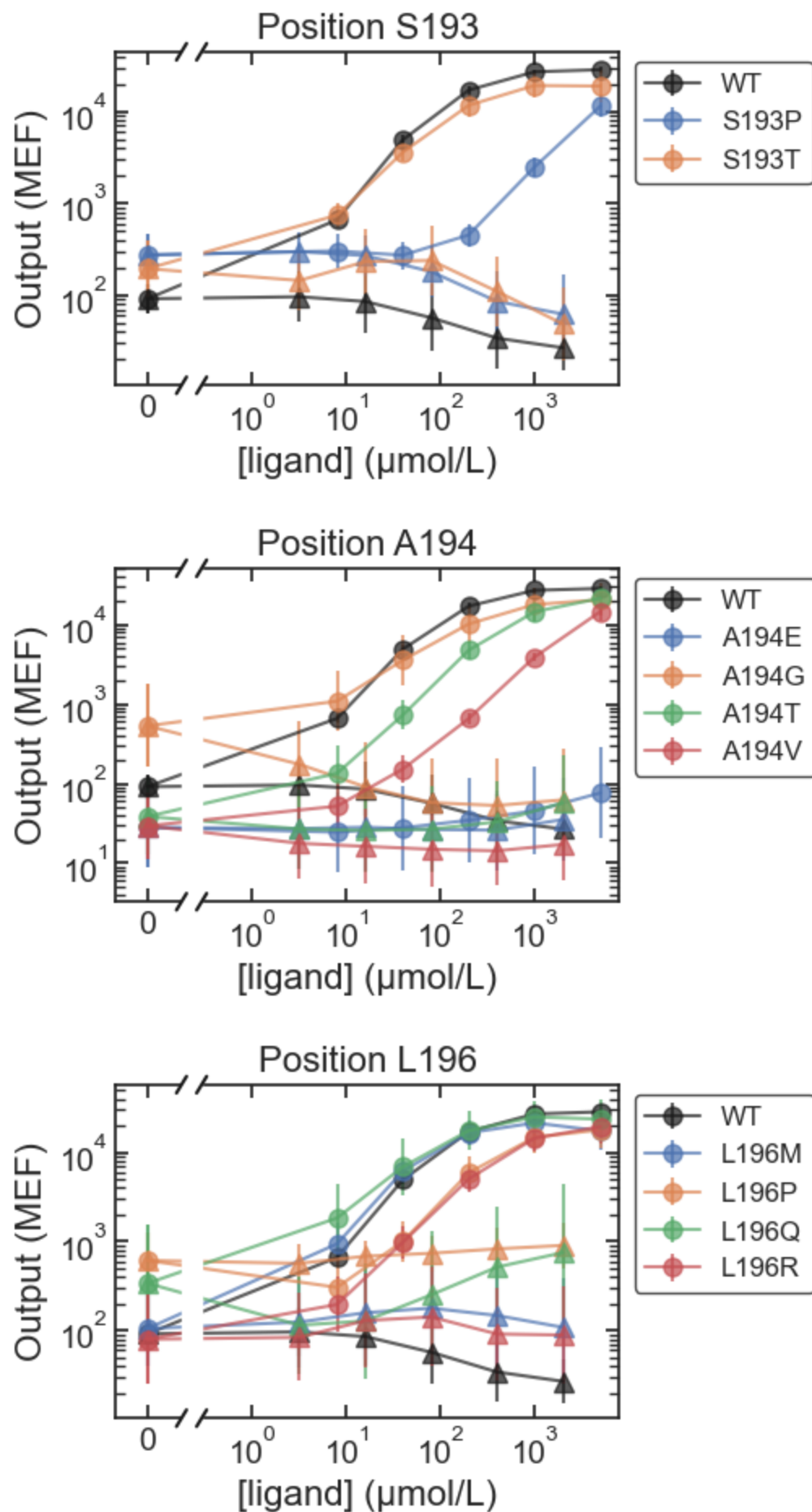


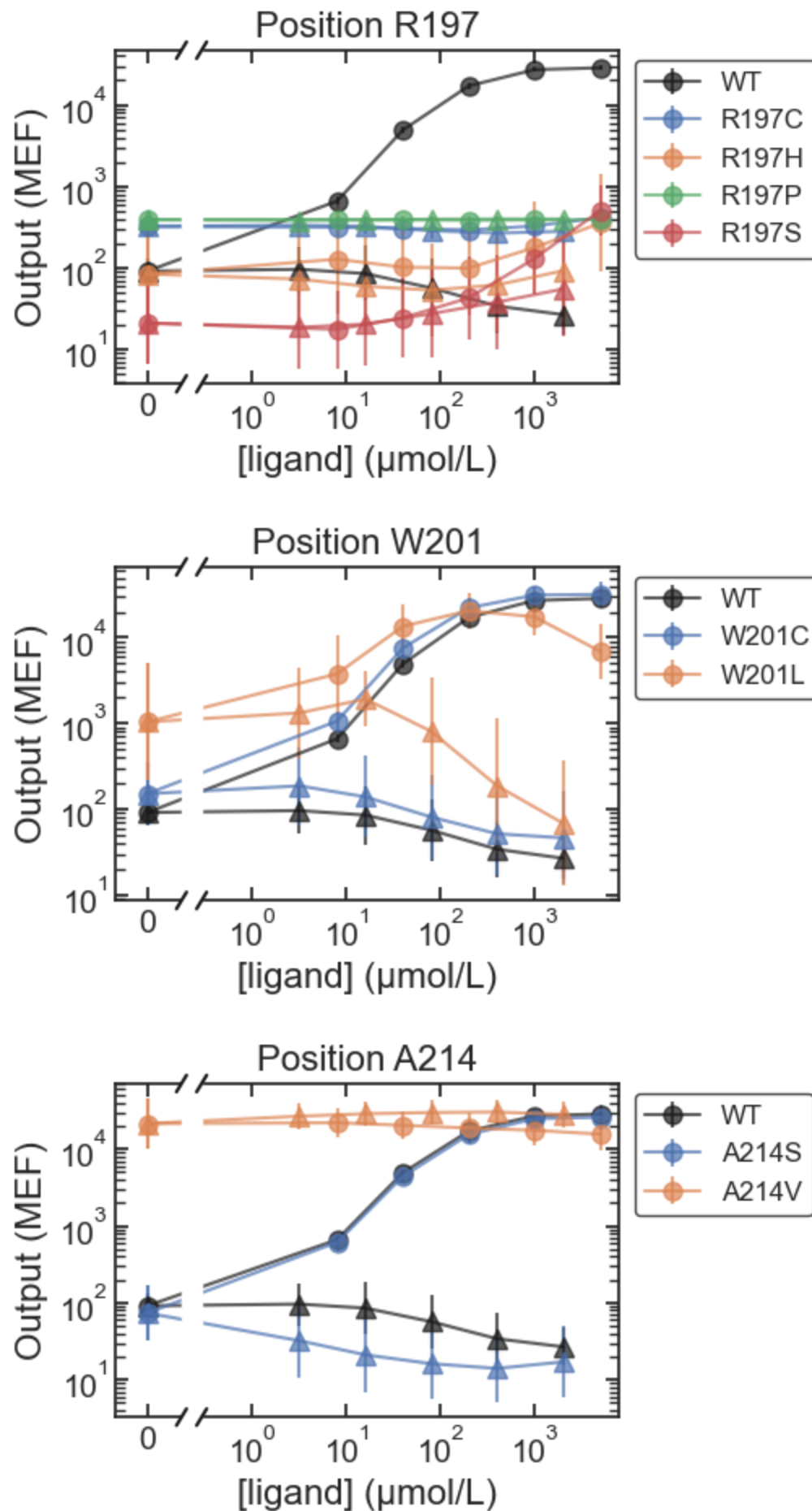


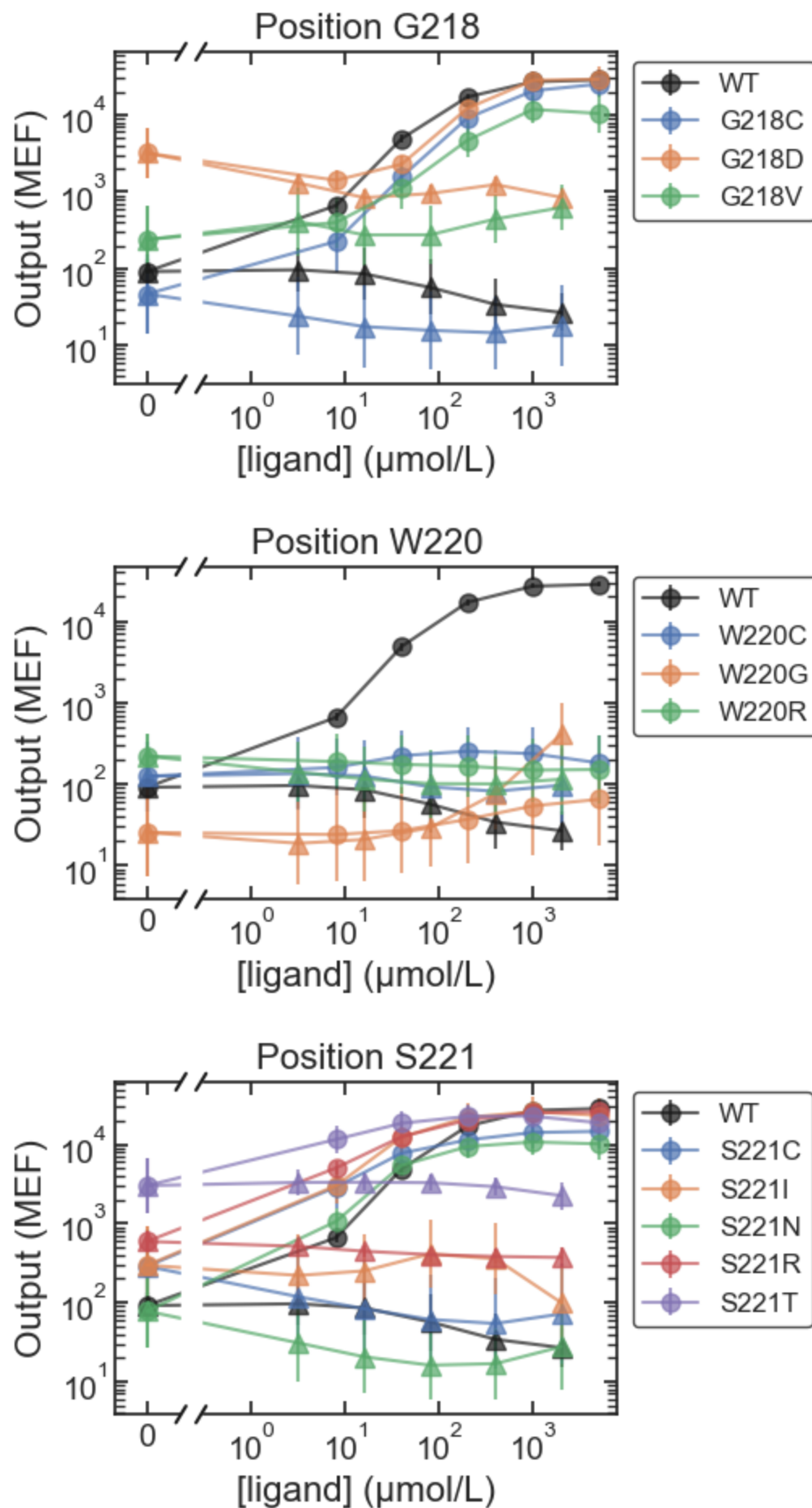


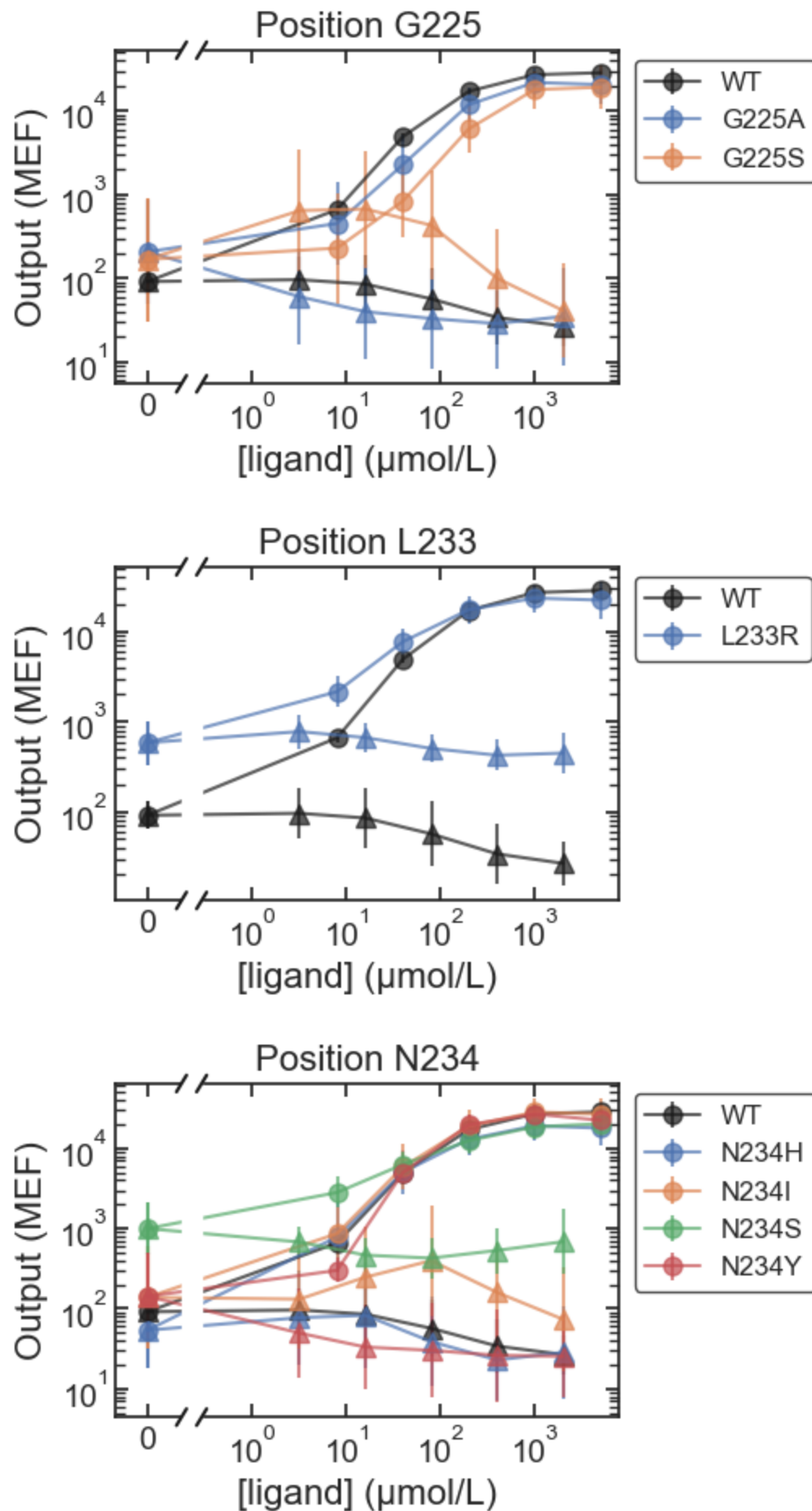


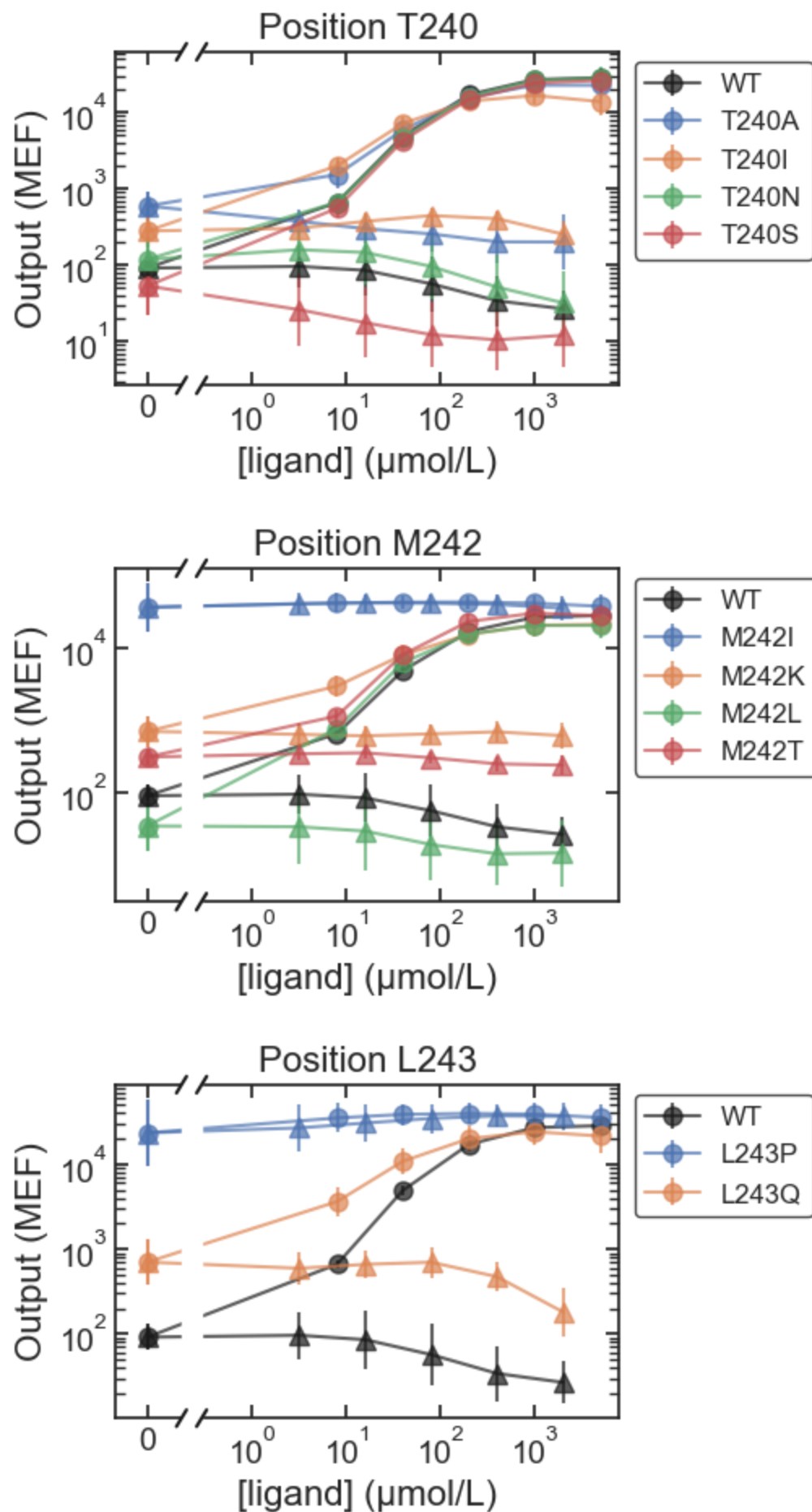


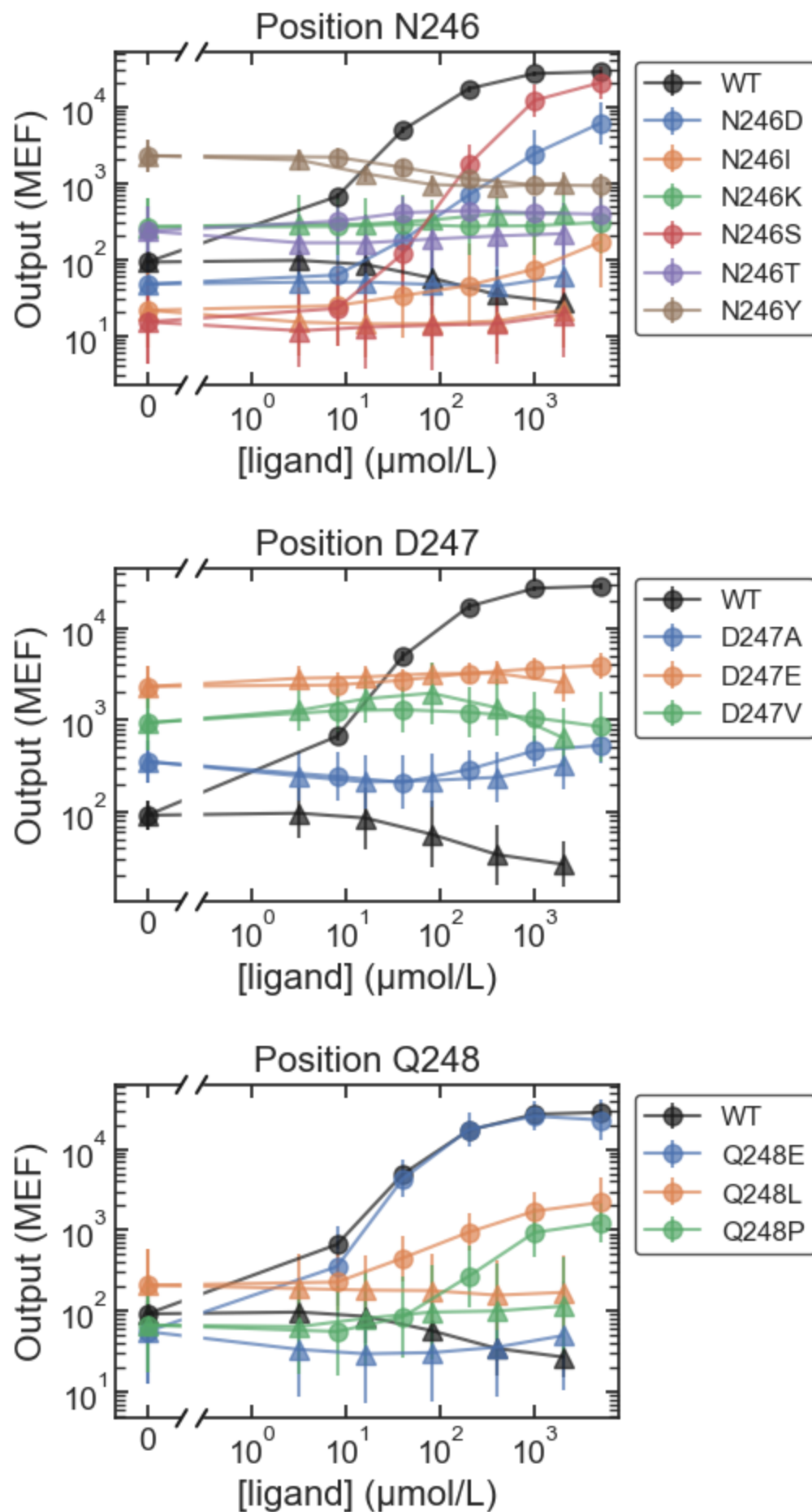


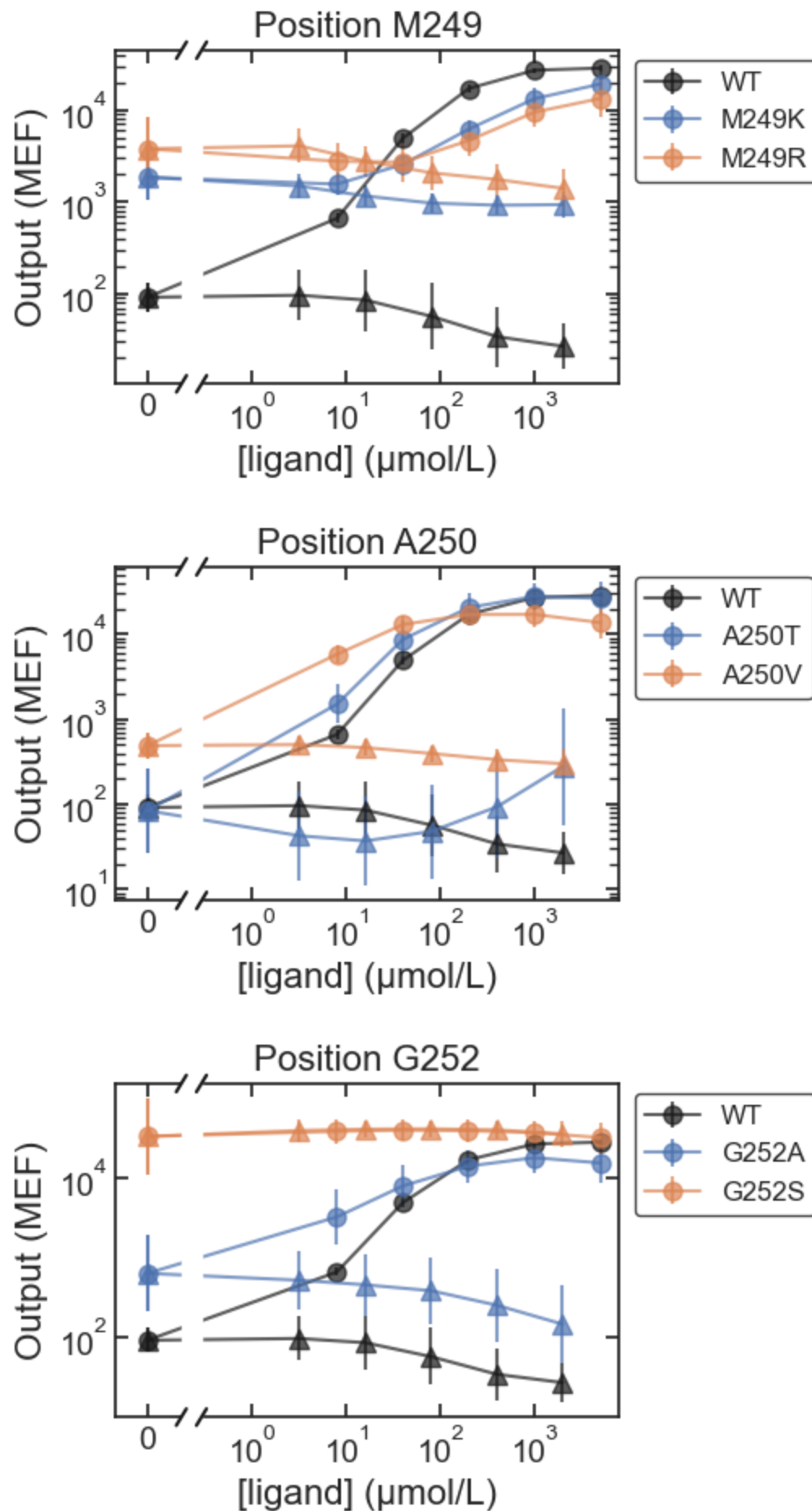


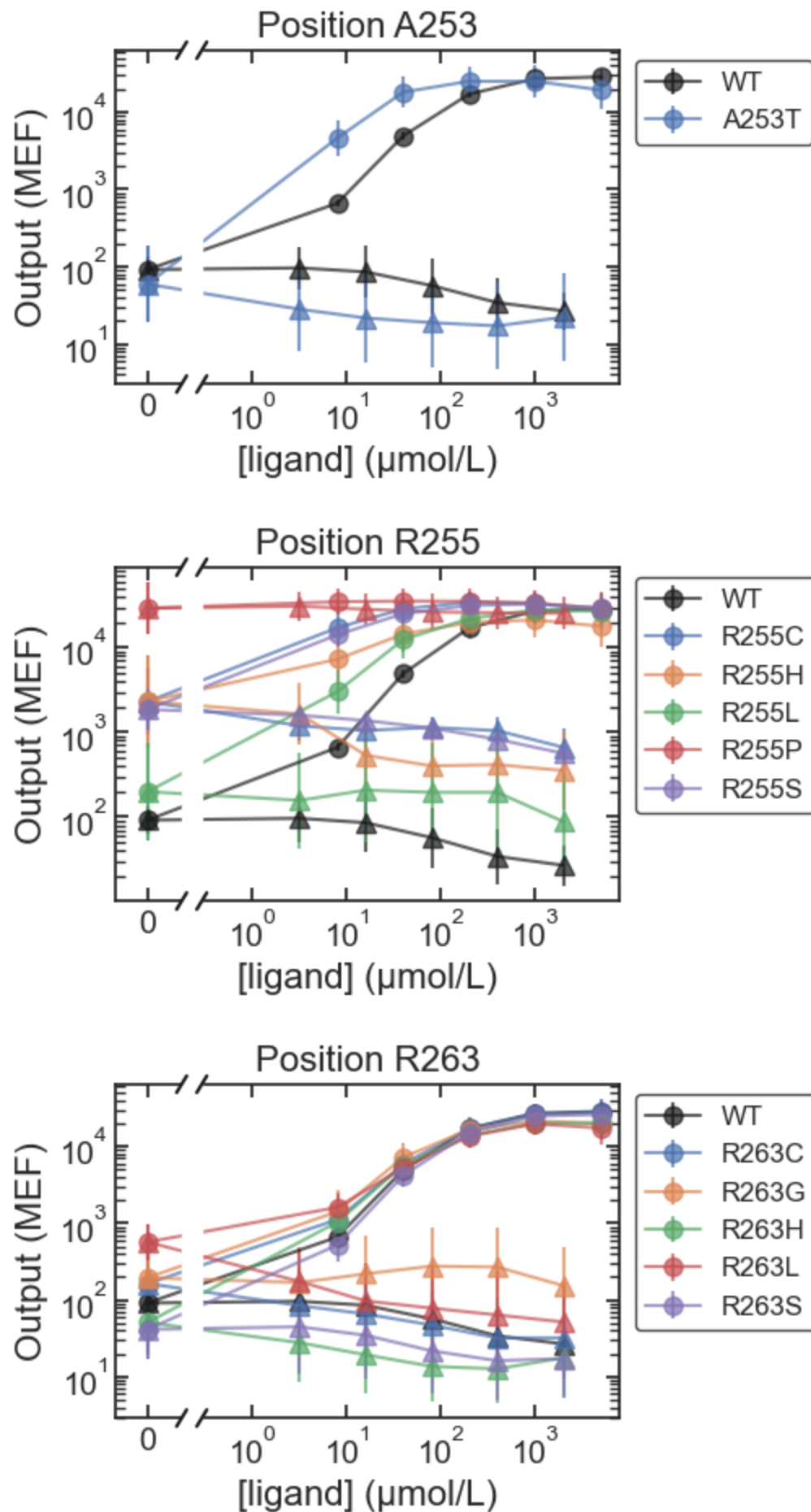


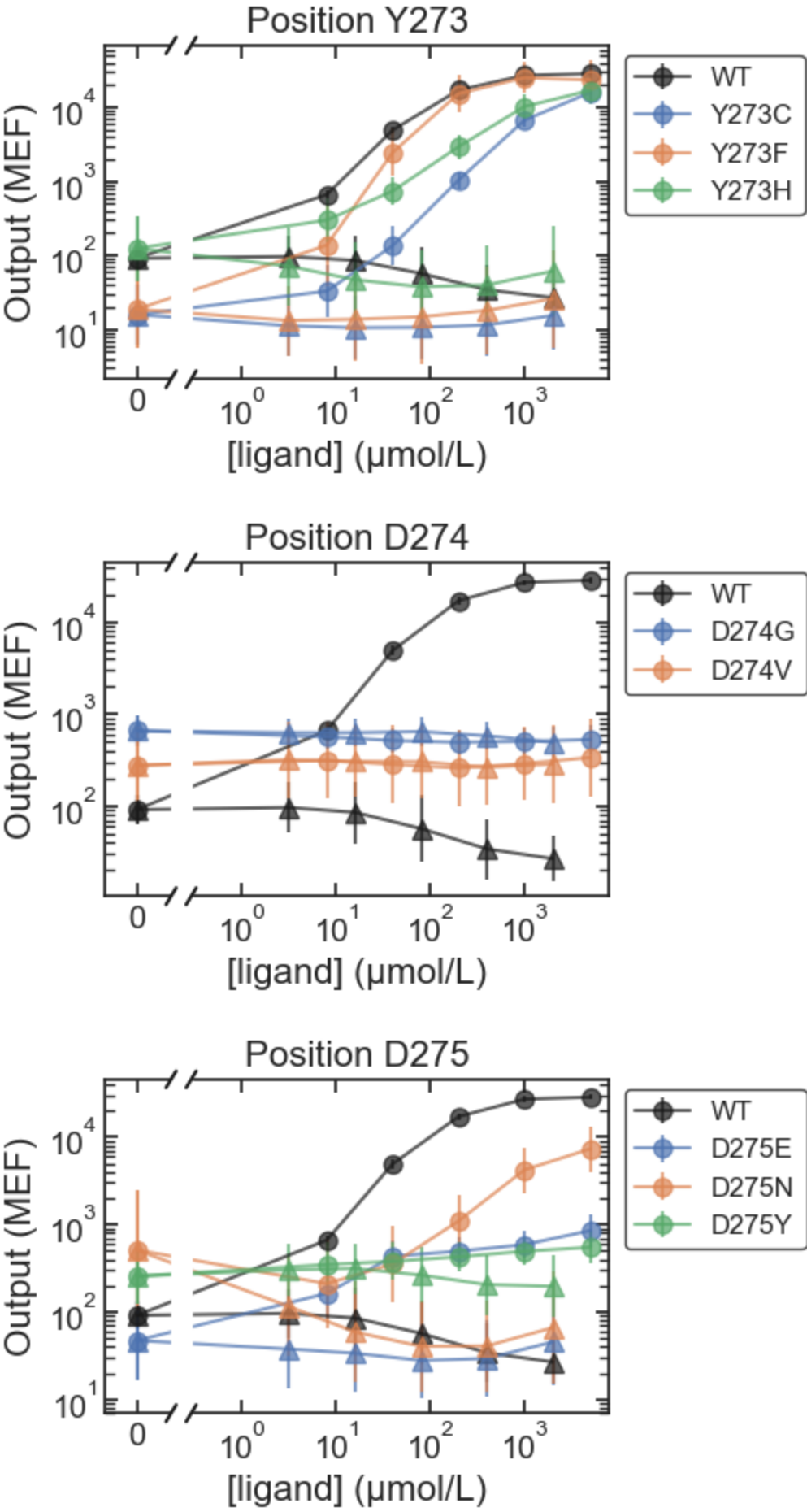


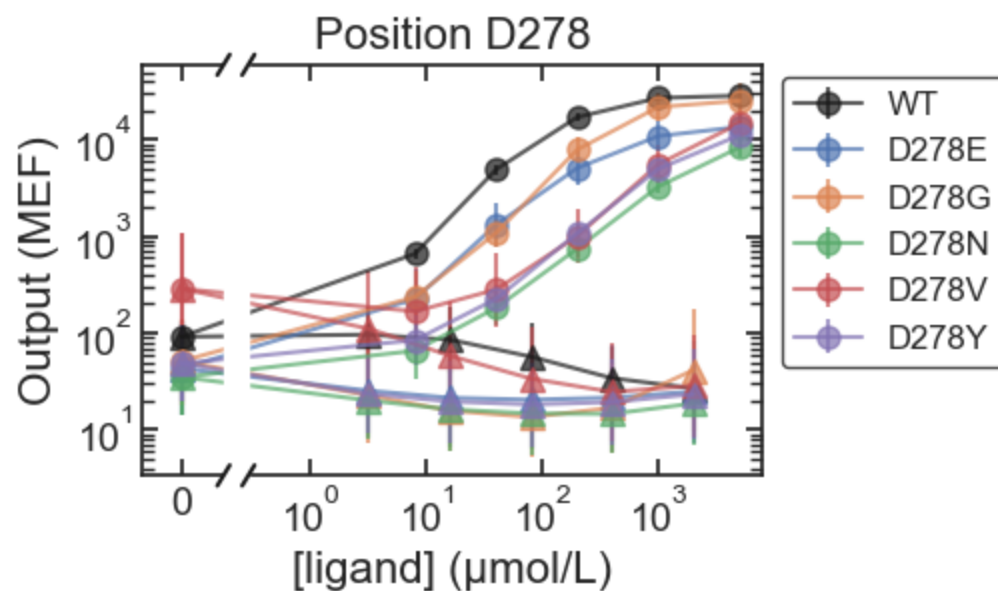
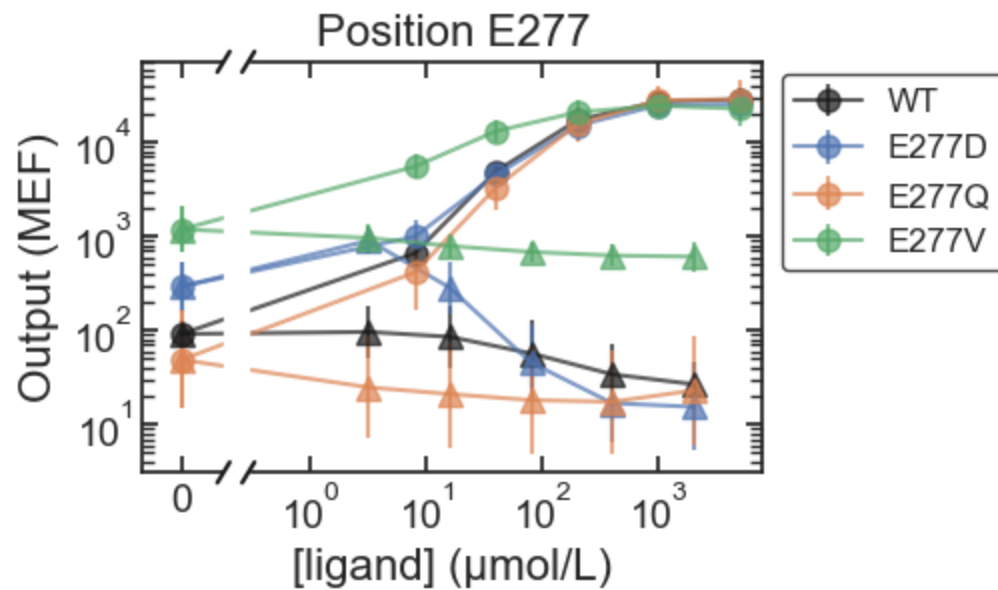
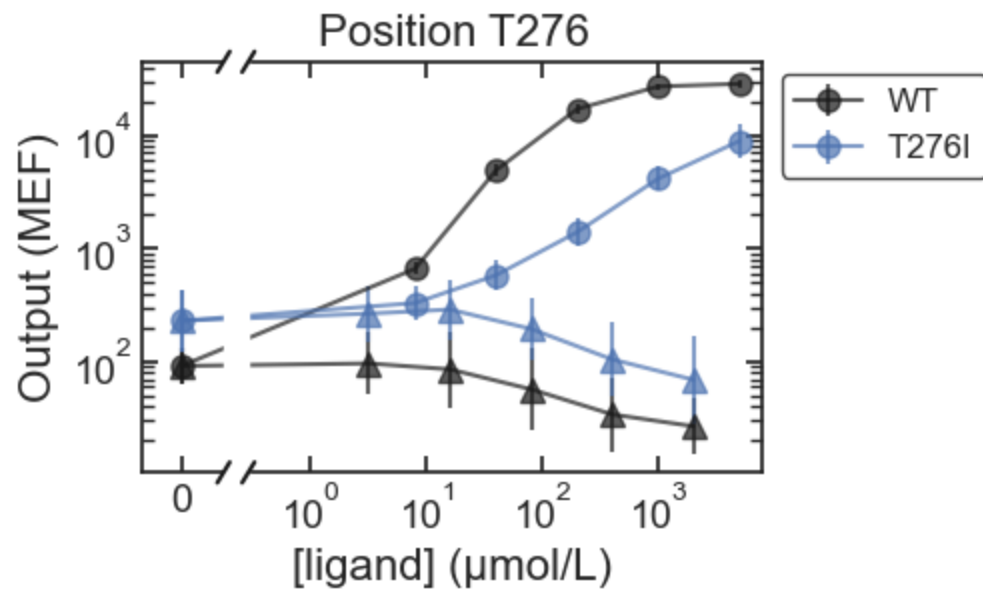


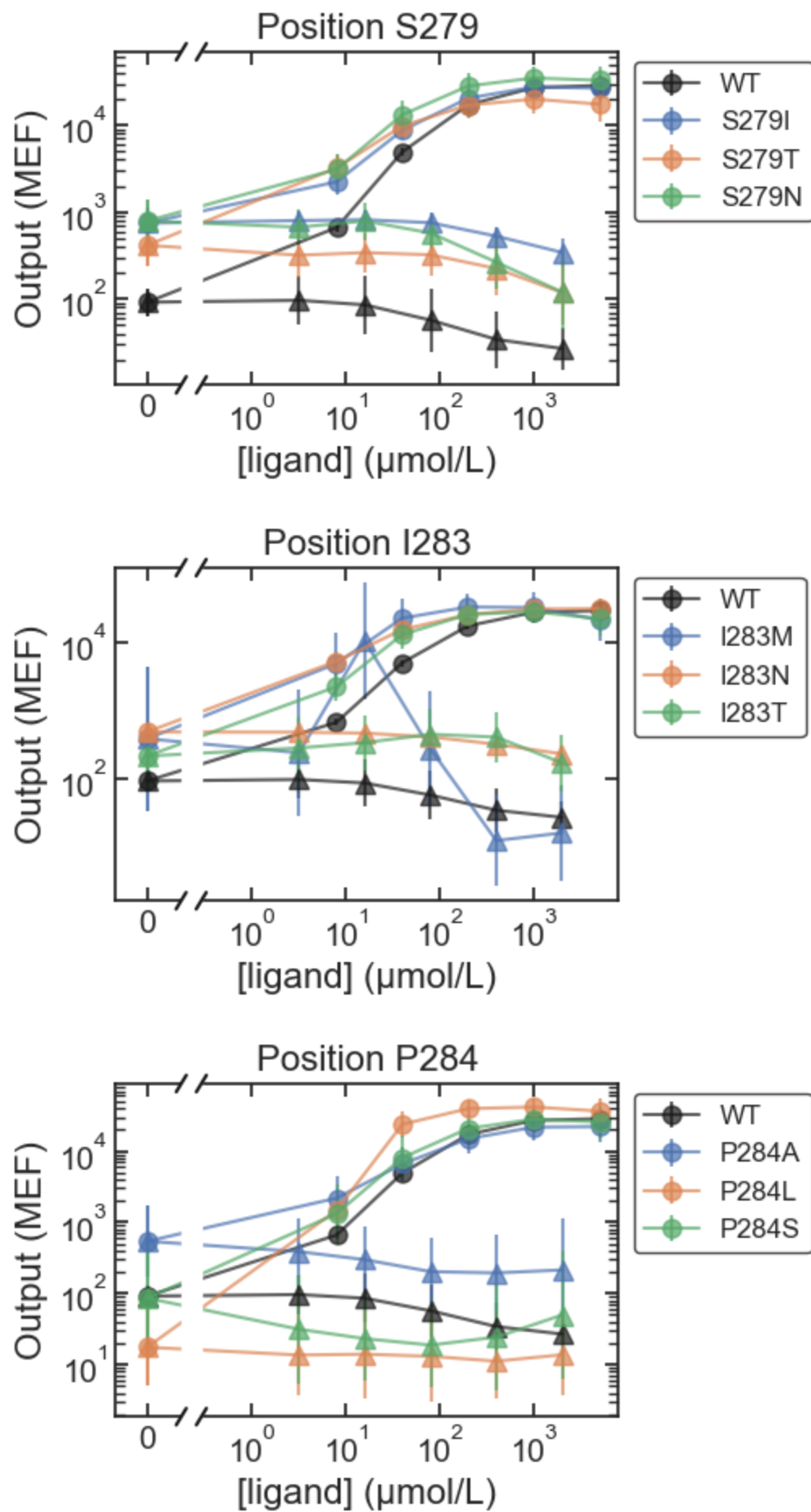


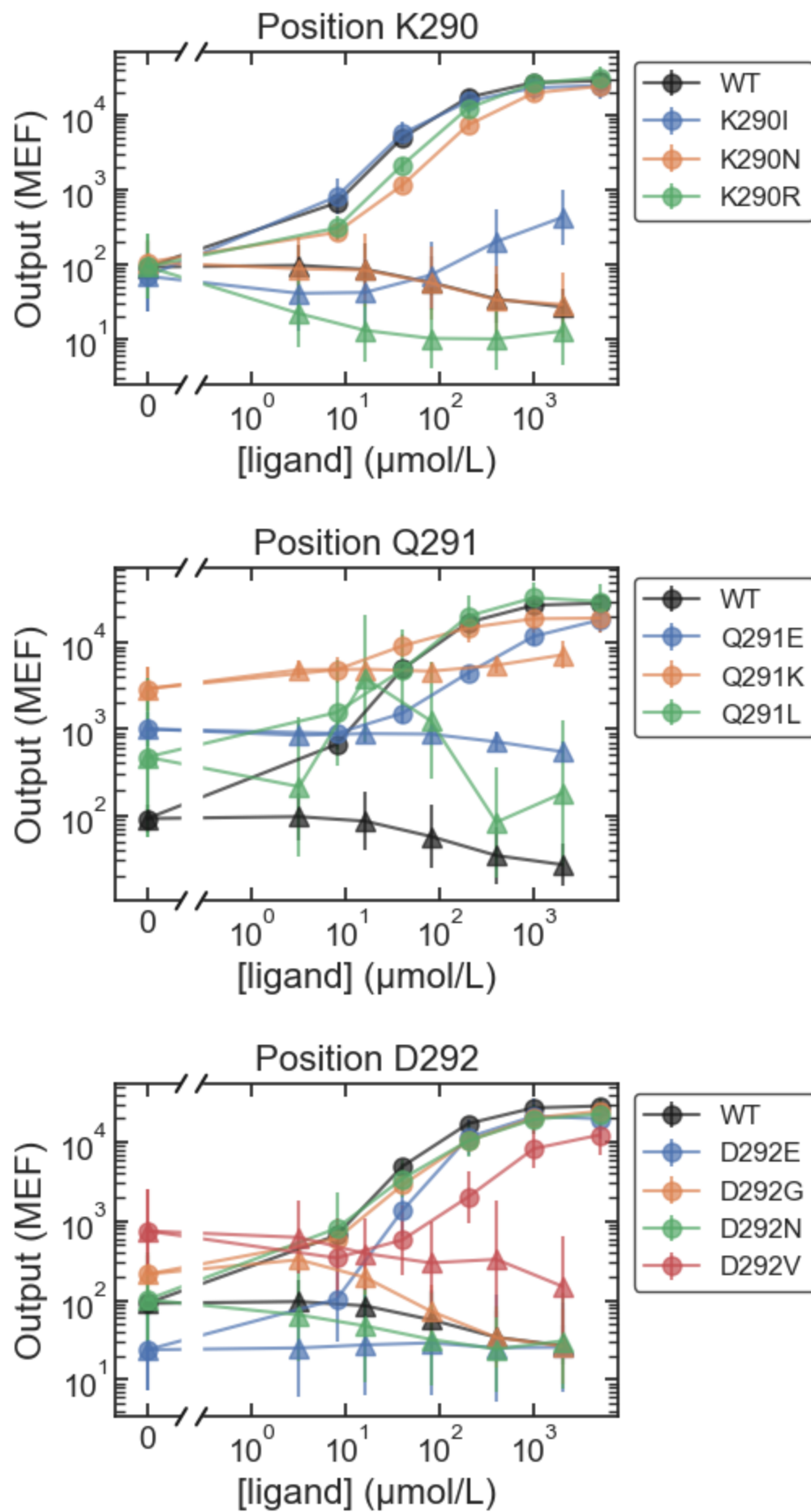


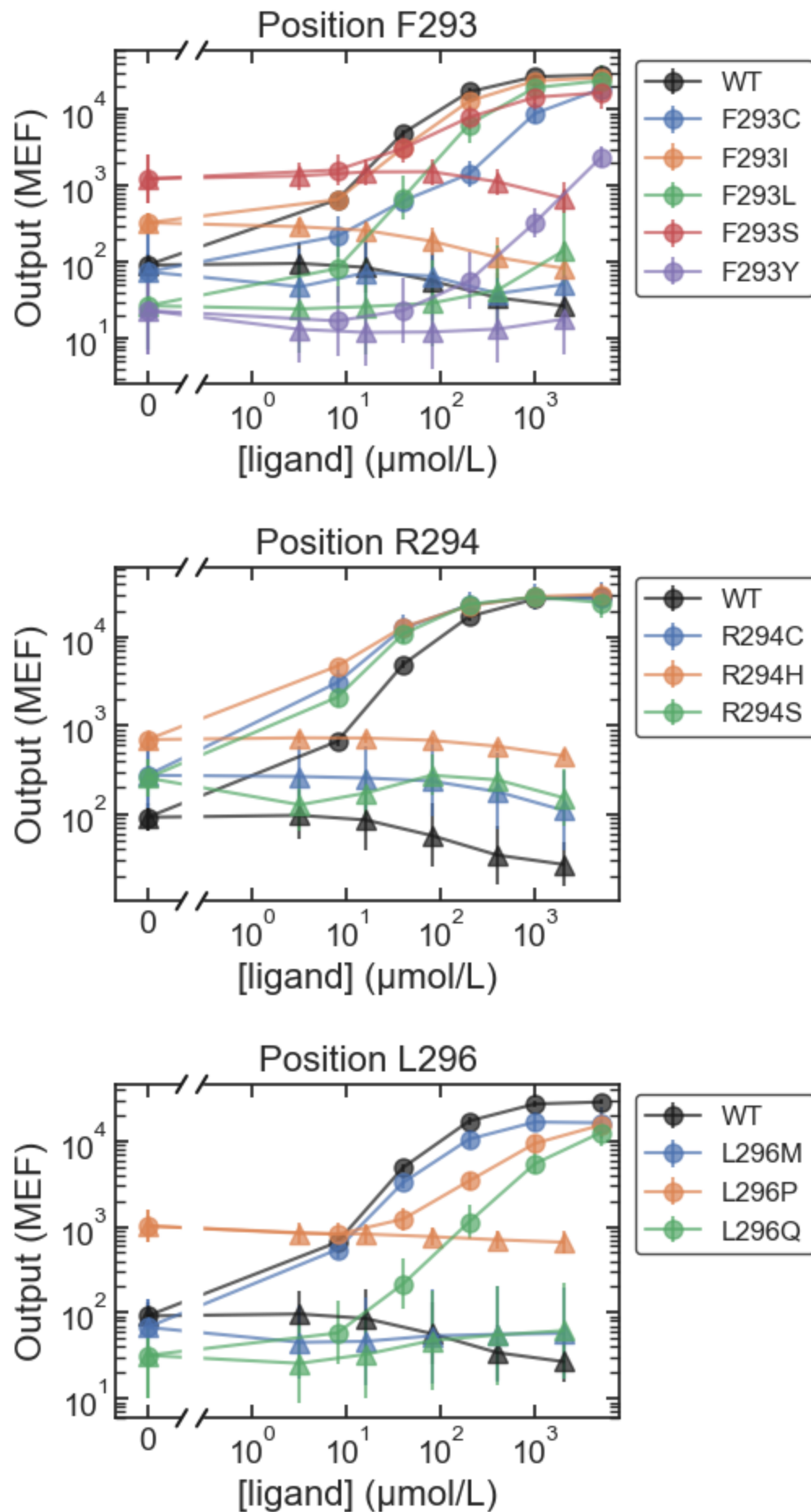


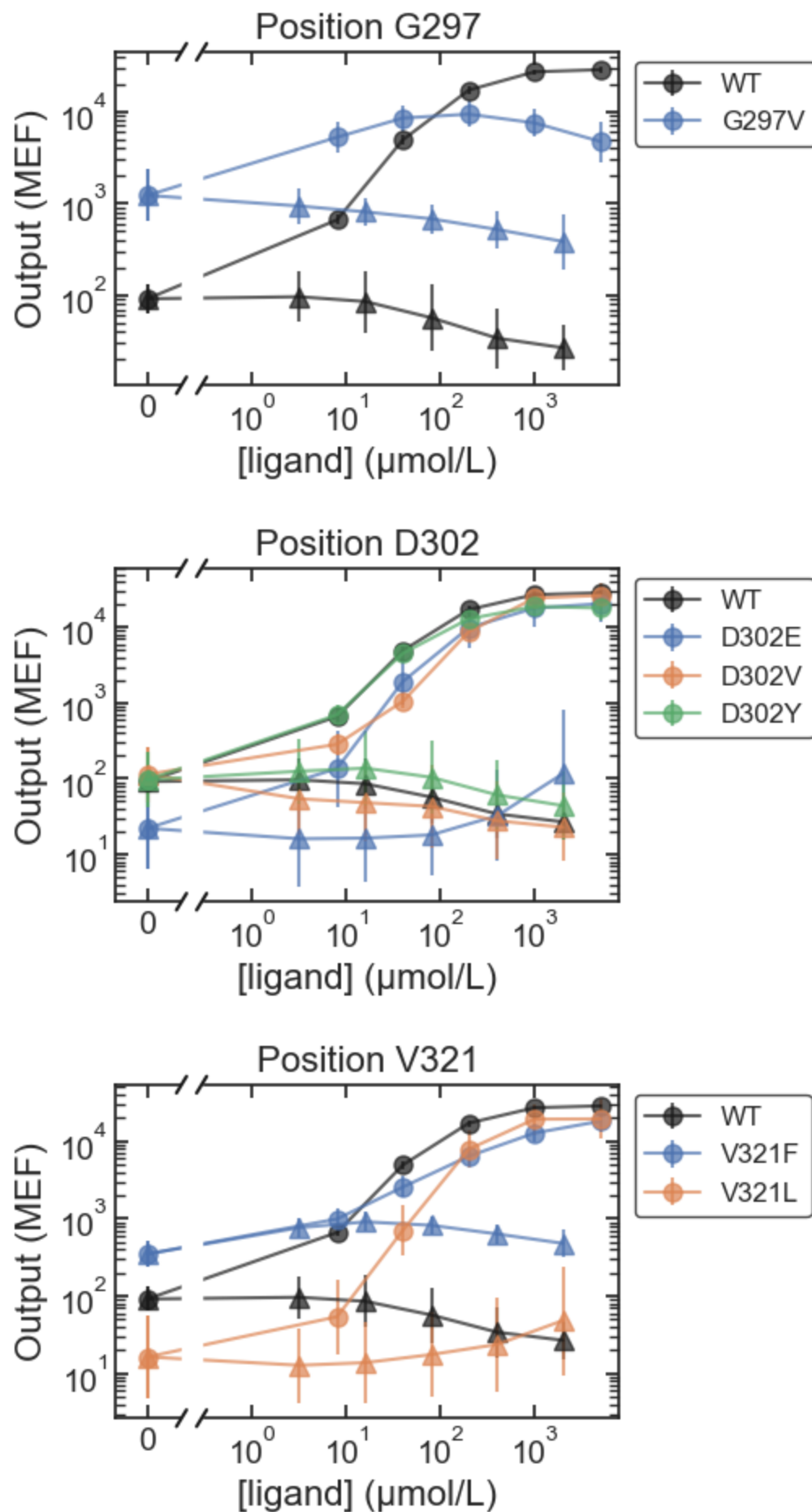


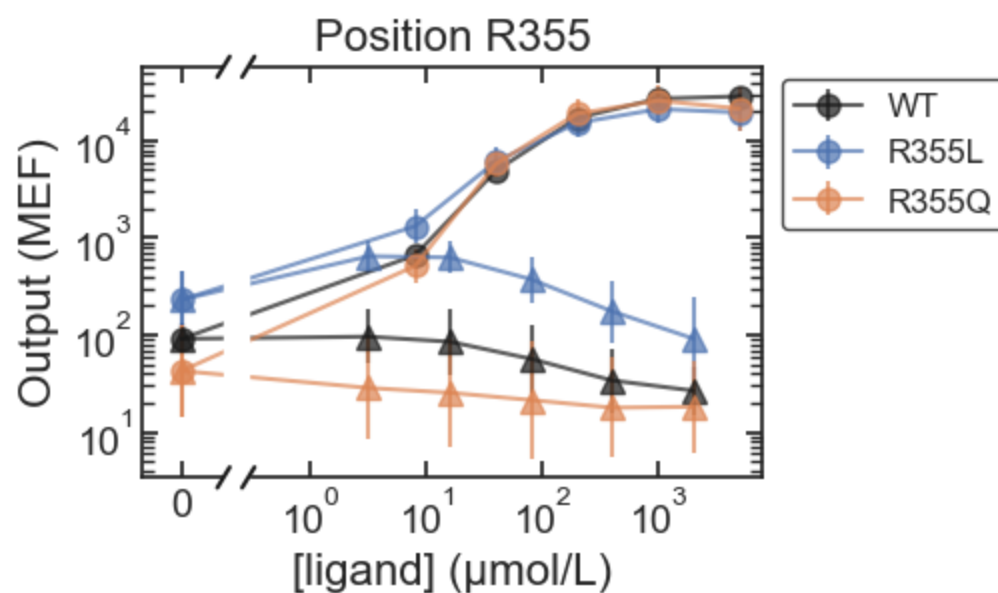
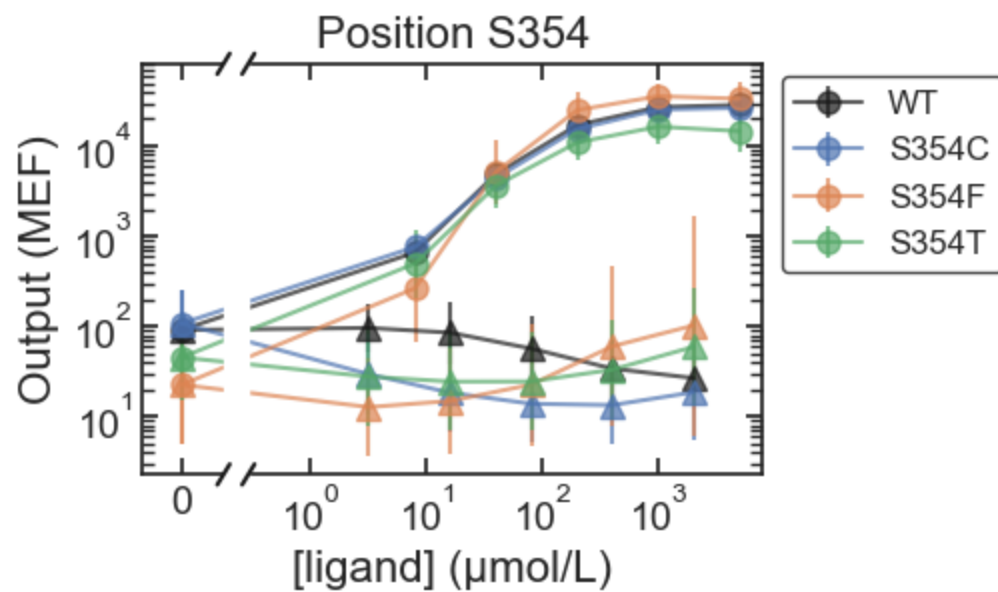
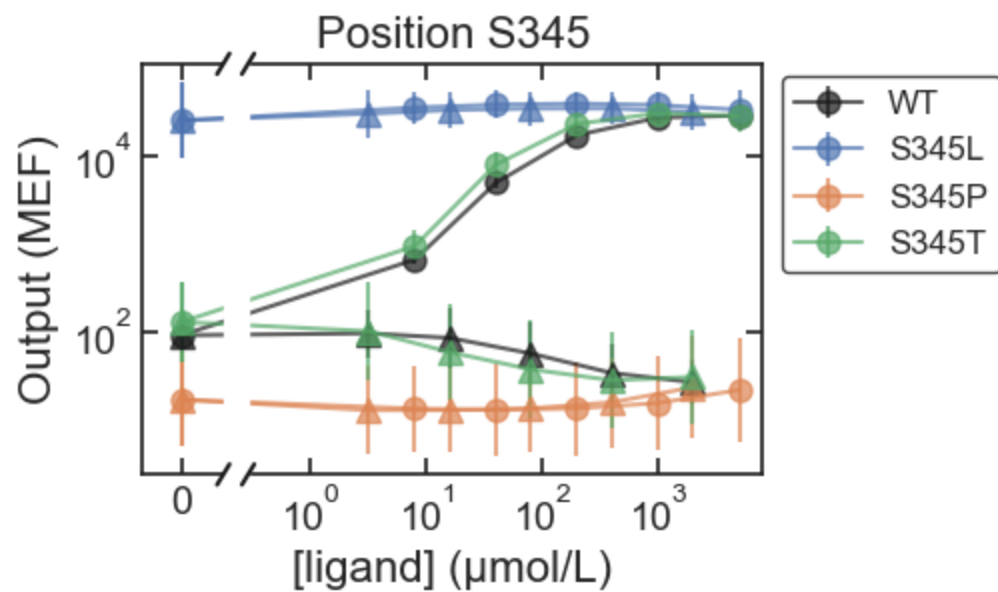


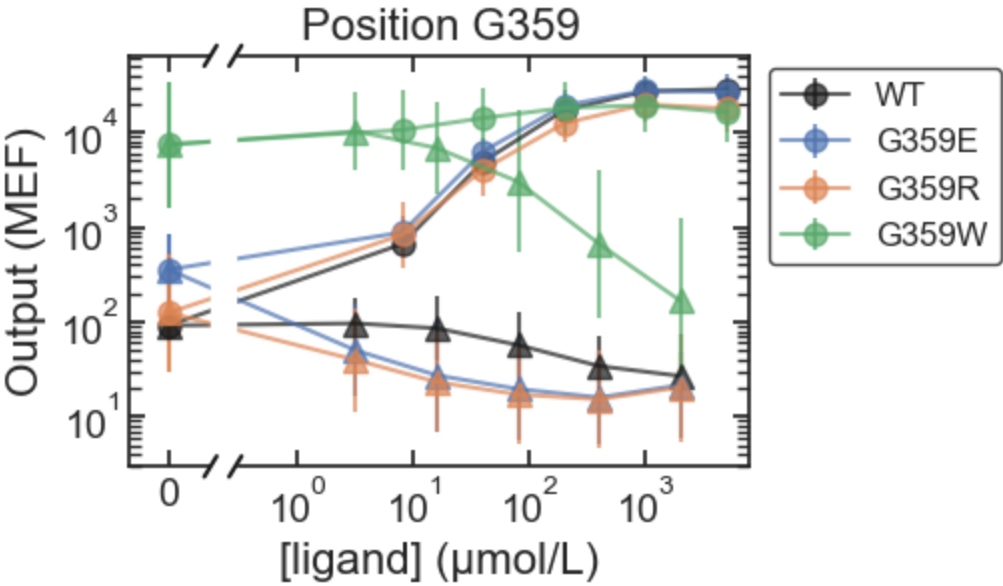




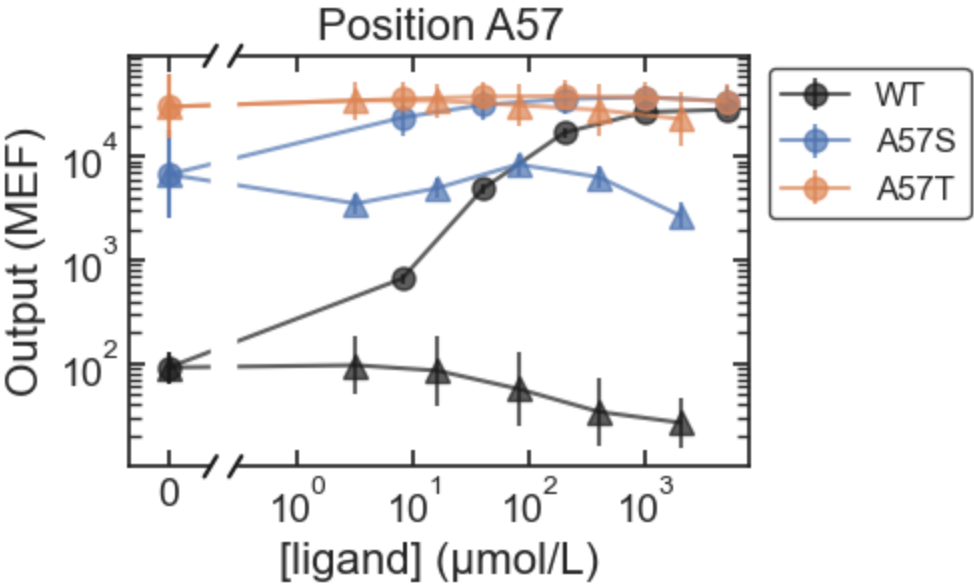


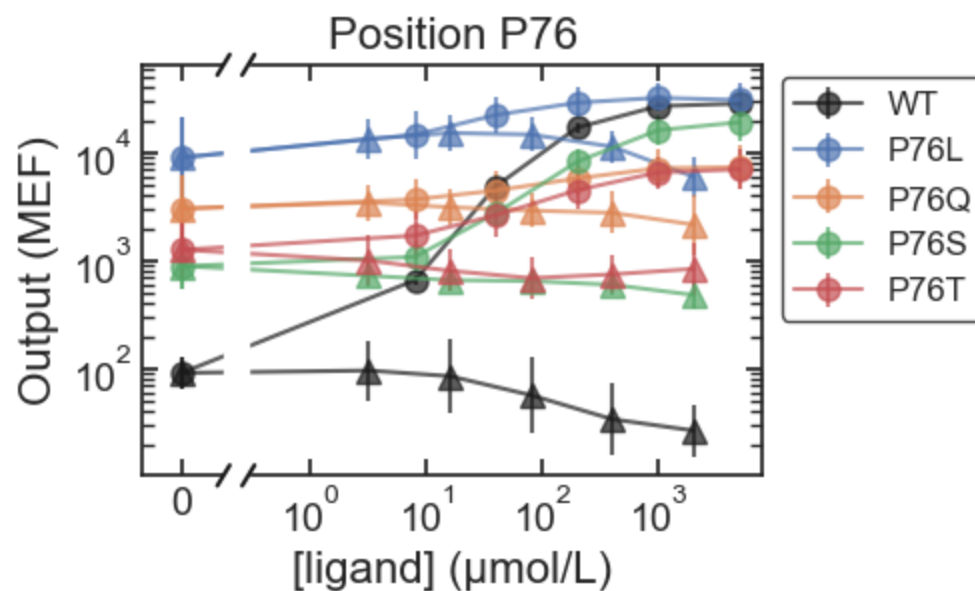
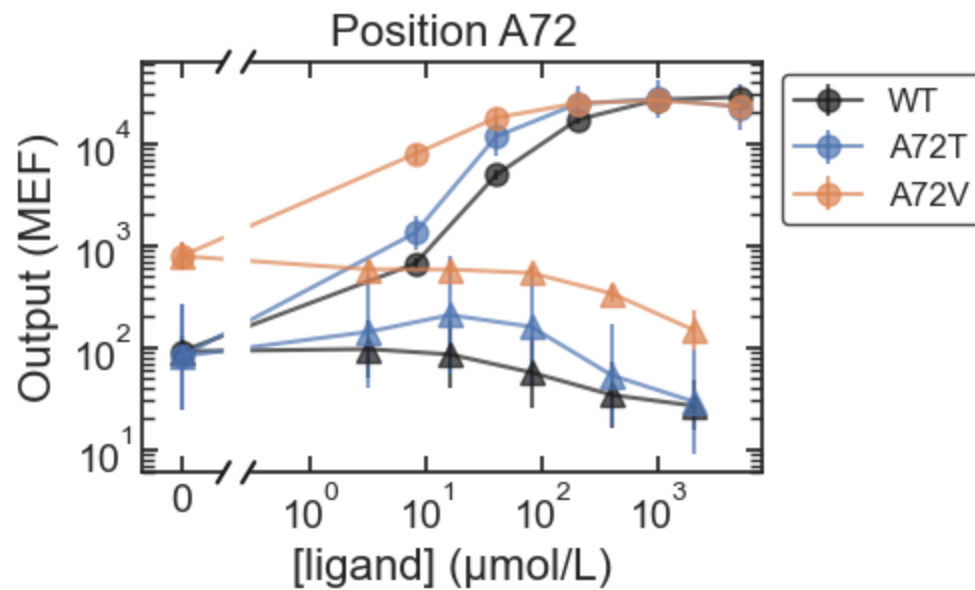
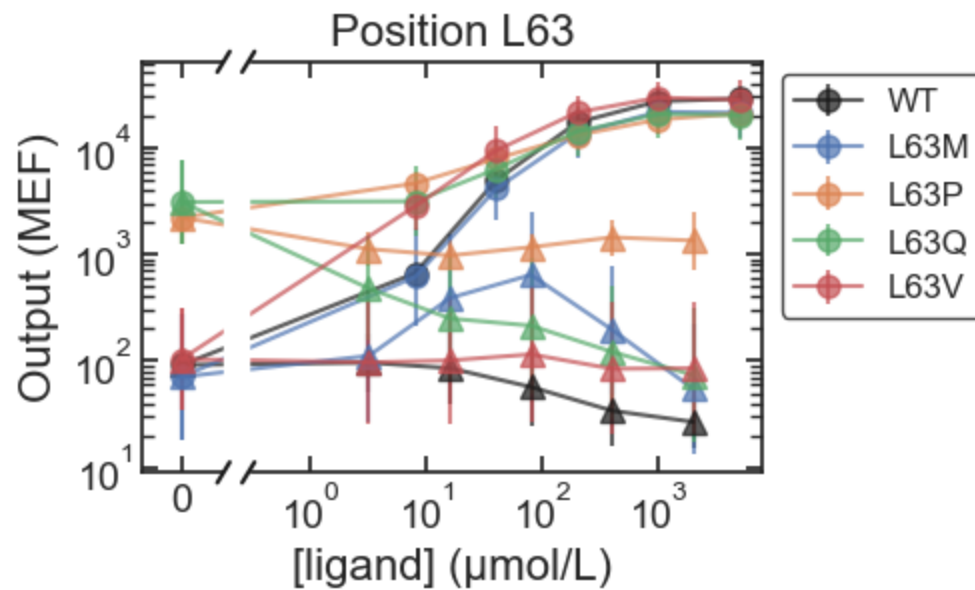


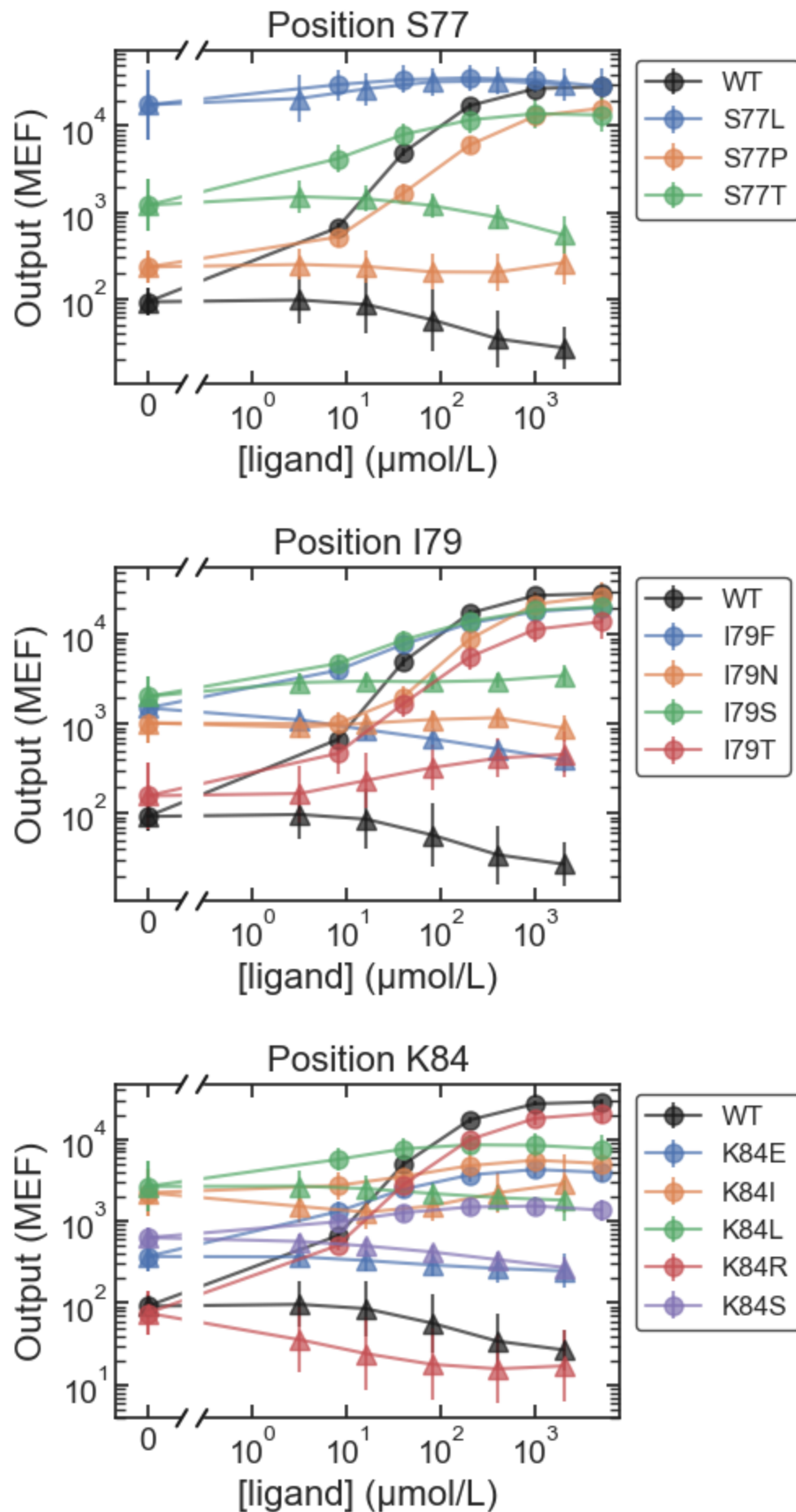


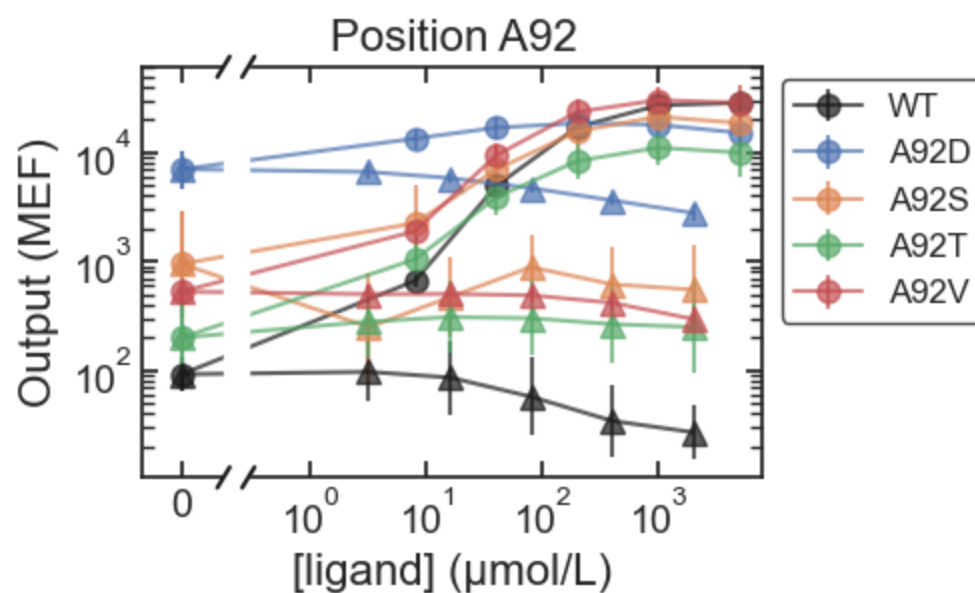
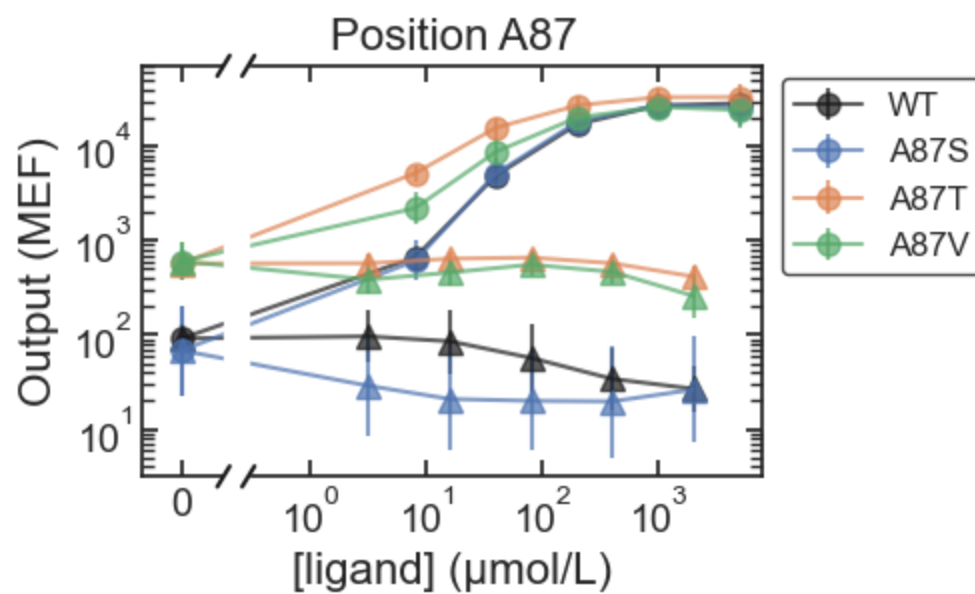
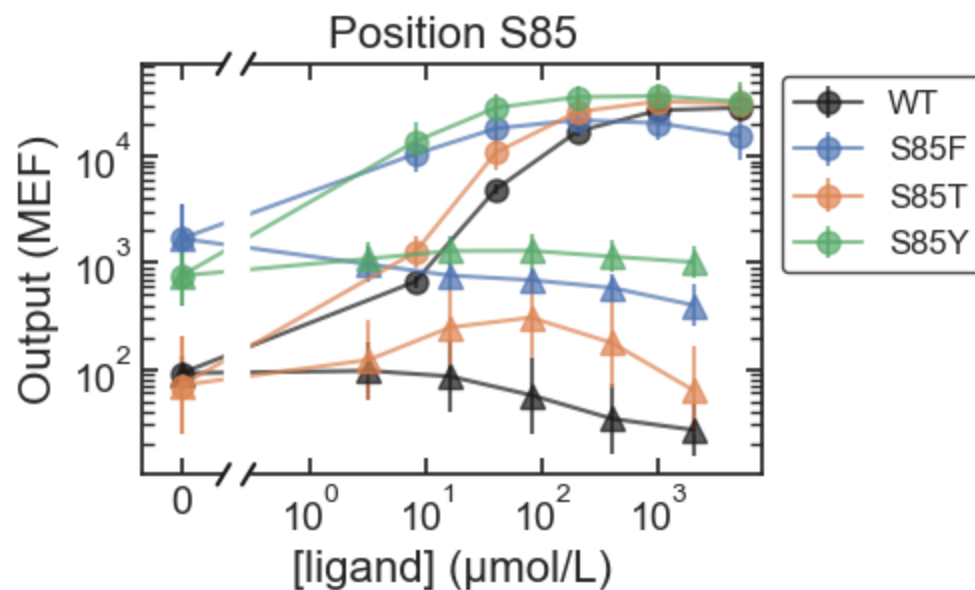


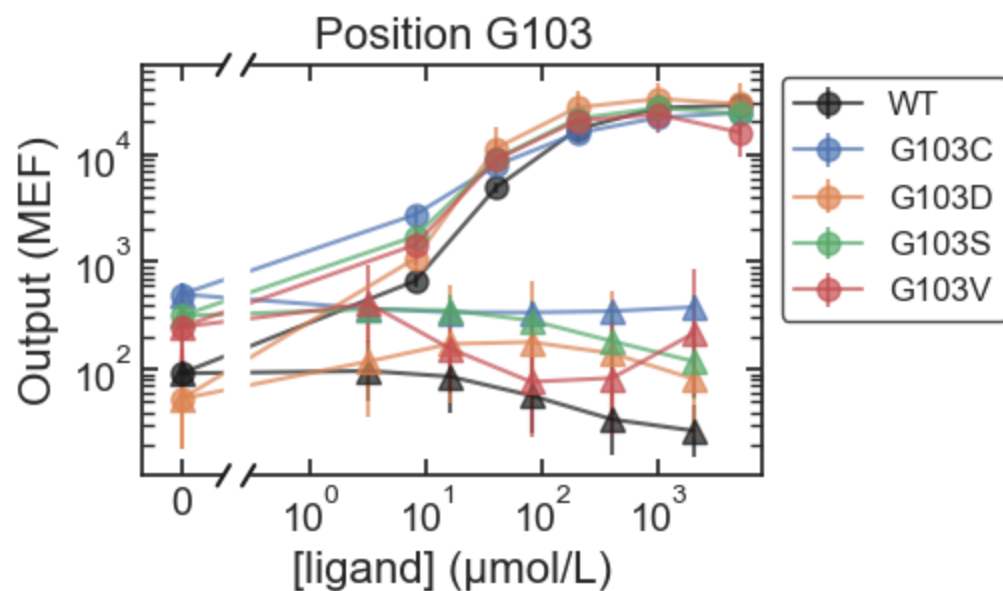
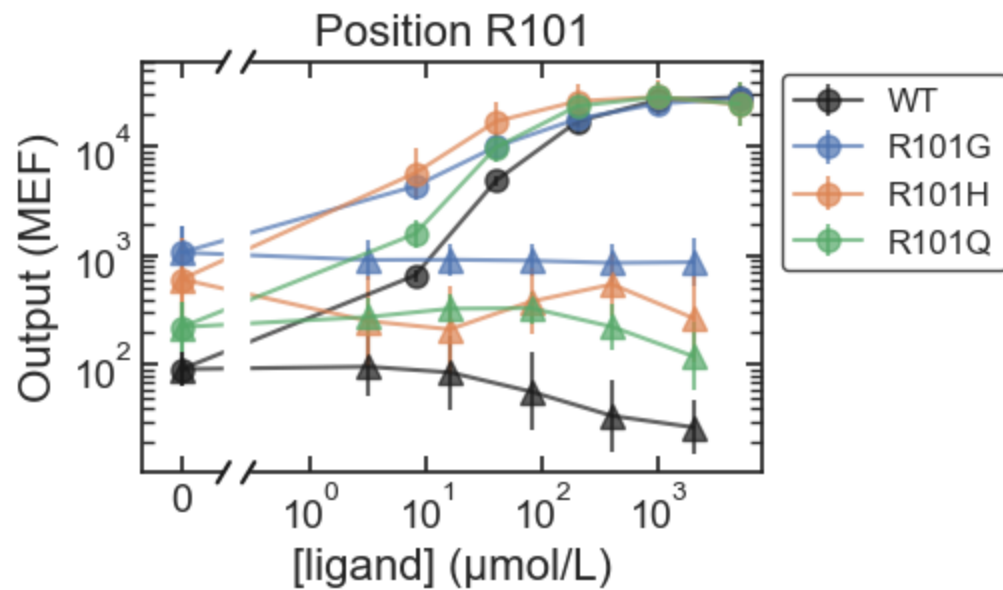
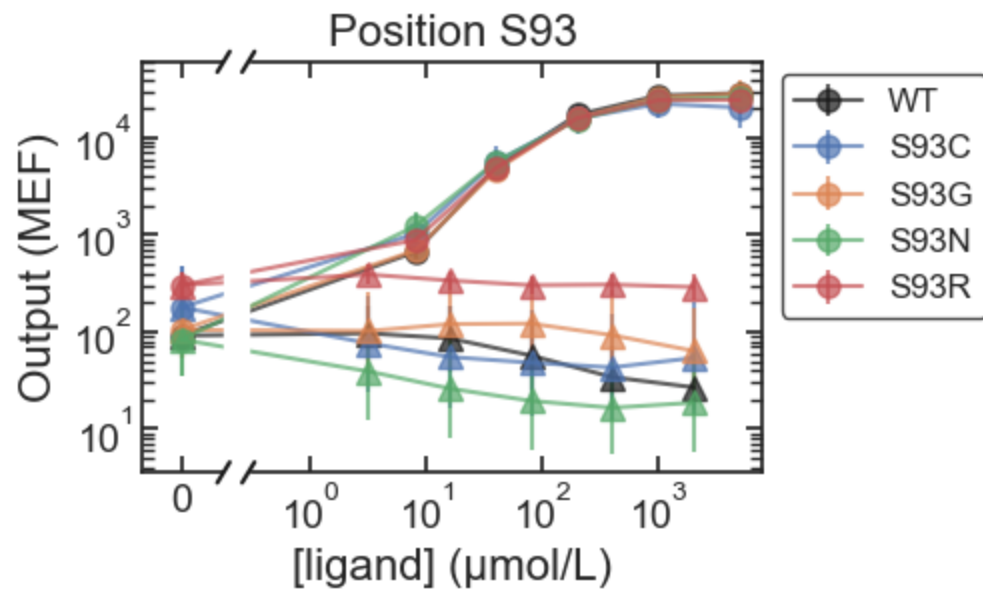
ONPF

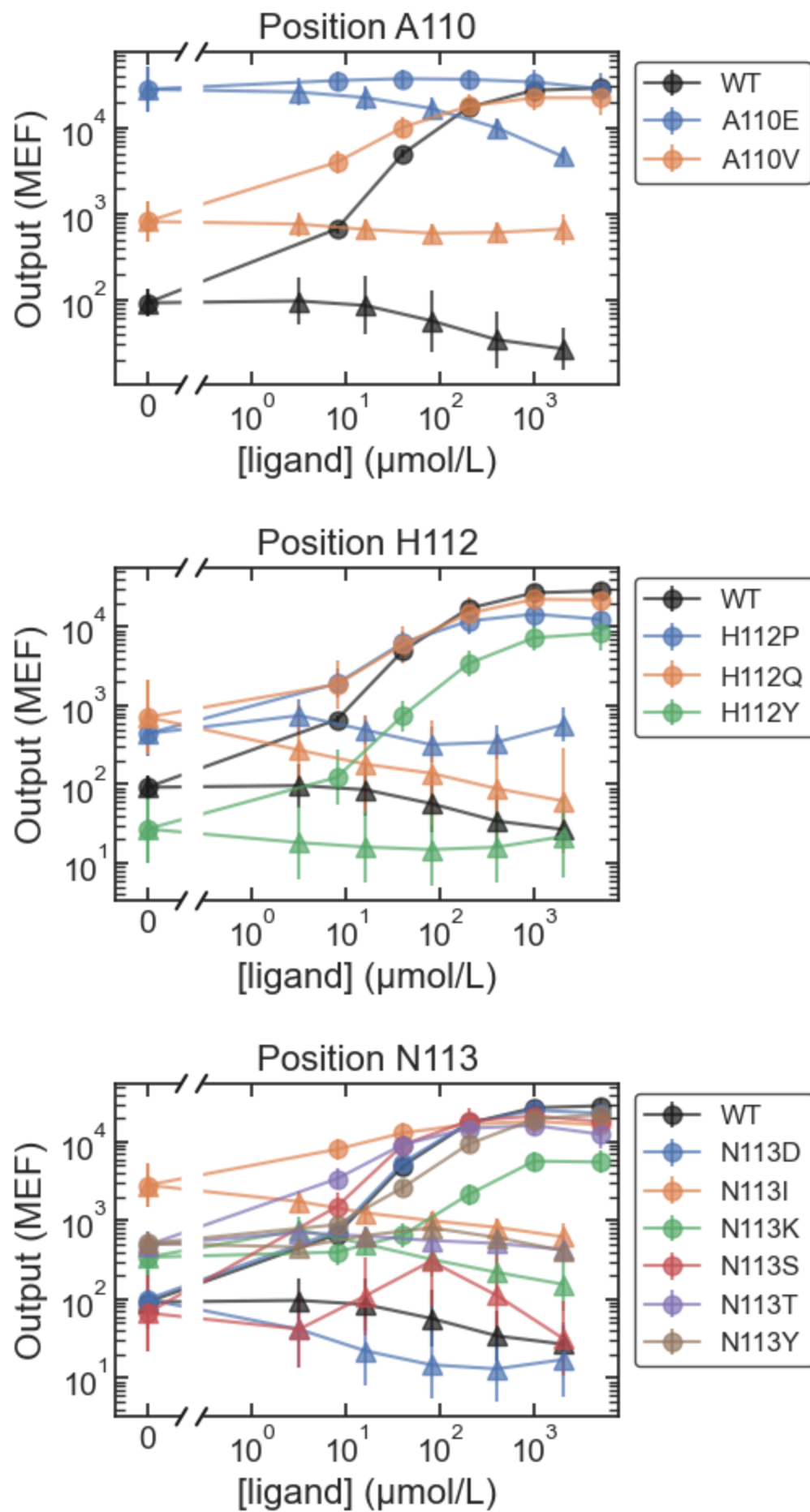


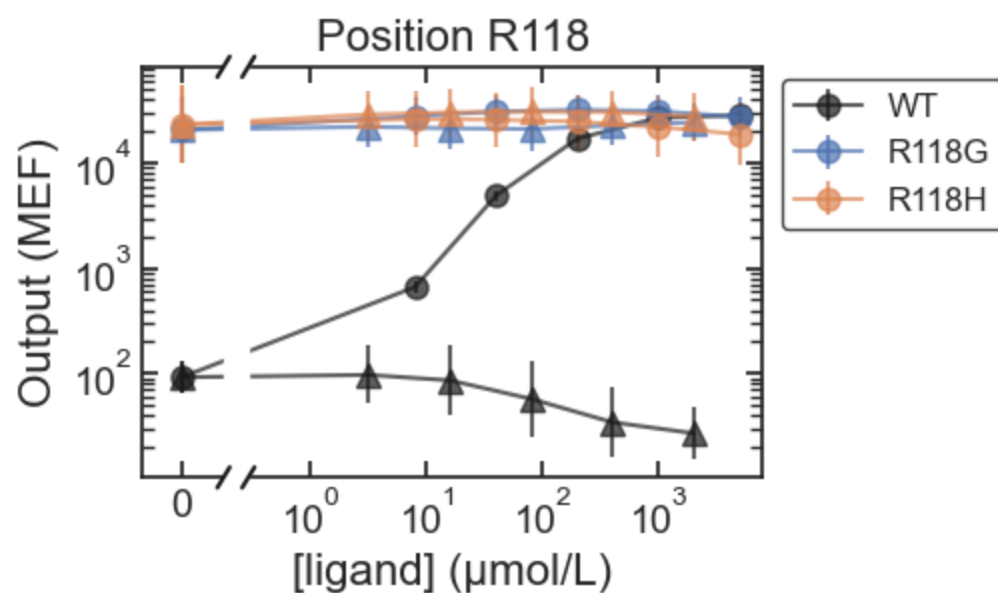
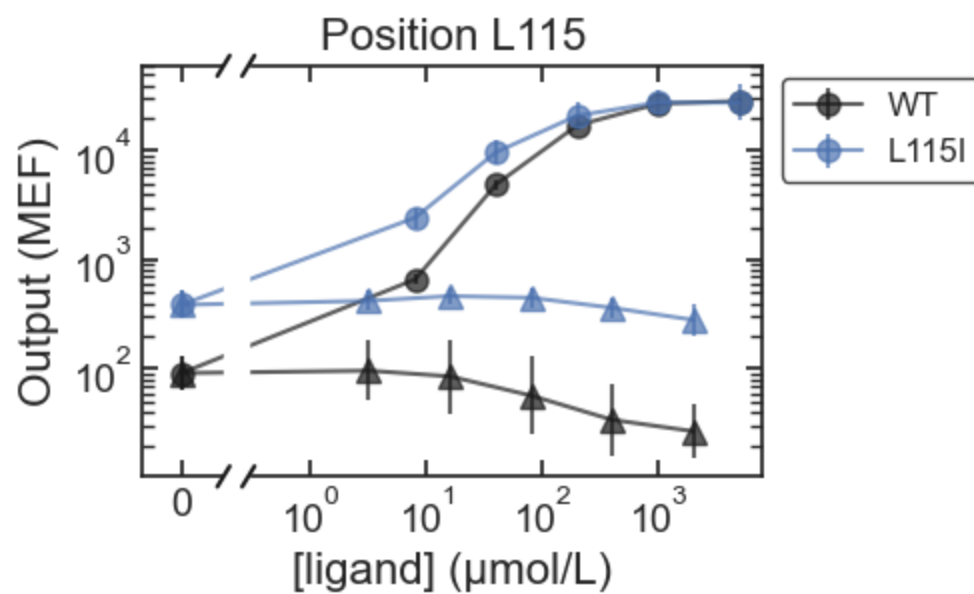
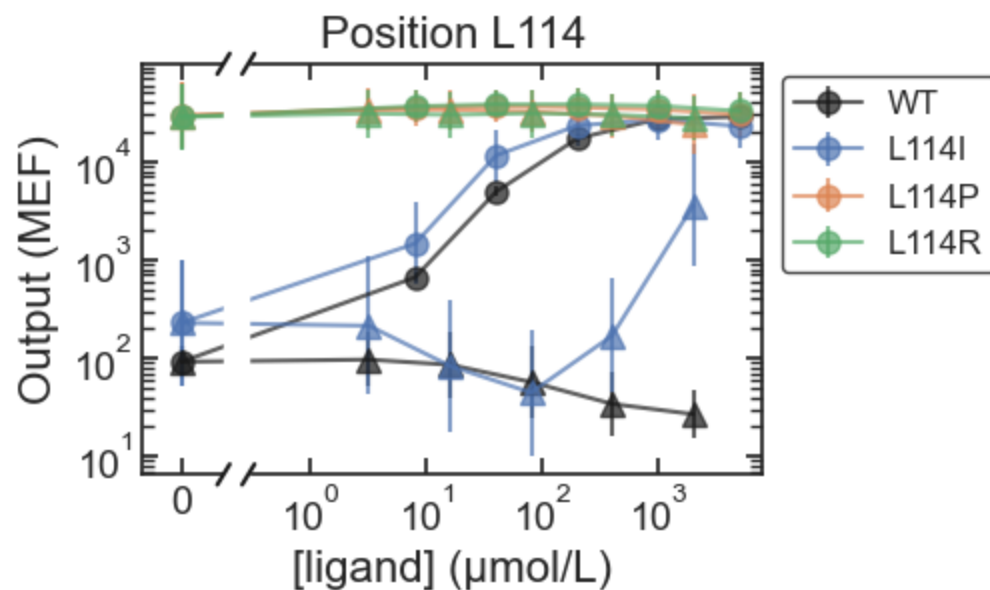


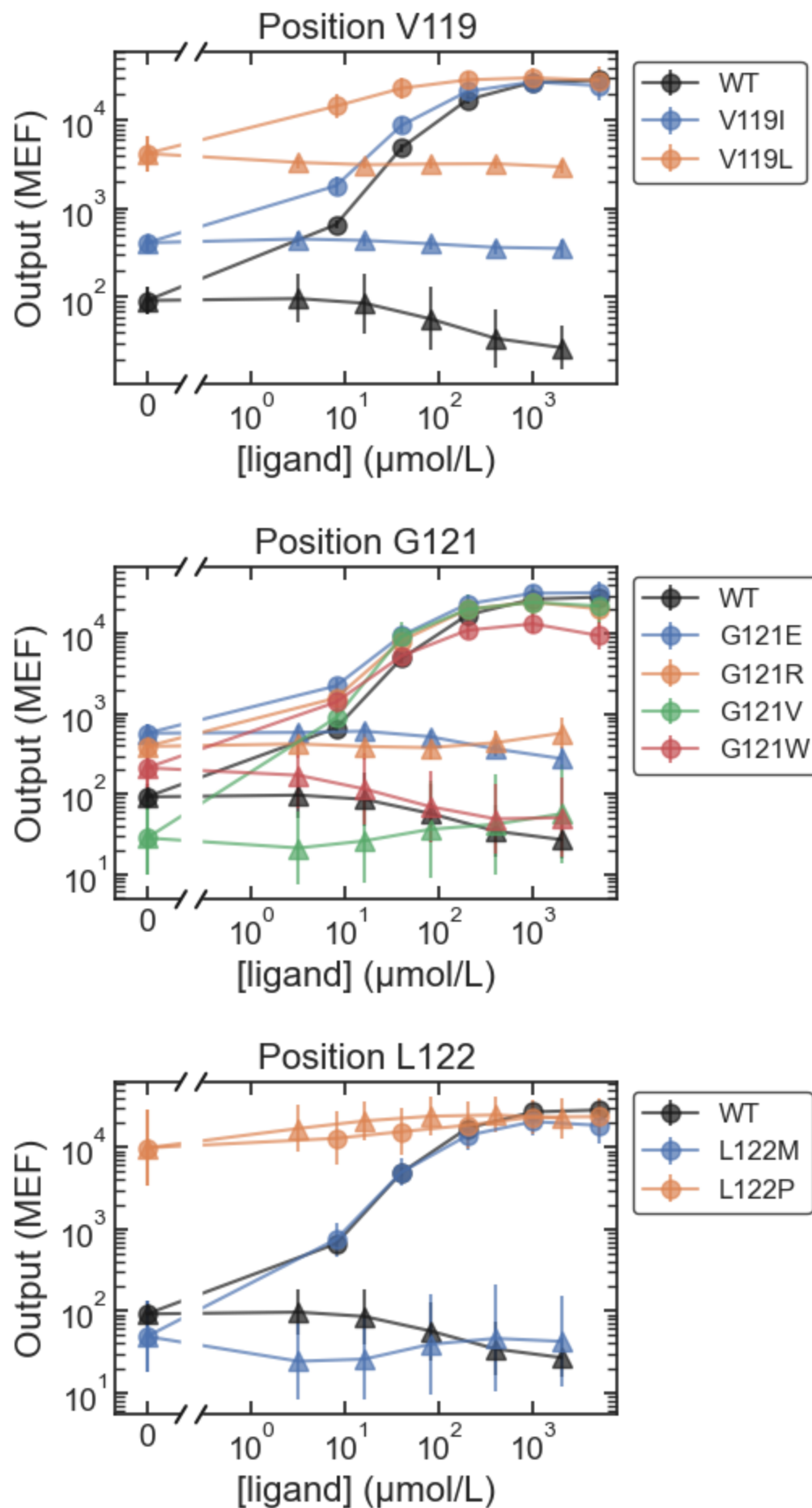


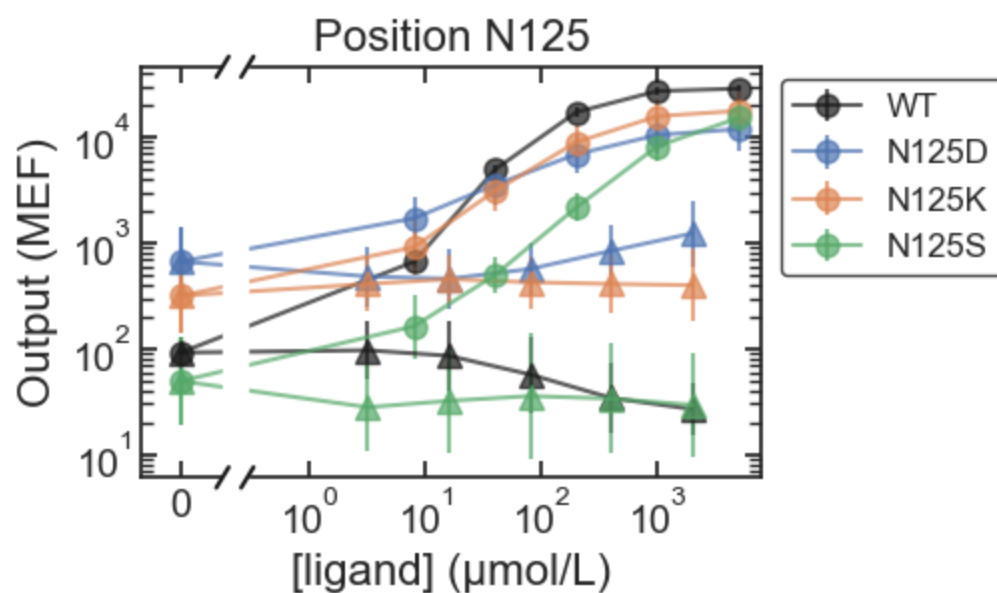
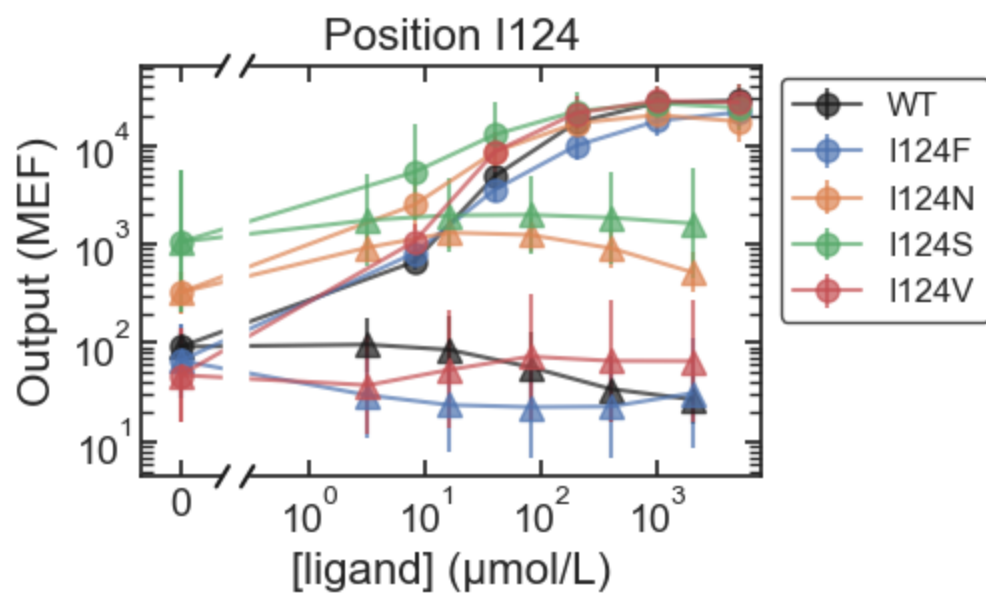
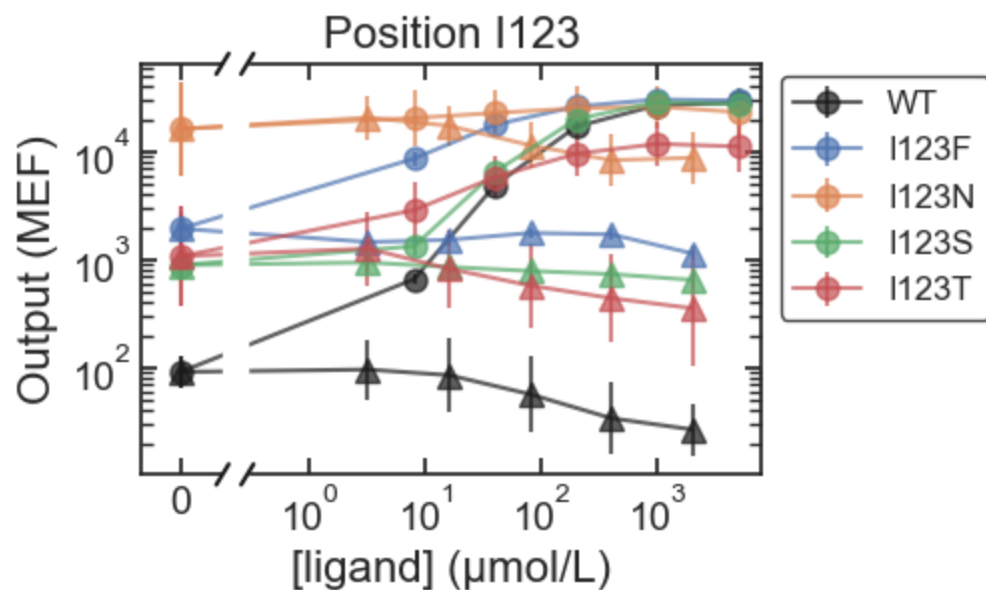


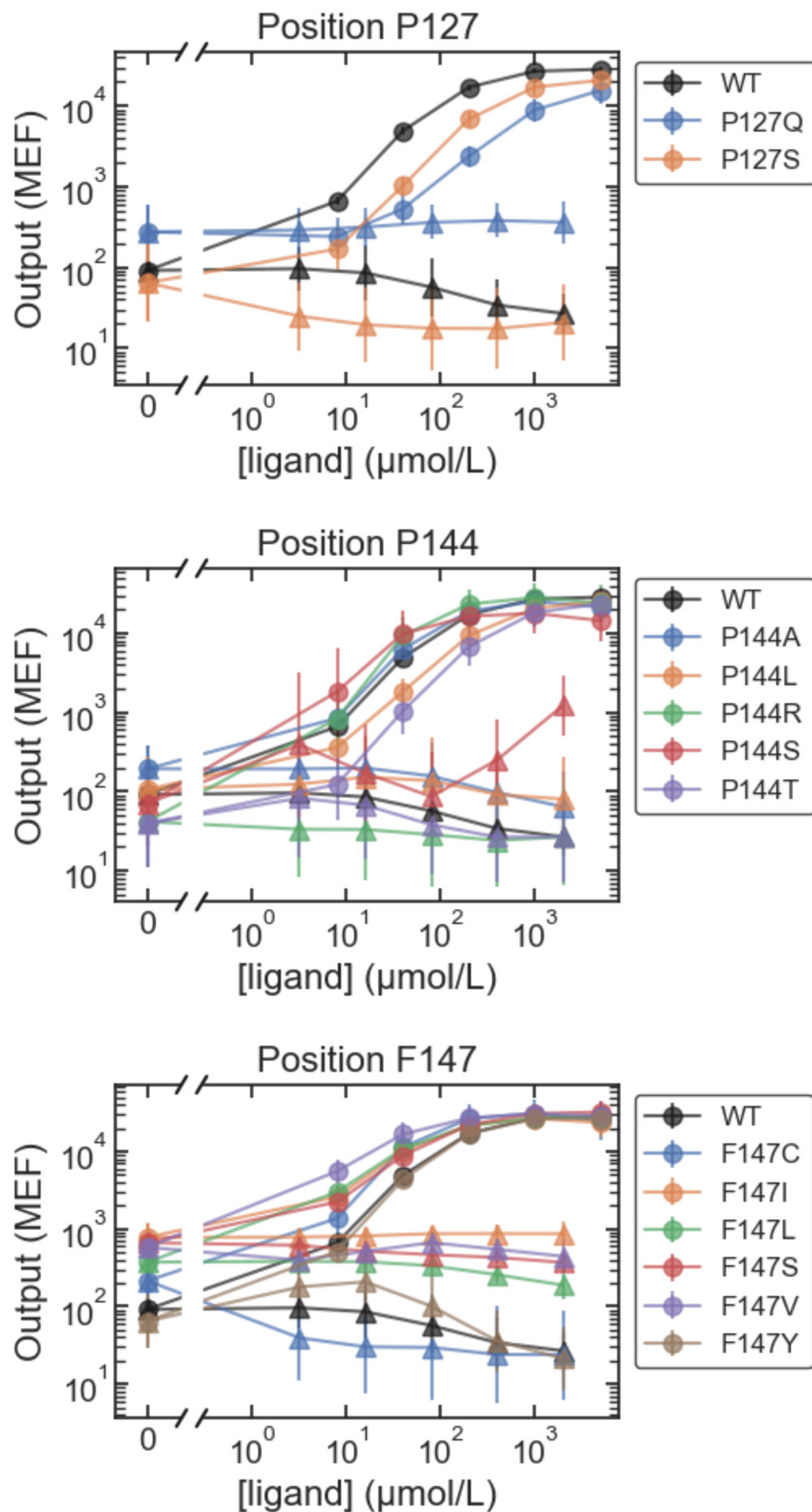


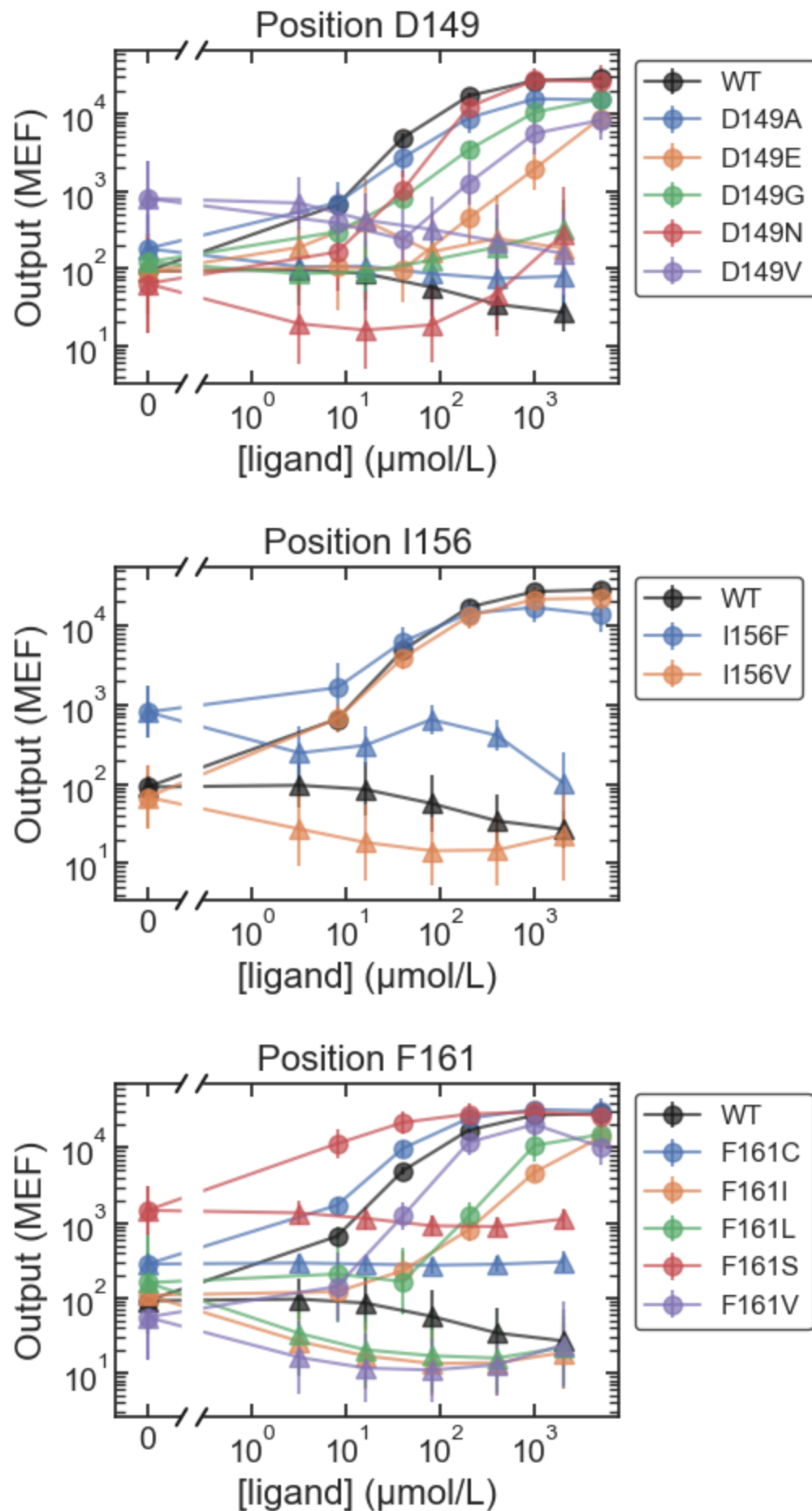


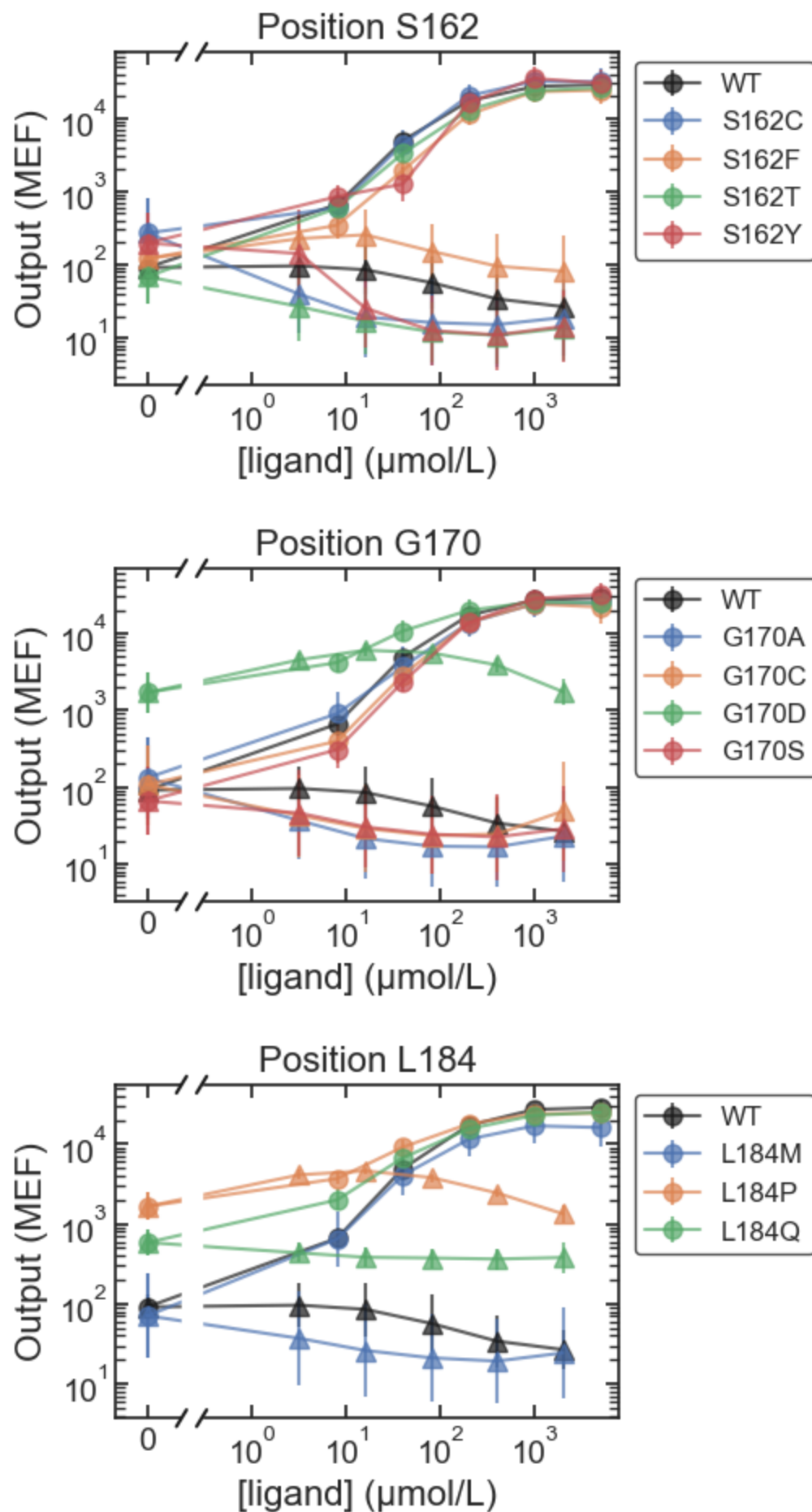


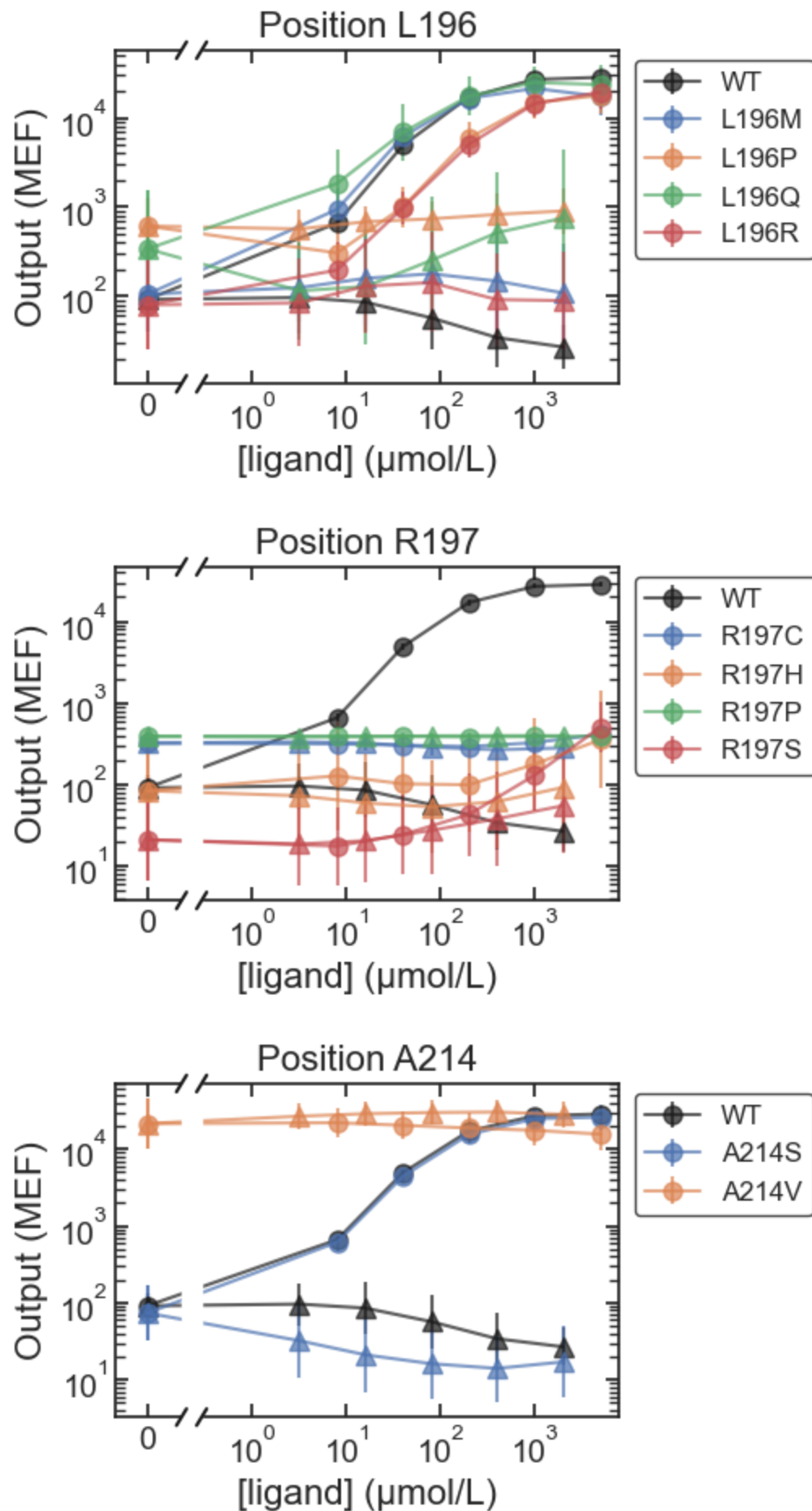


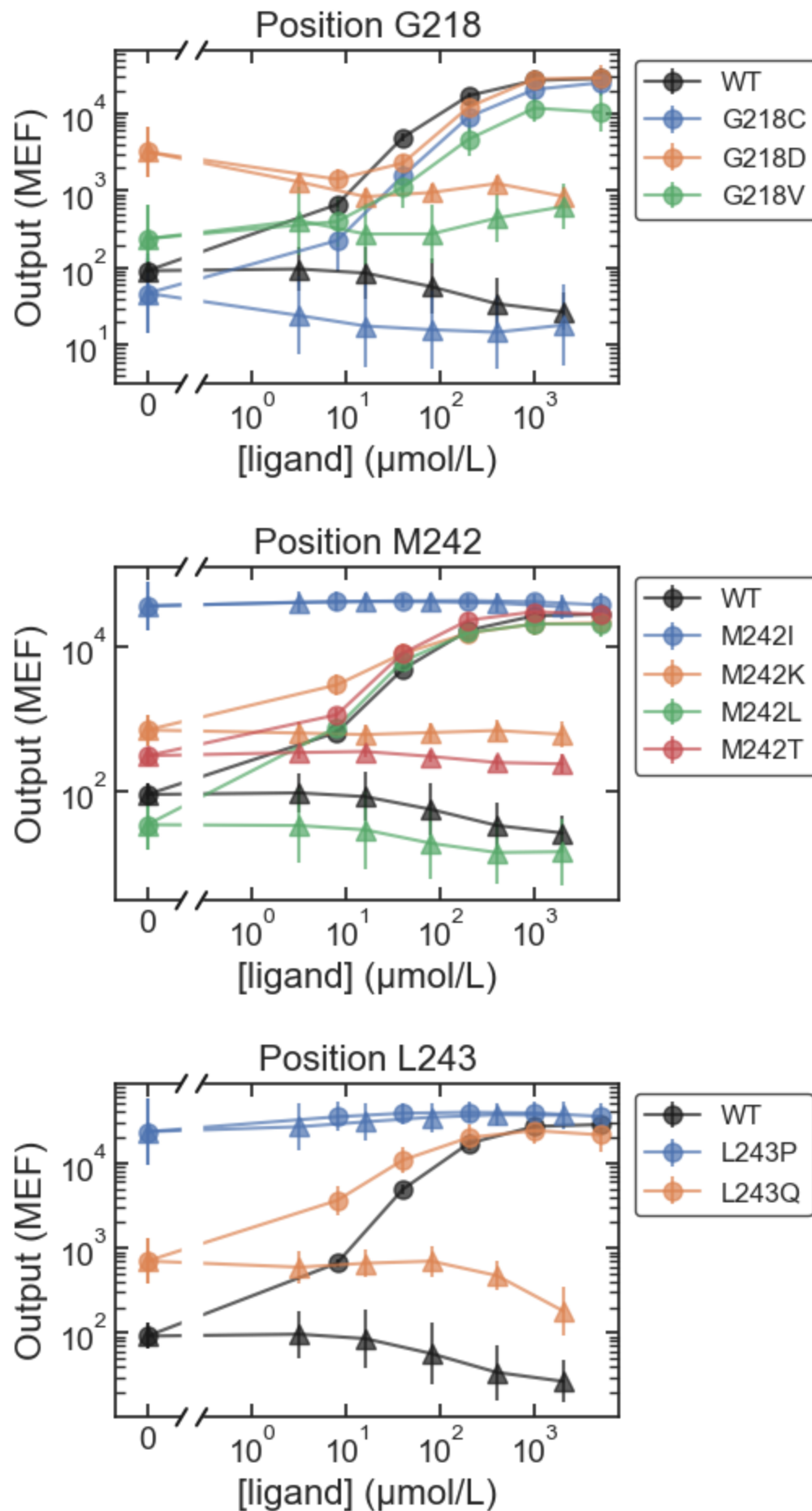


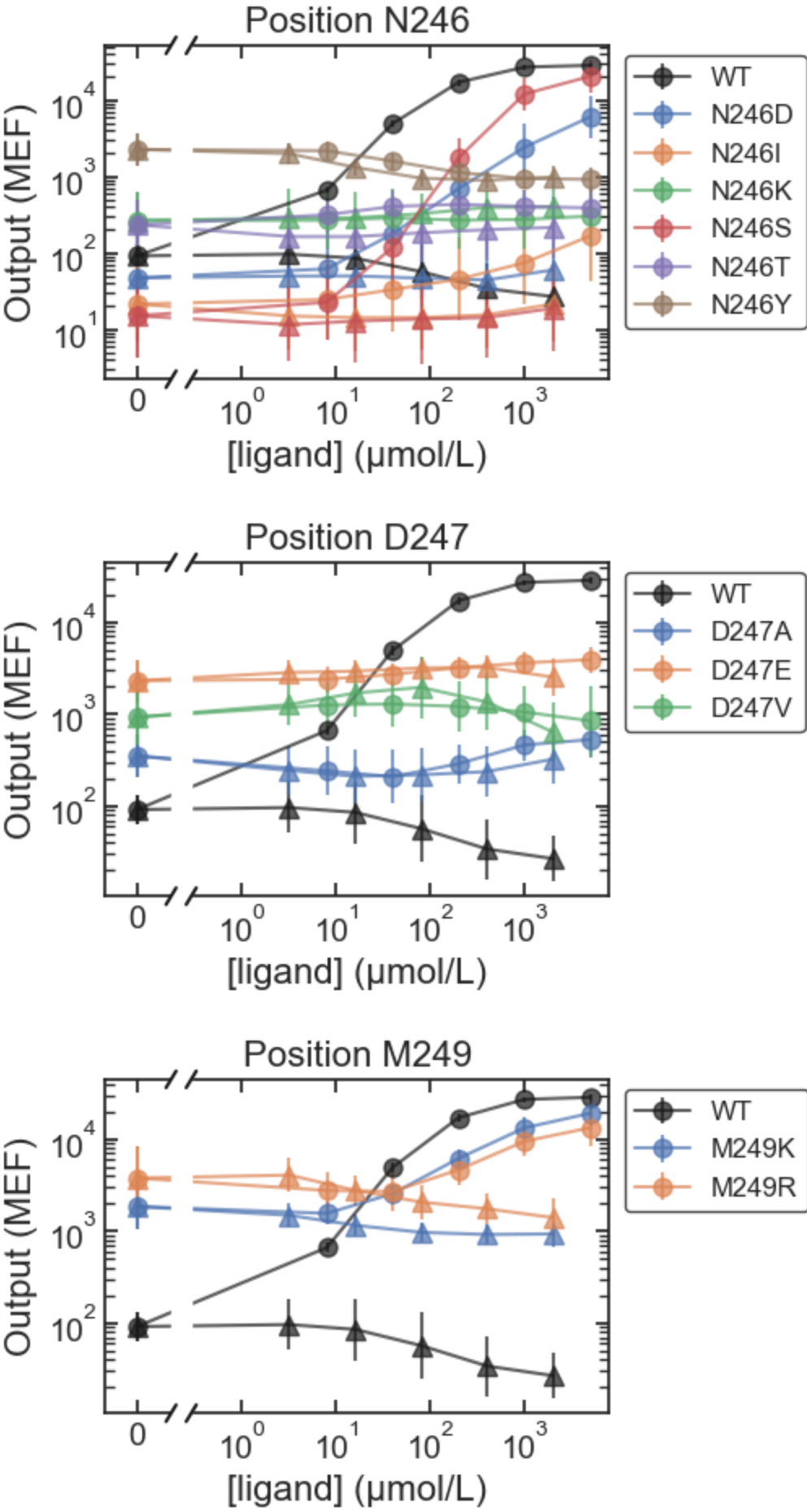


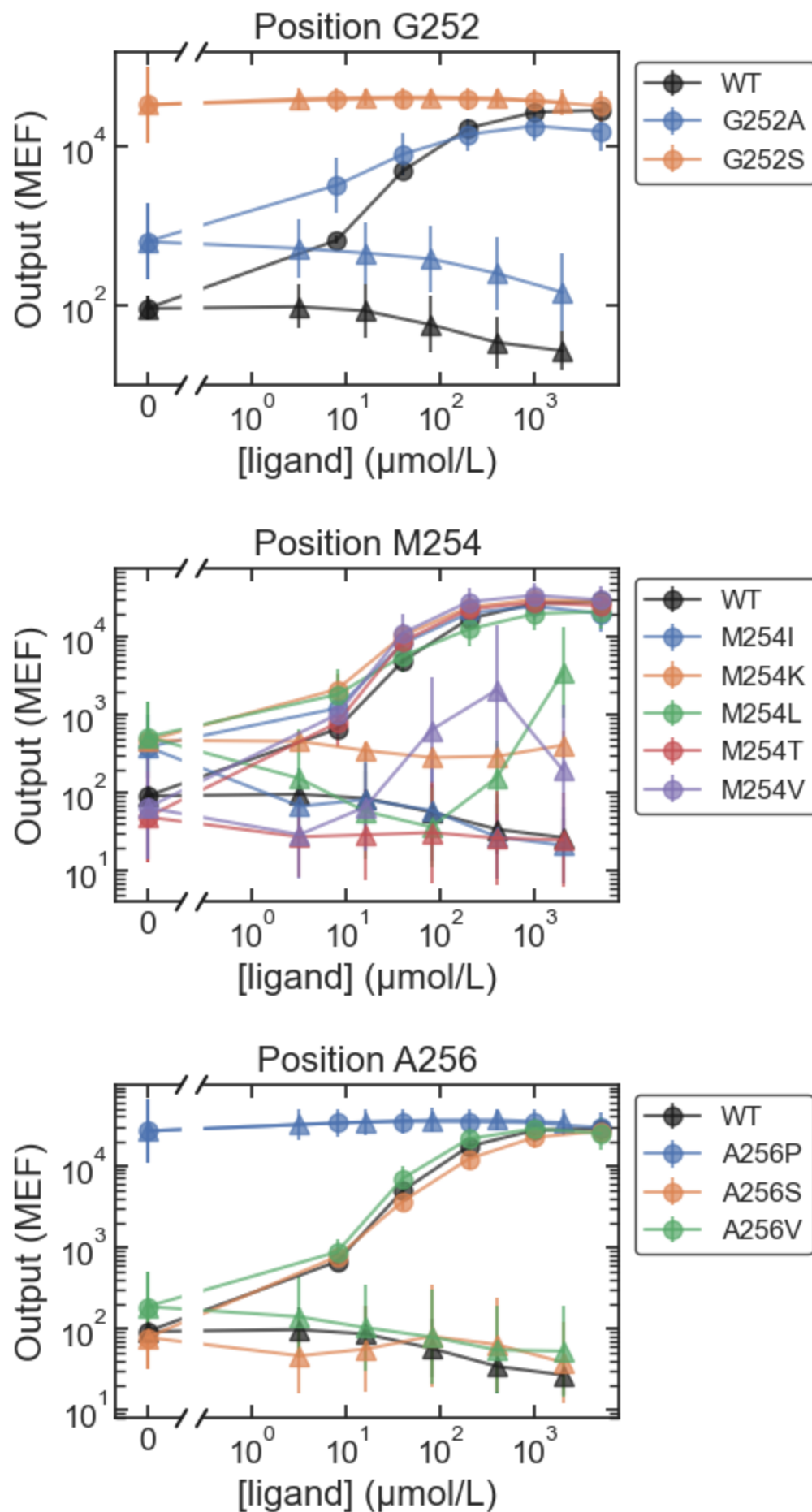


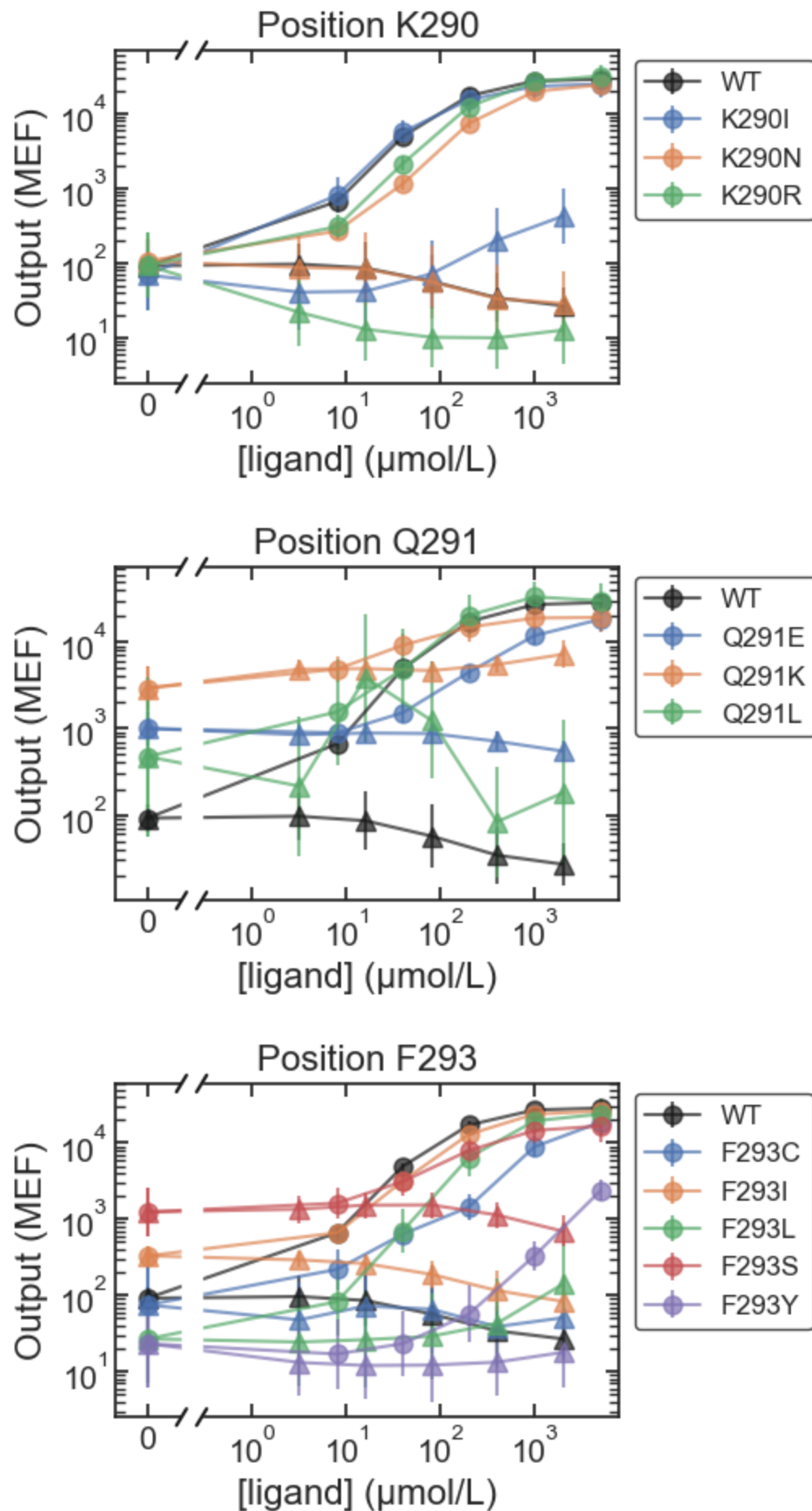


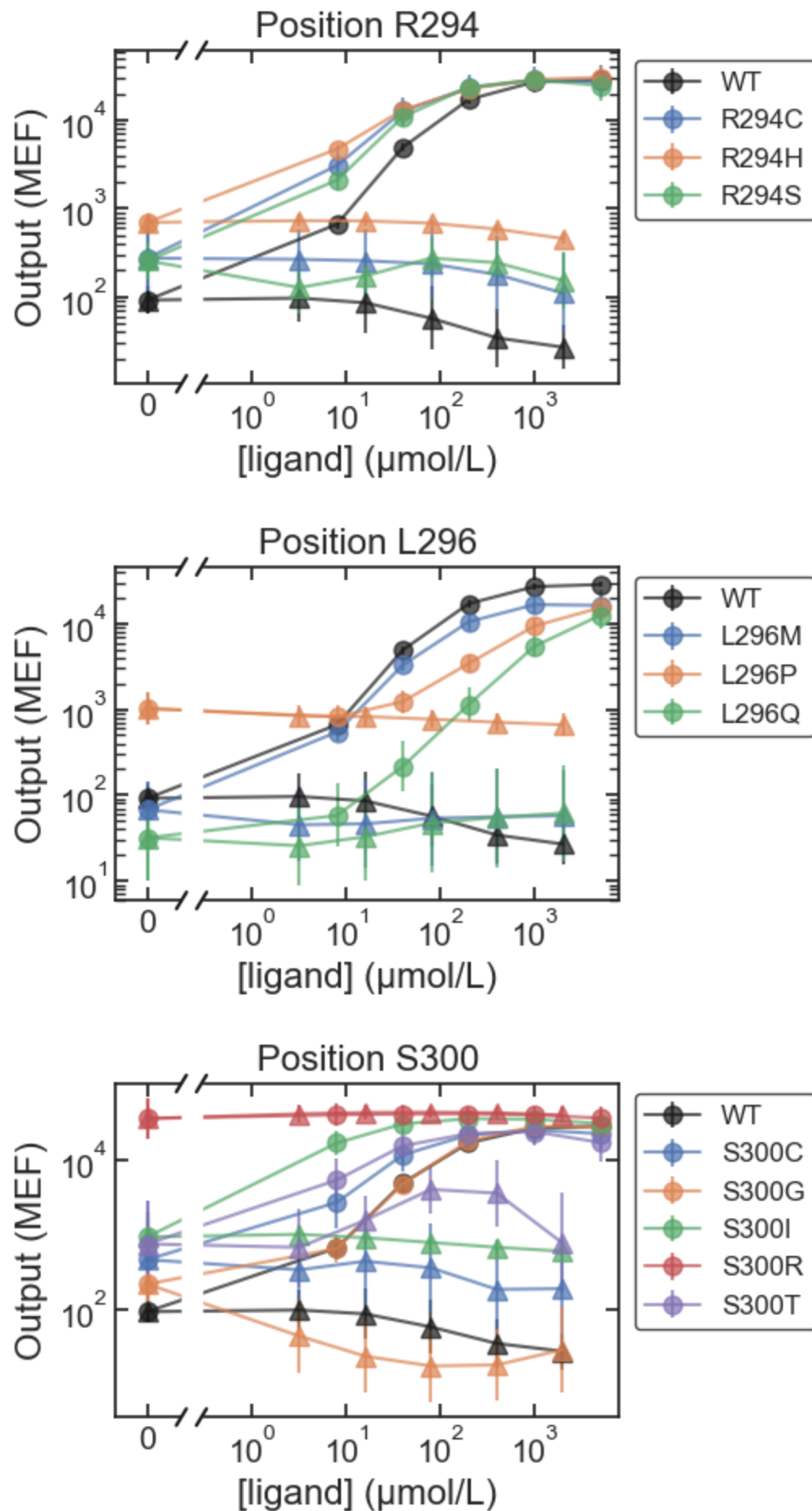


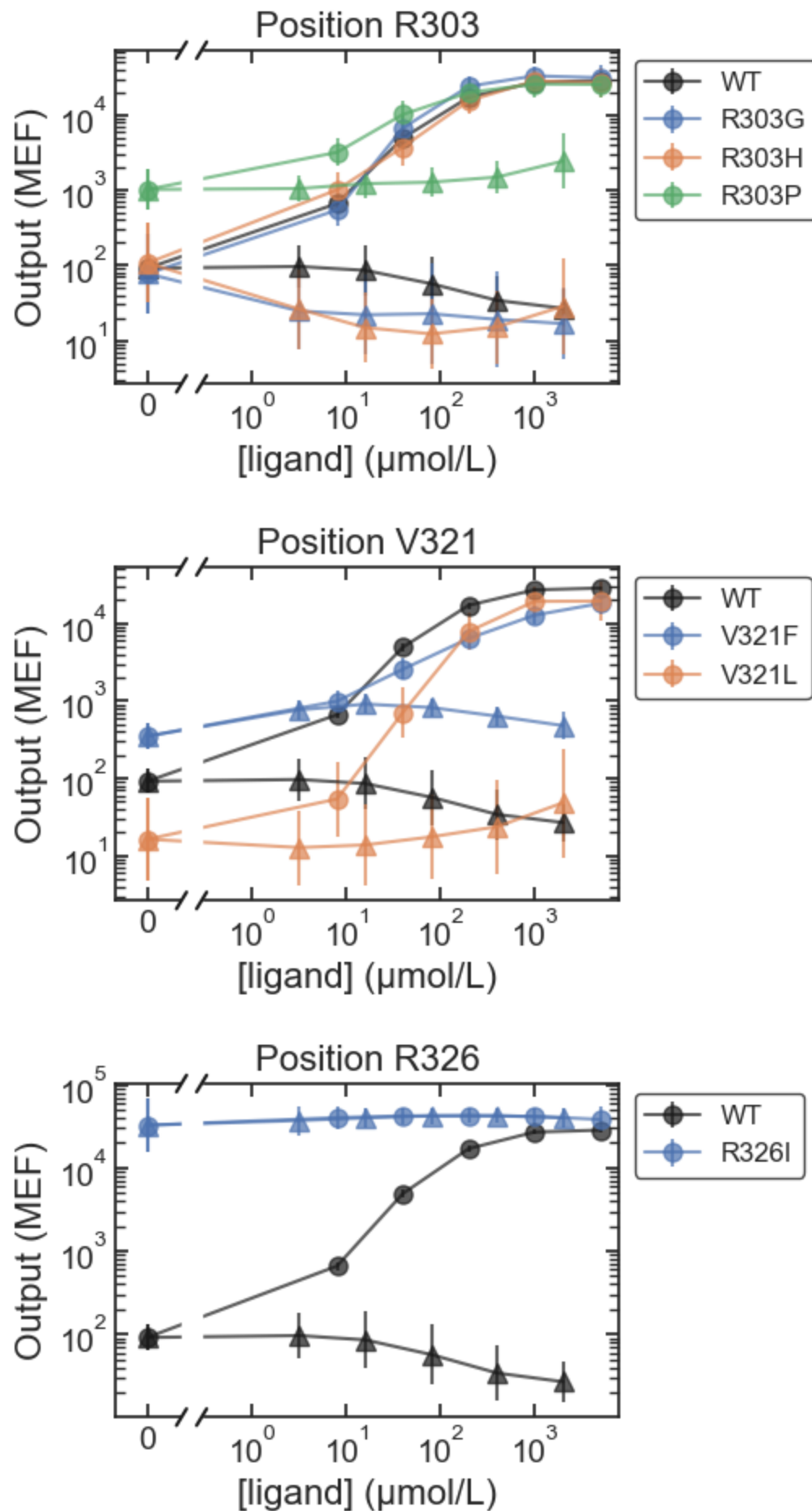


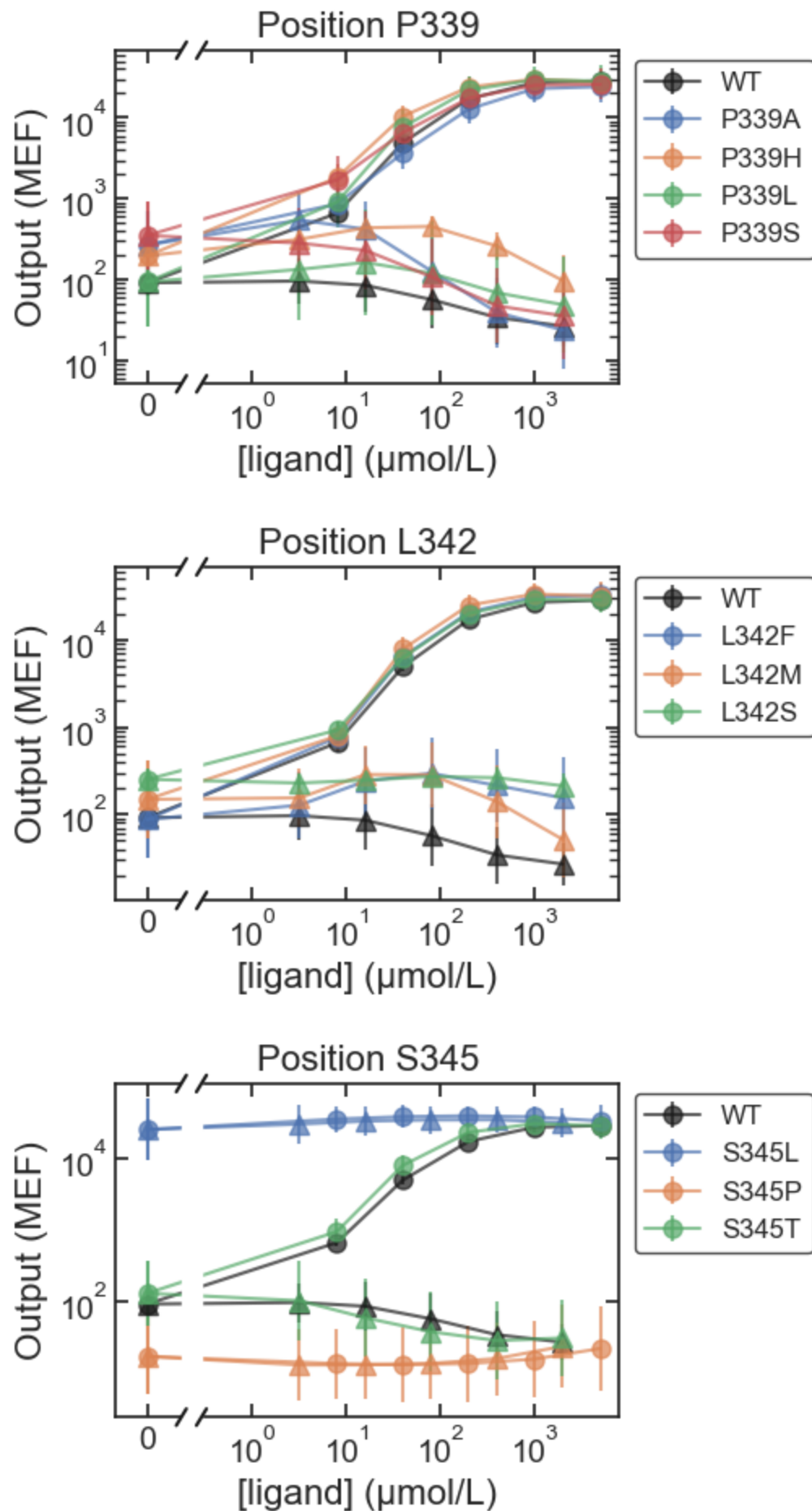




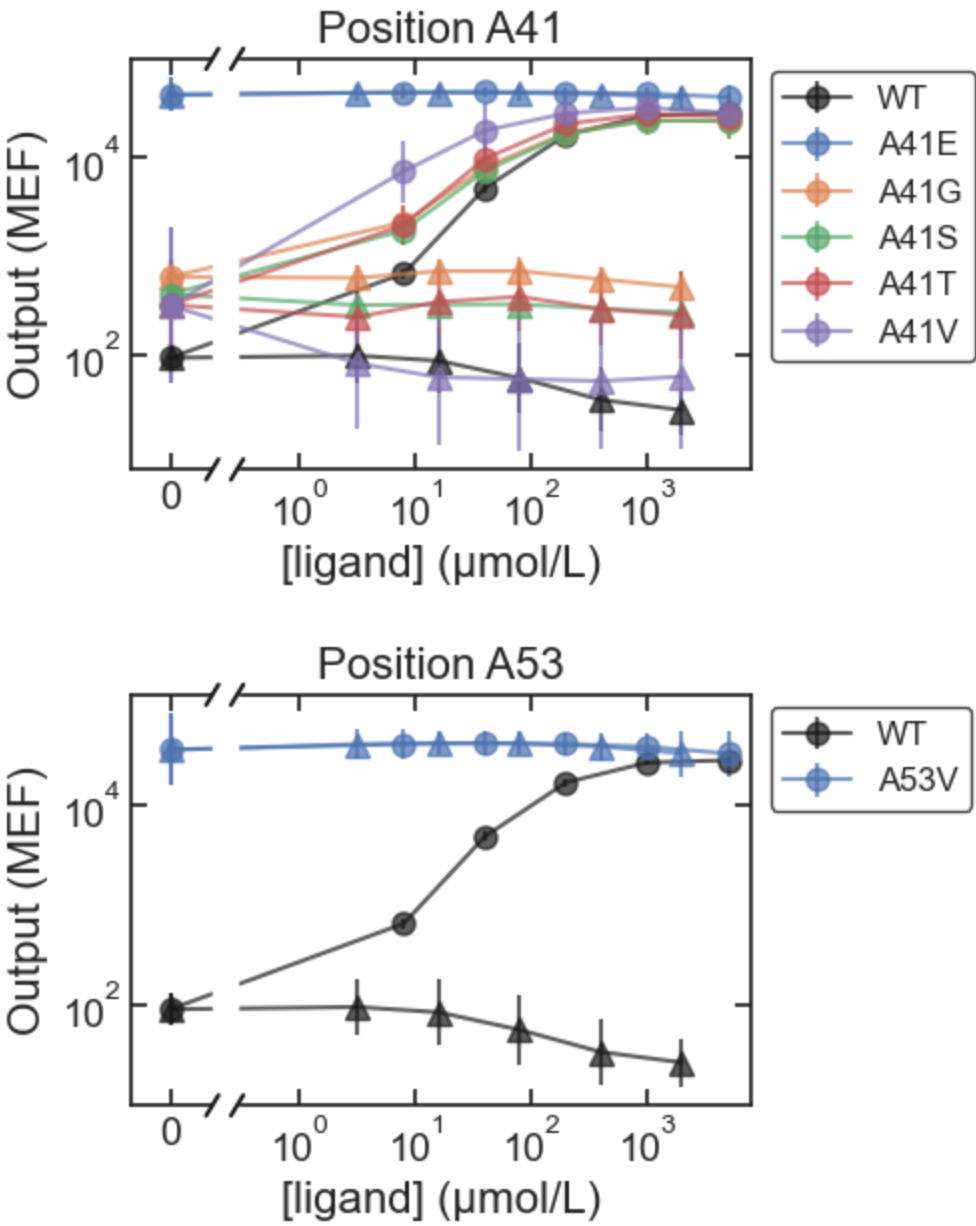


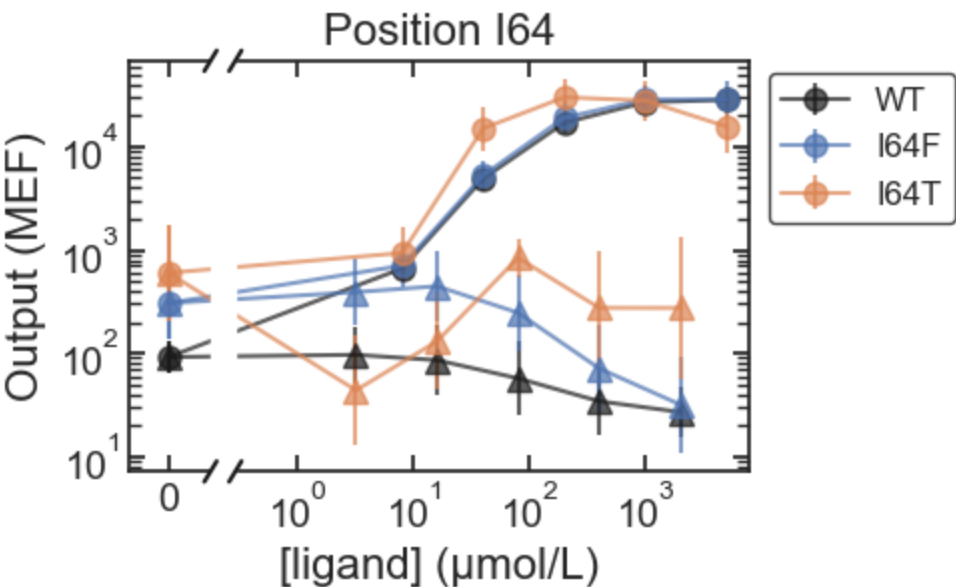
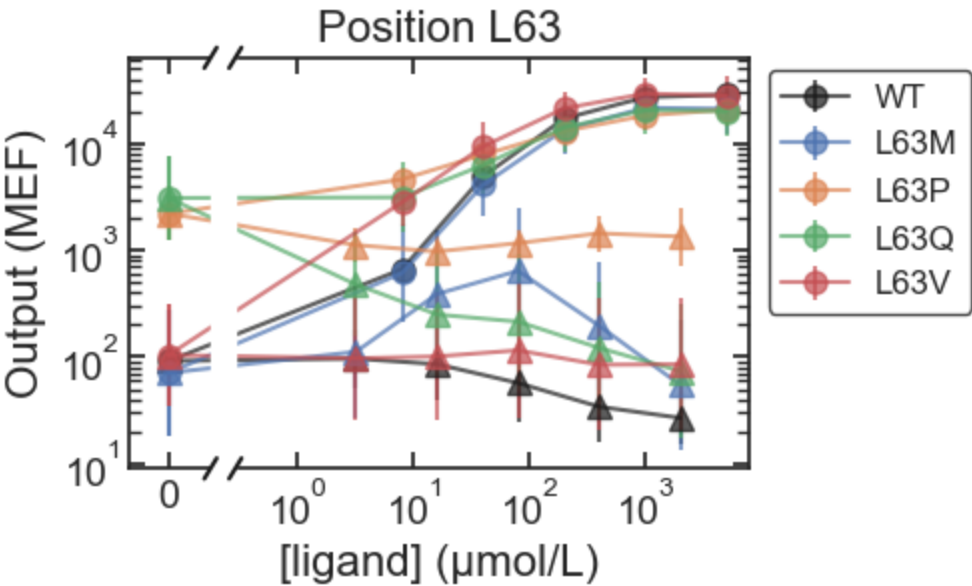
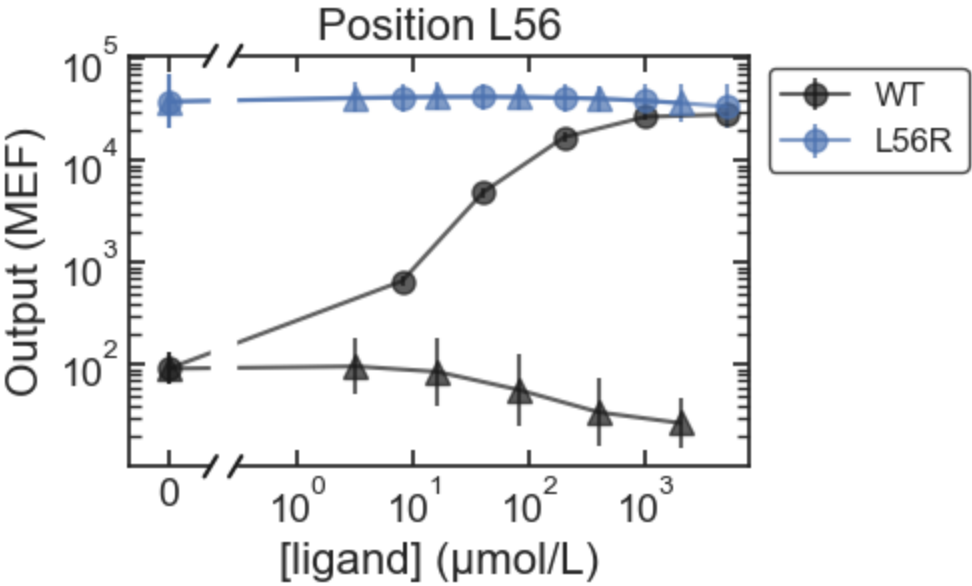


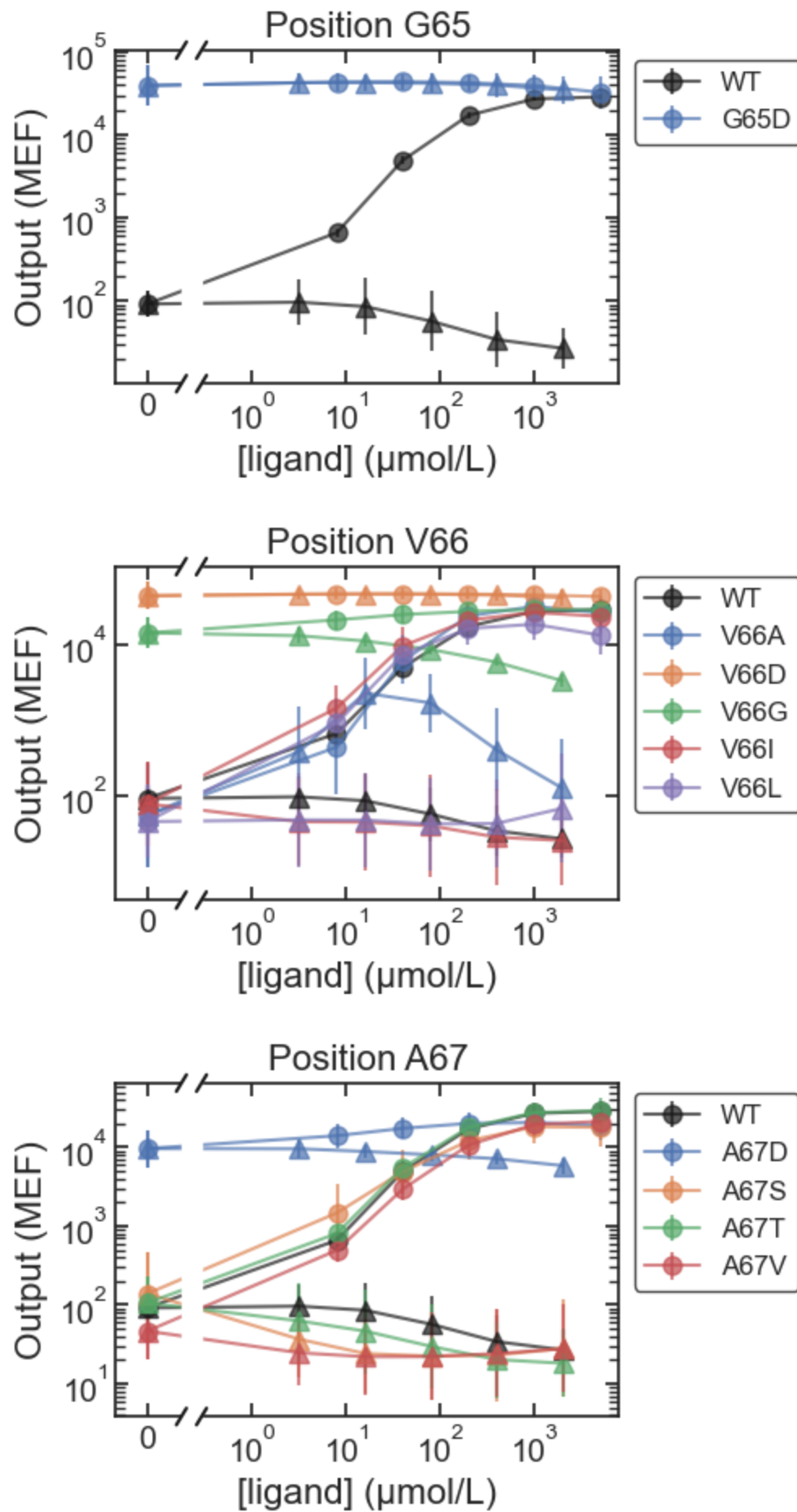


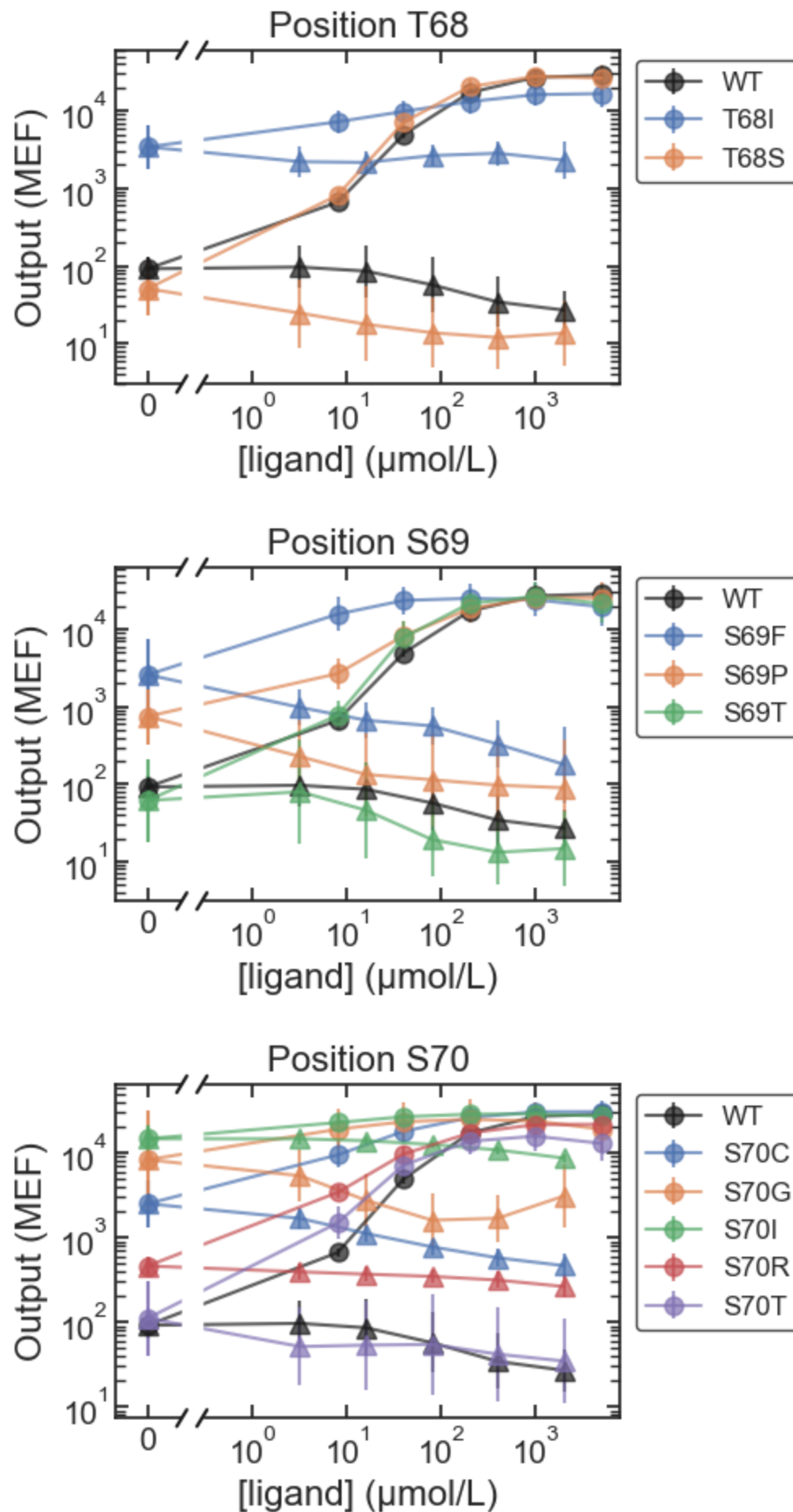


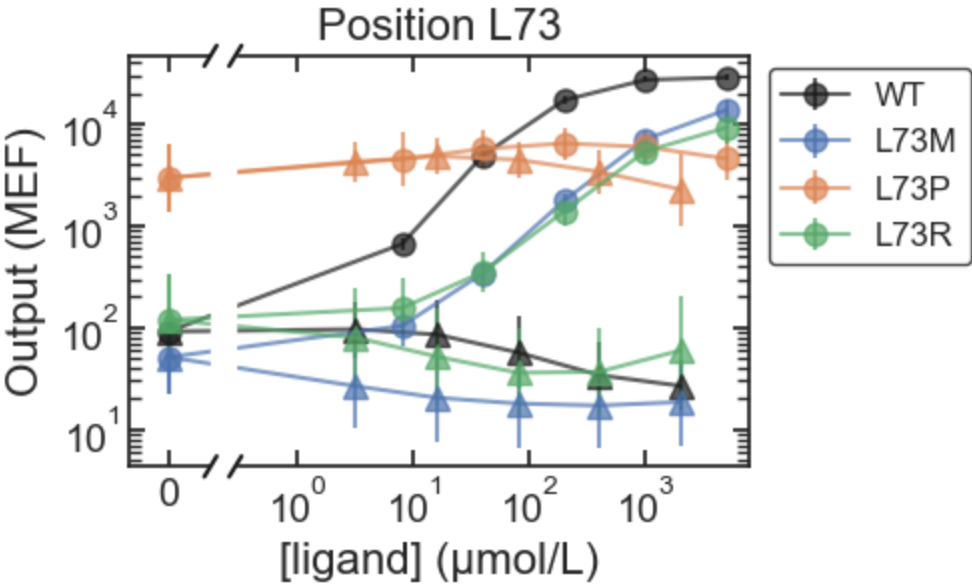
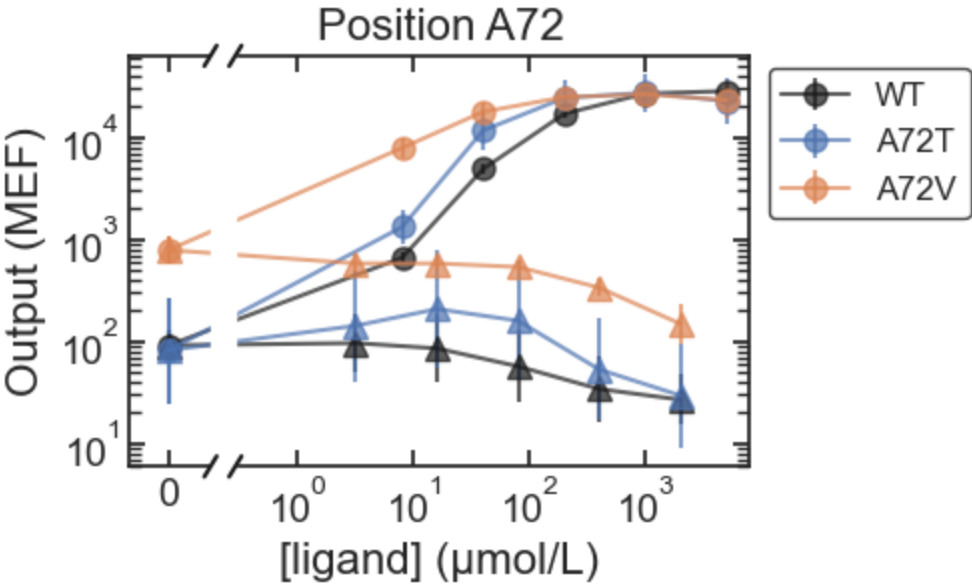
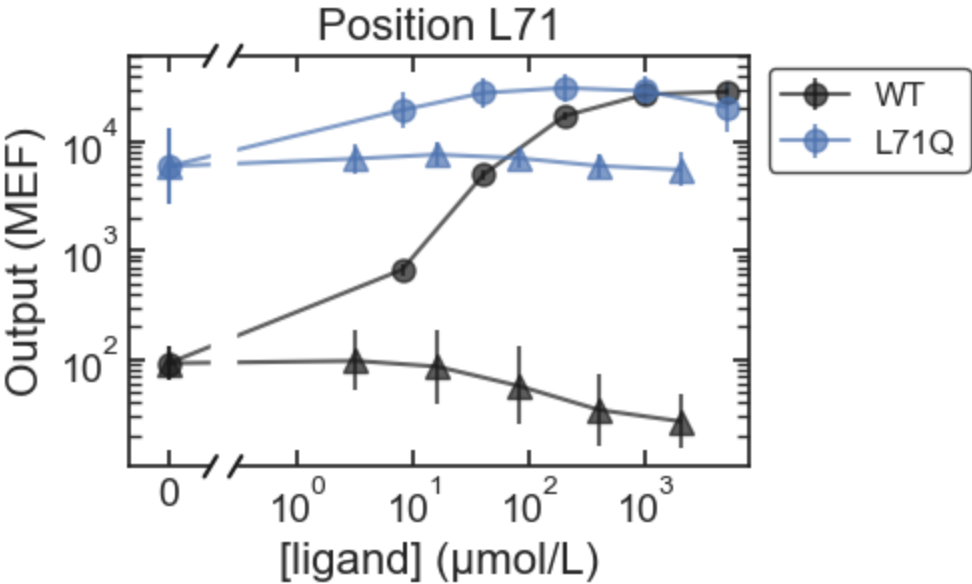
MWC

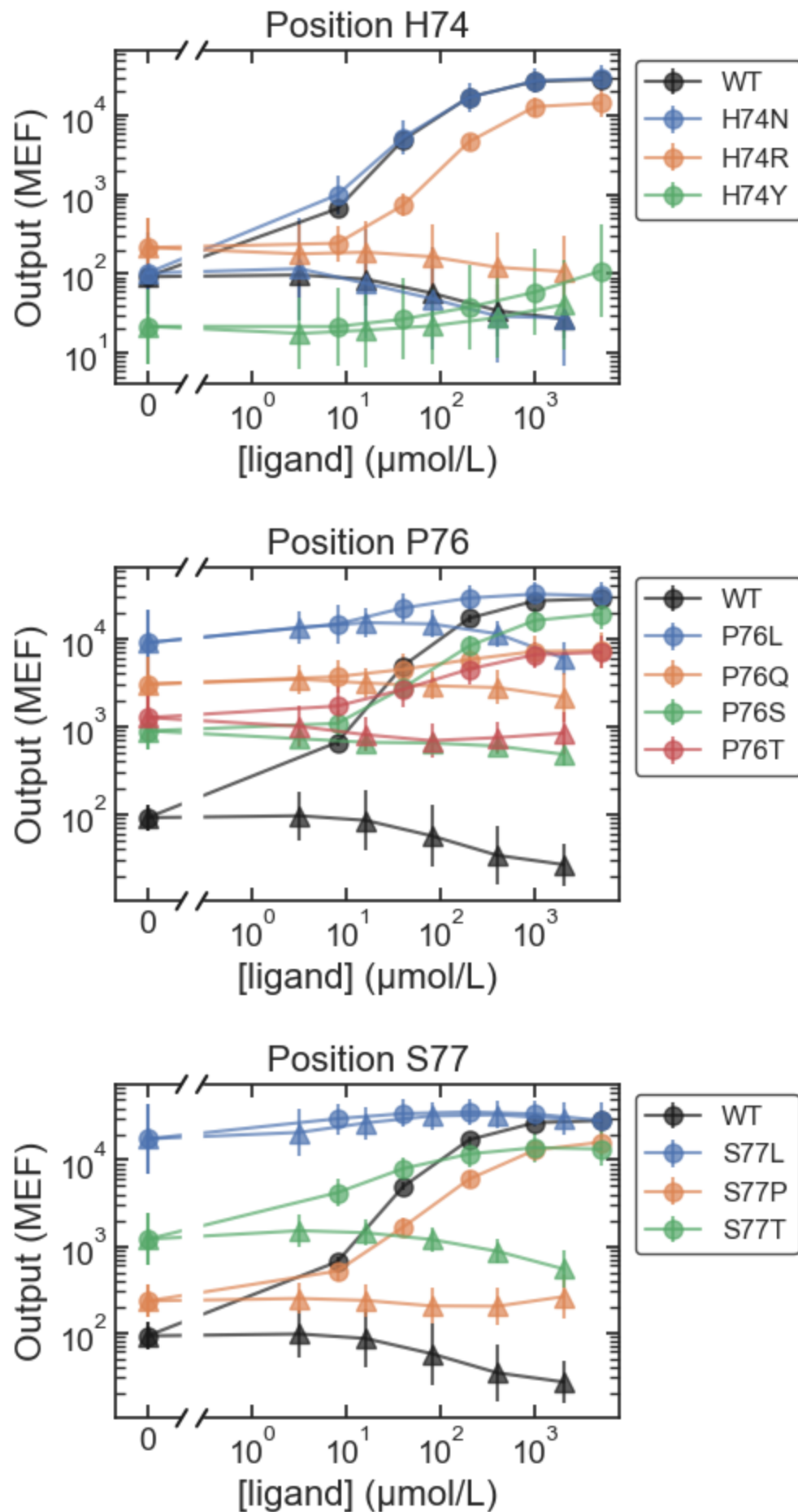


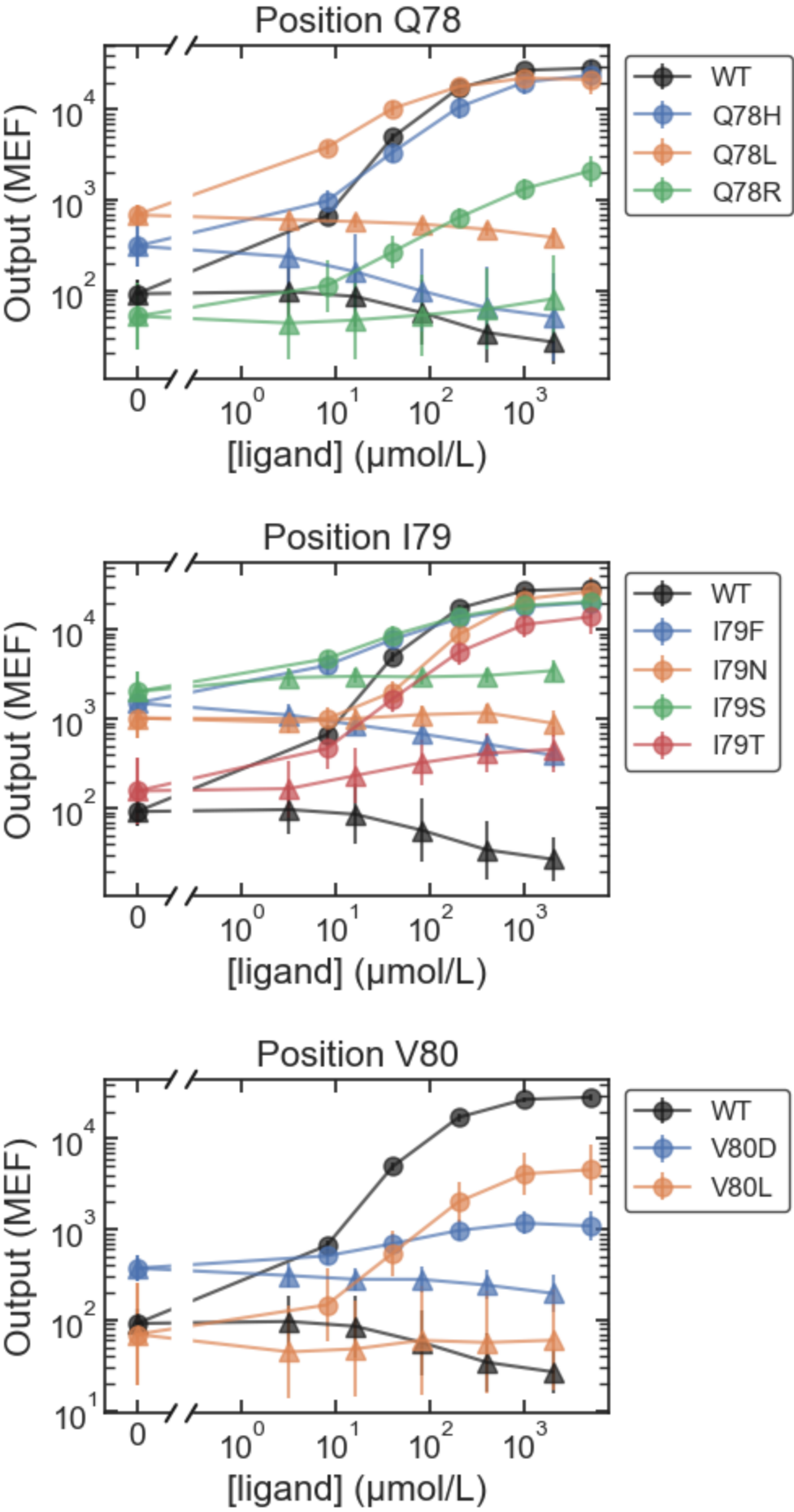


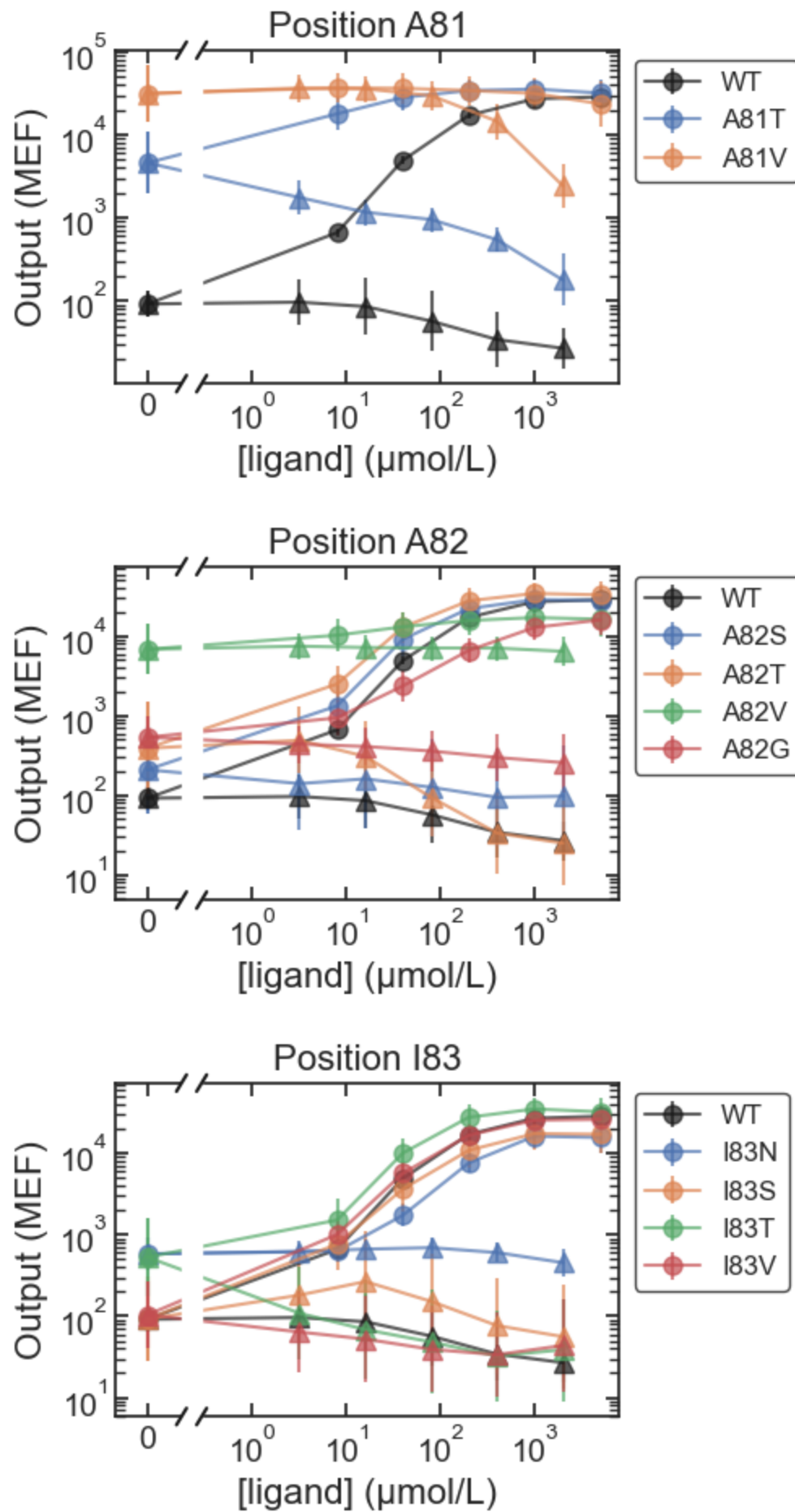


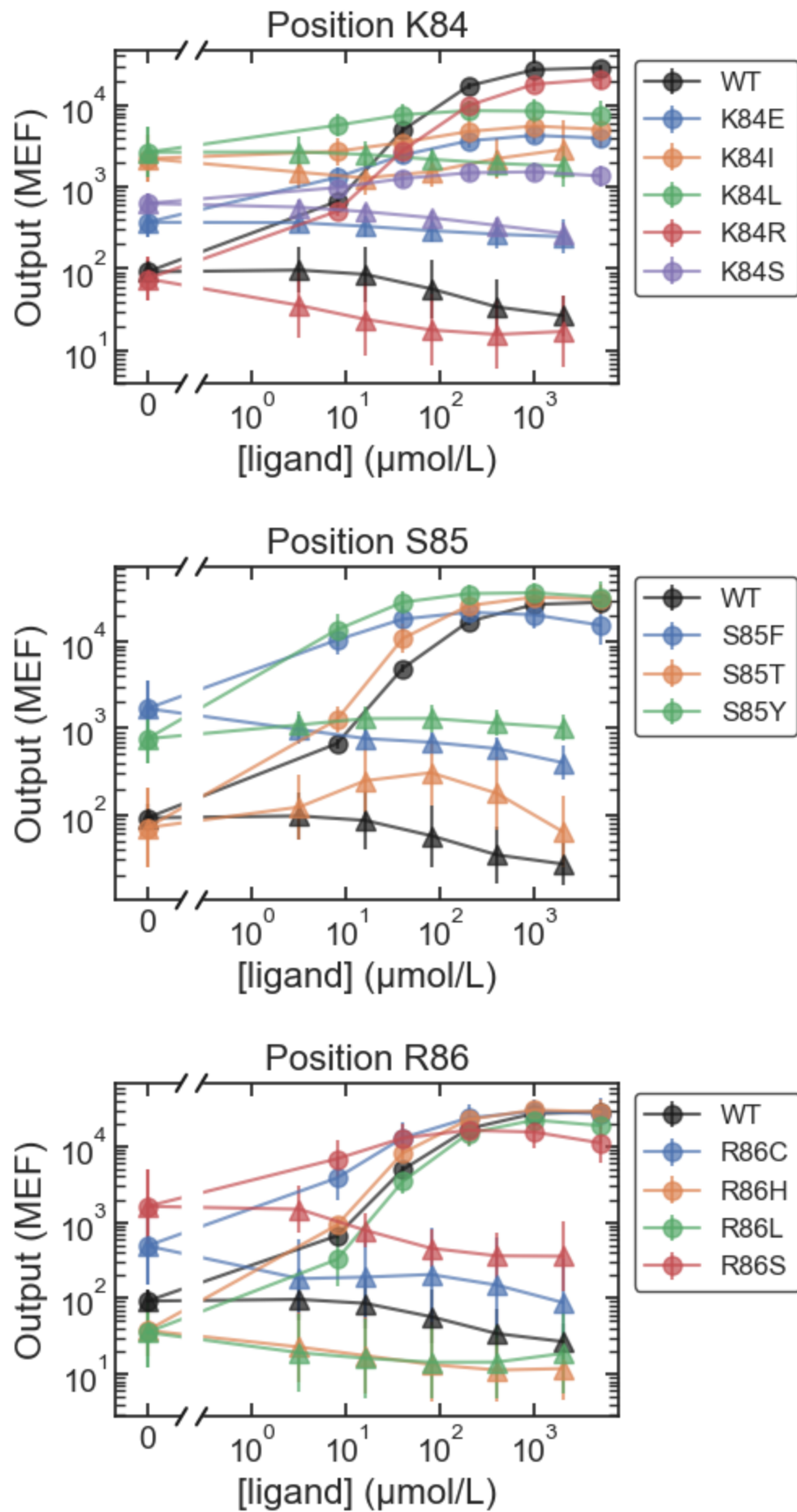


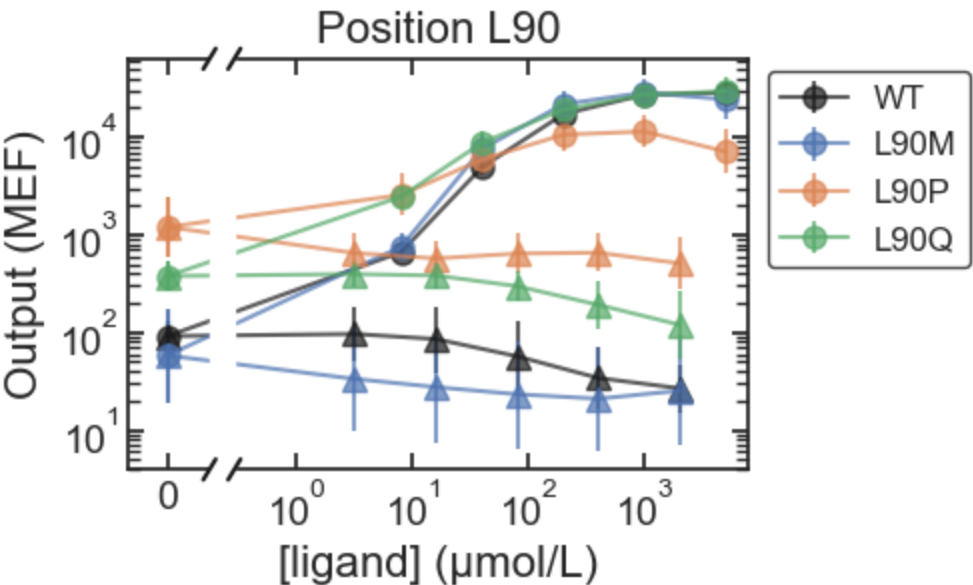
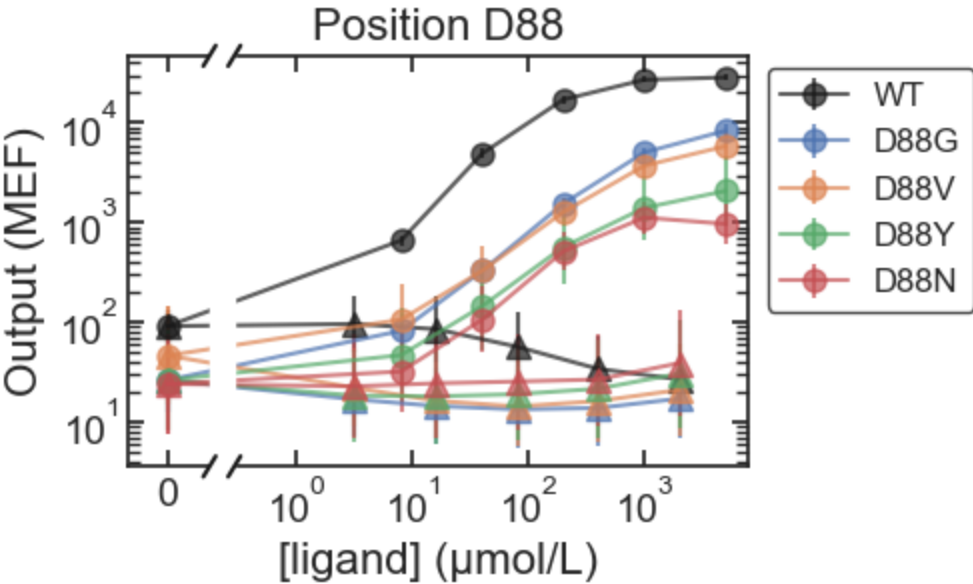
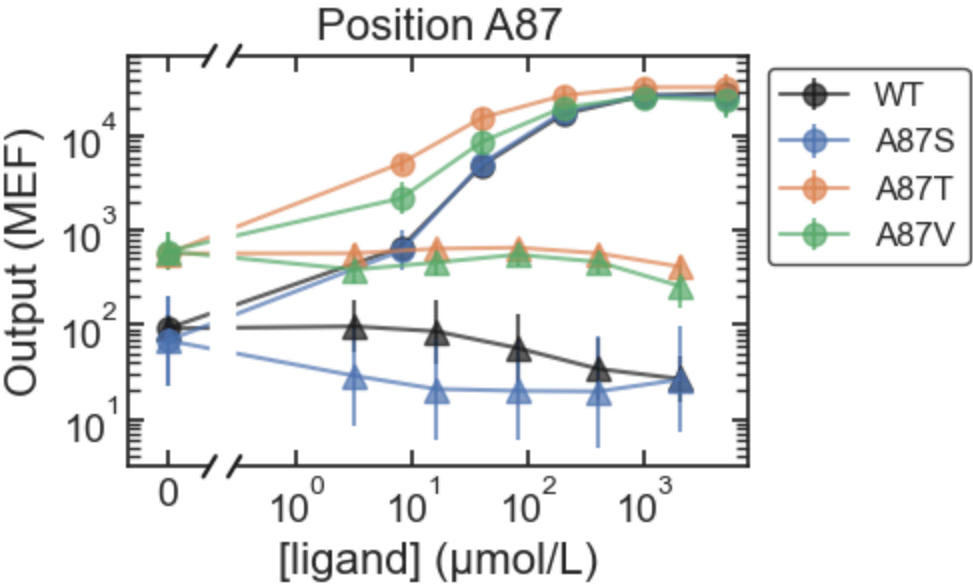


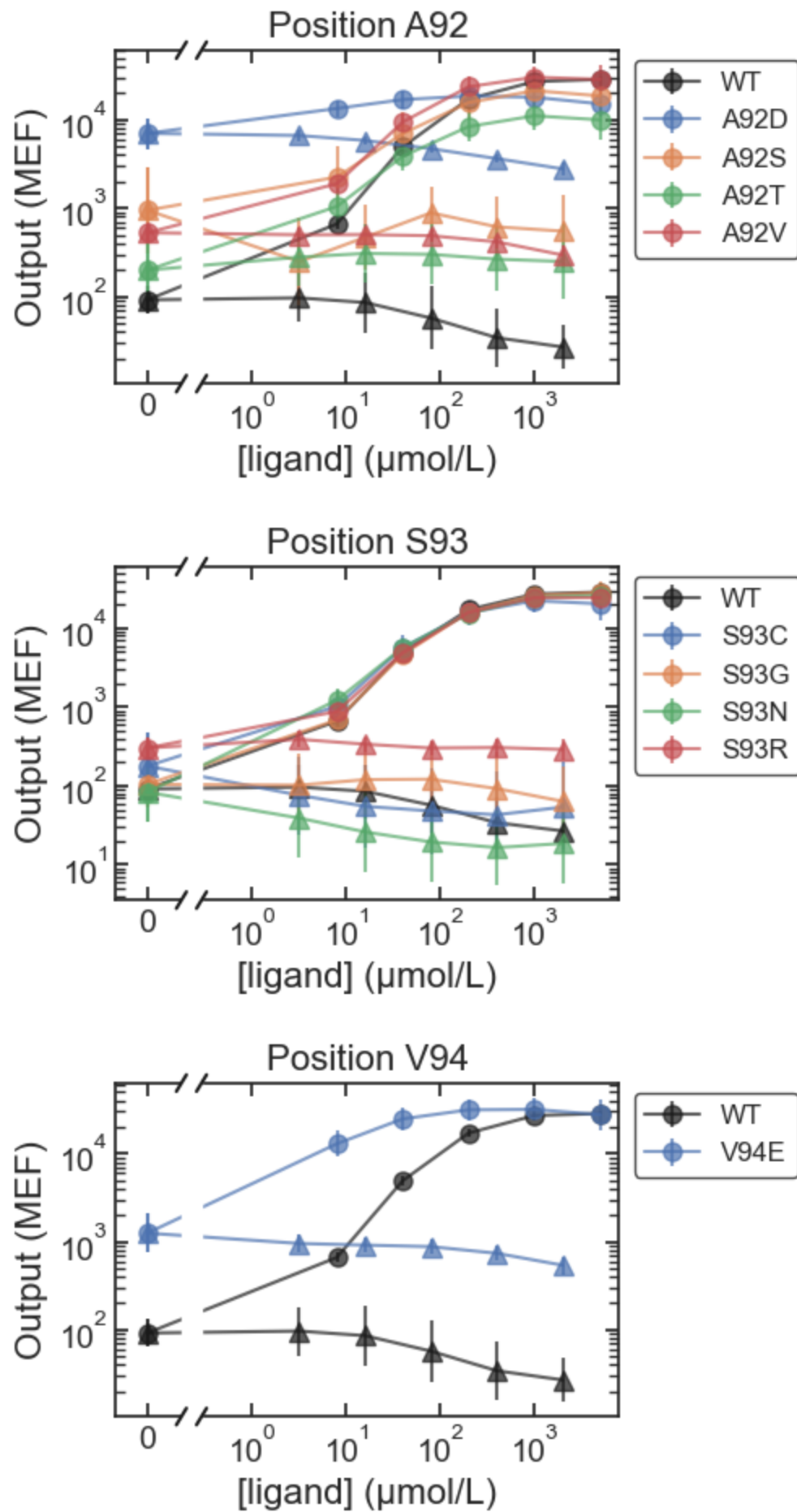


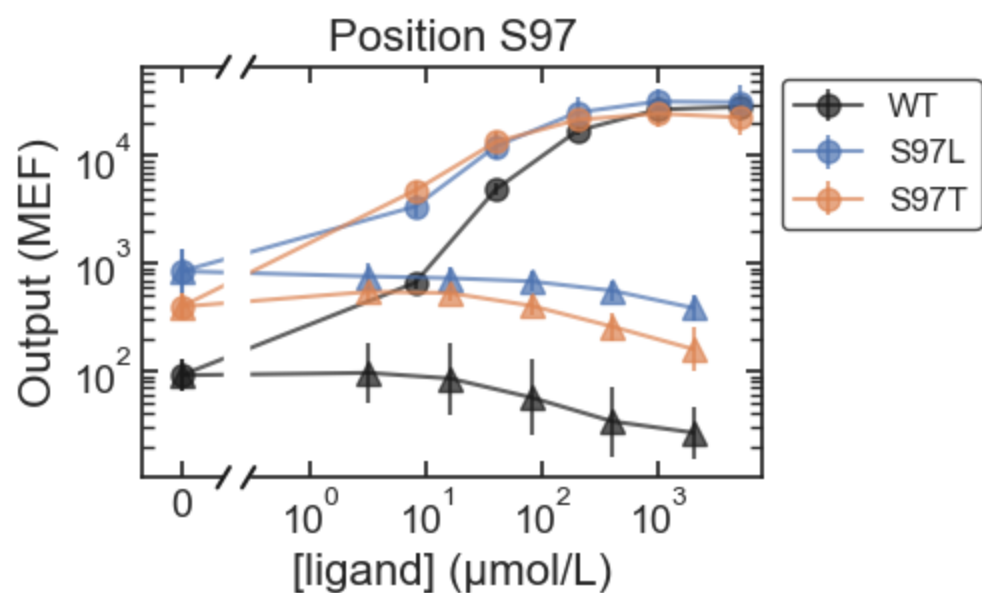
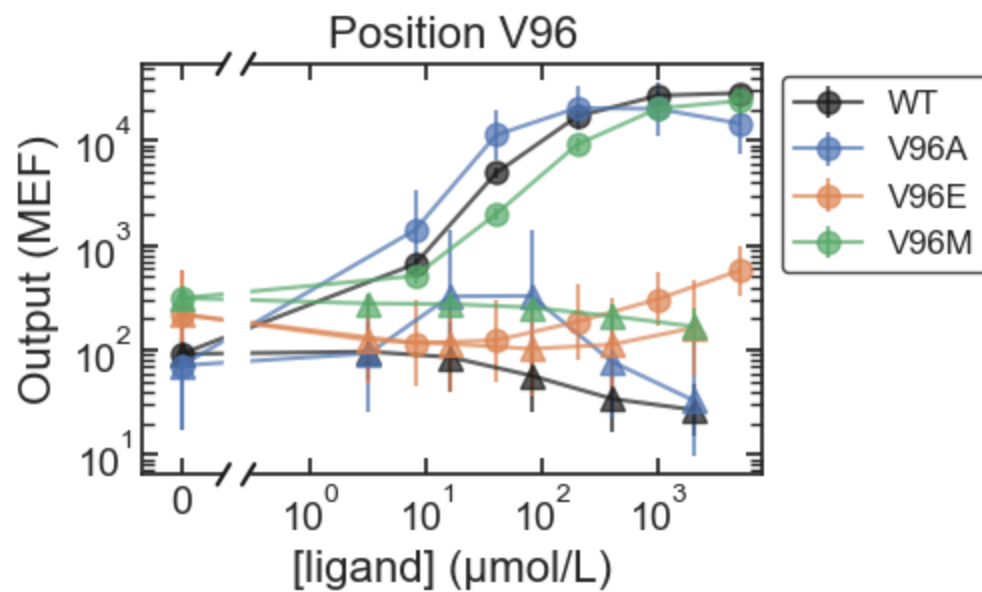
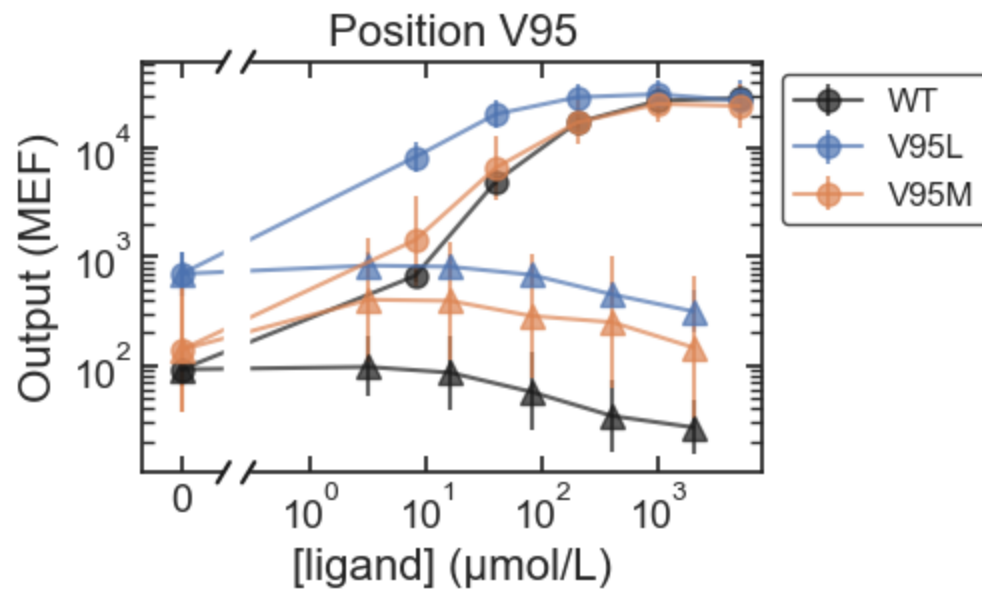


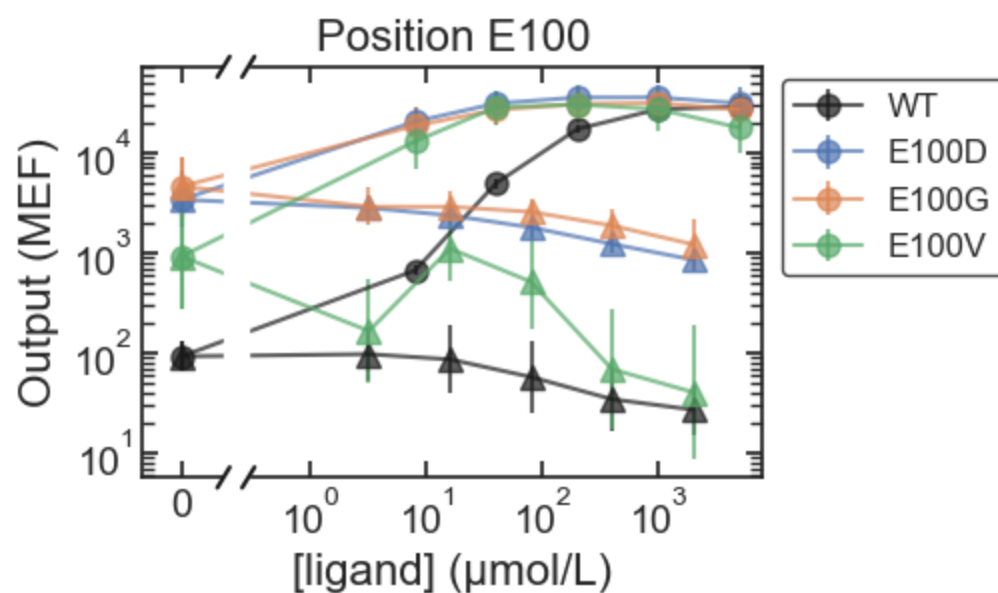
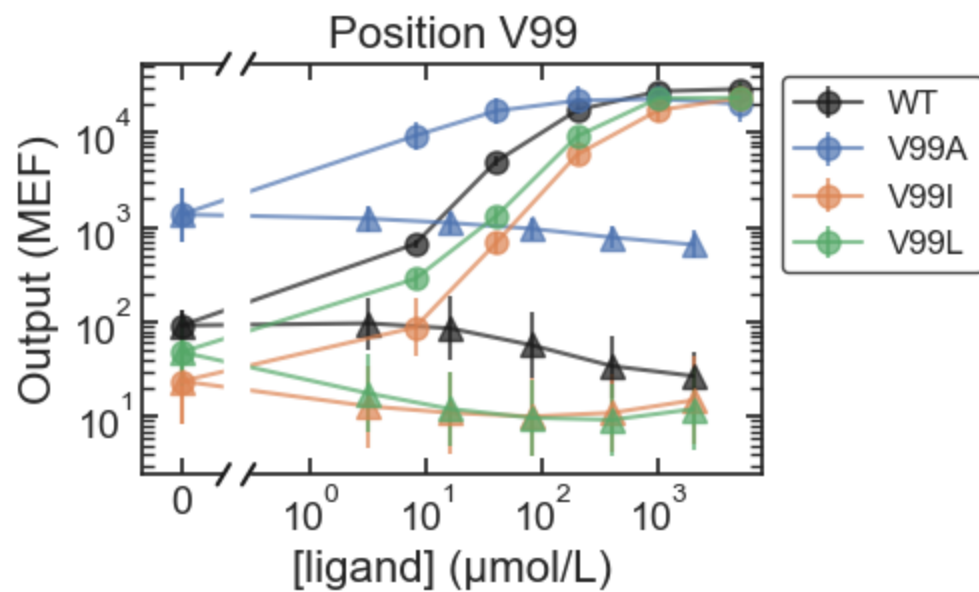
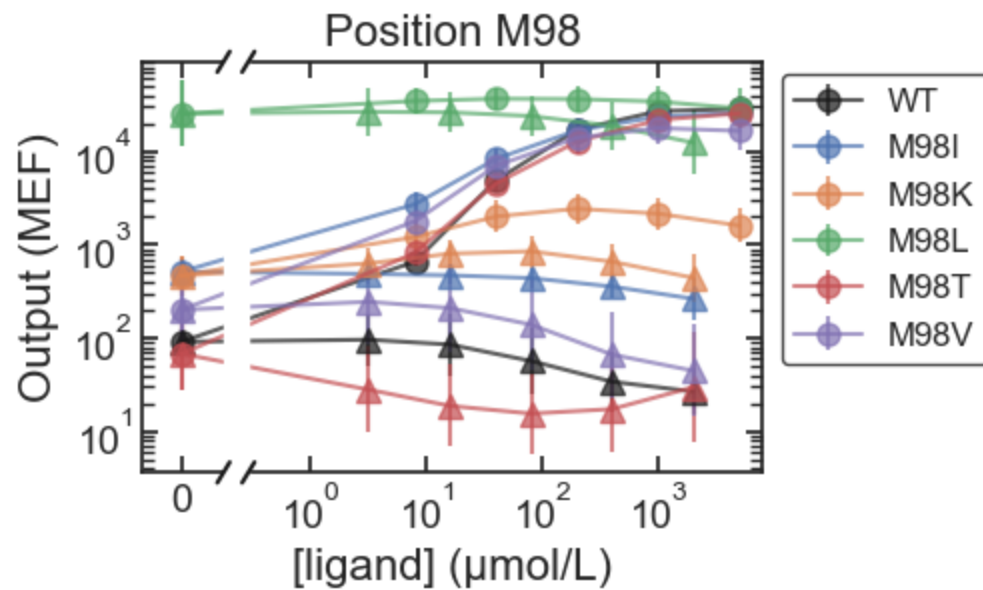


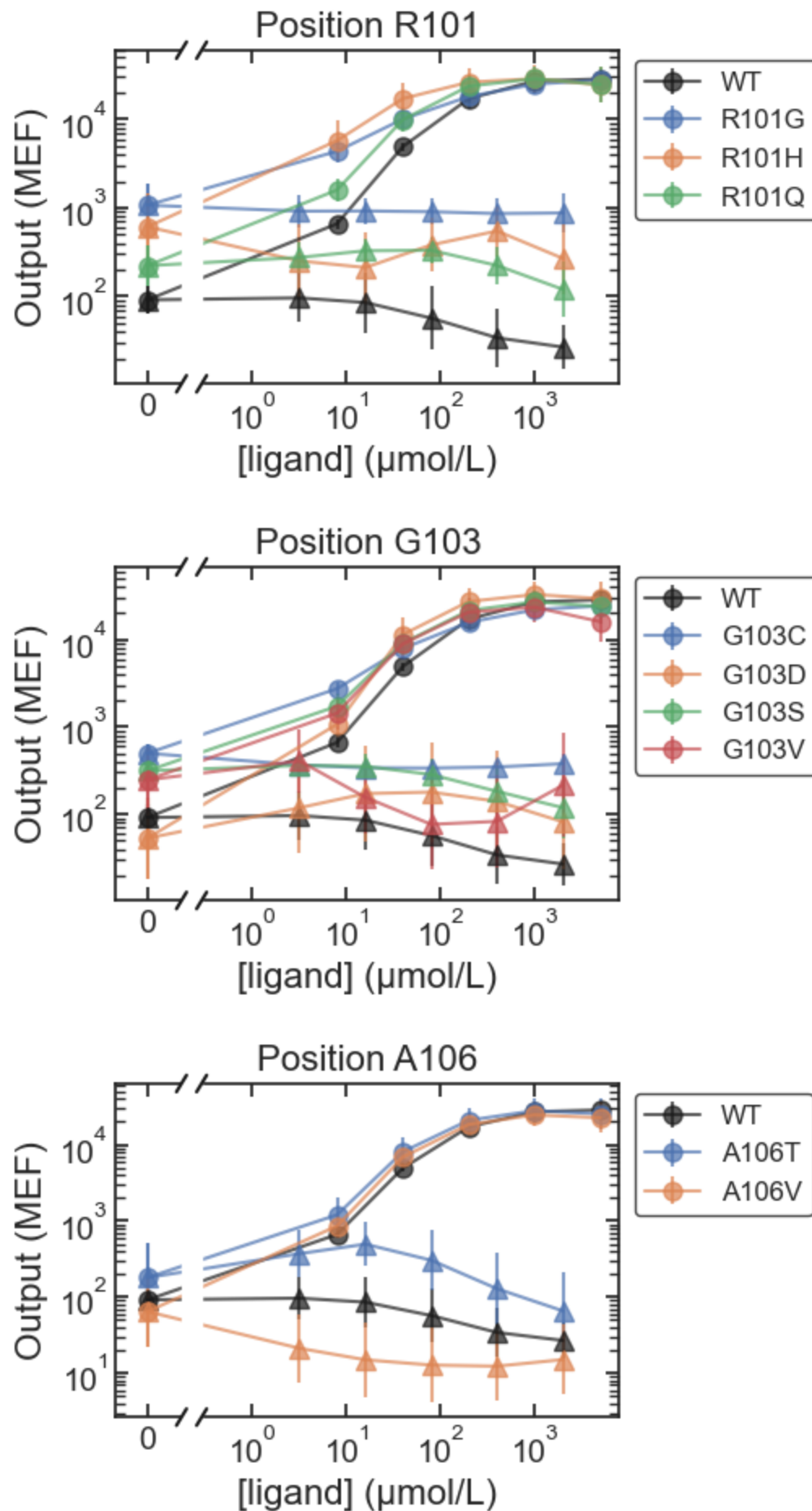


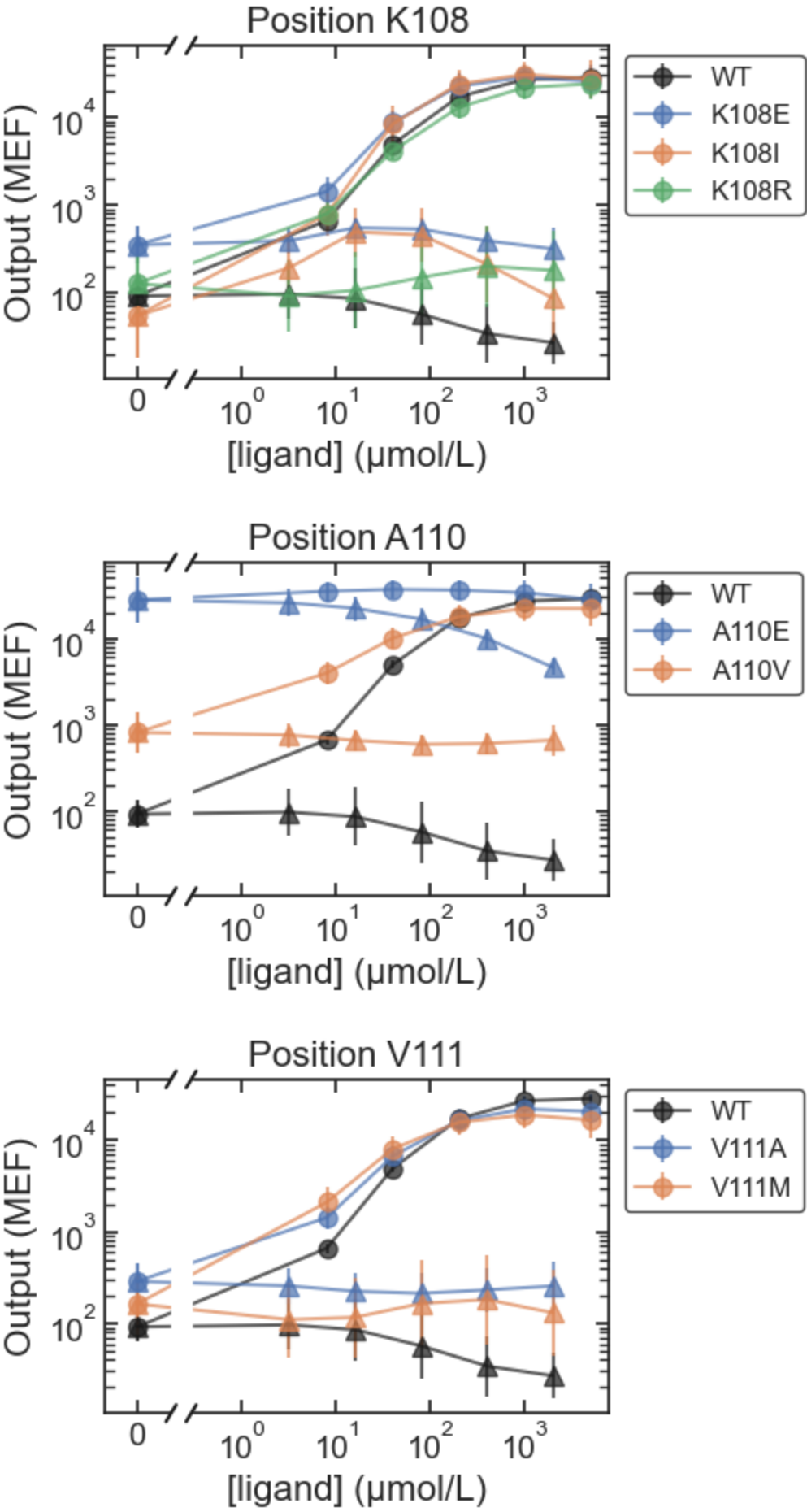


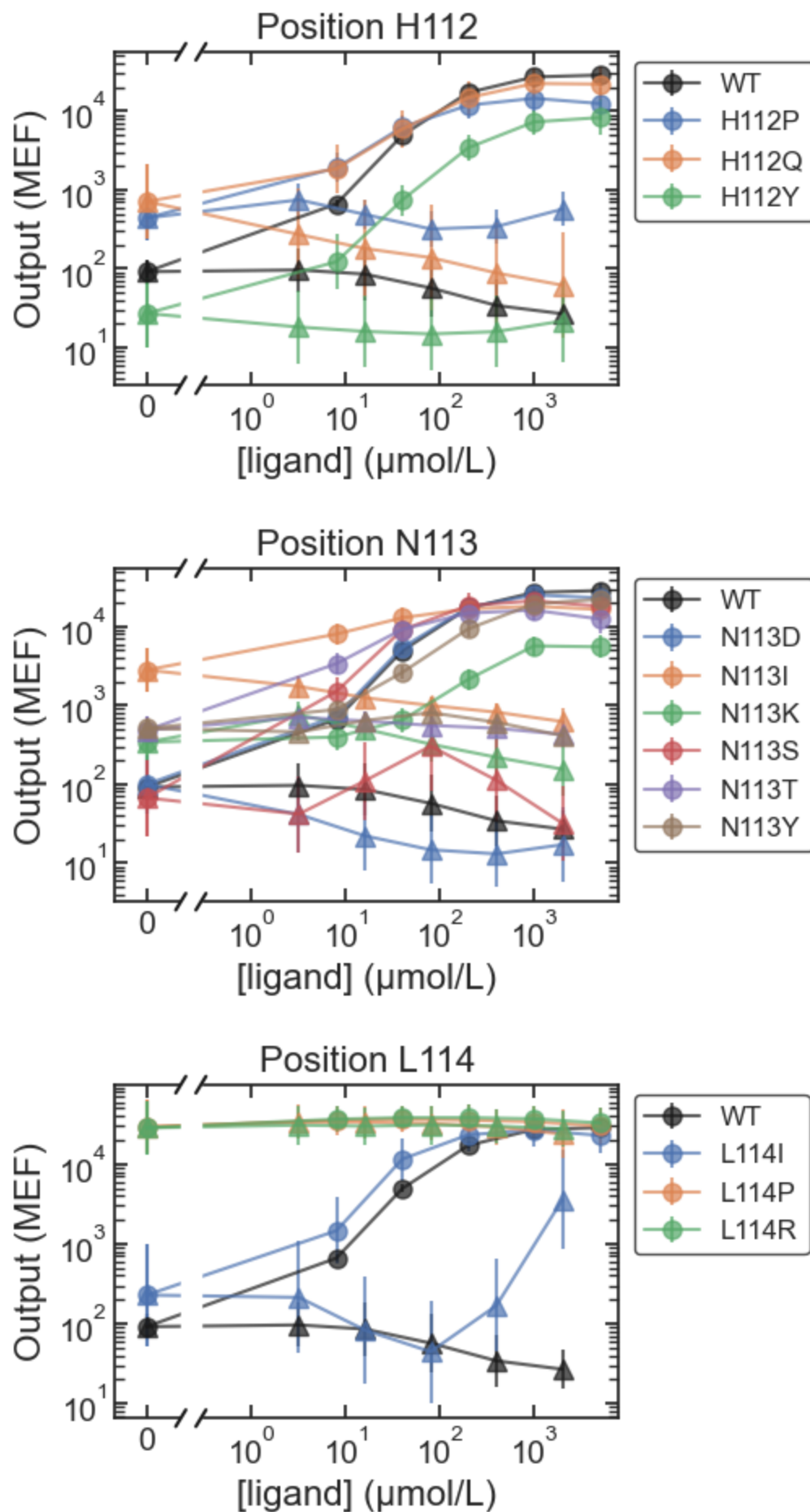


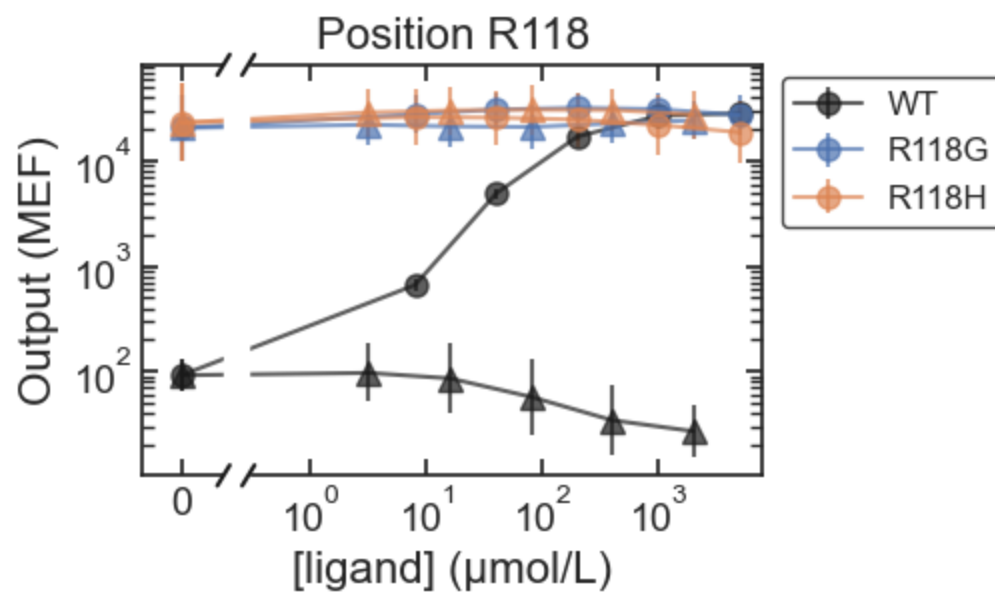
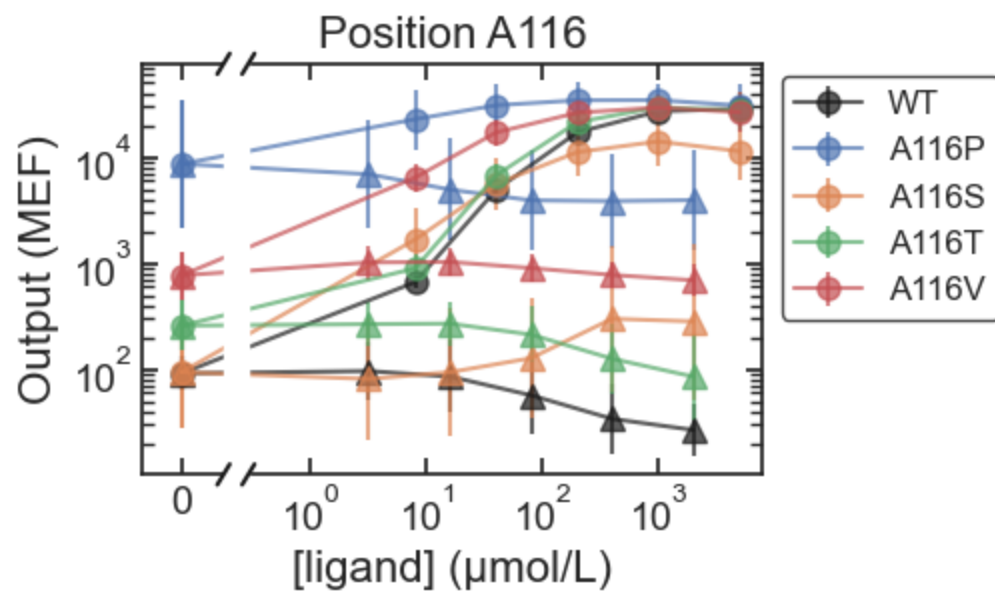
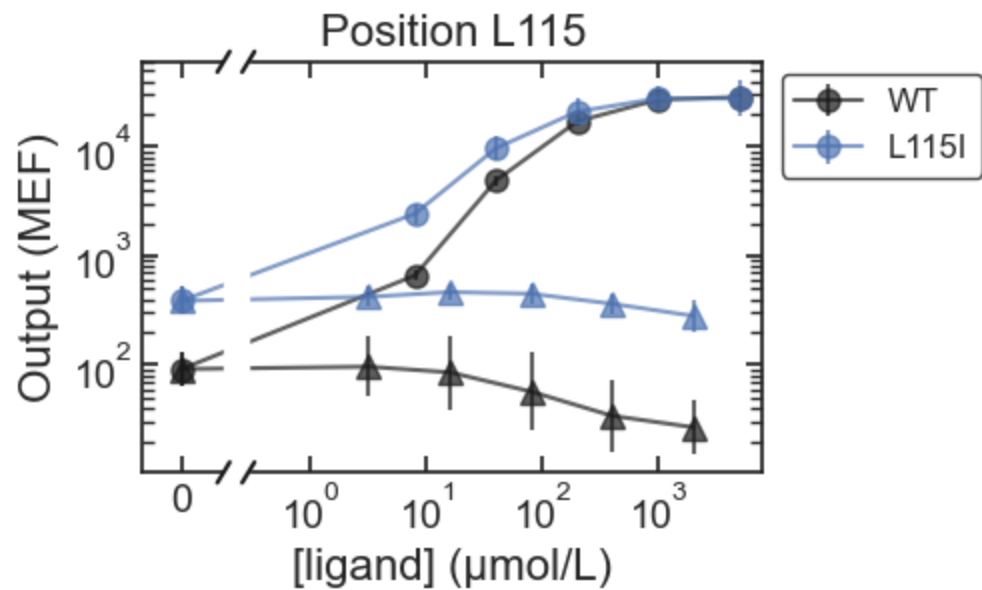


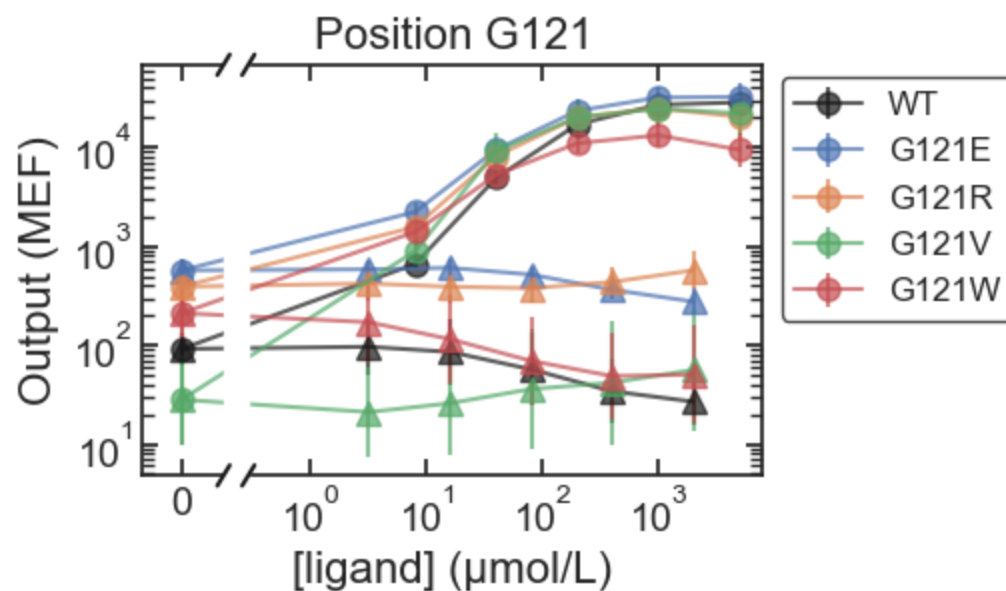
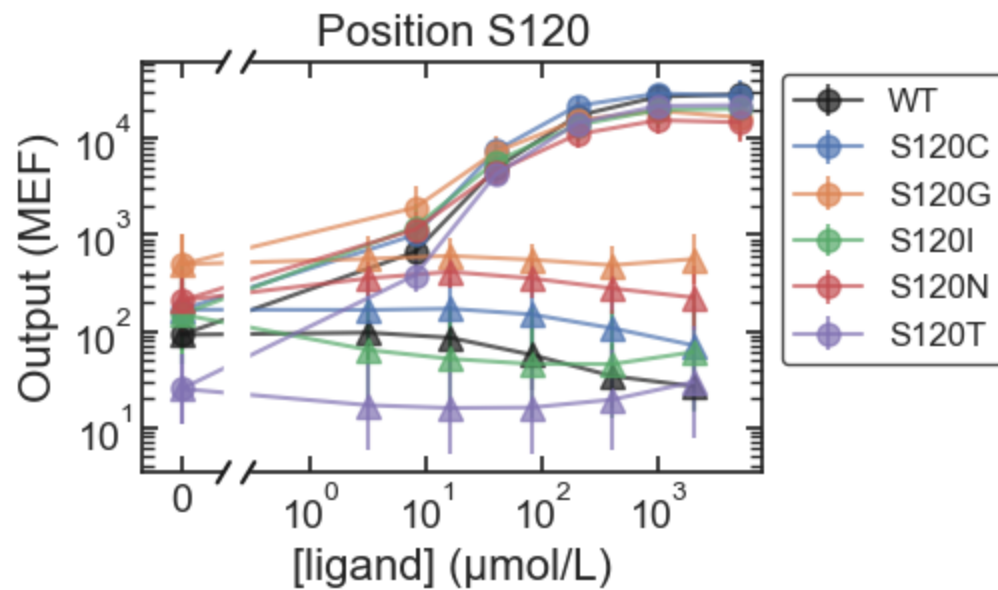
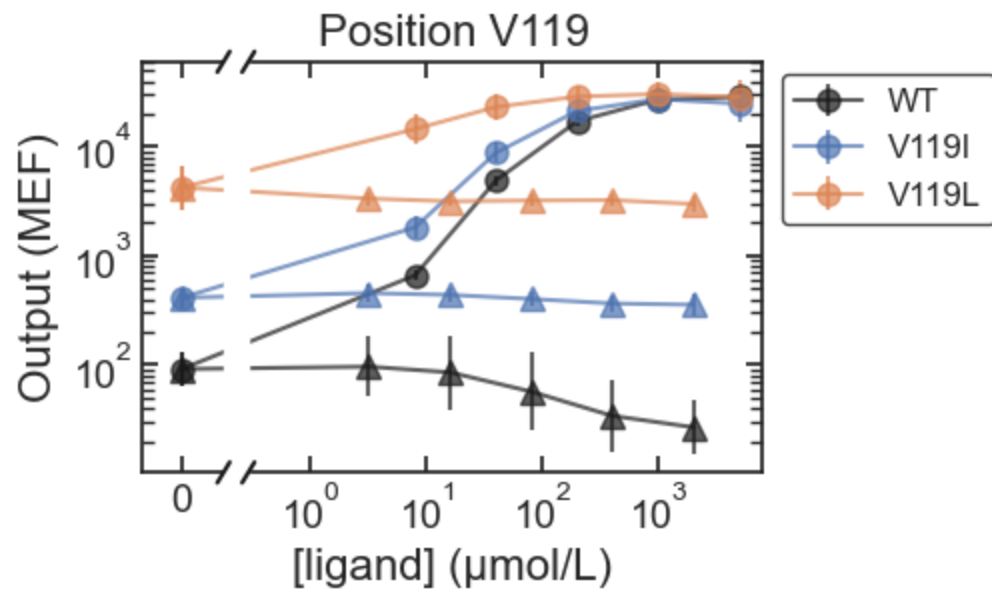


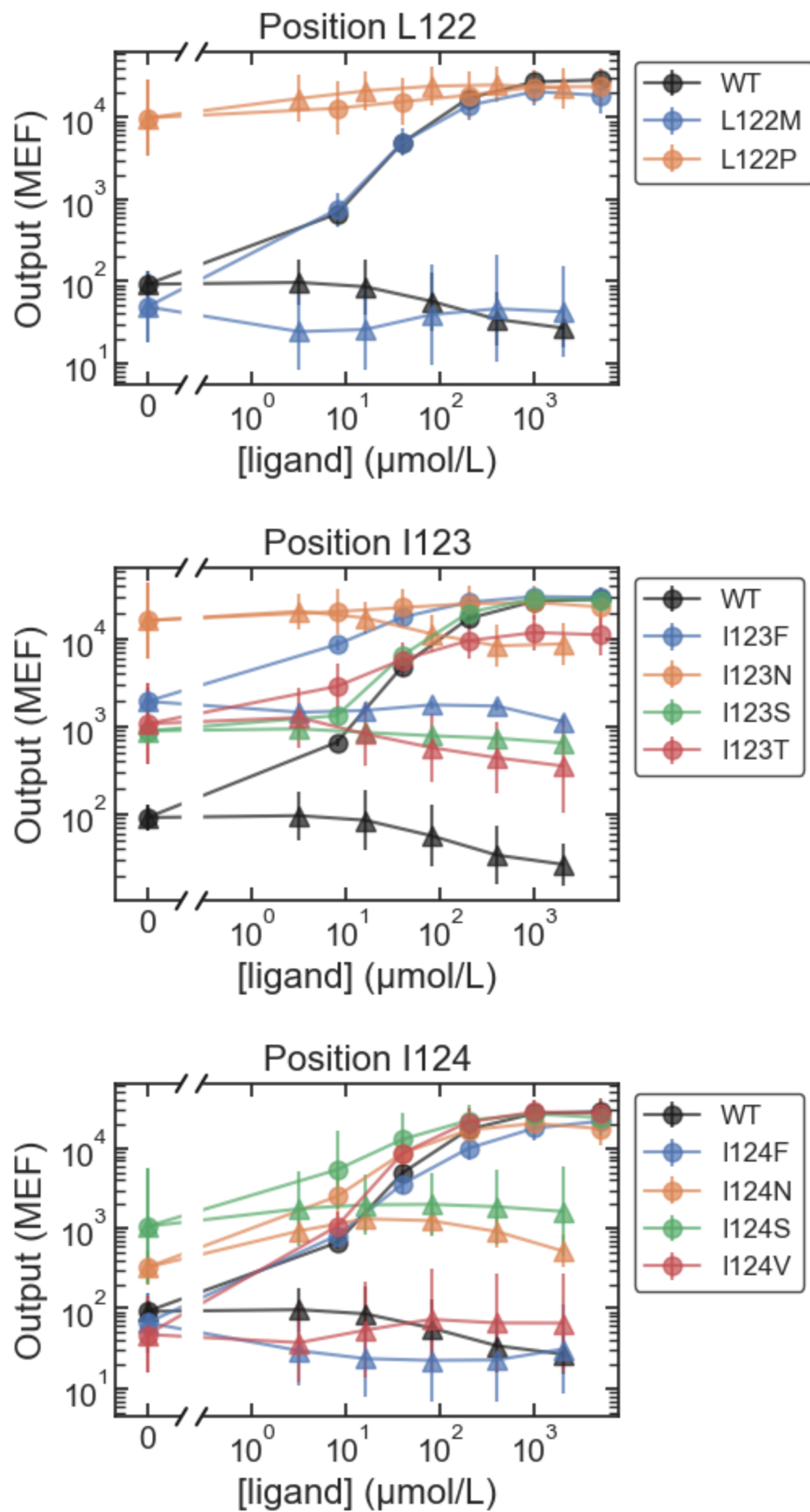


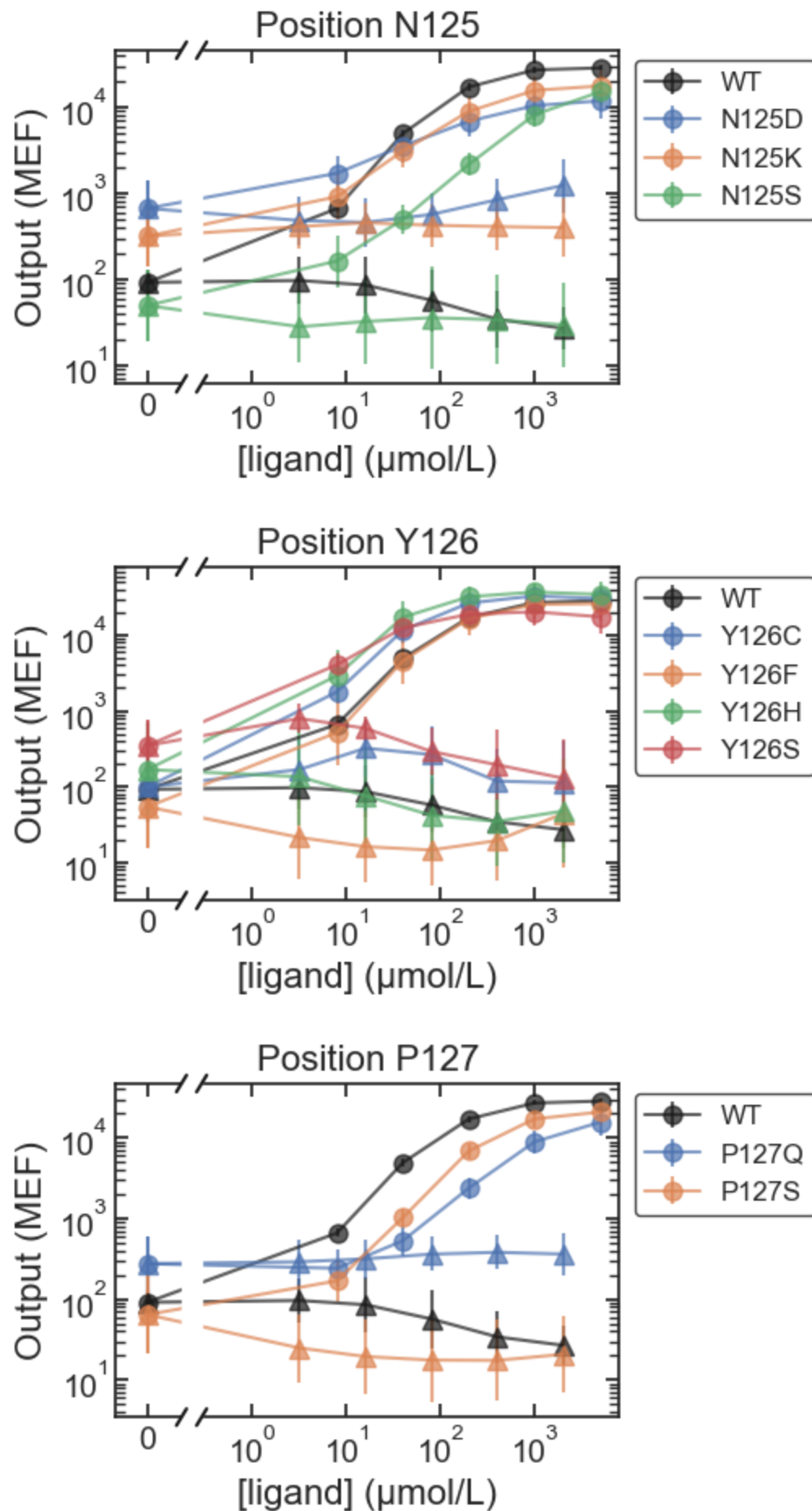


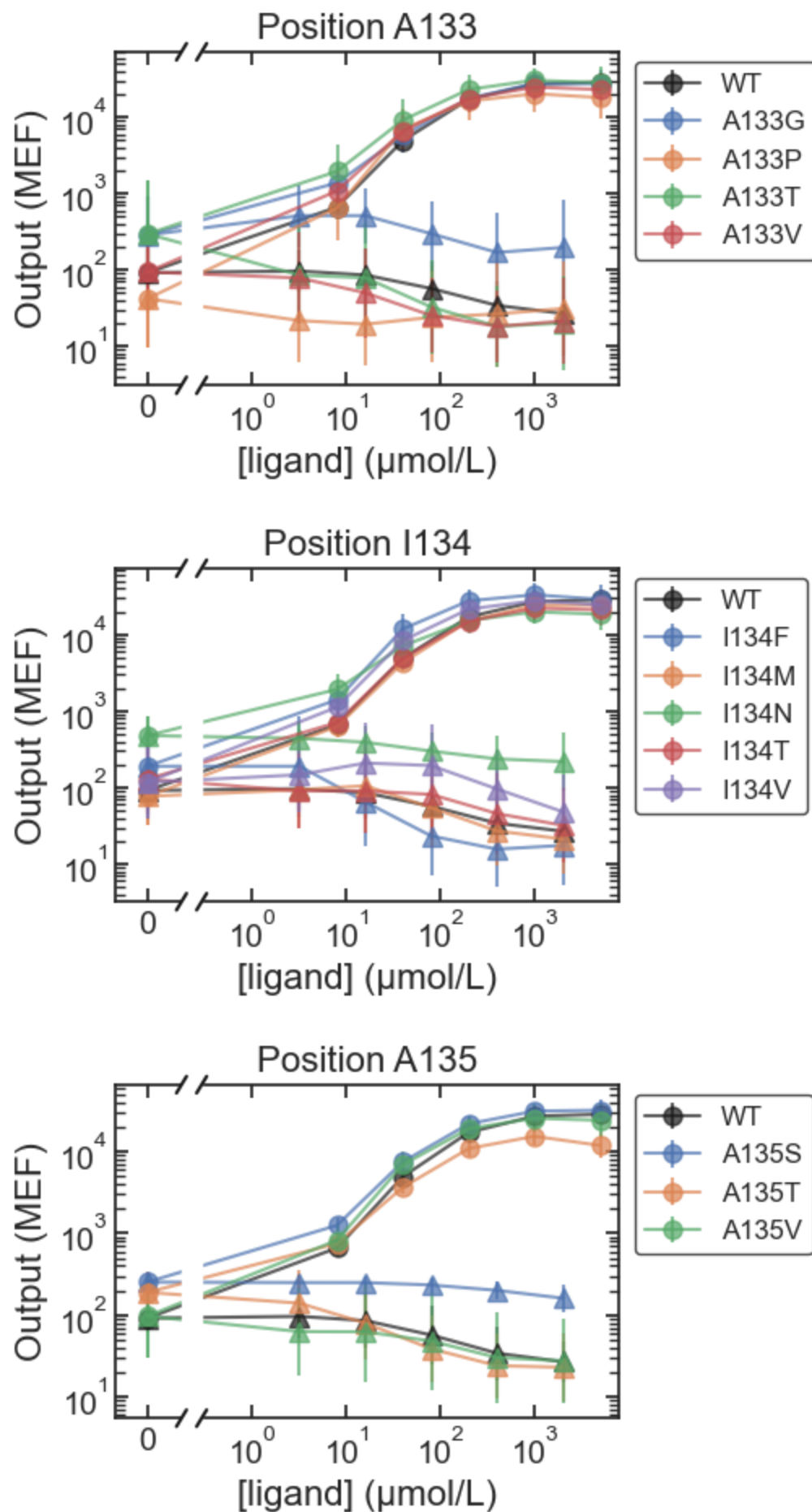


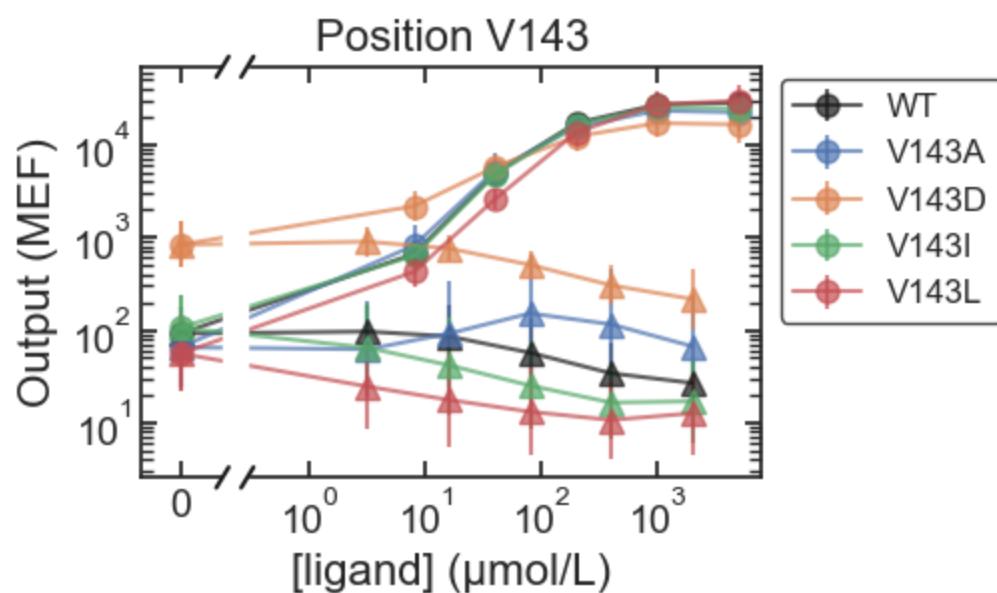
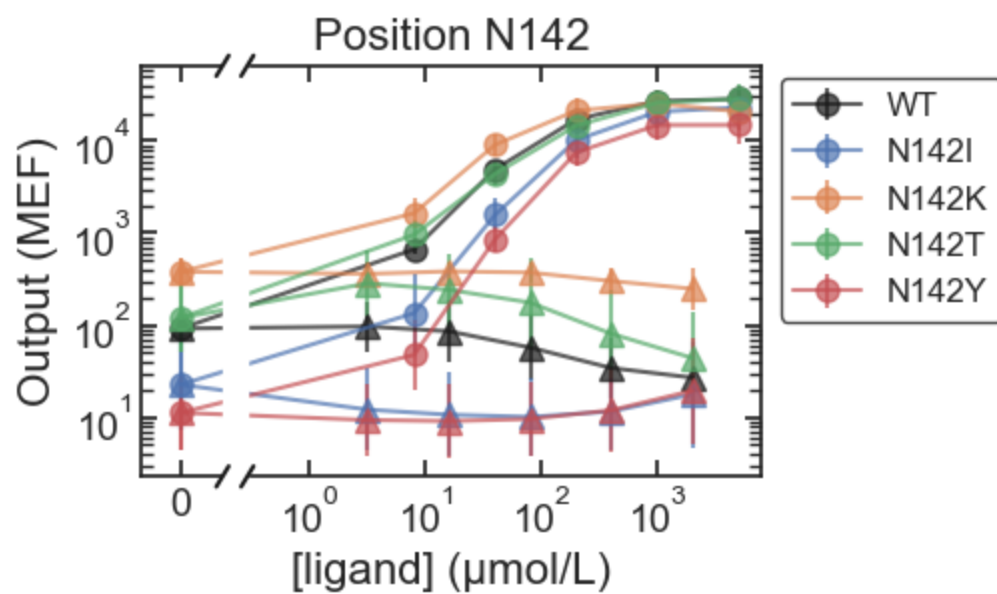
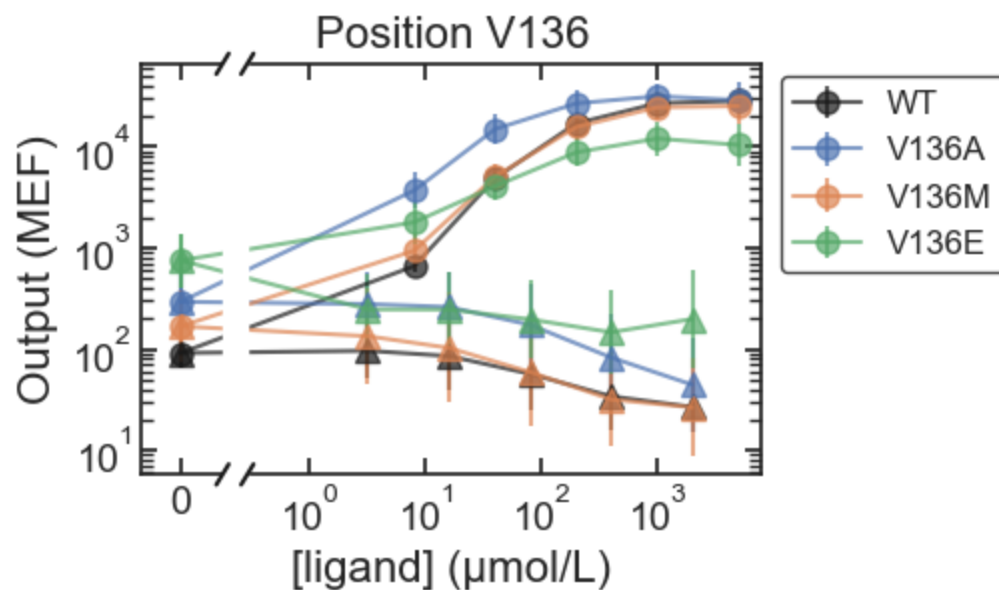


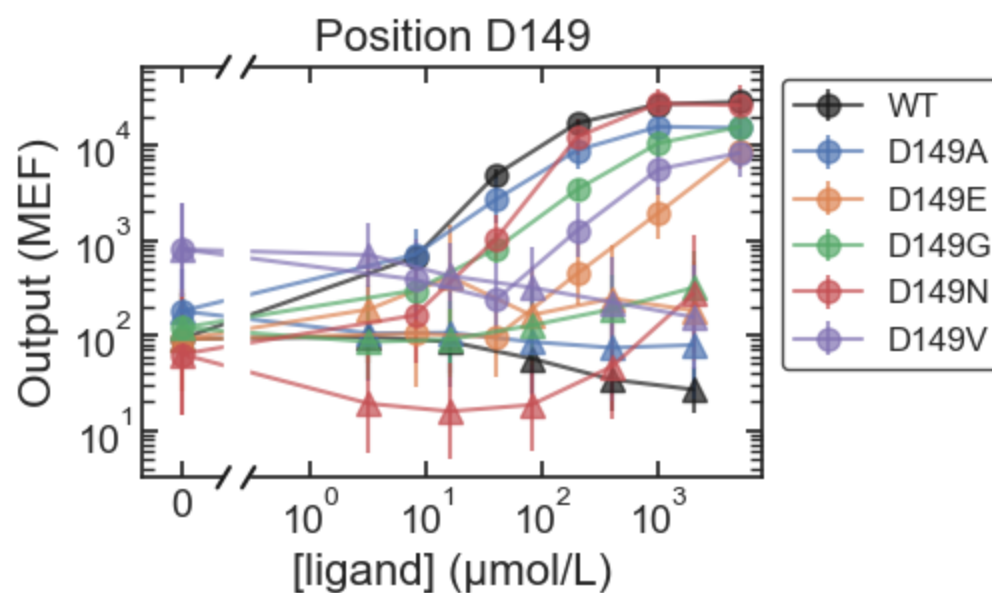
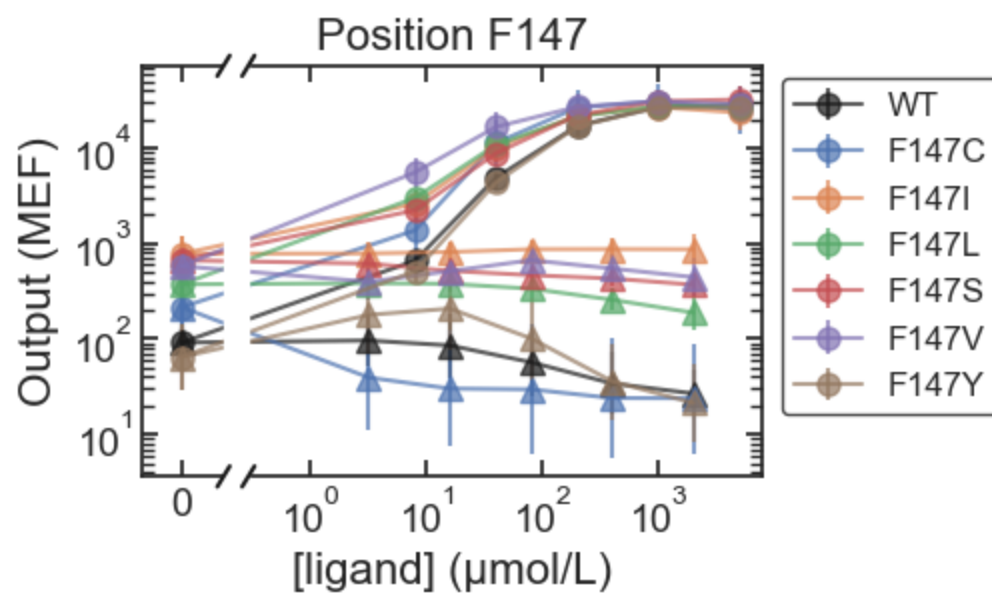
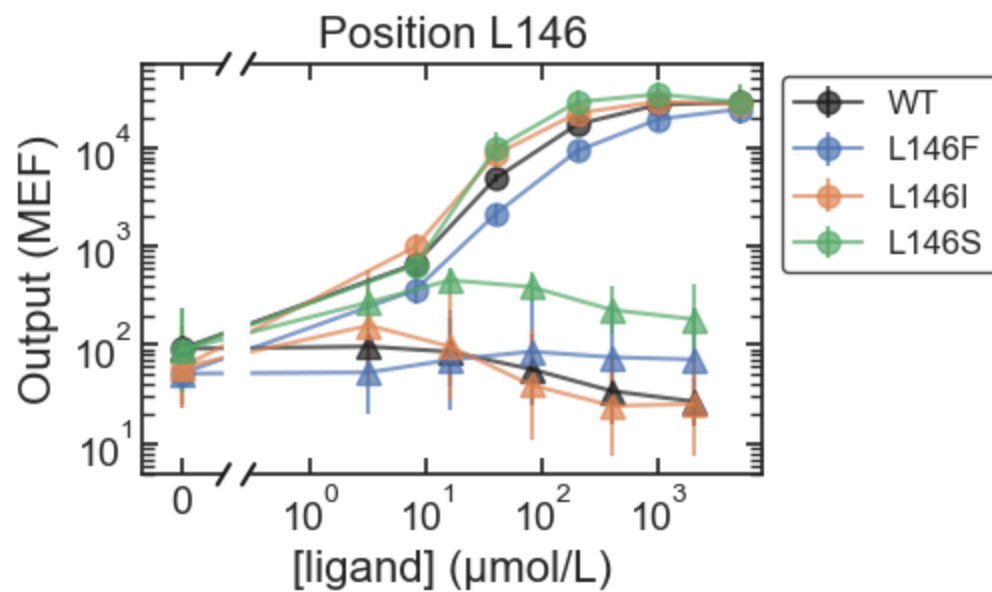


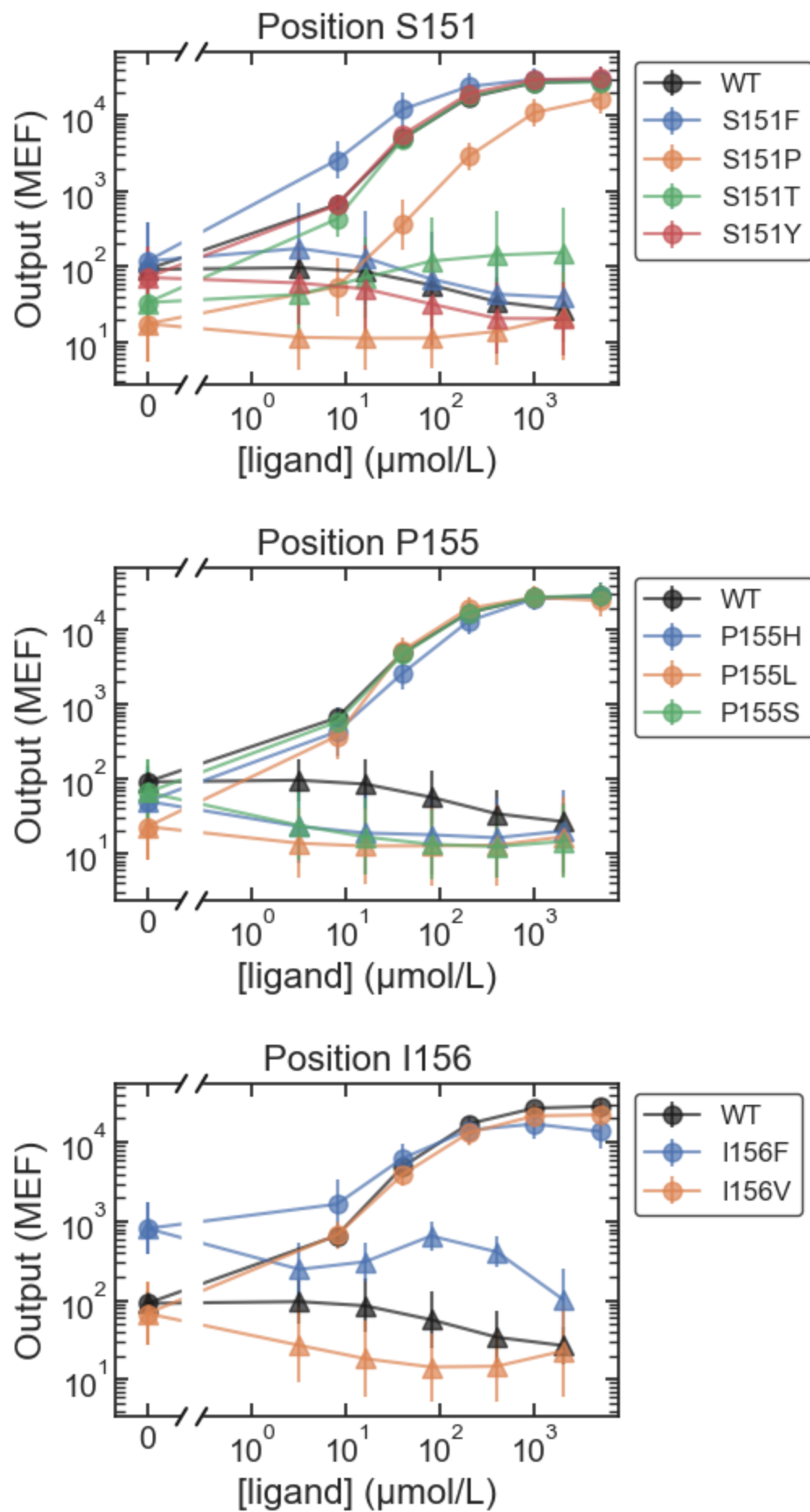


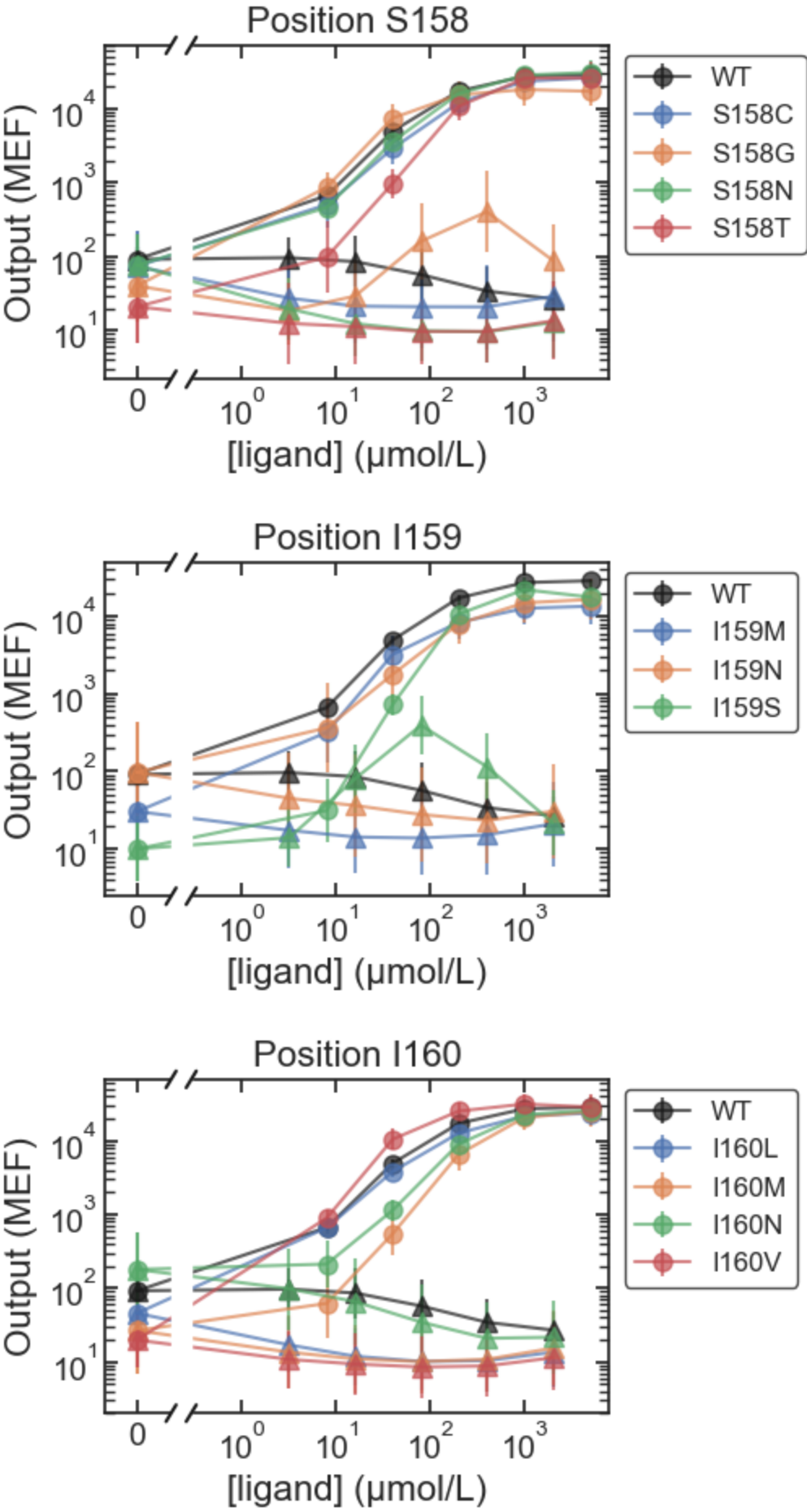


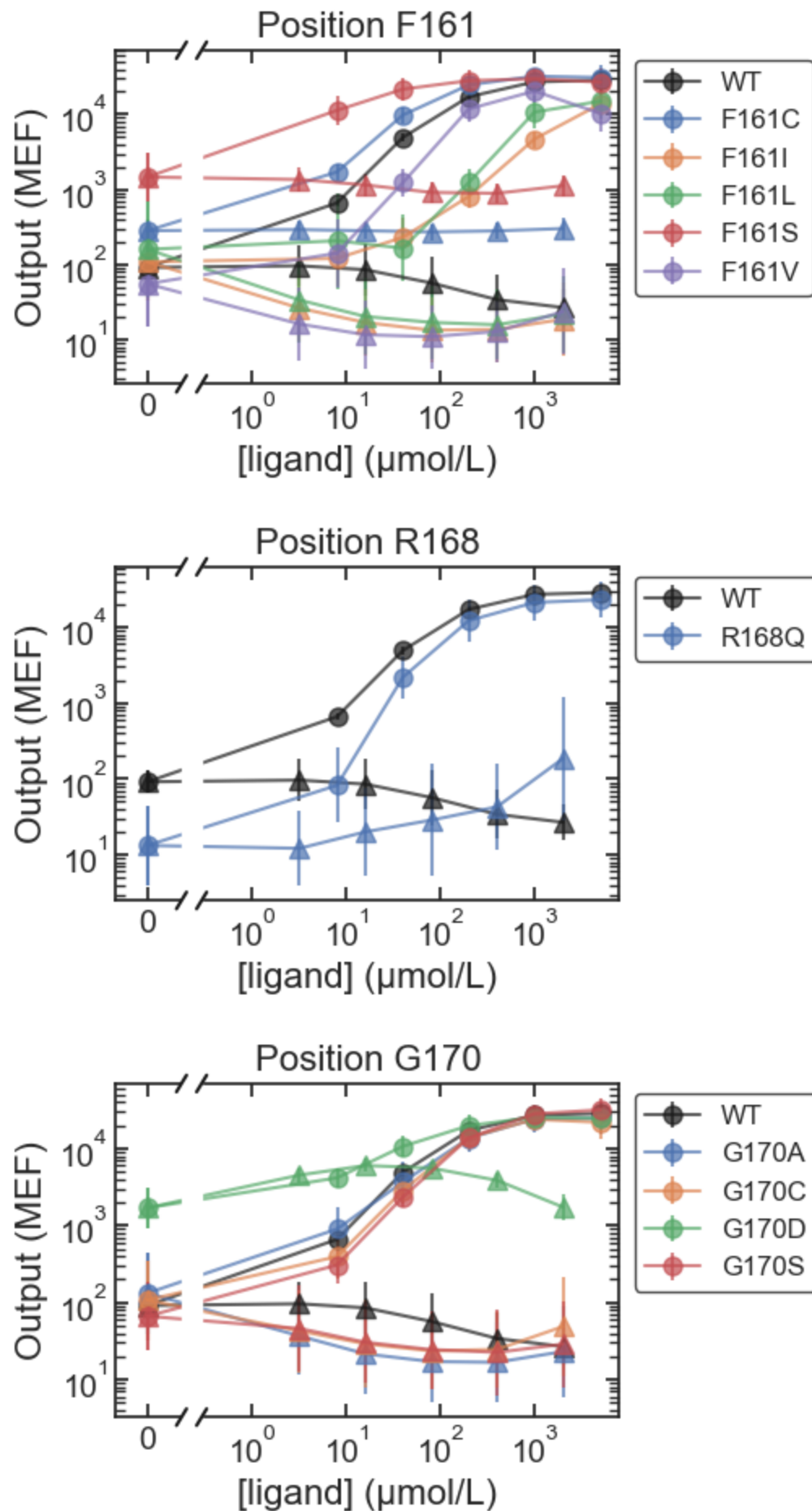


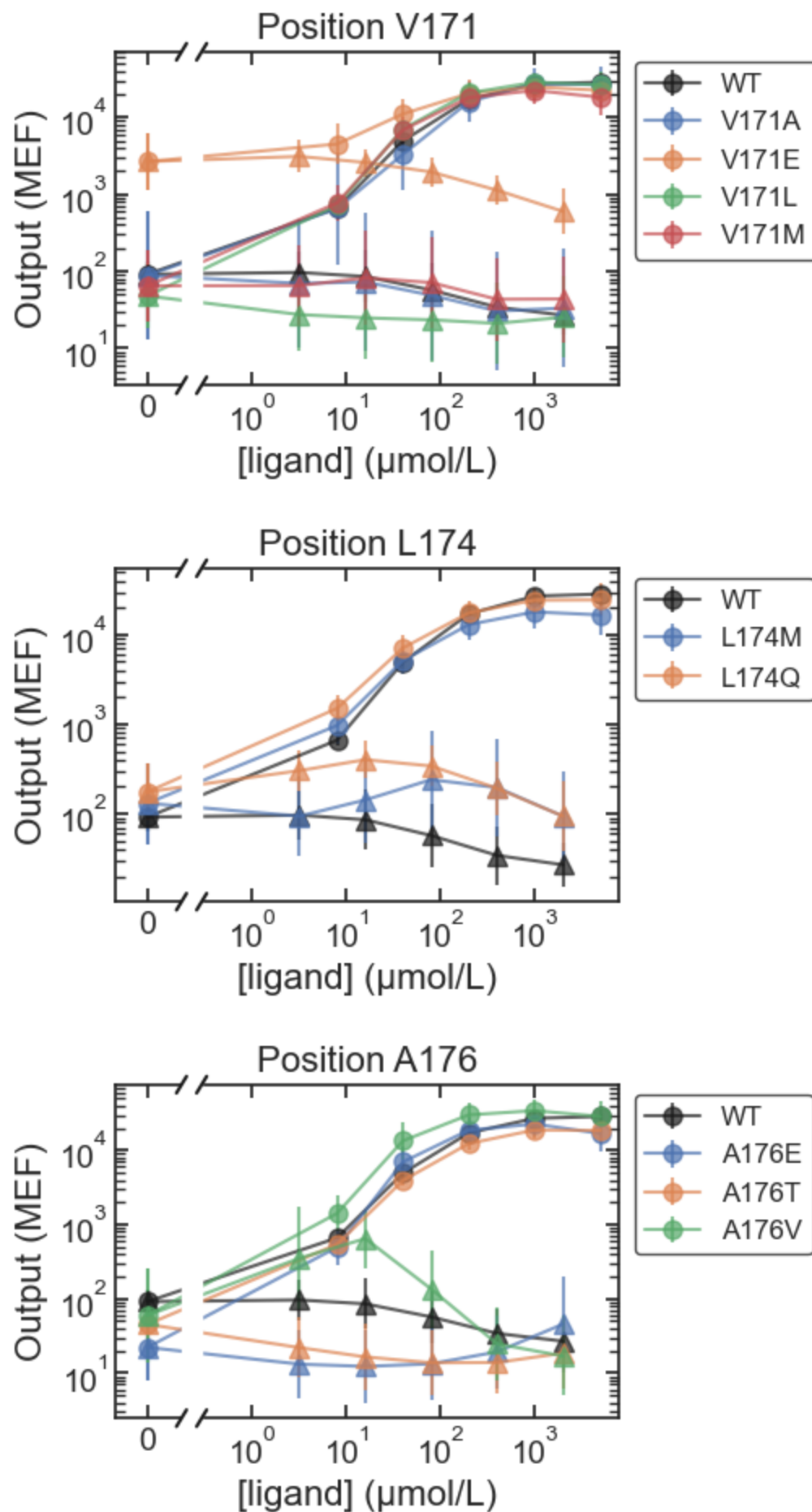


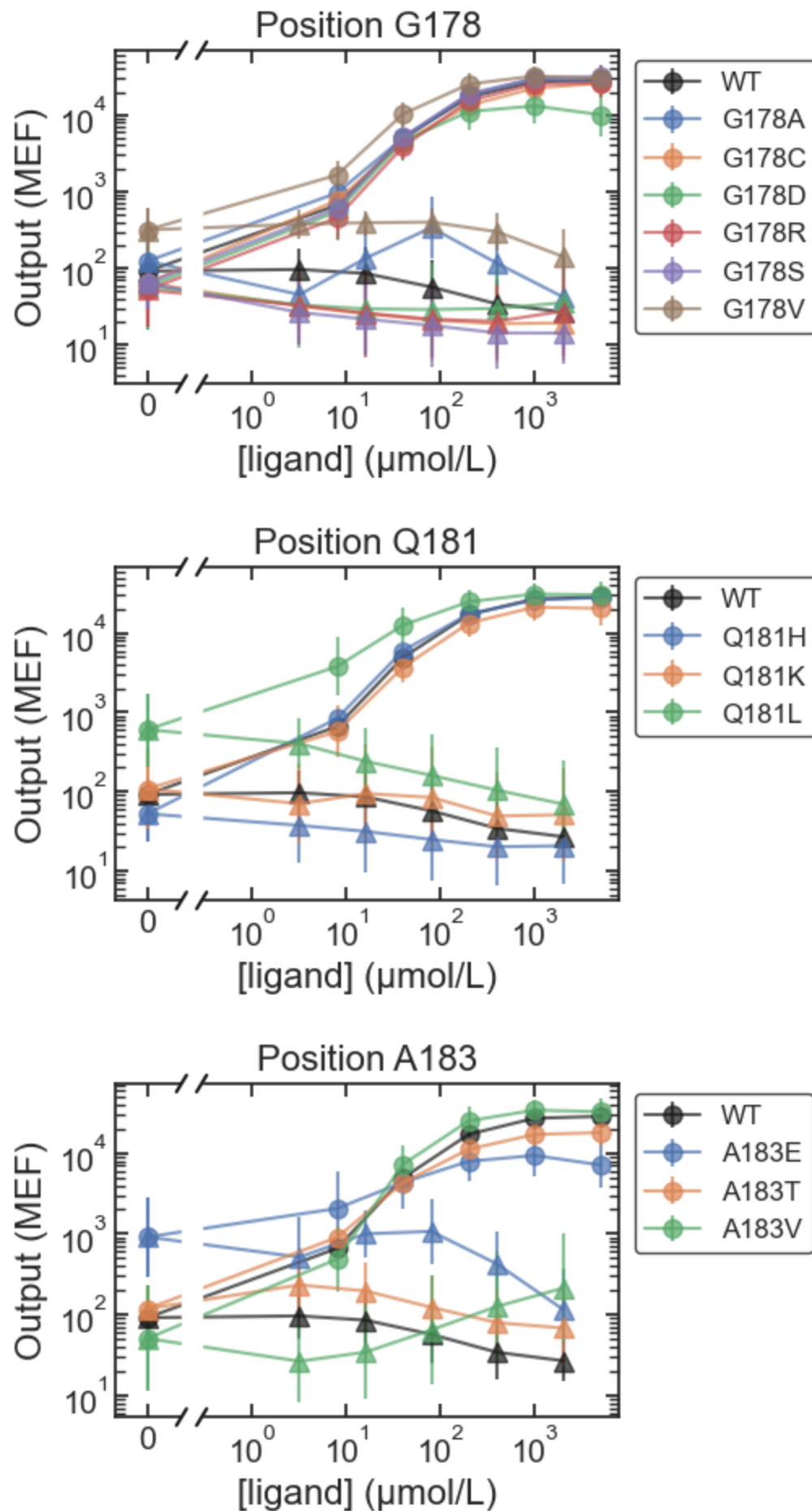


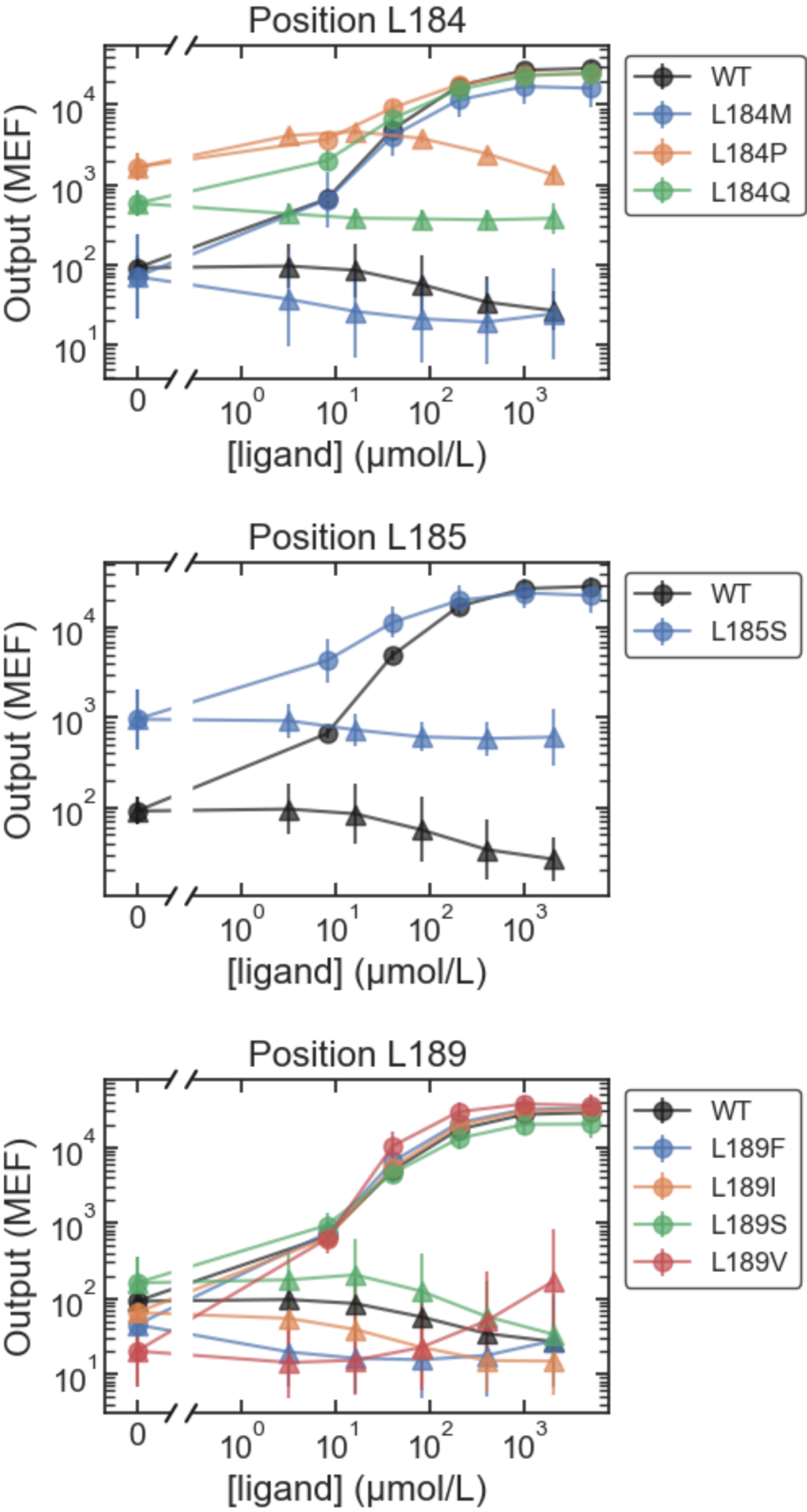


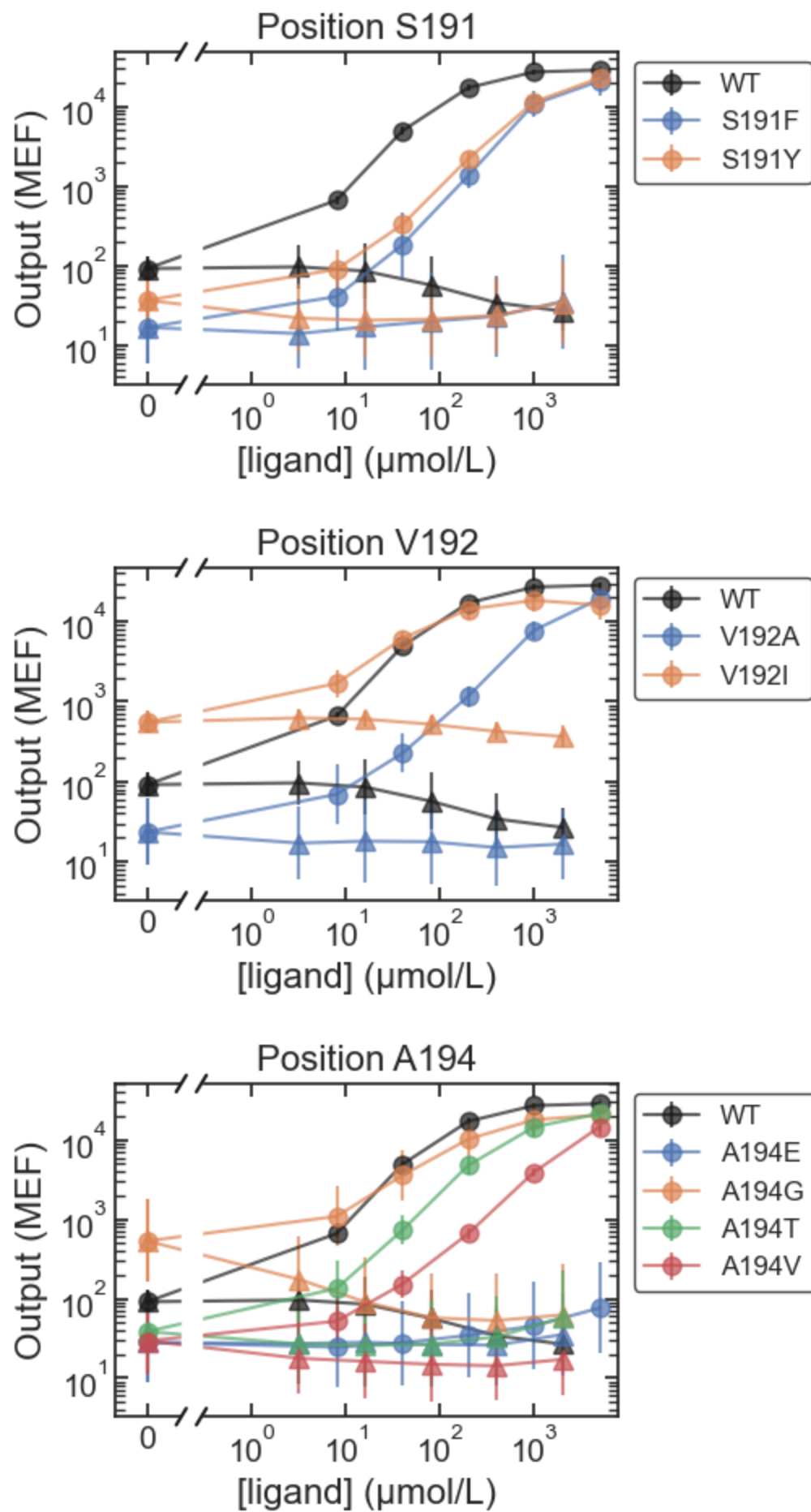


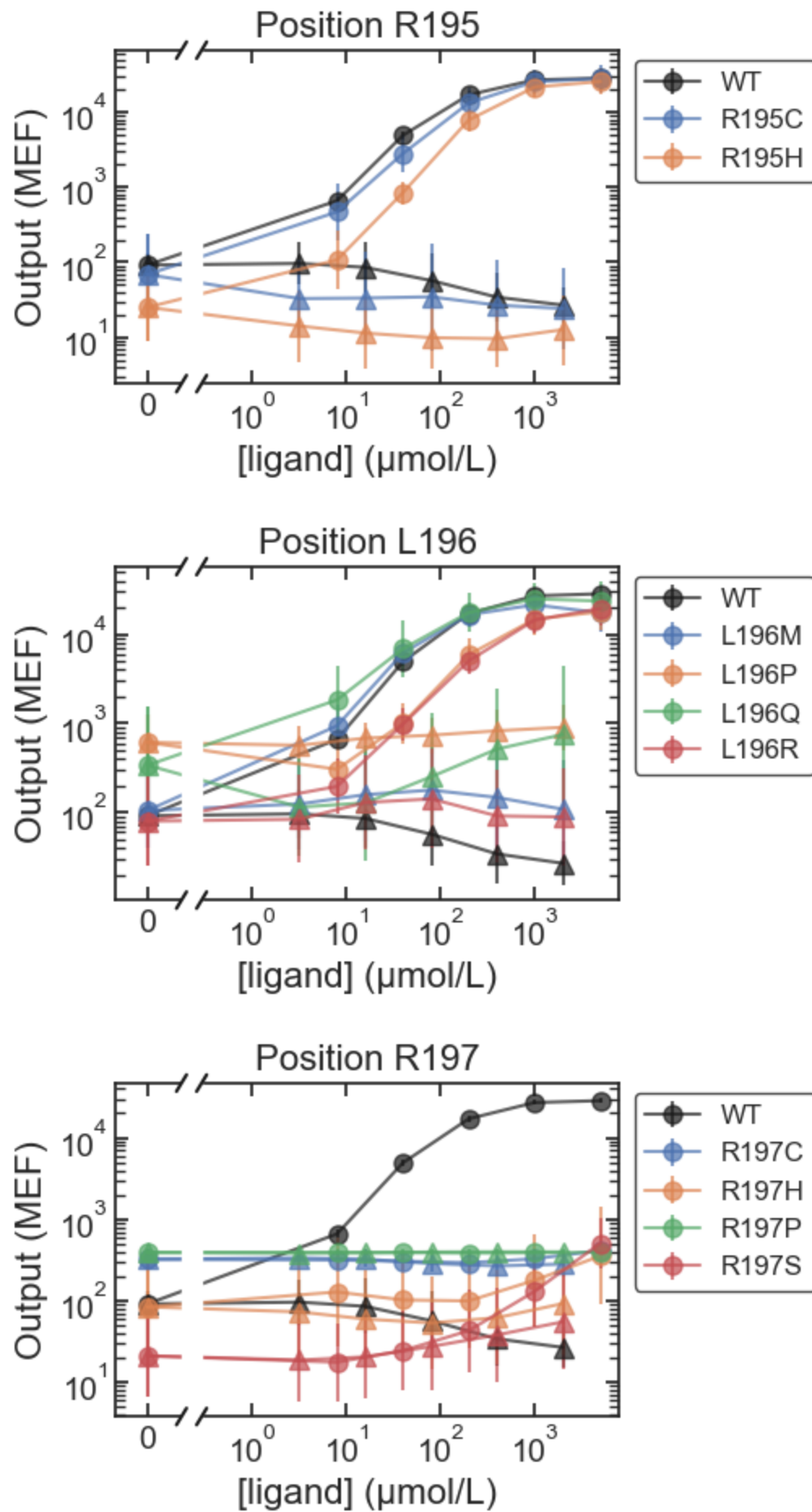


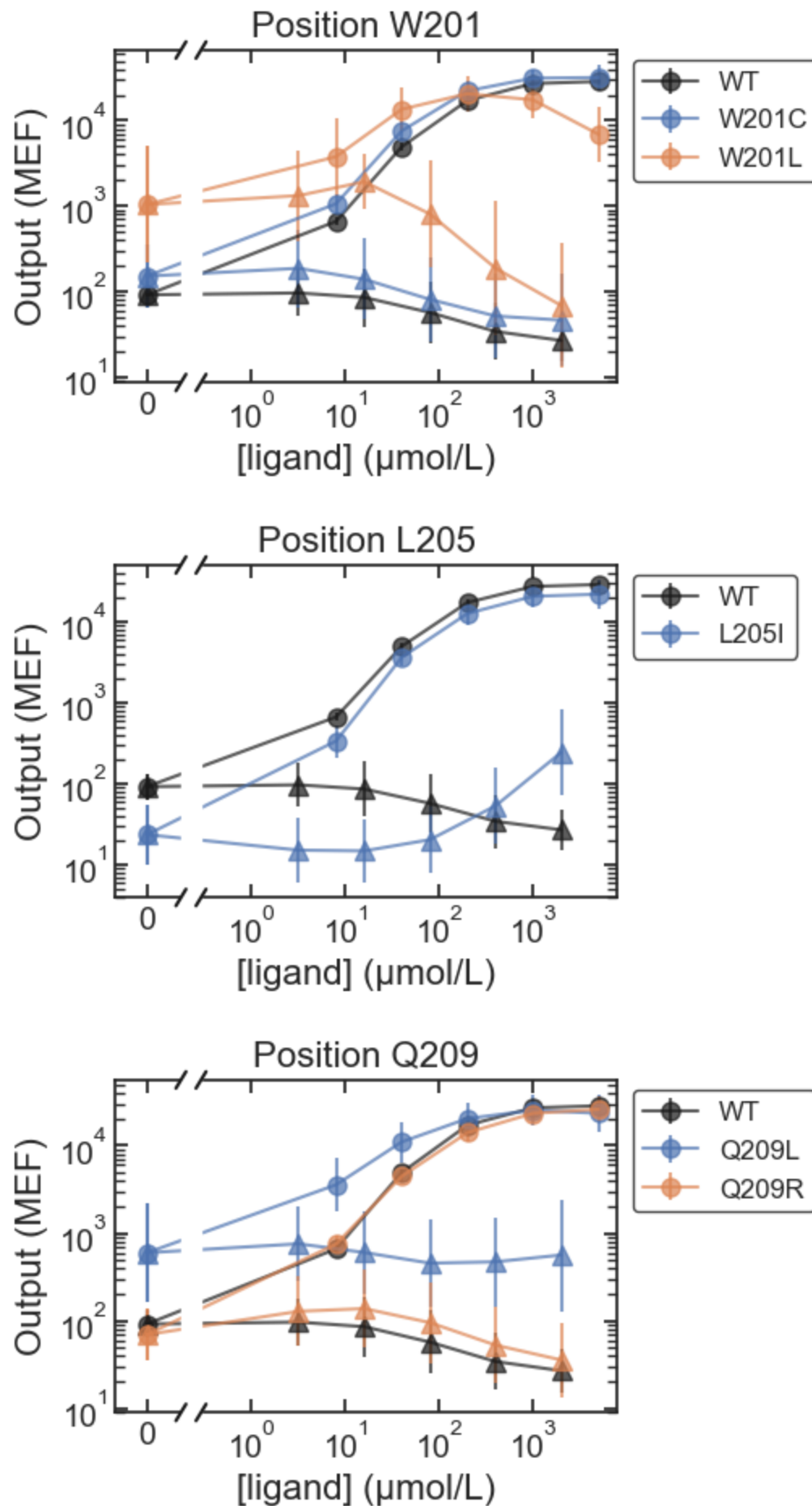


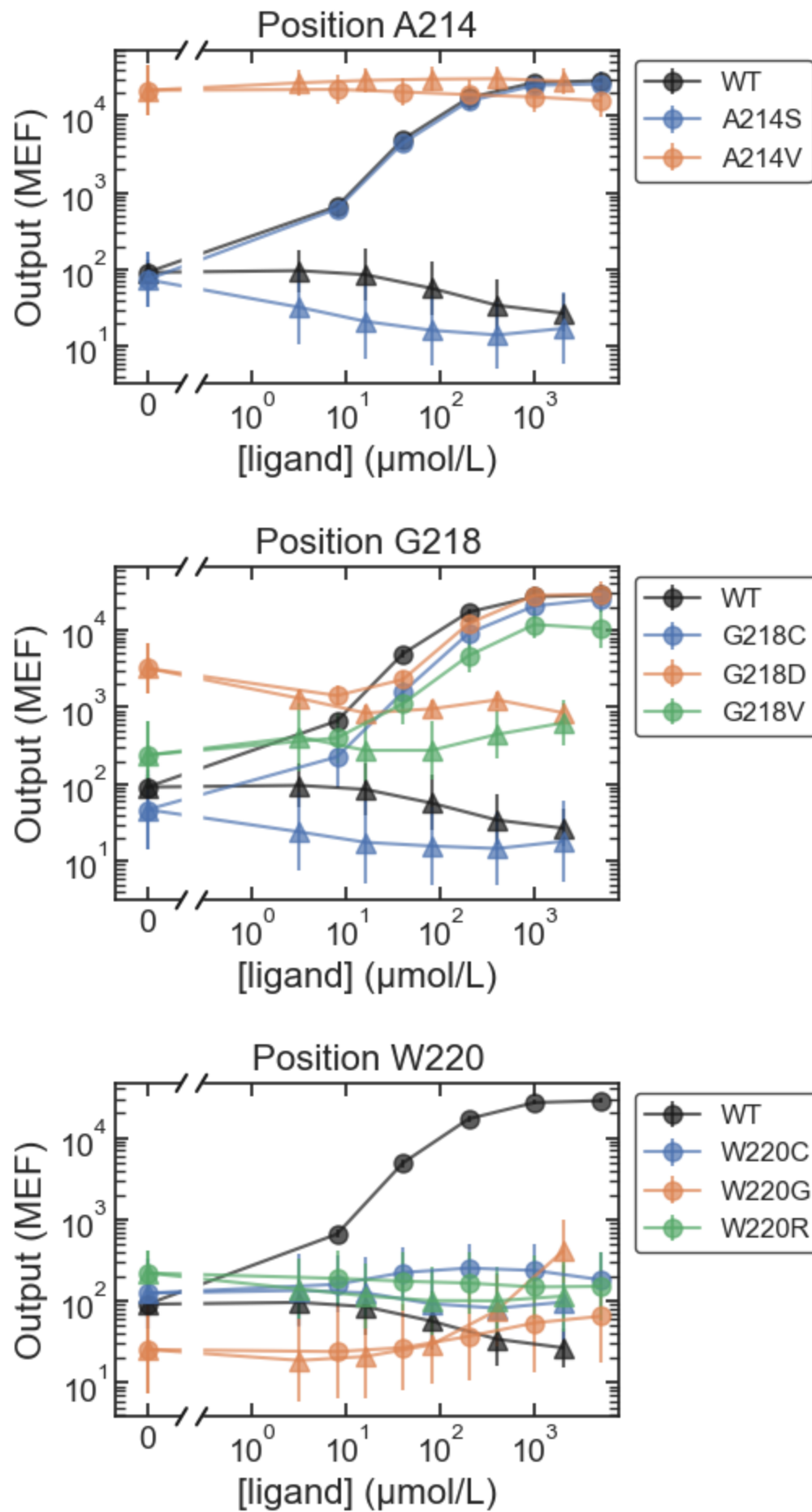


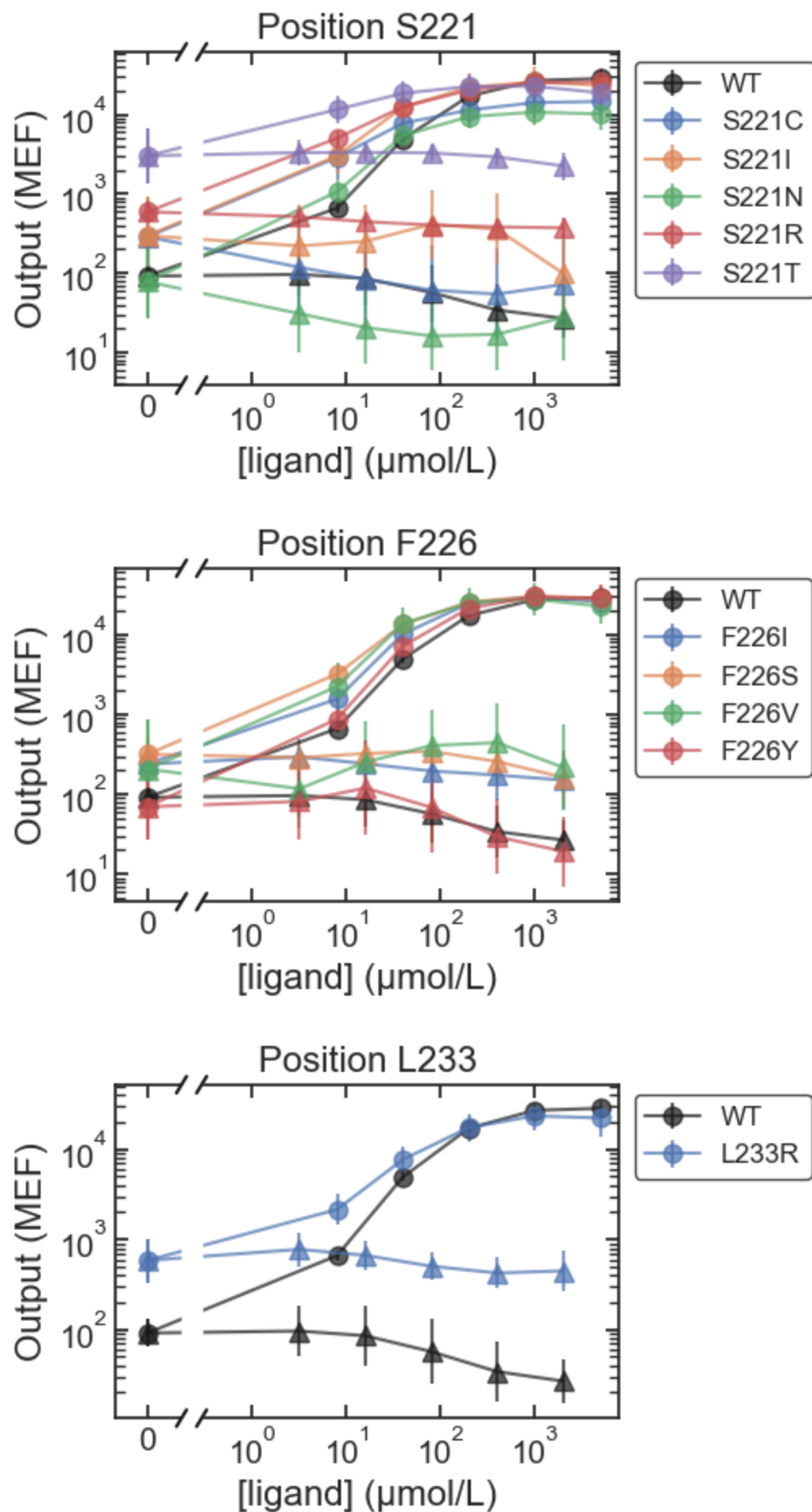


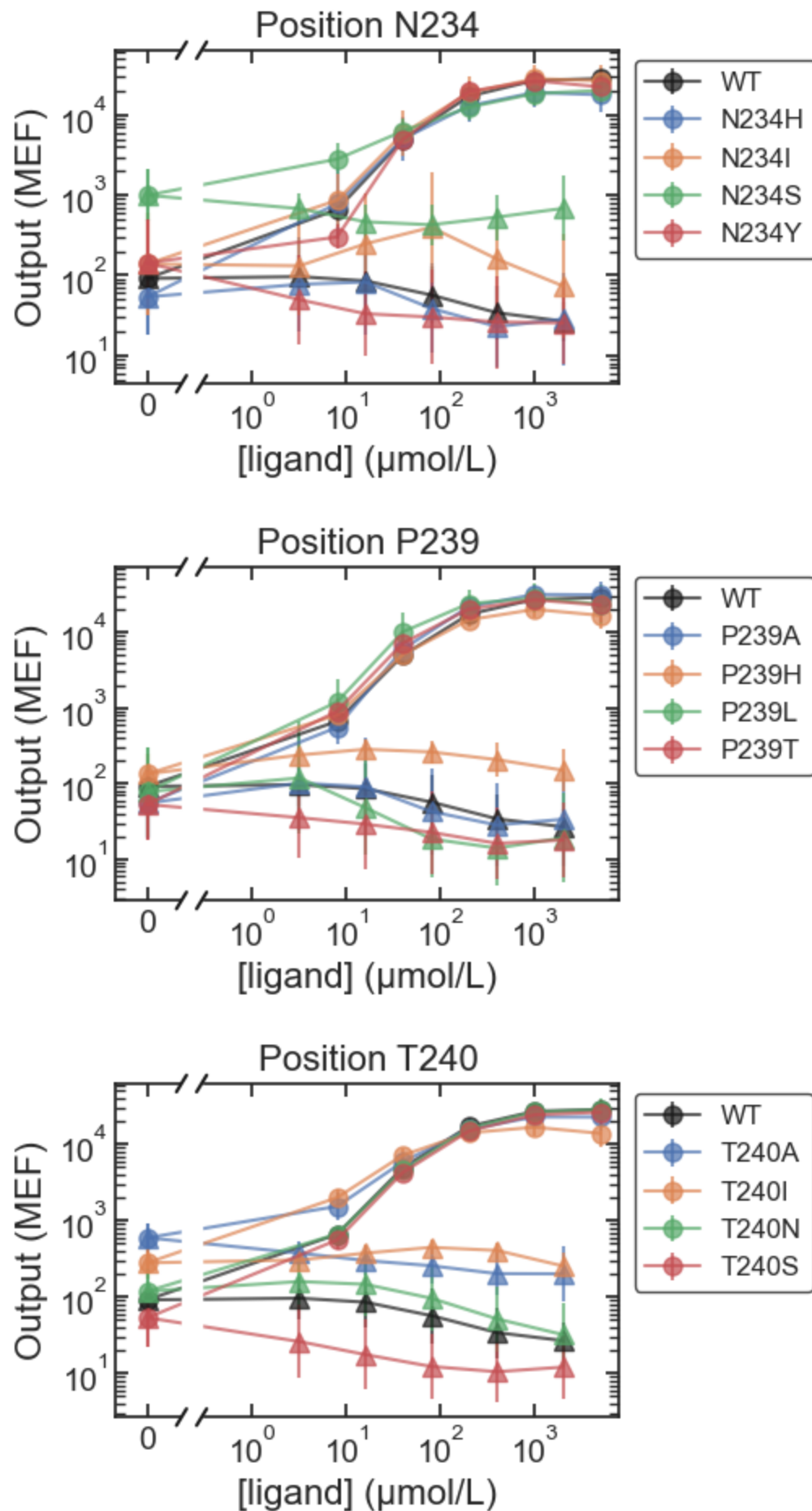


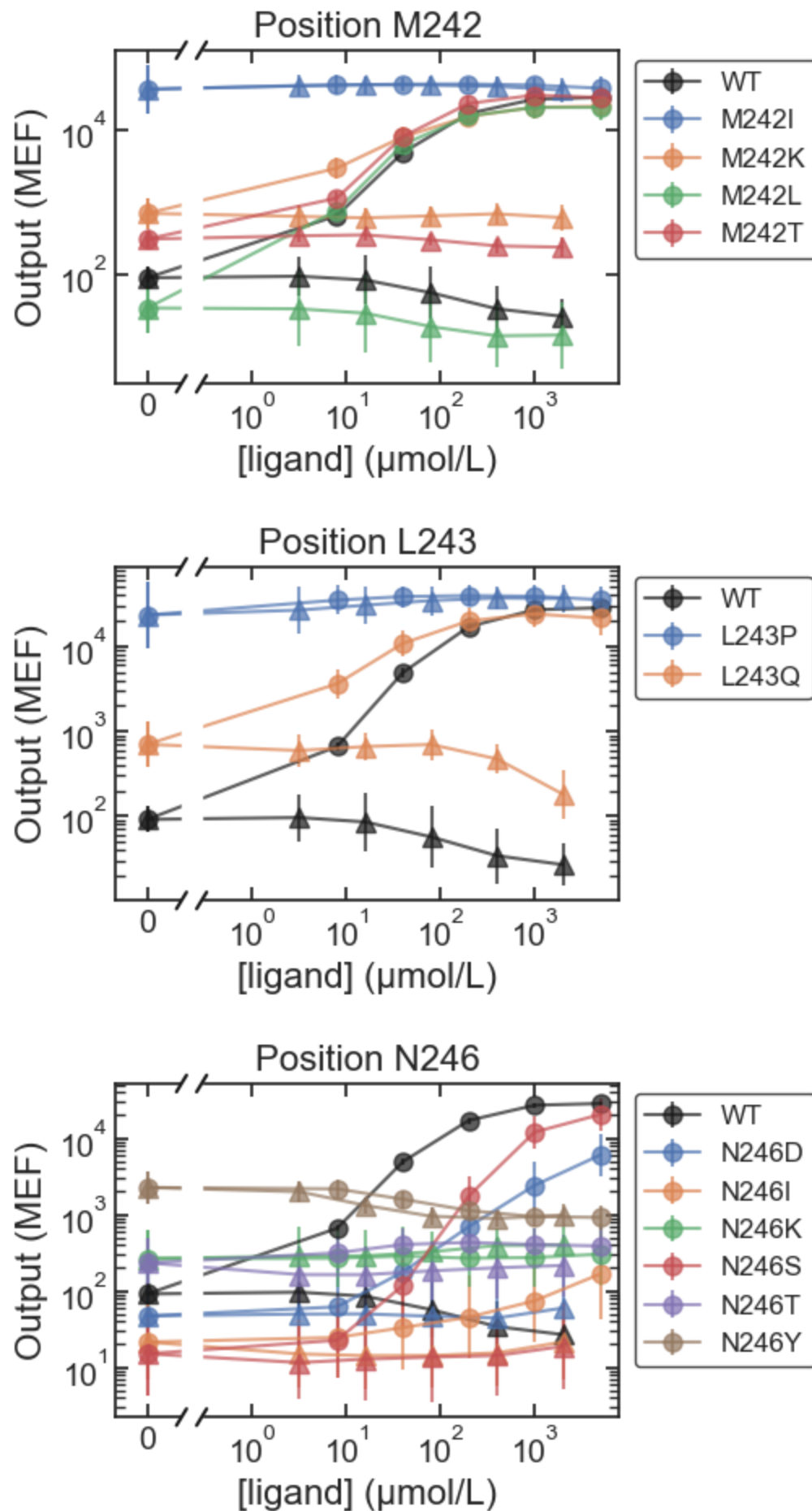


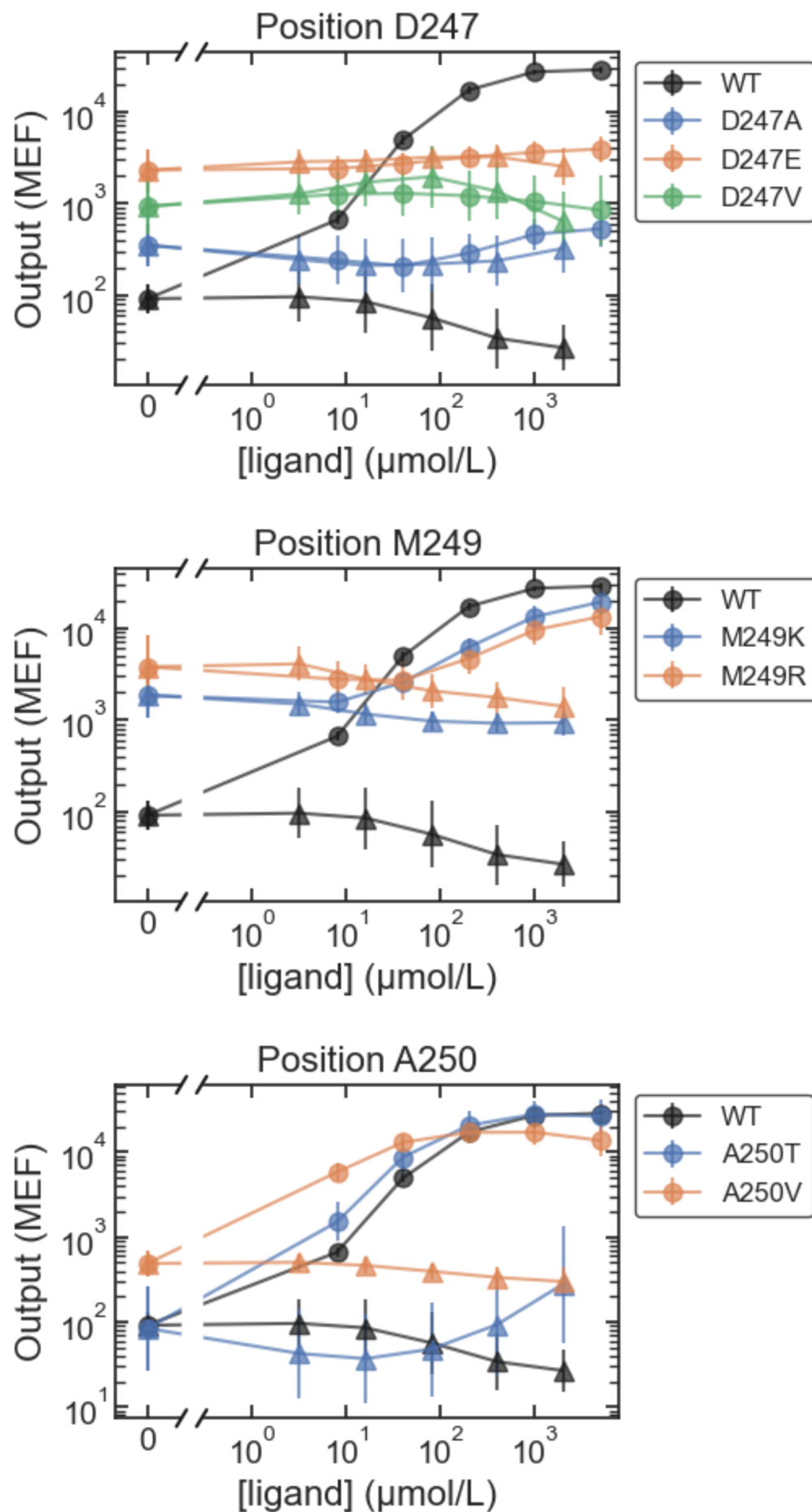


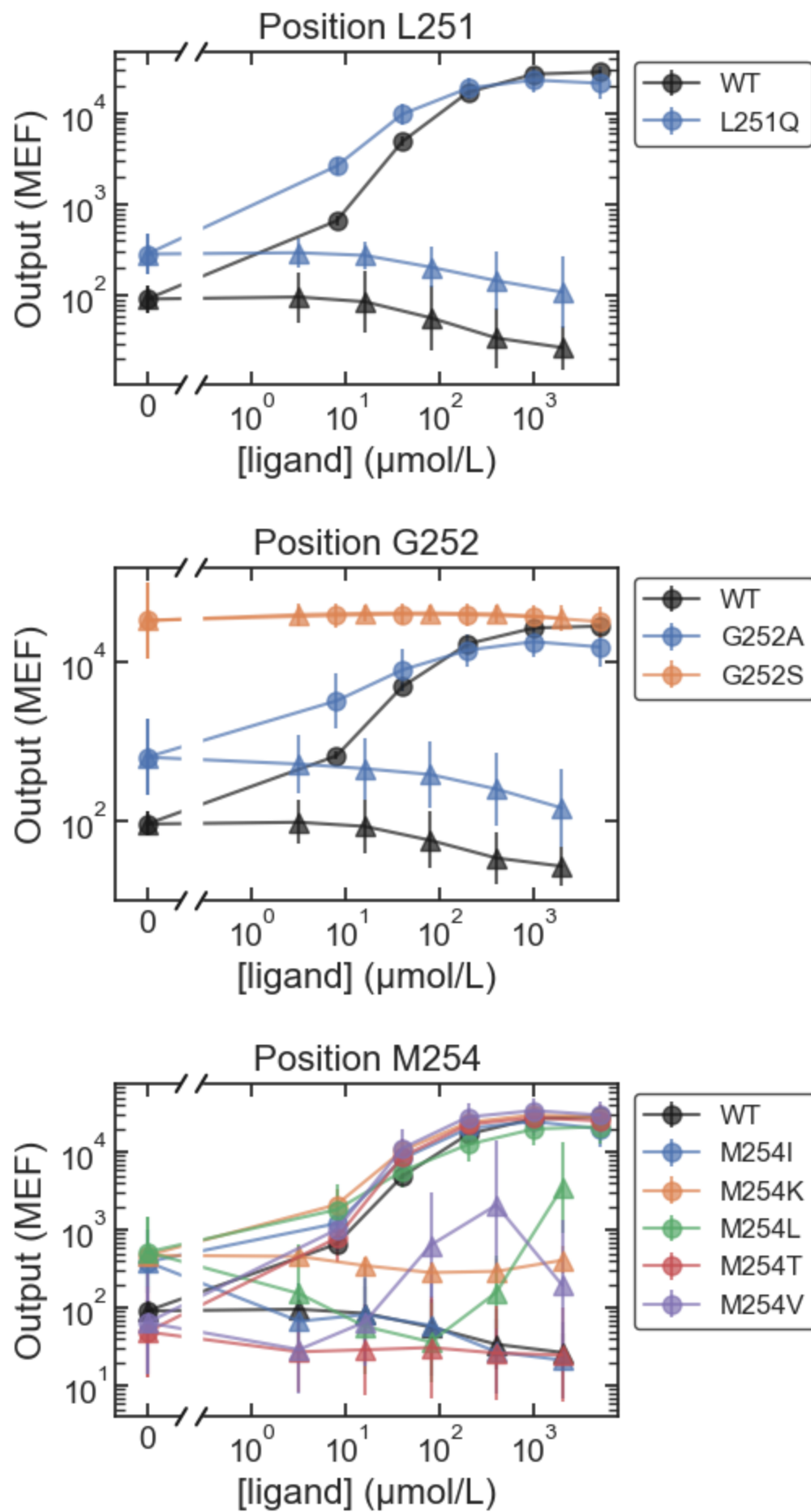


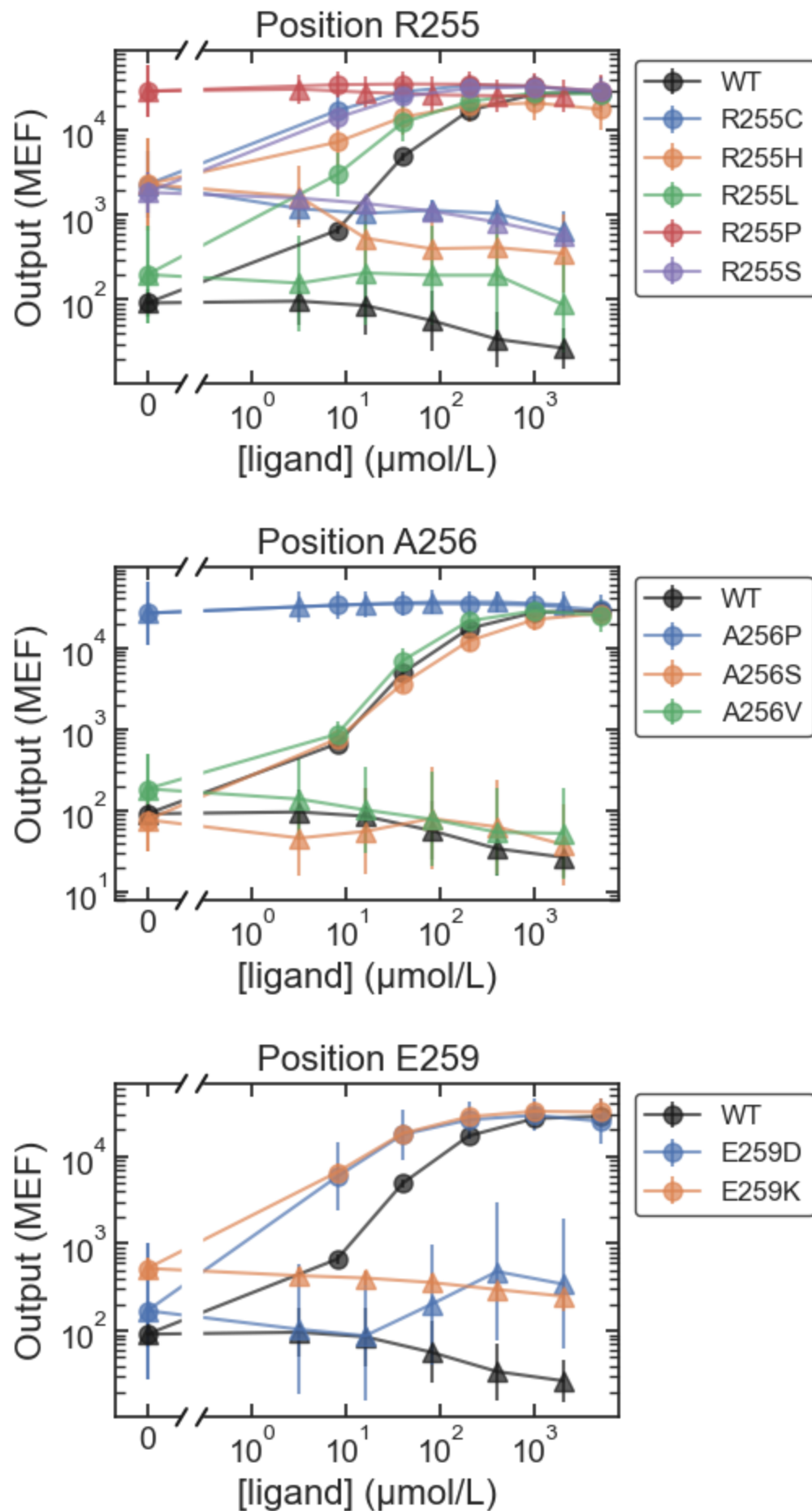


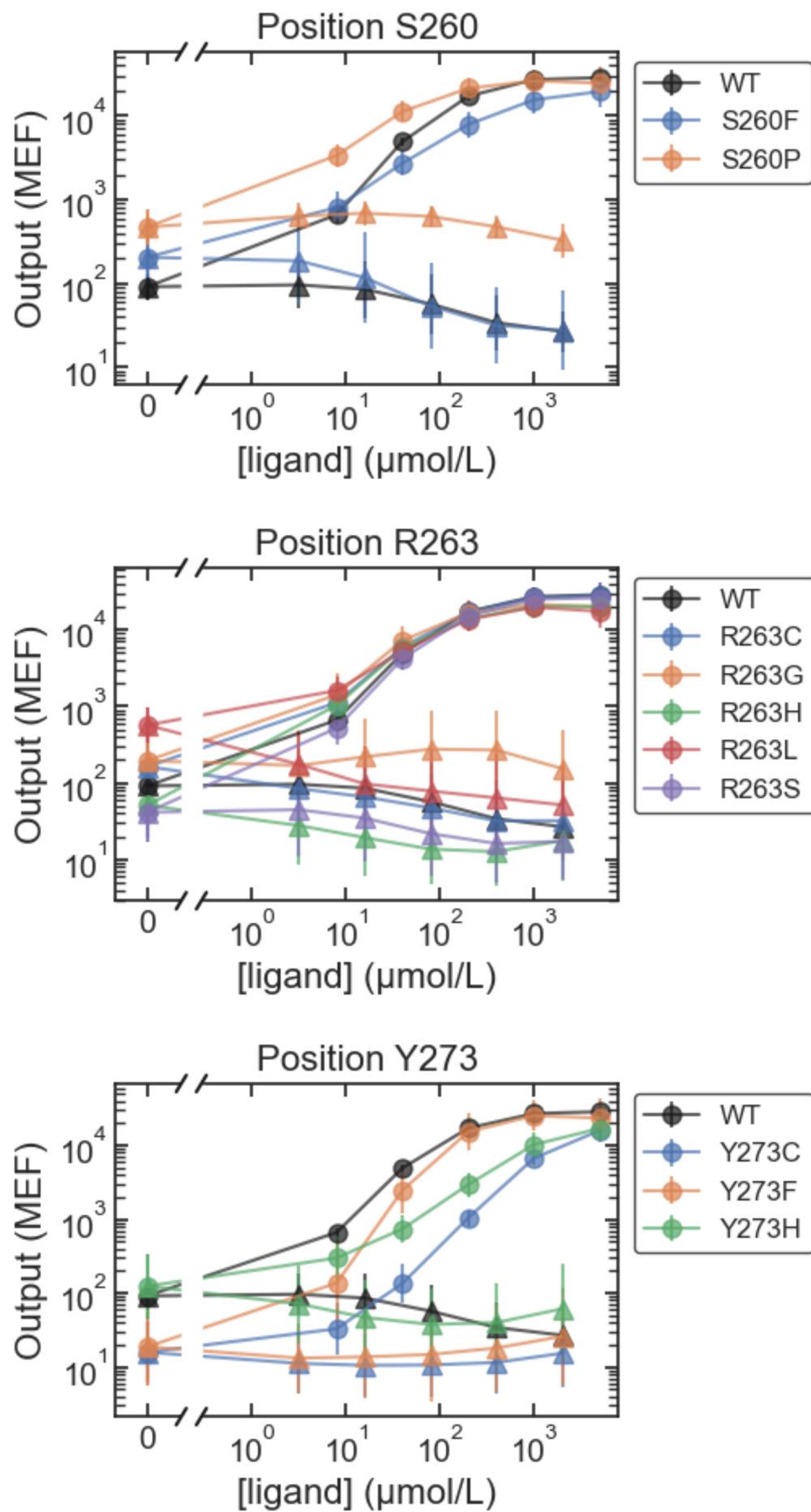


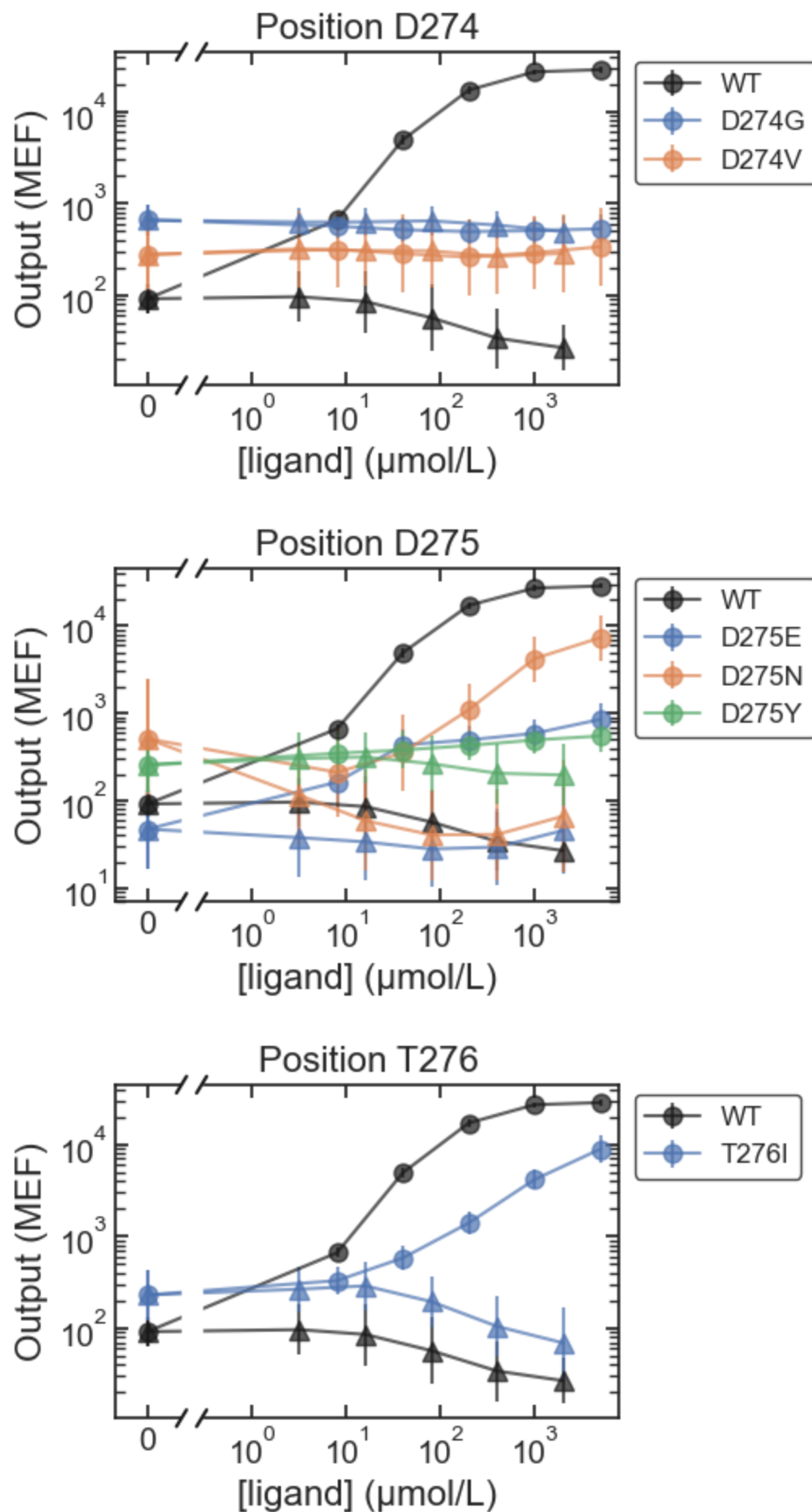


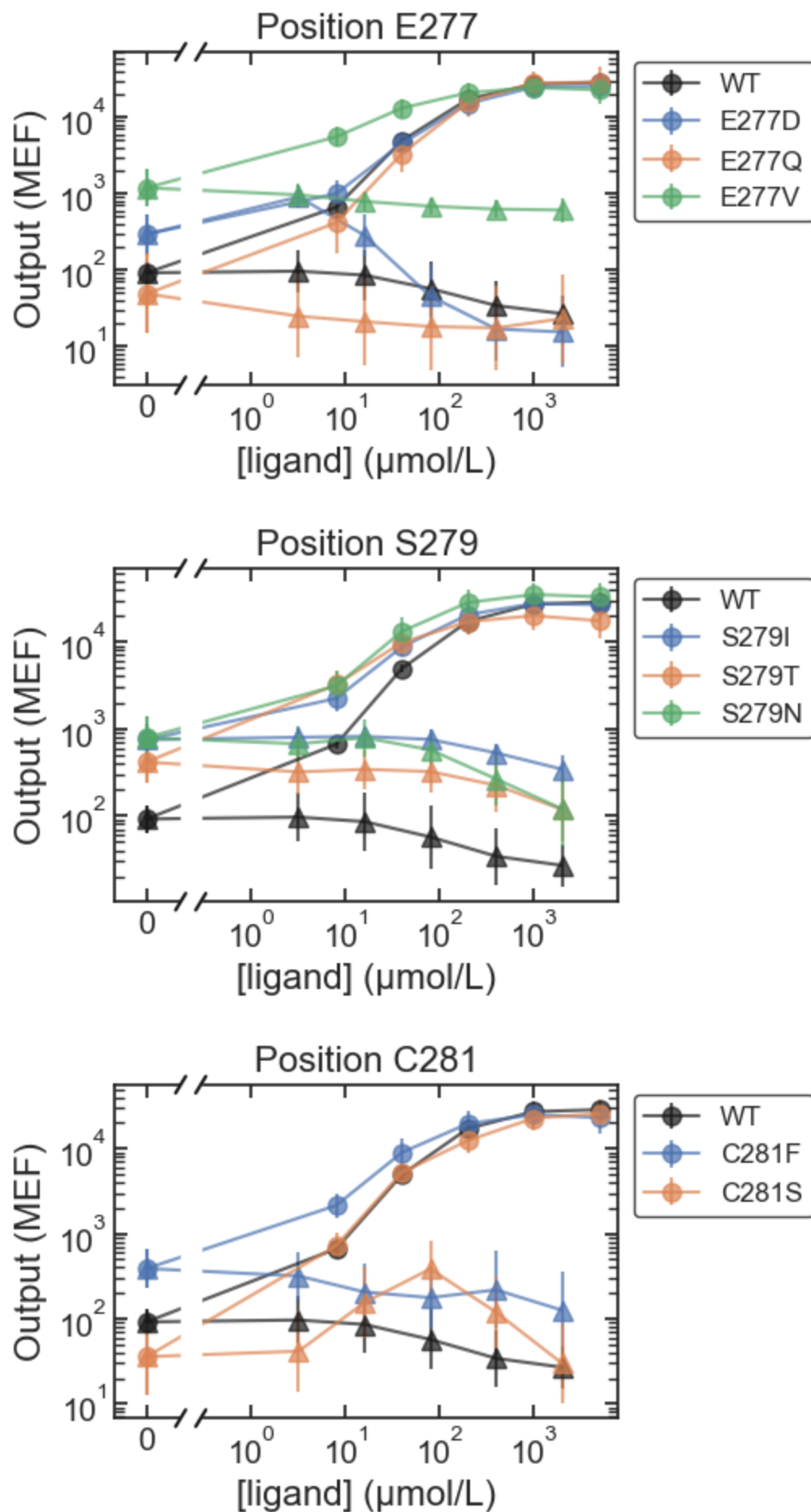


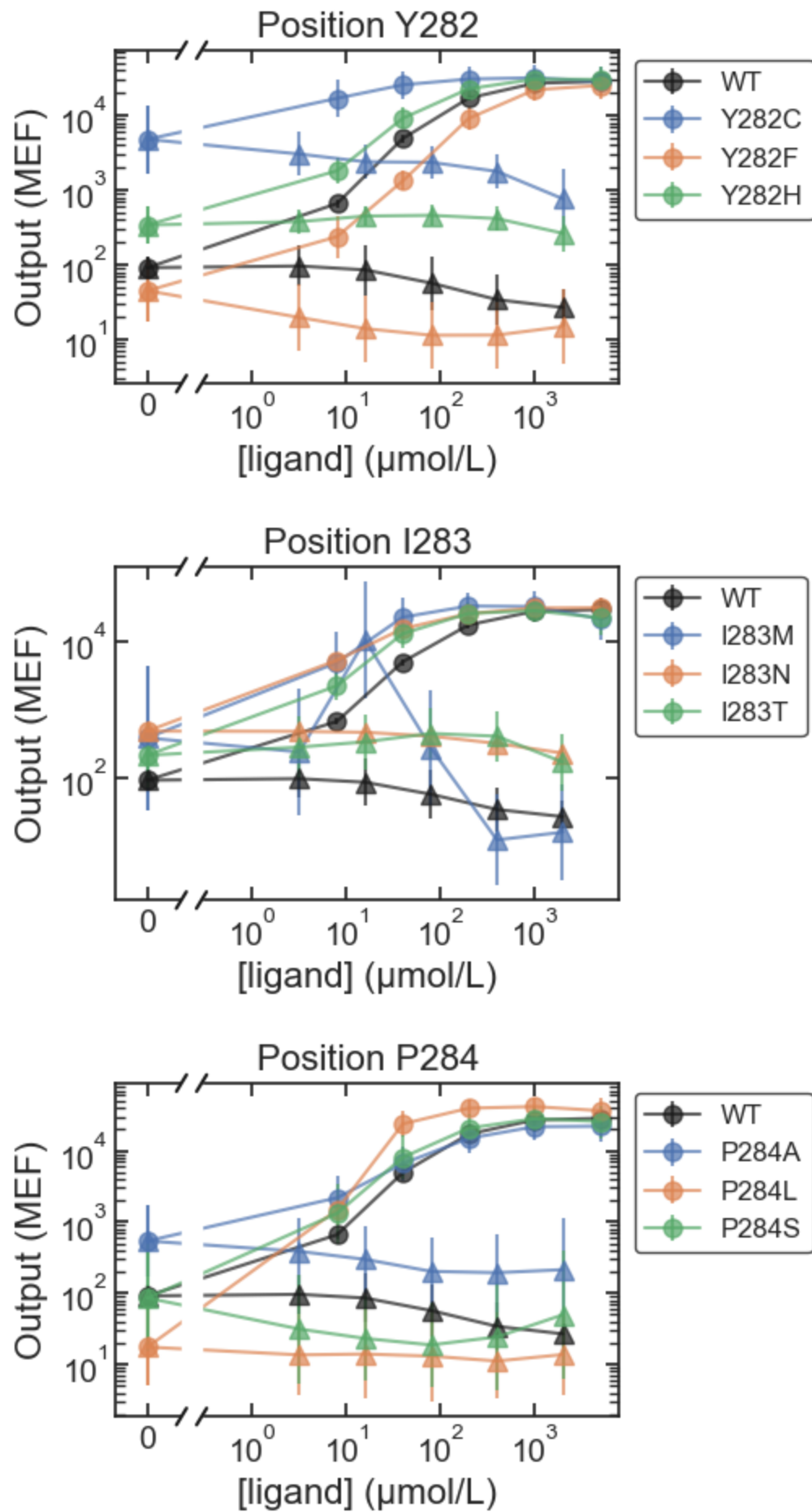


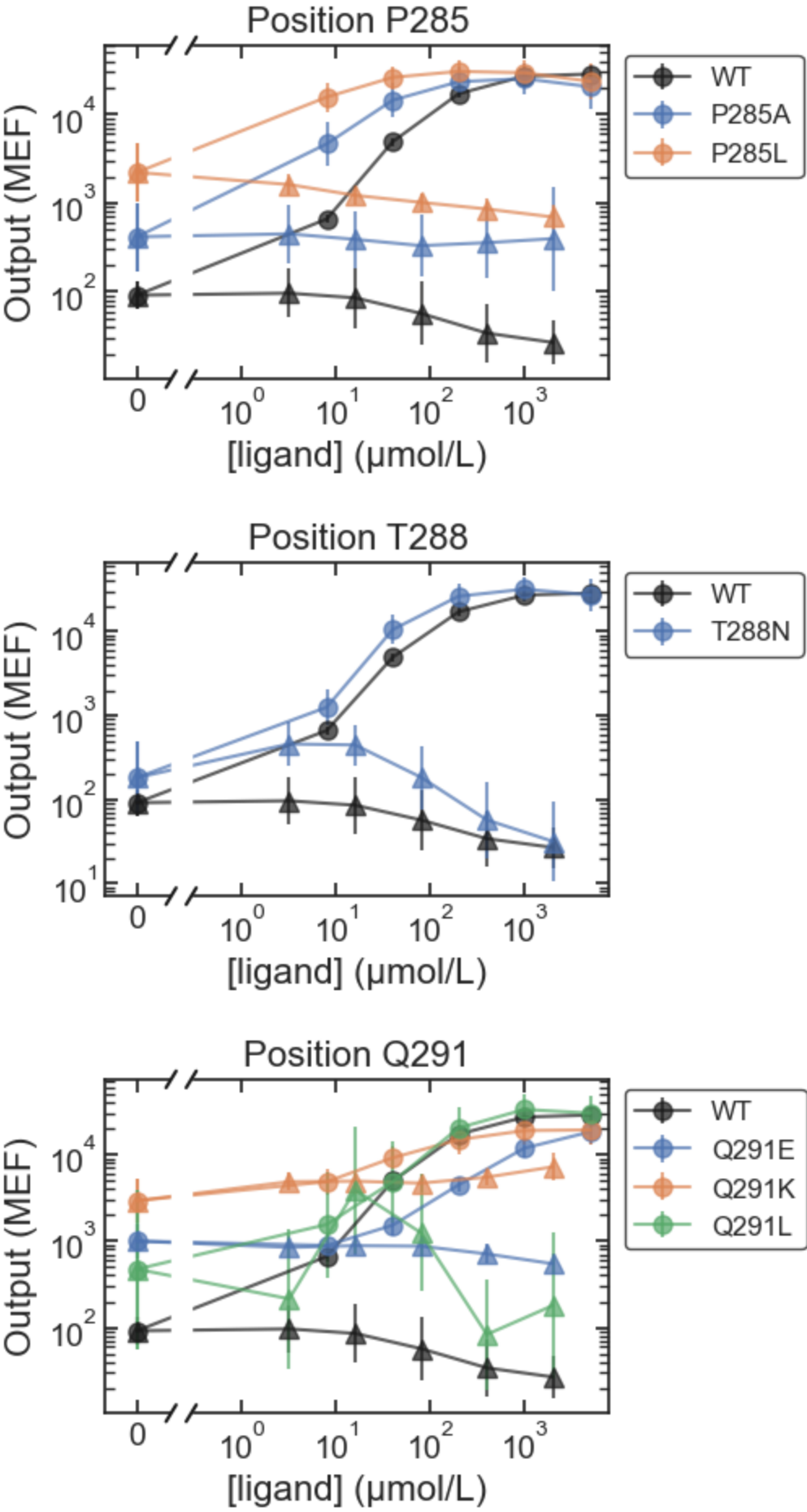


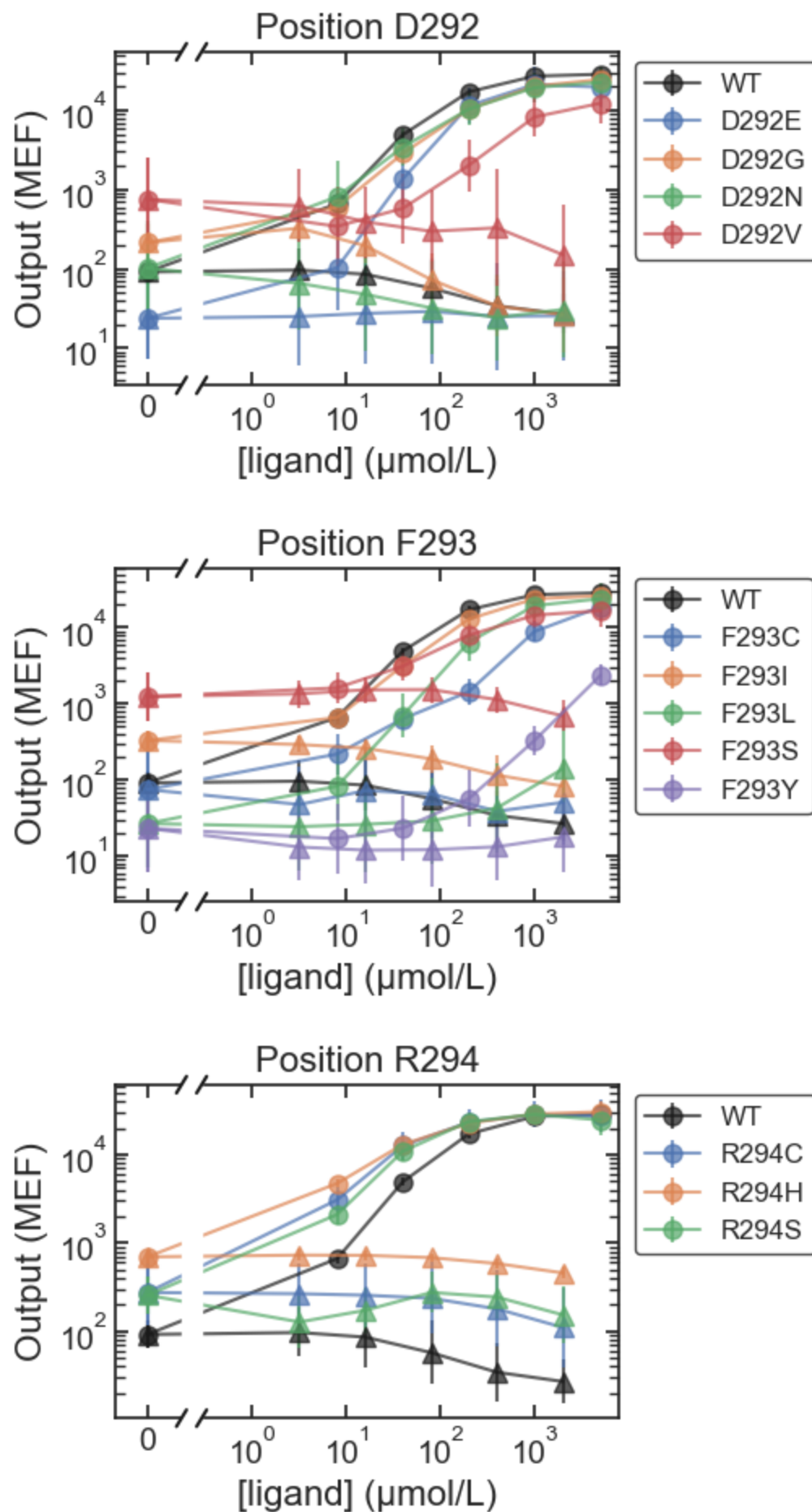


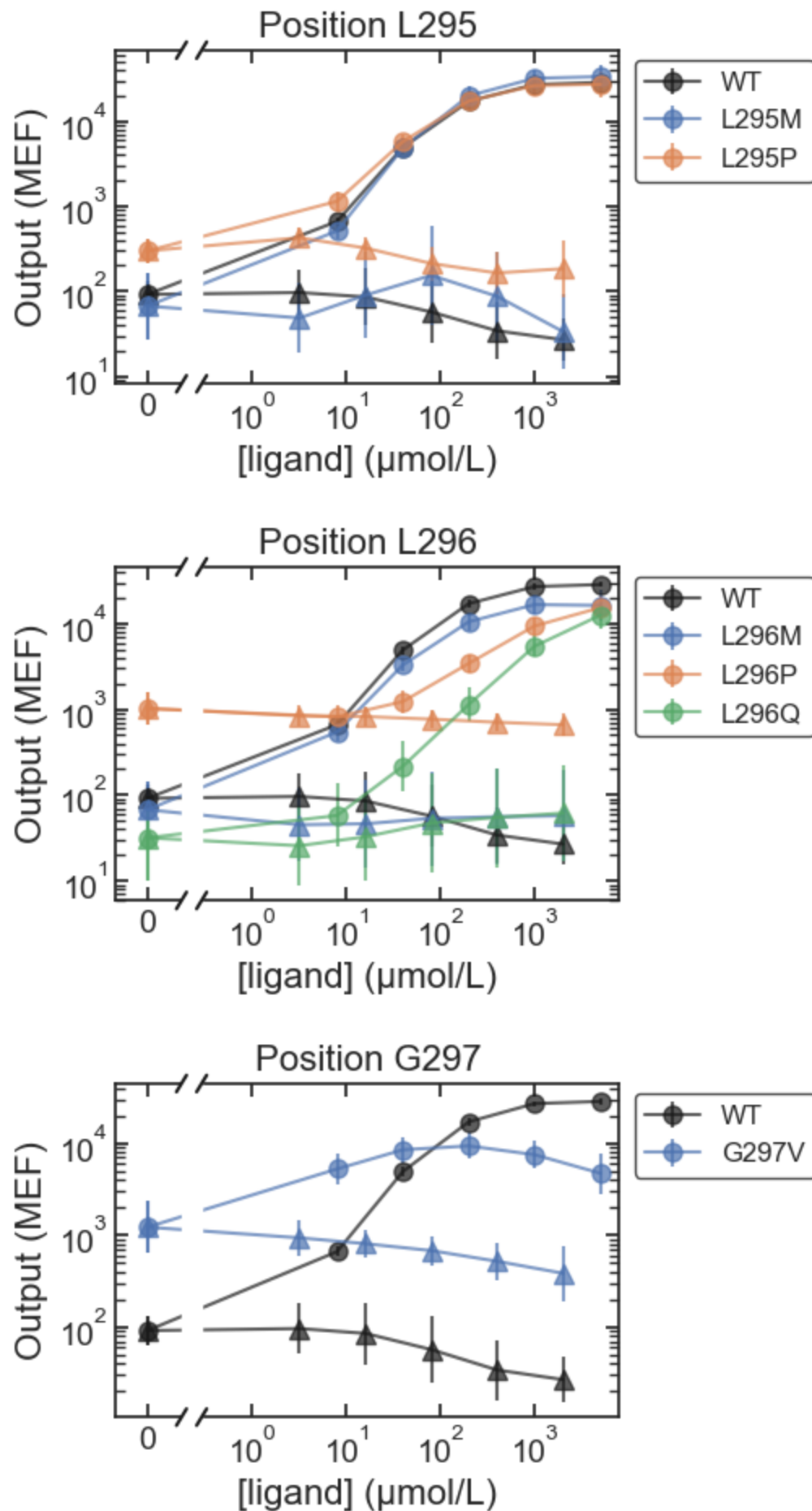


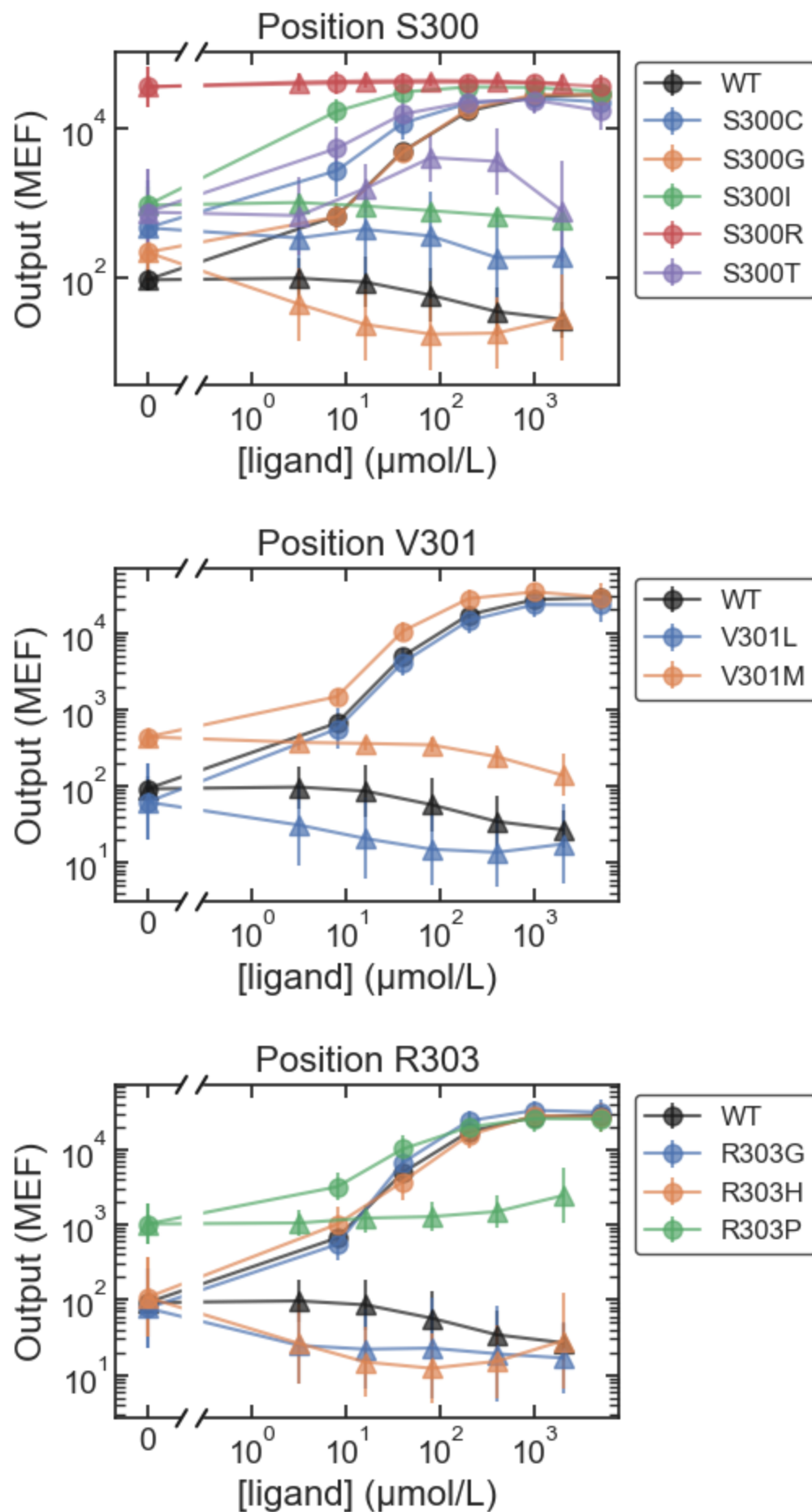


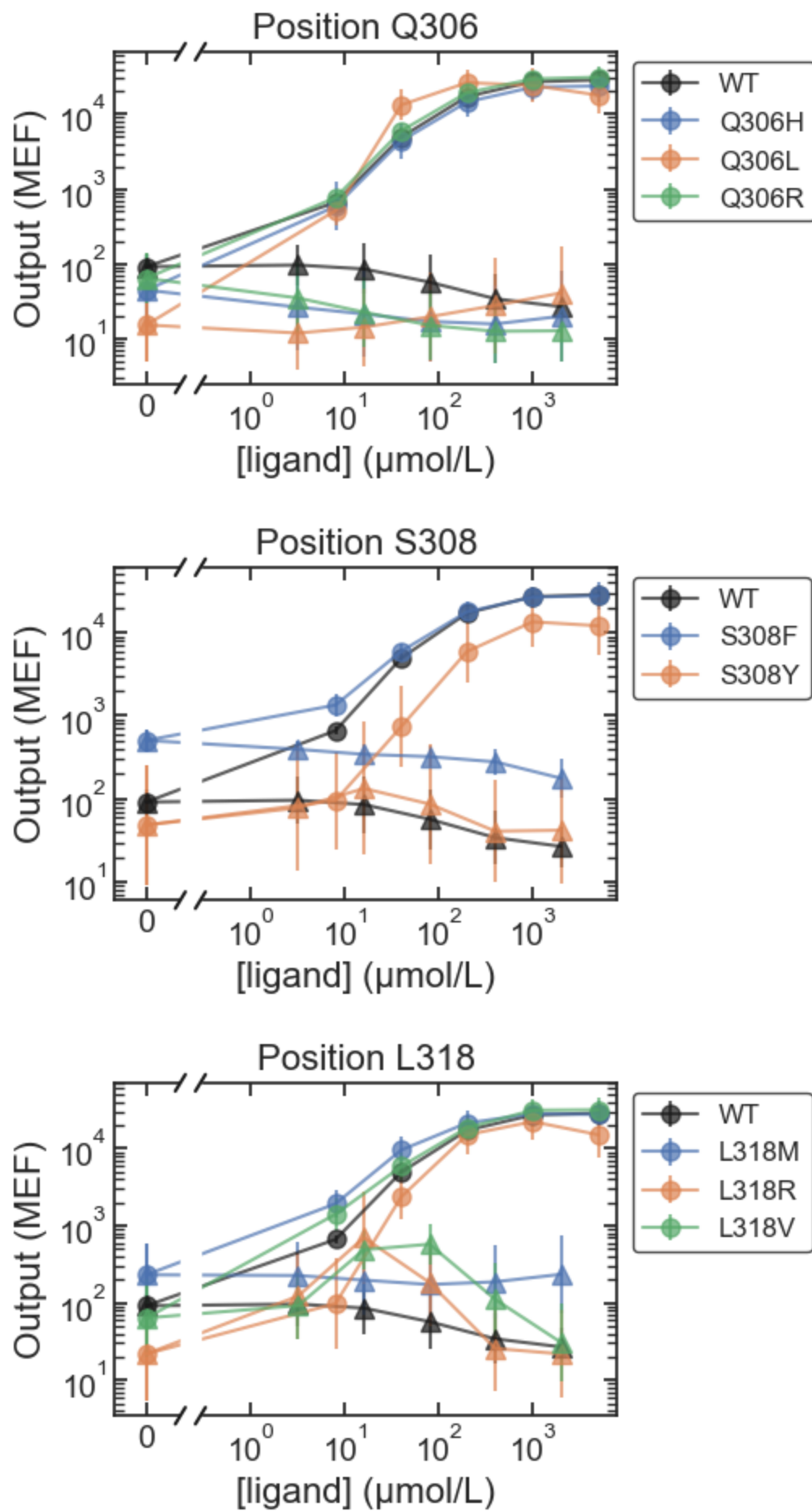


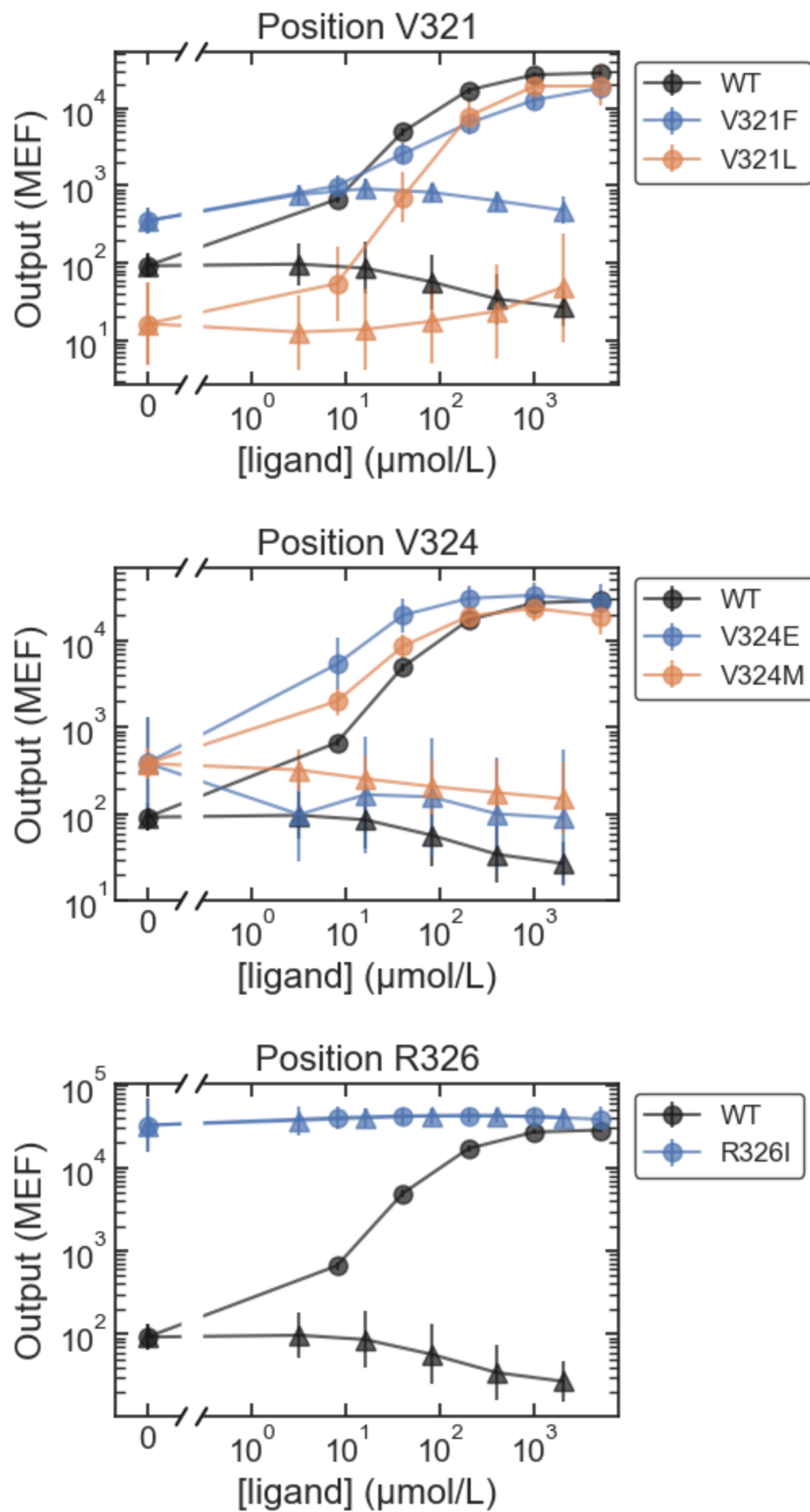


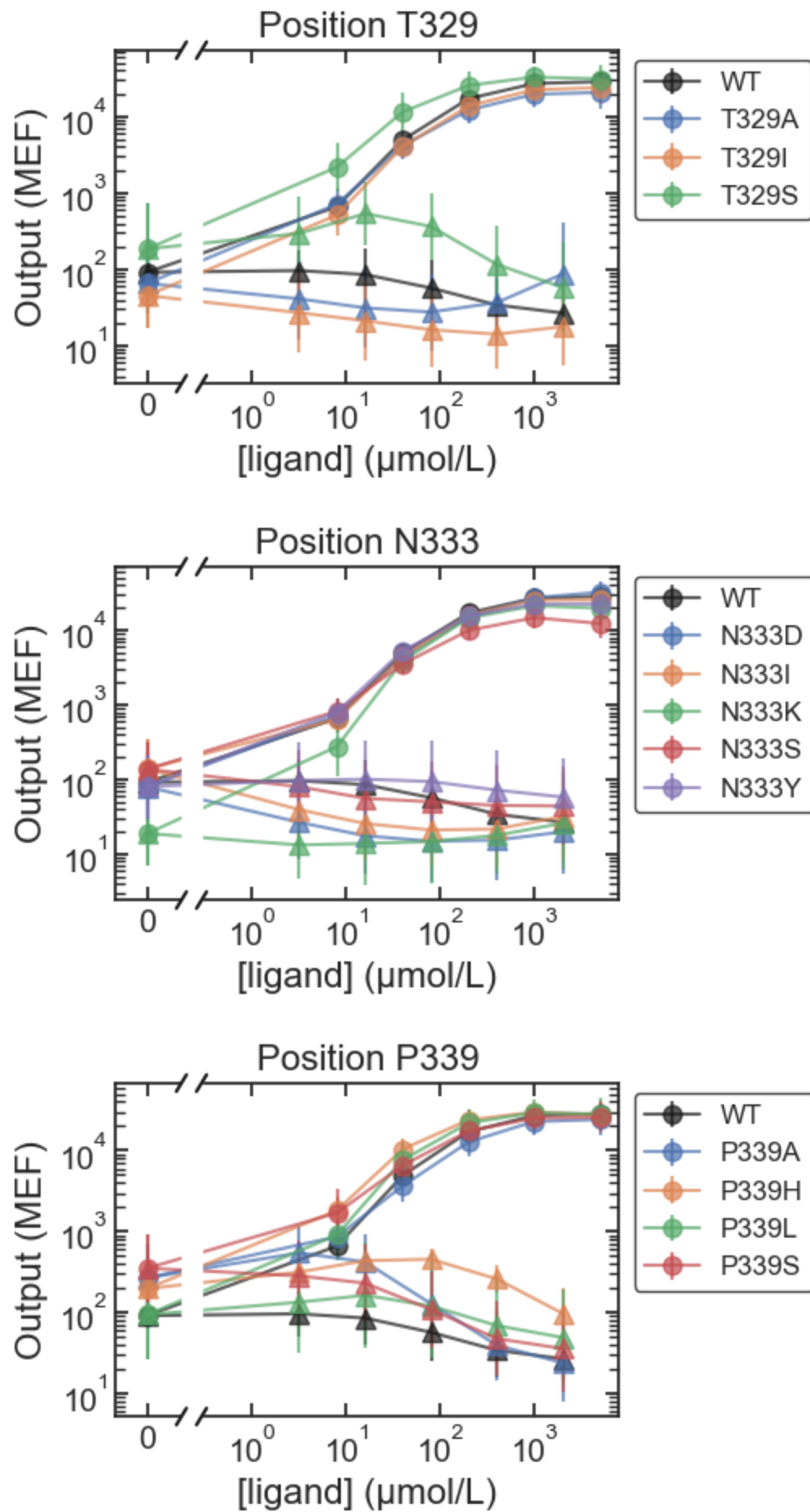


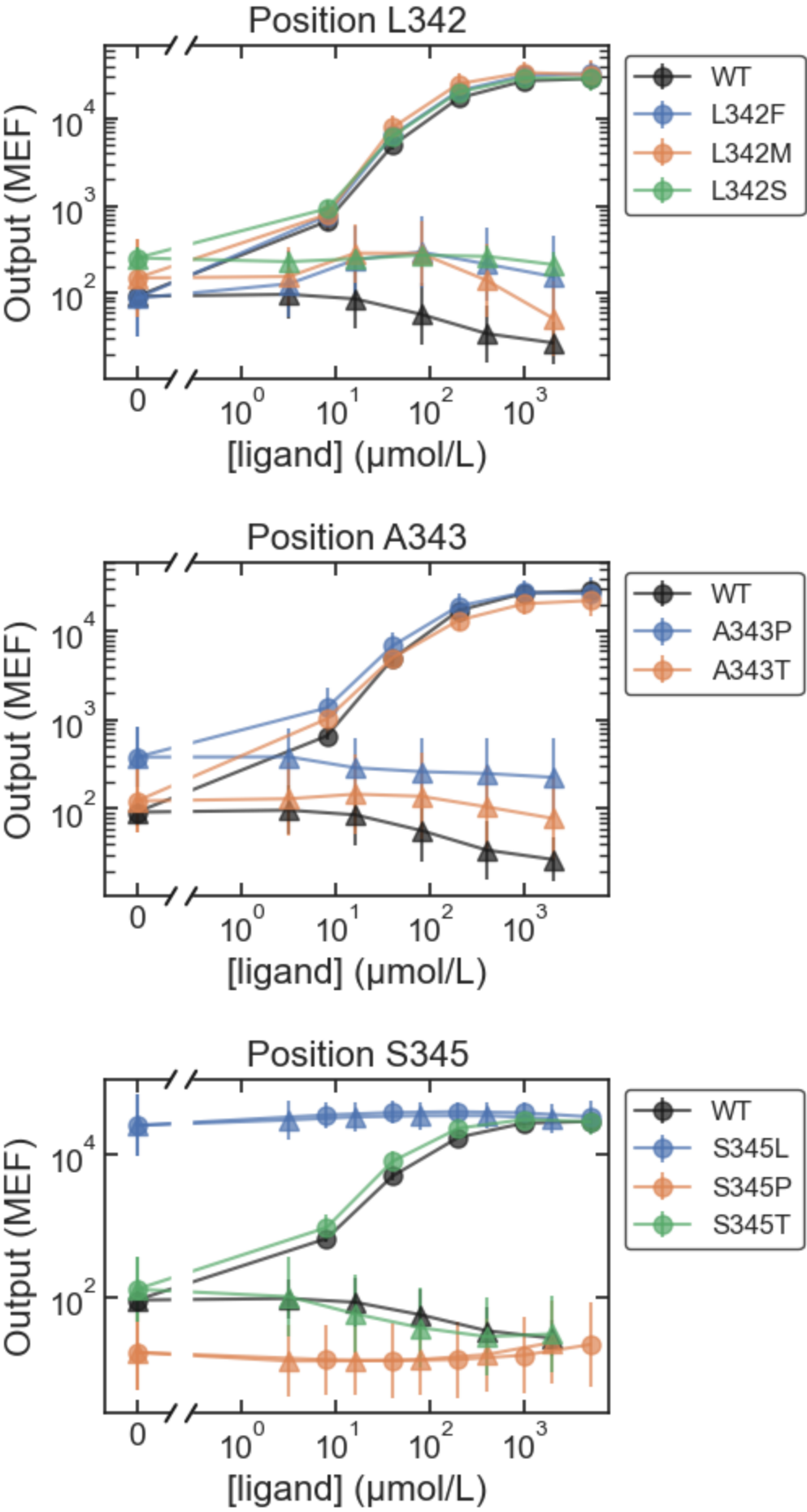


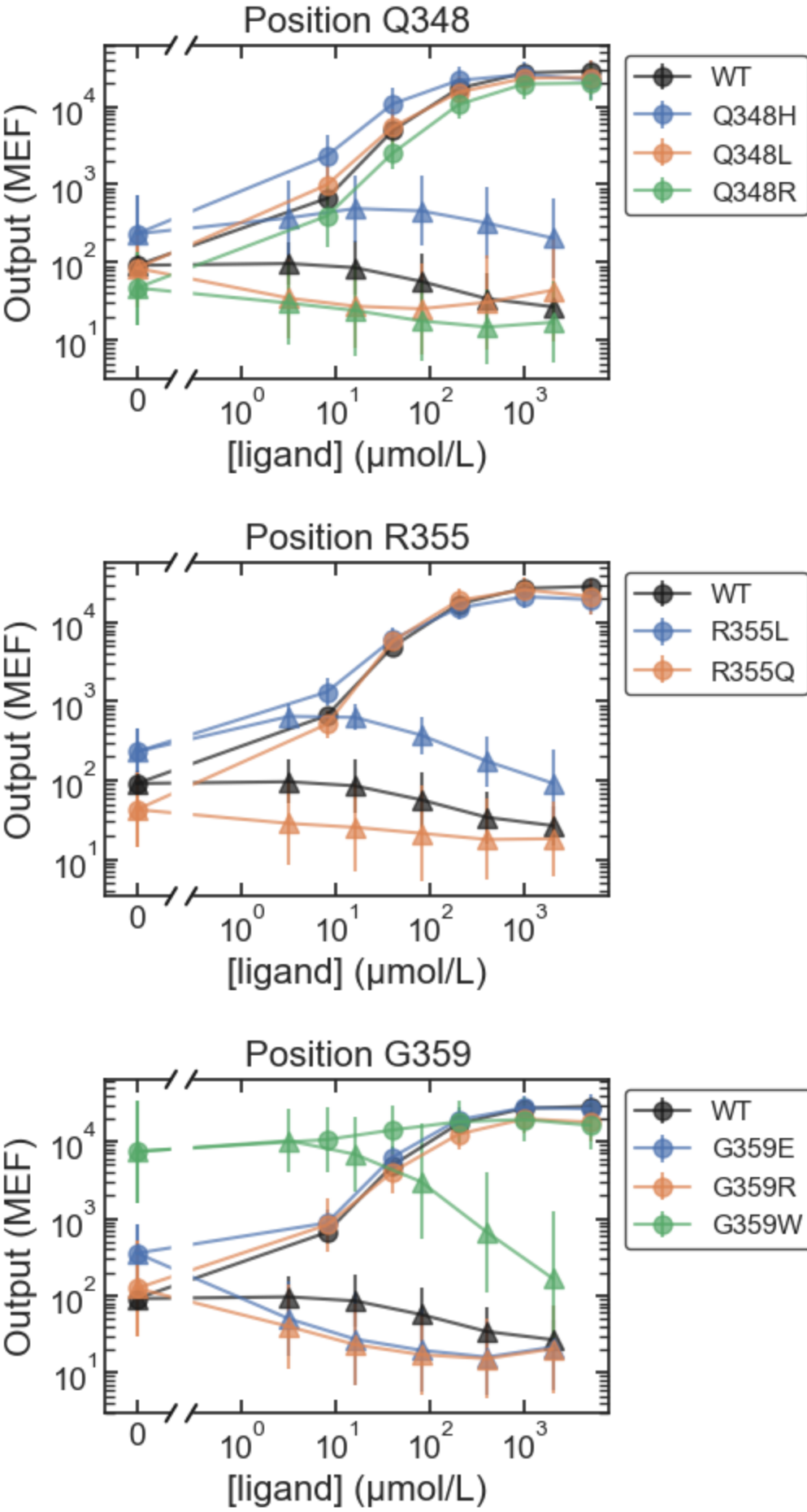




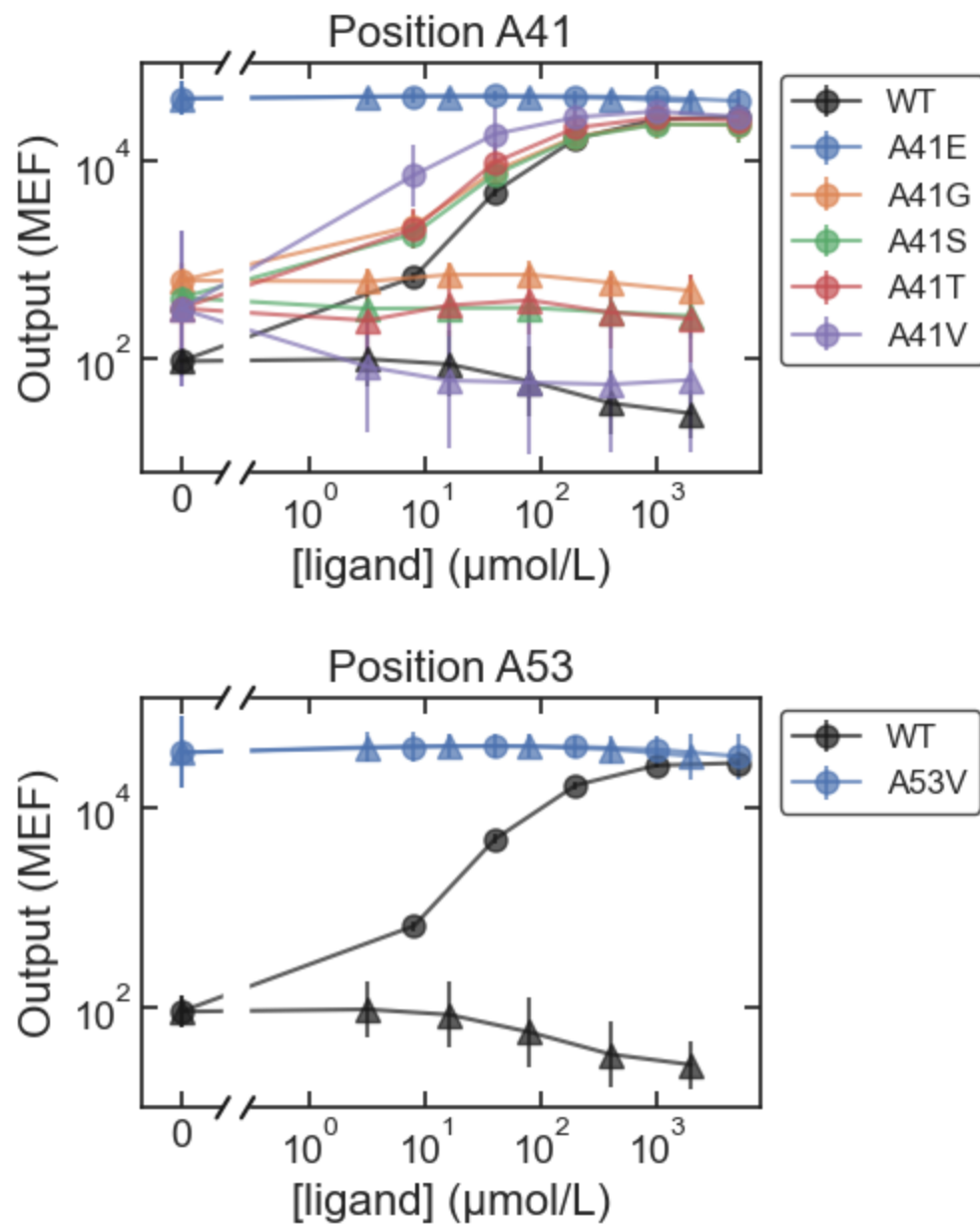


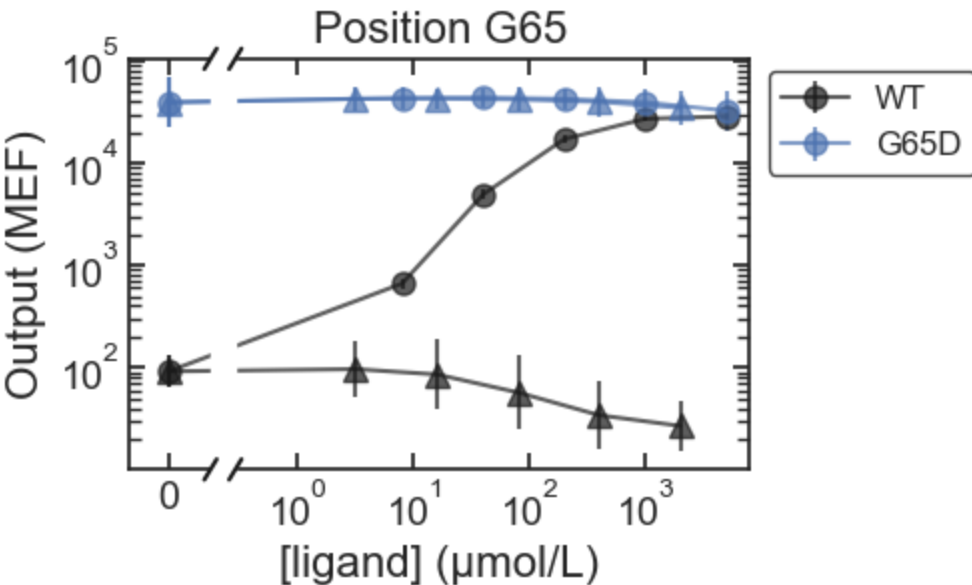
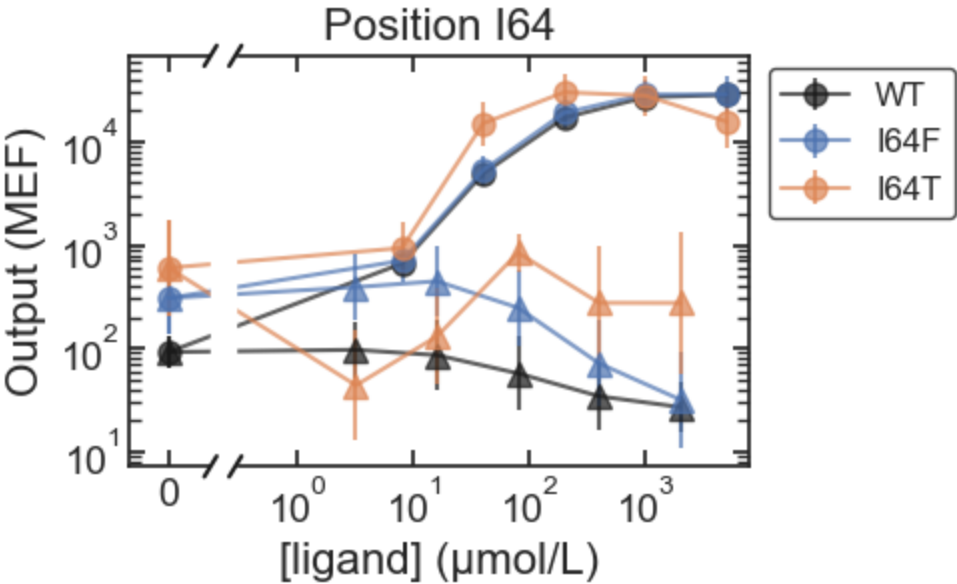
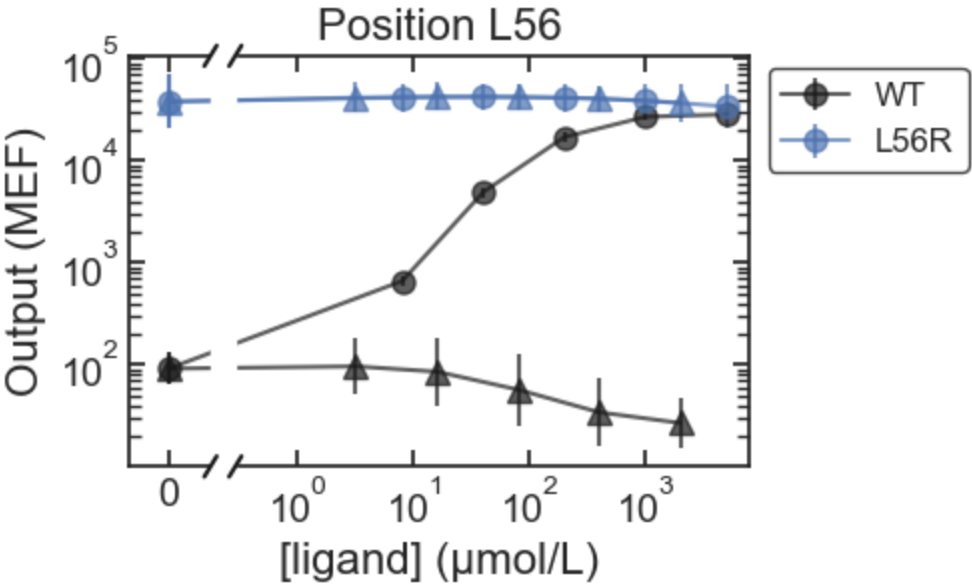


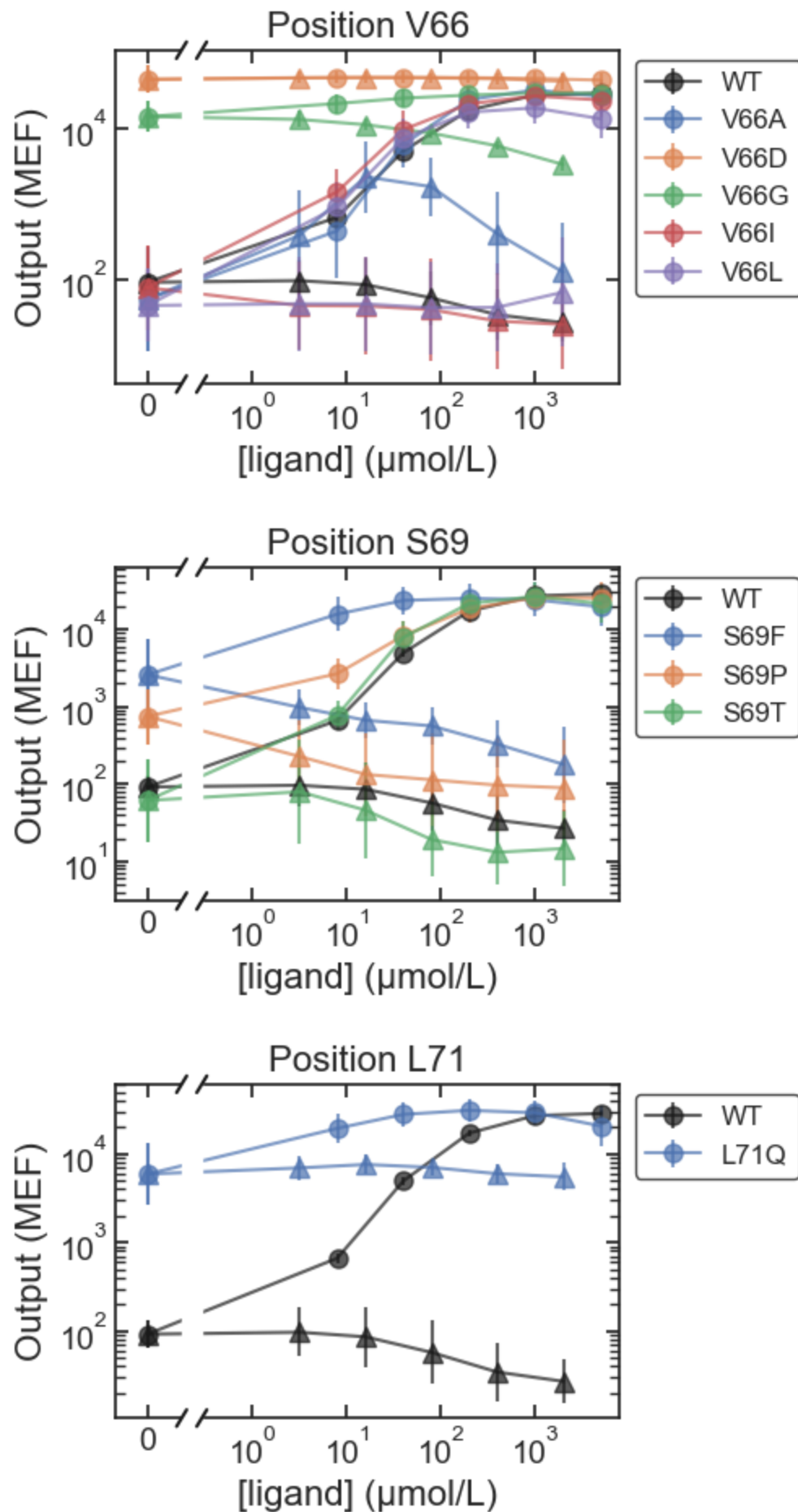


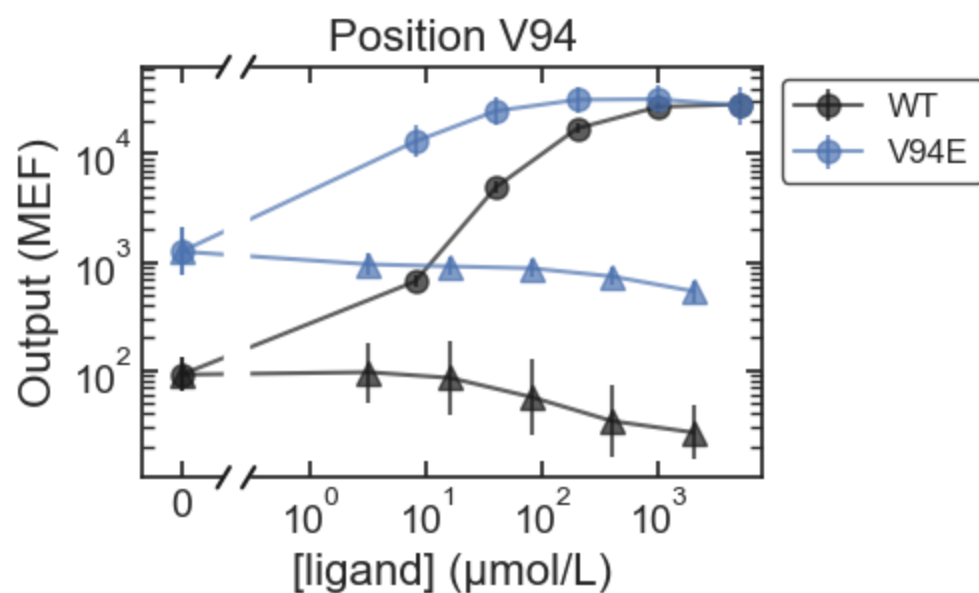
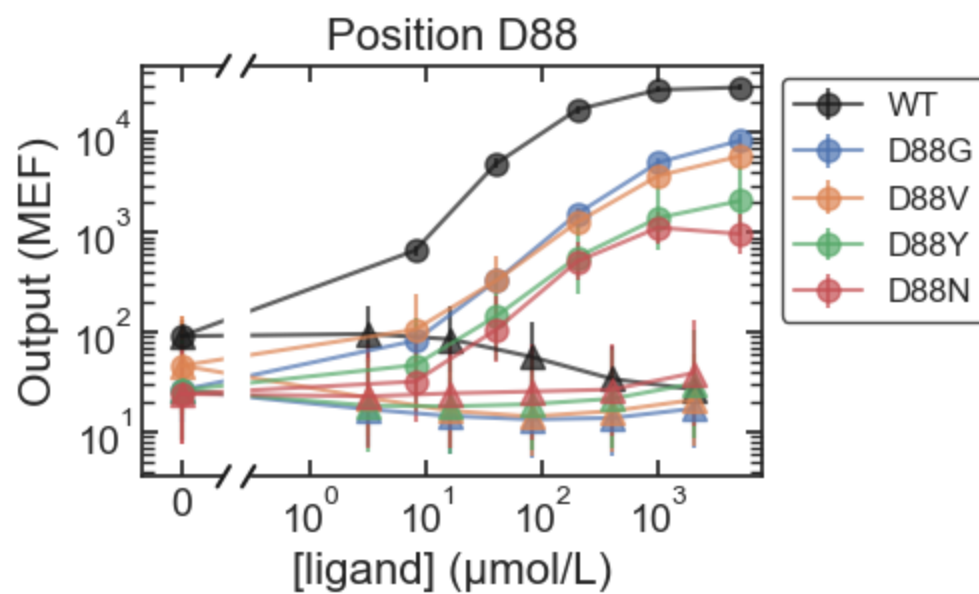
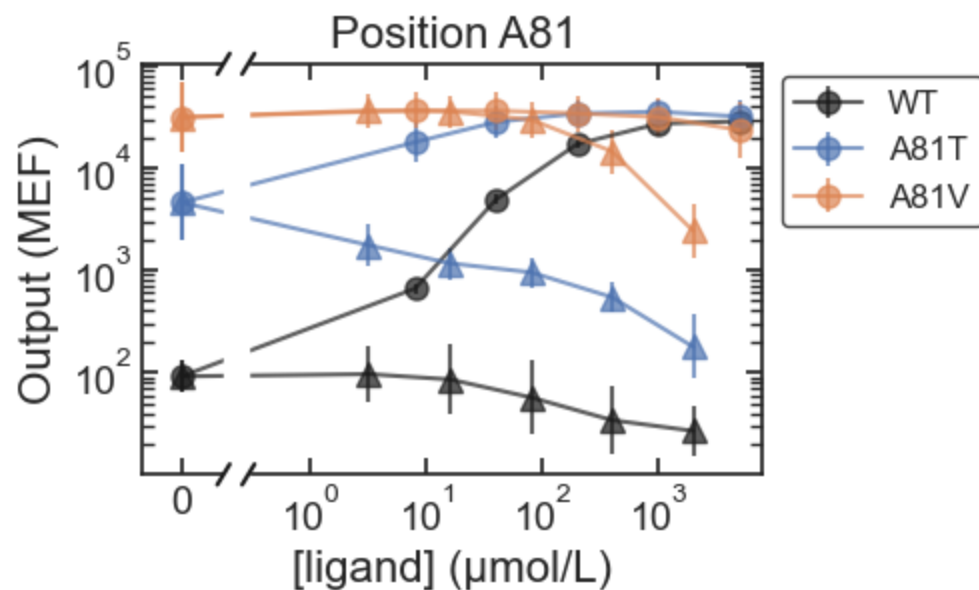


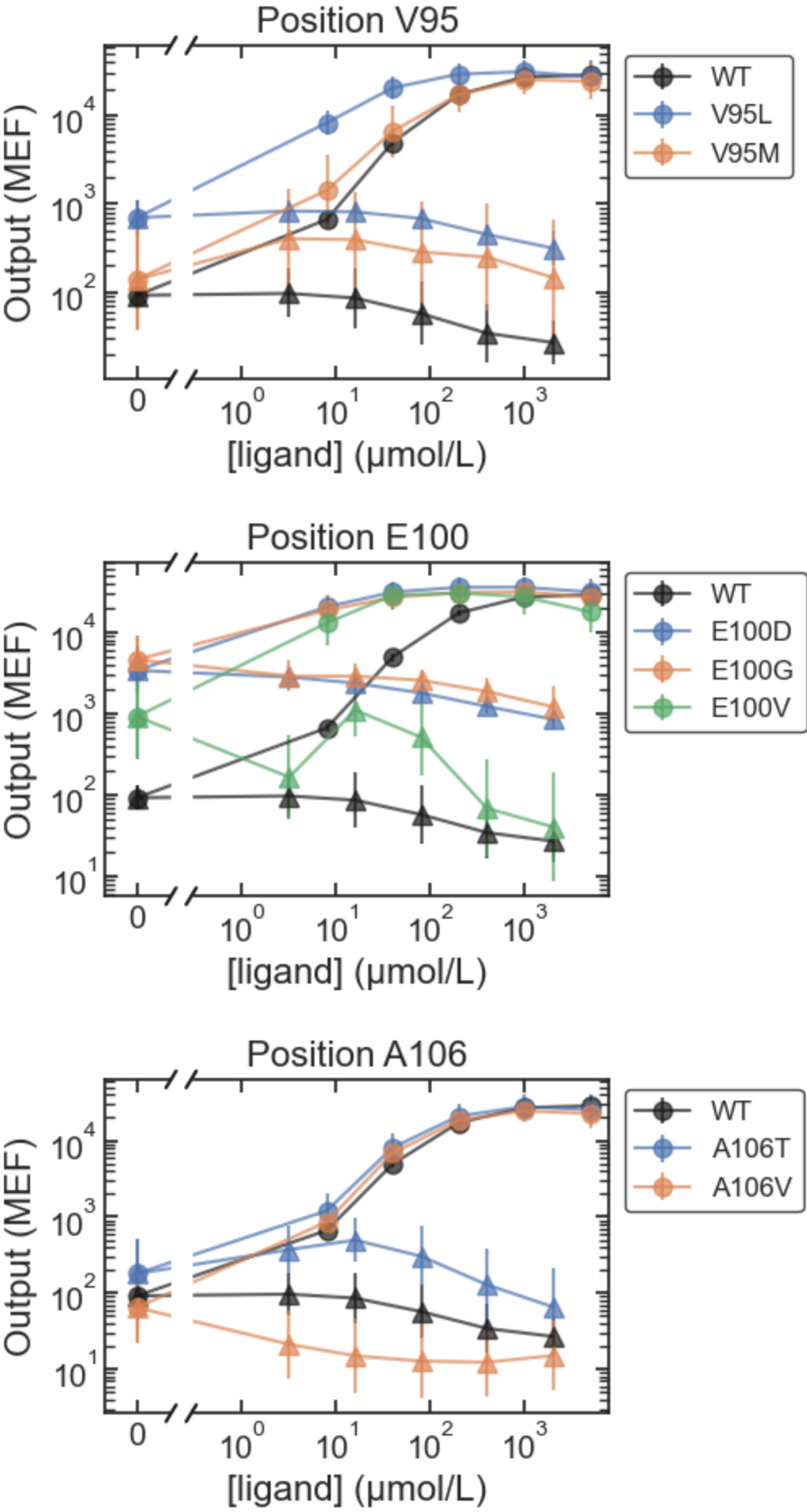
MWC_only

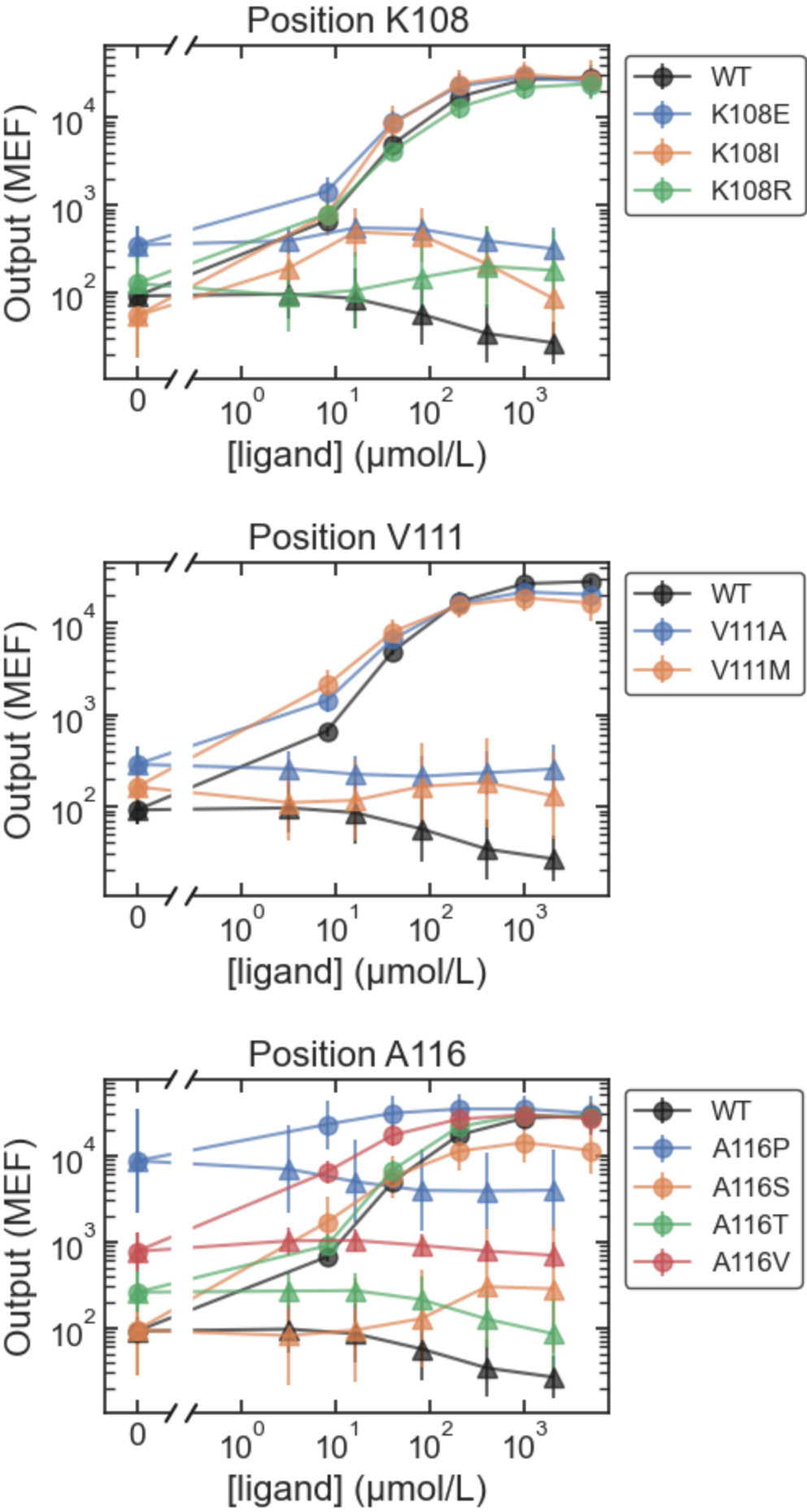


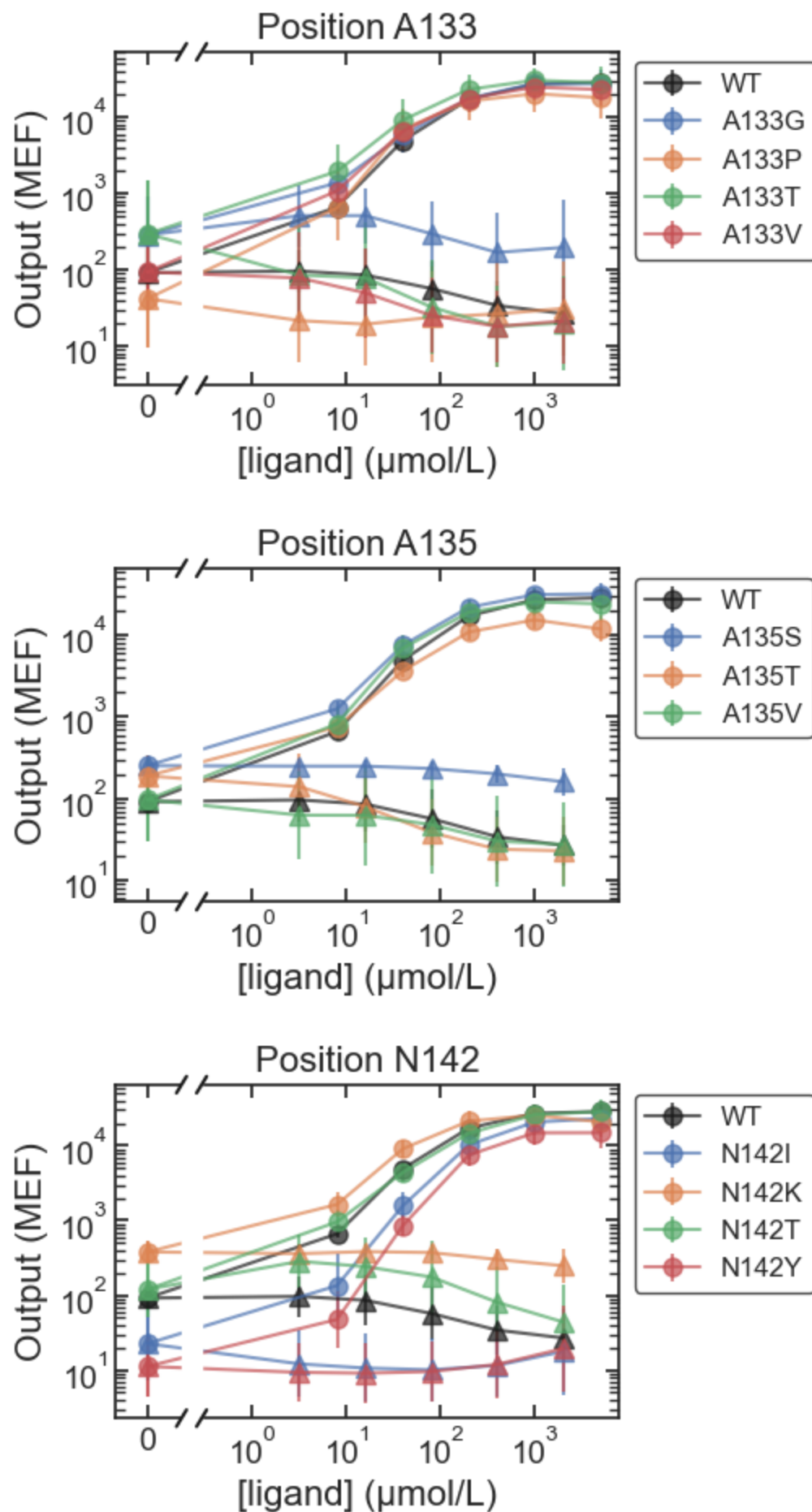


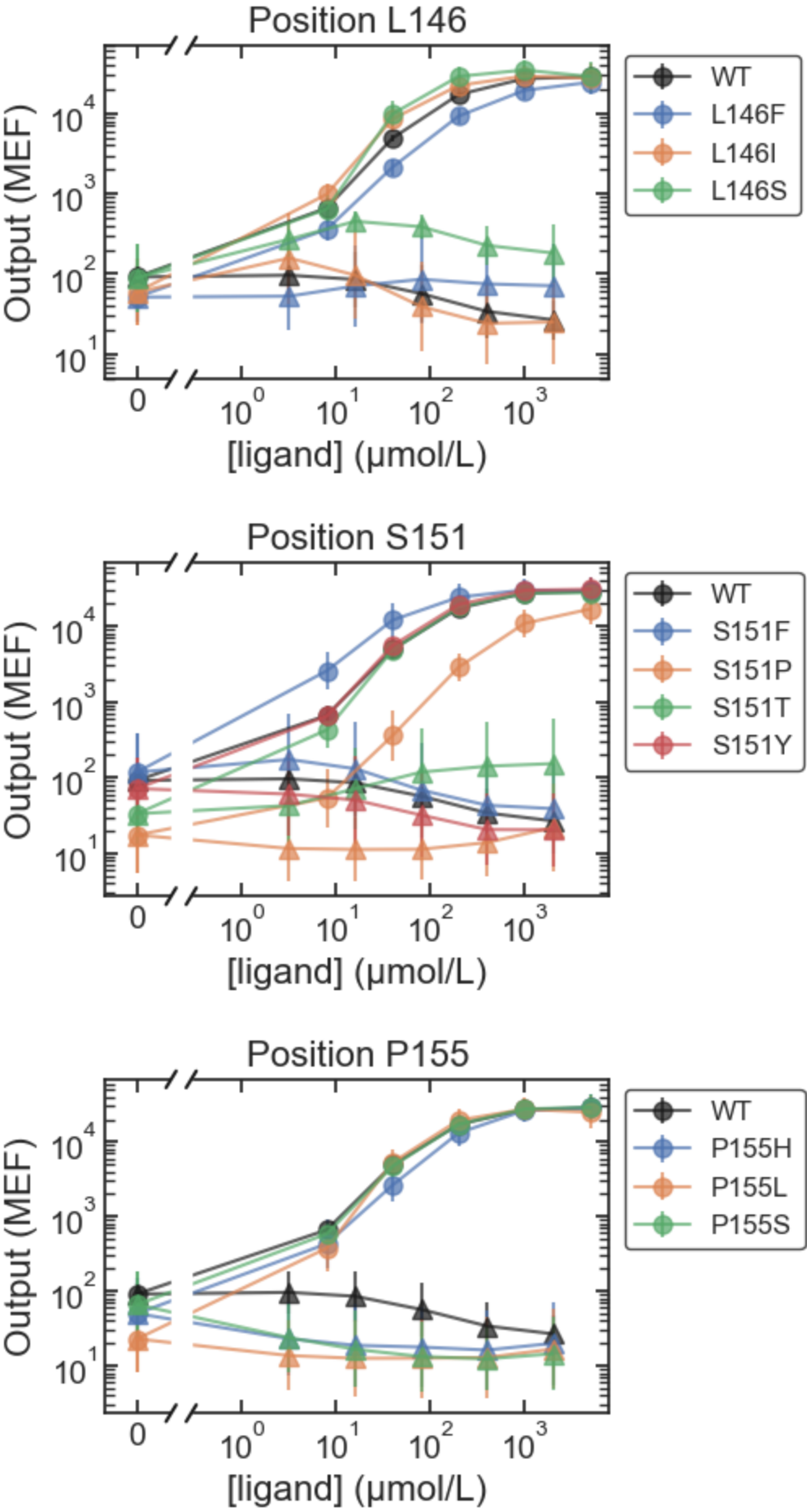


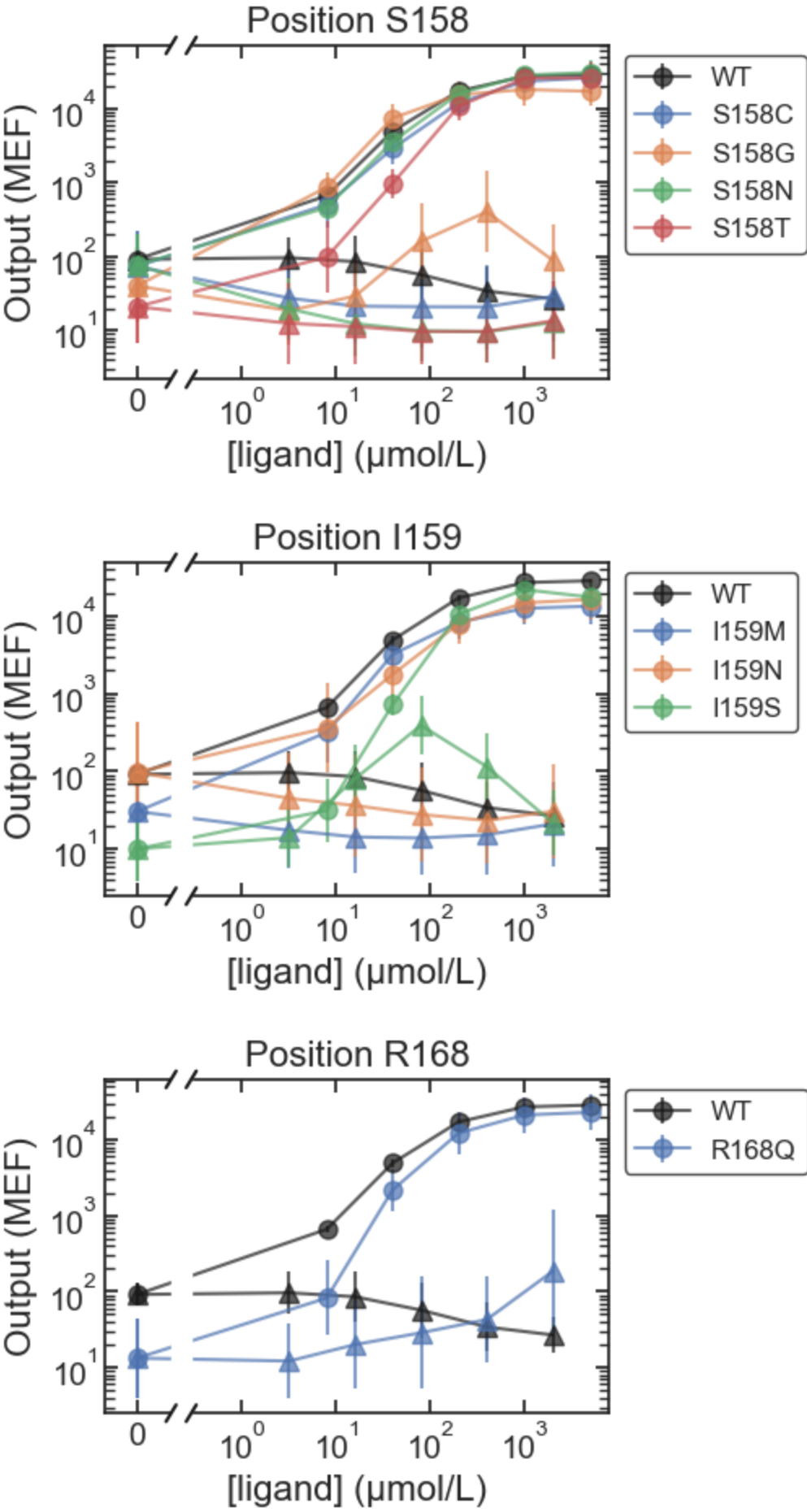


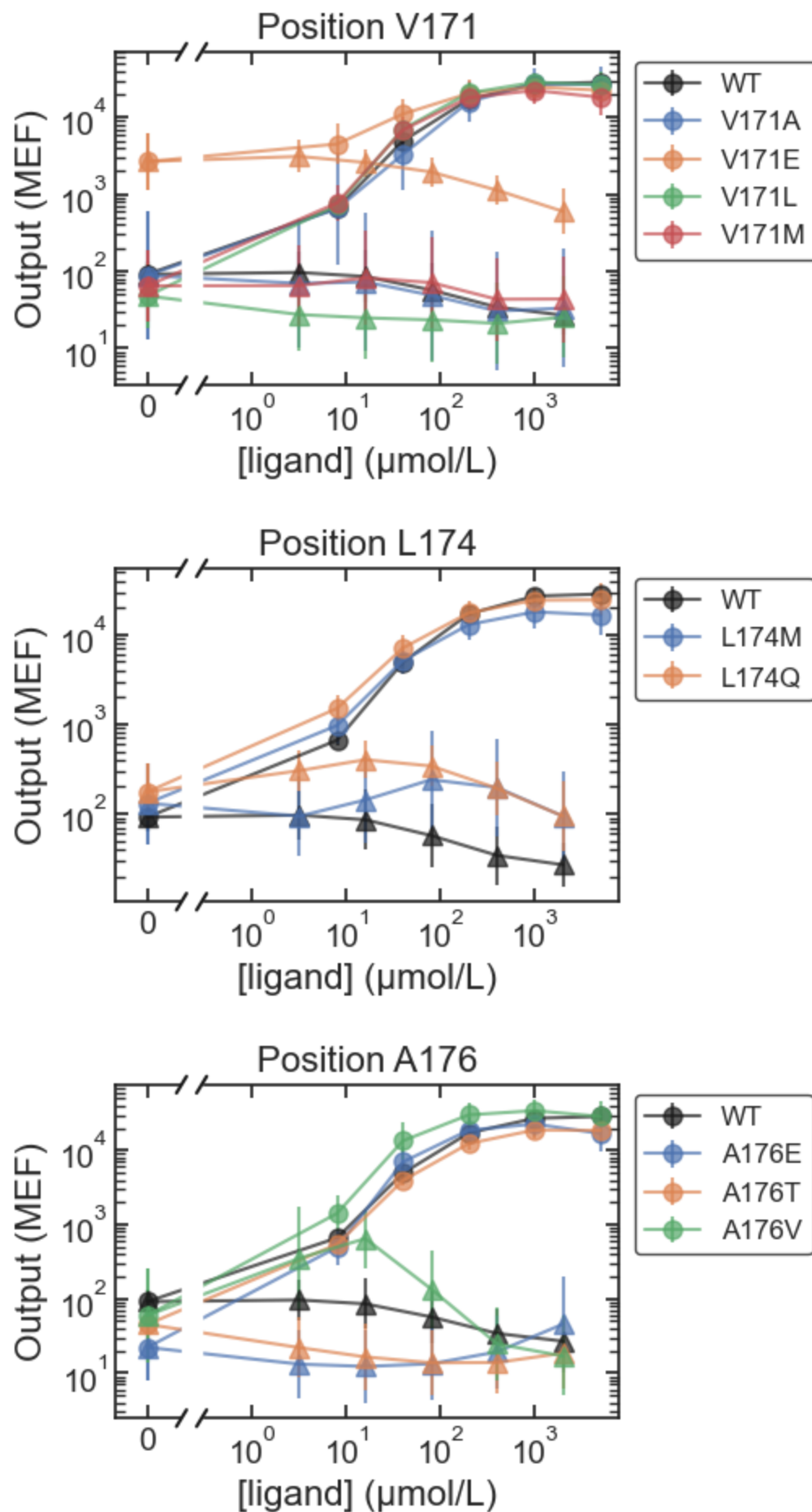


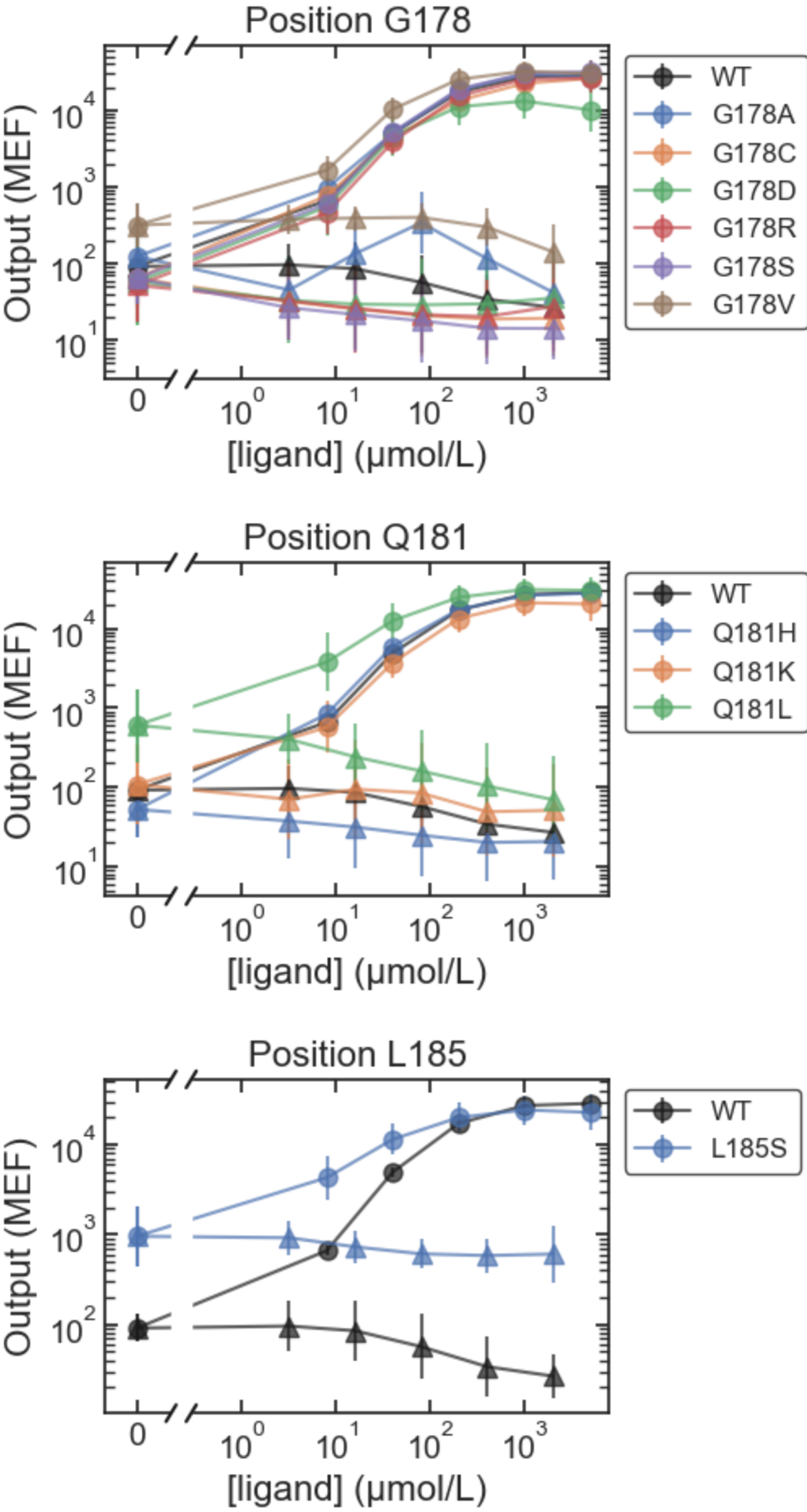


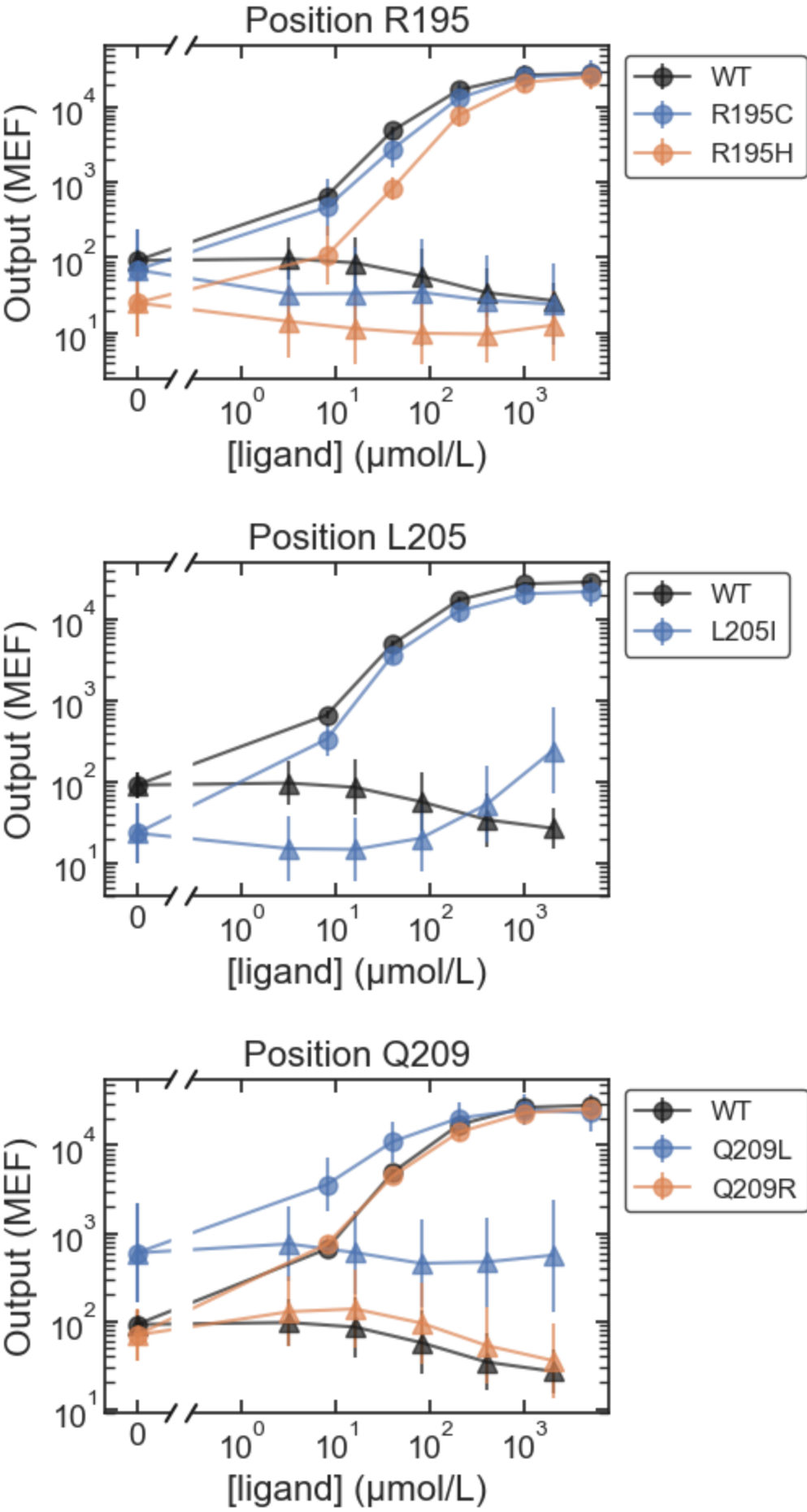


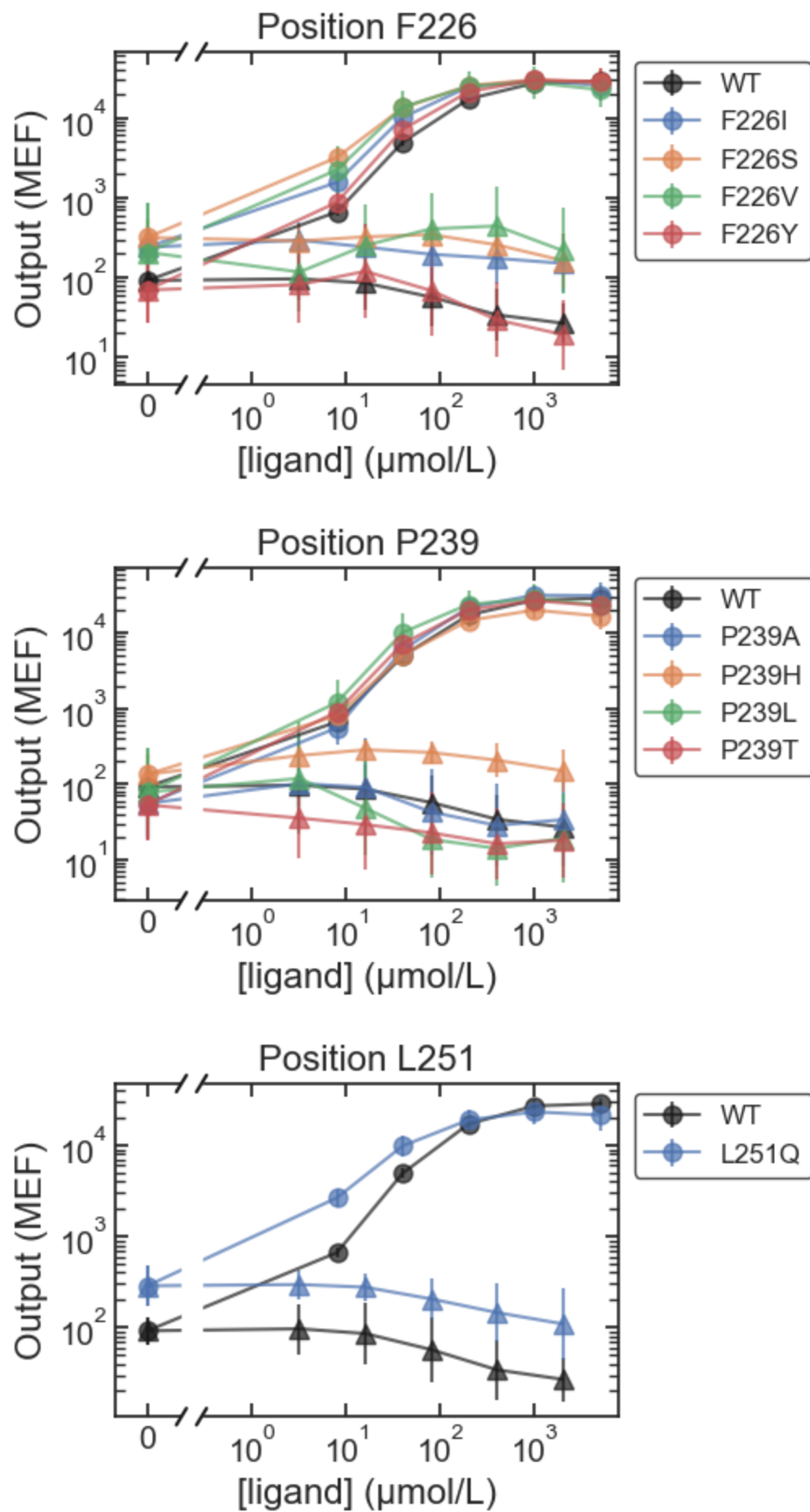


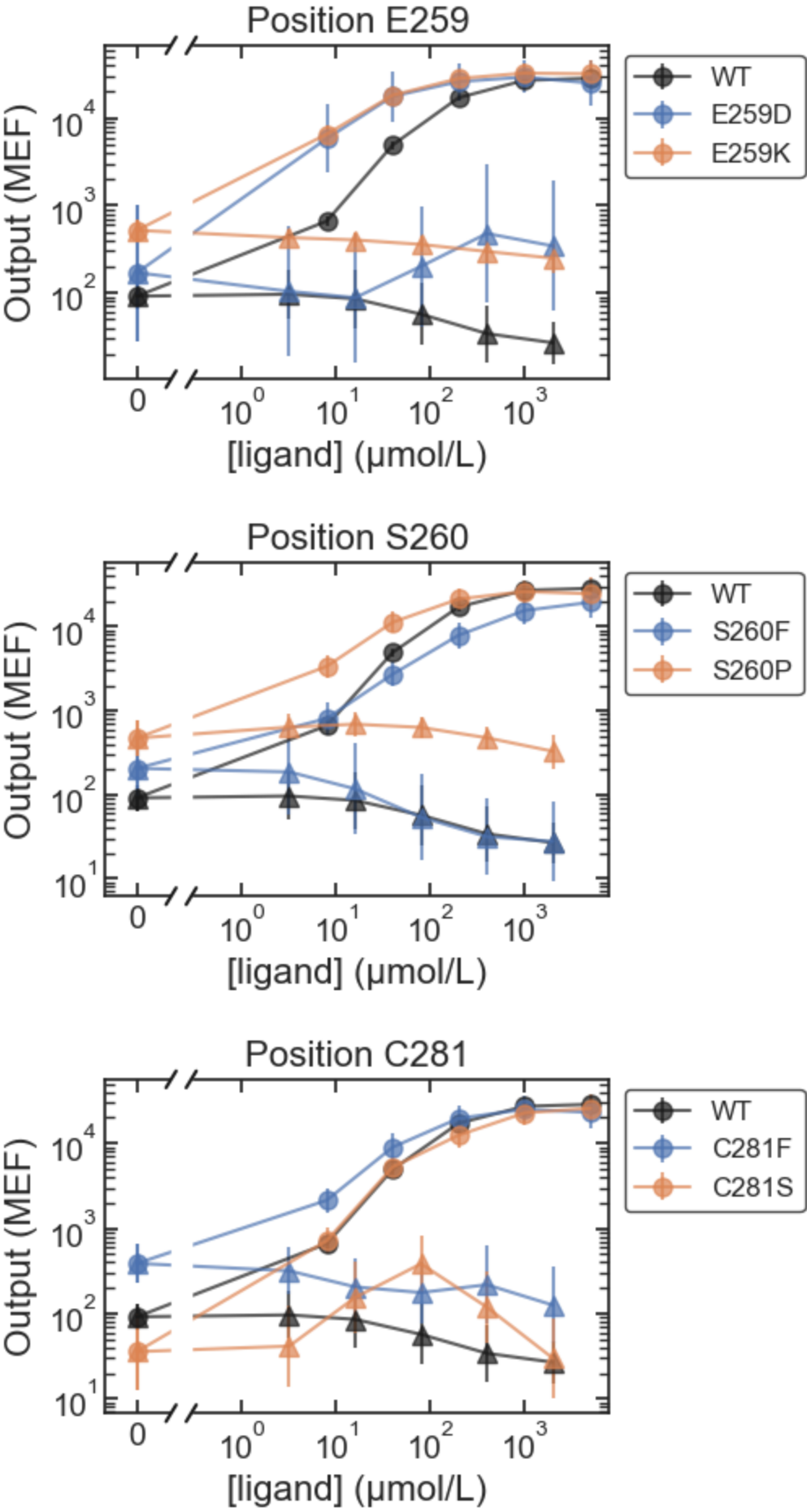


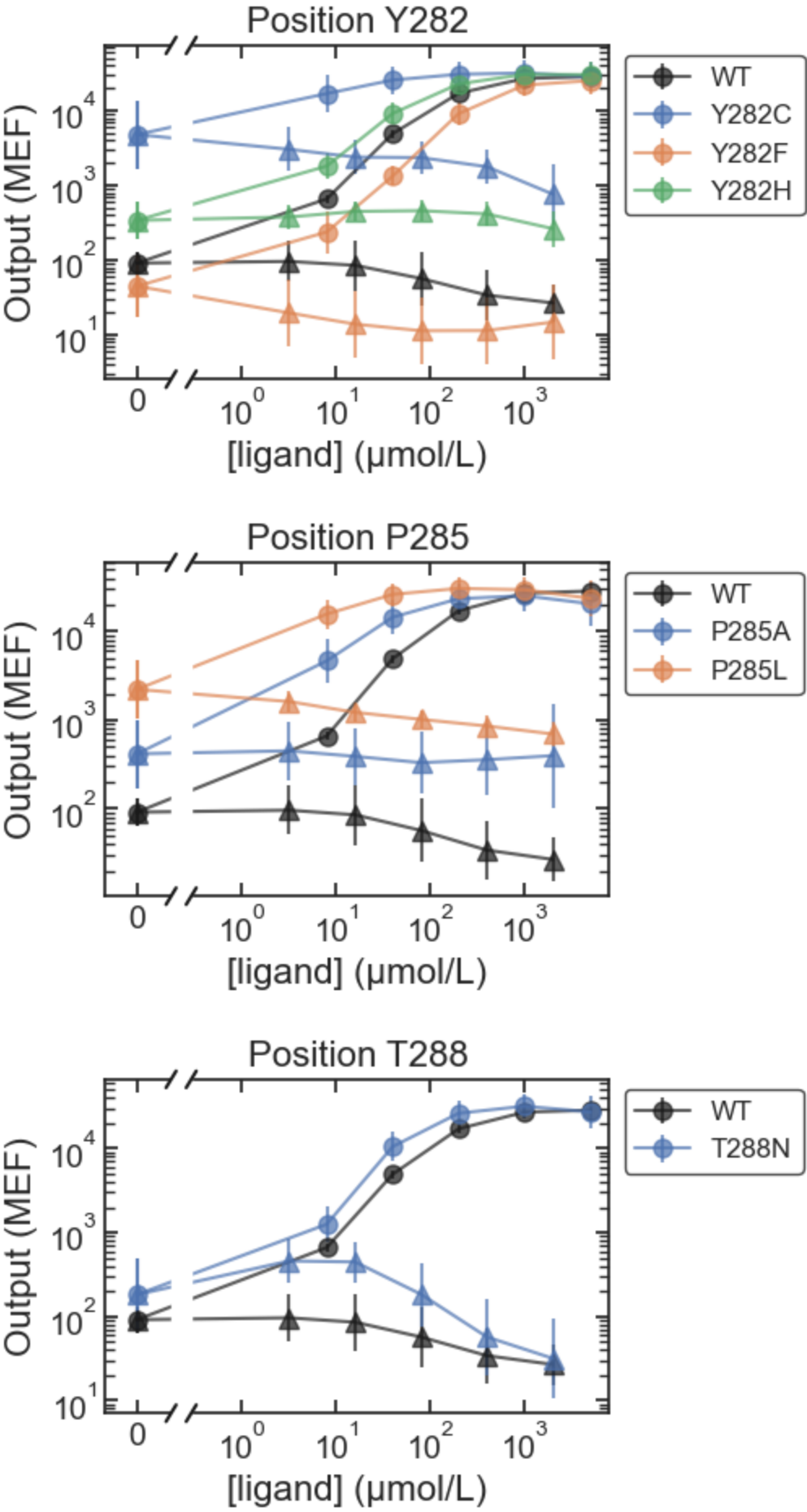


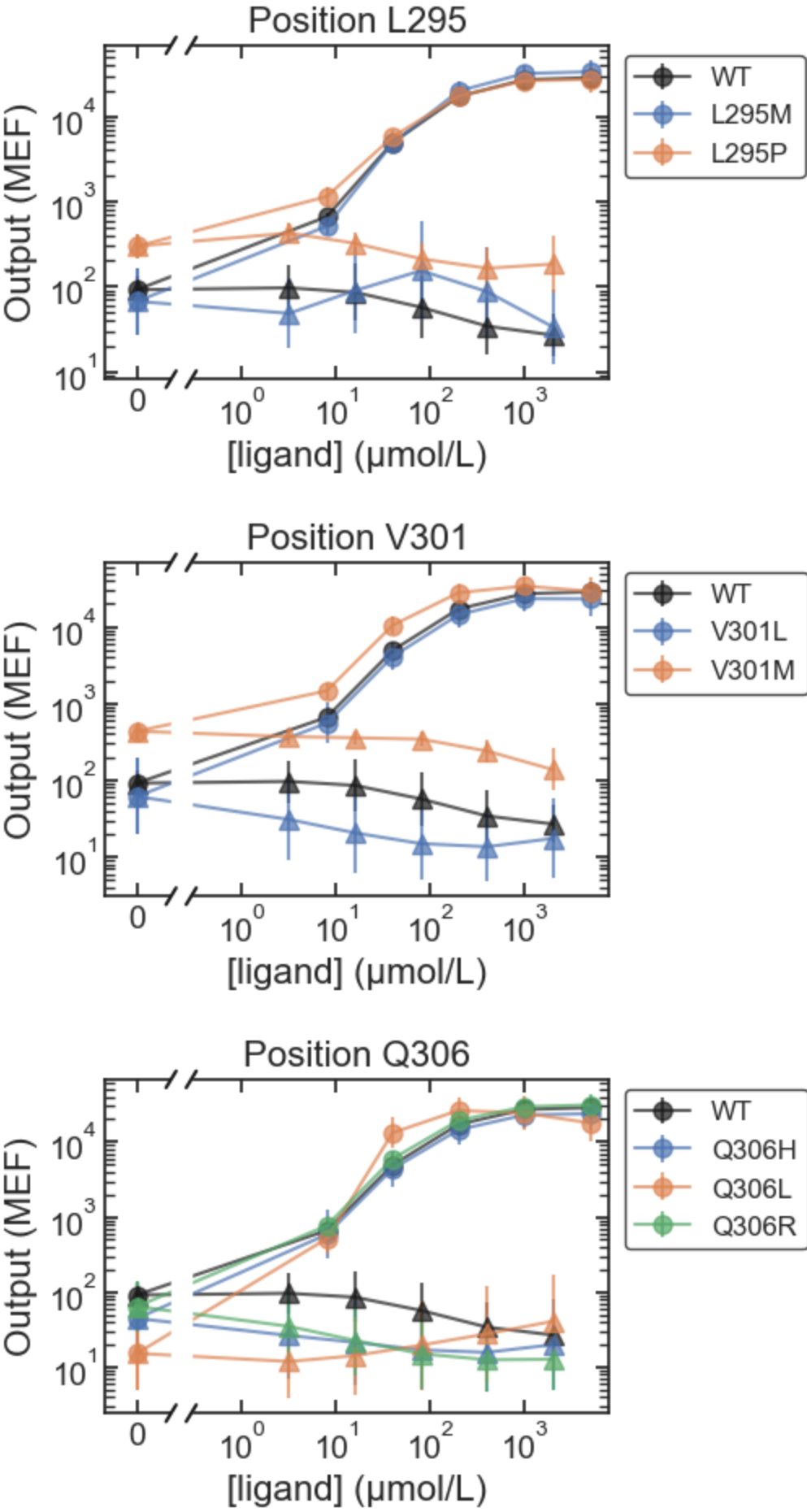


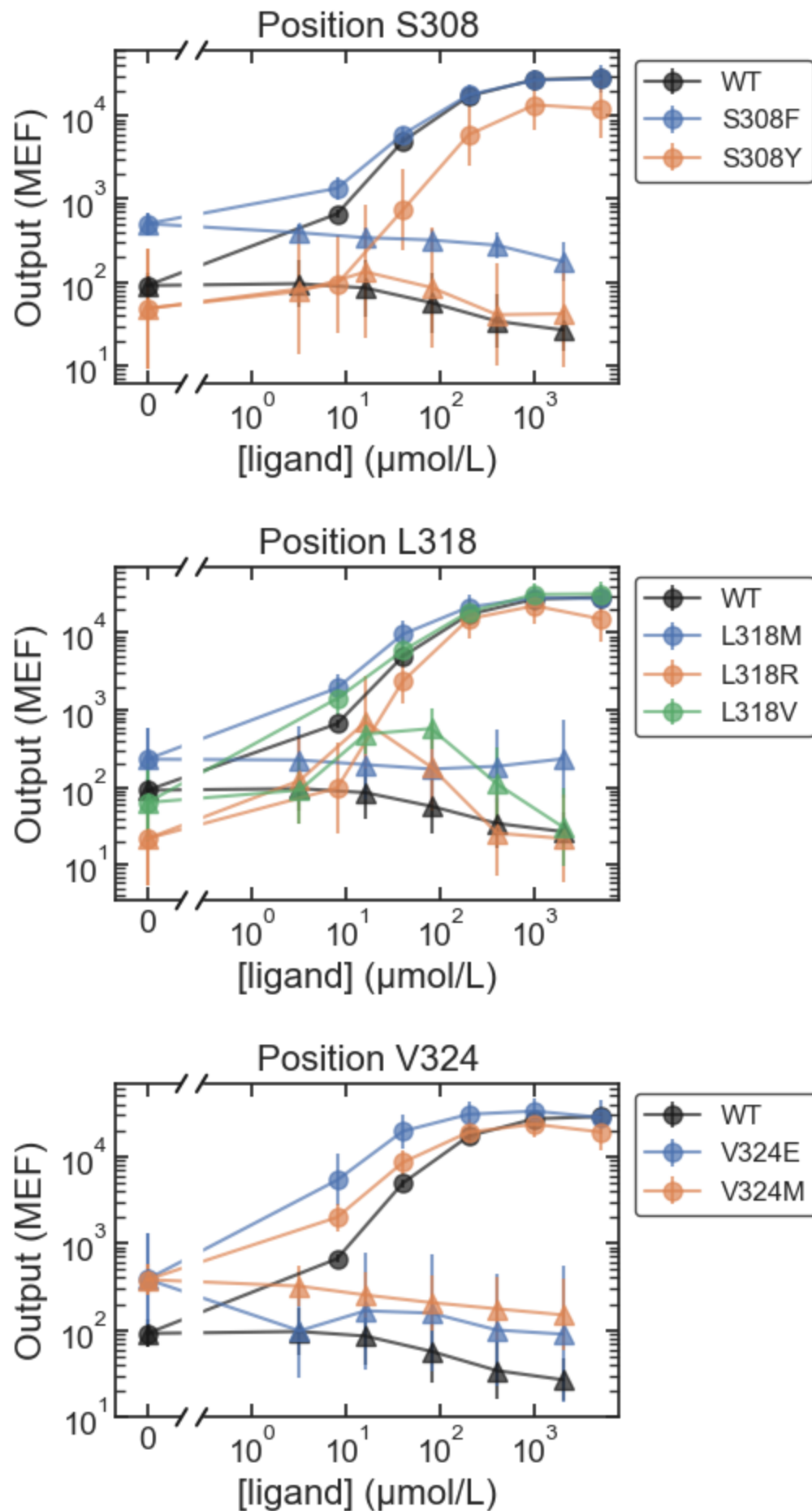


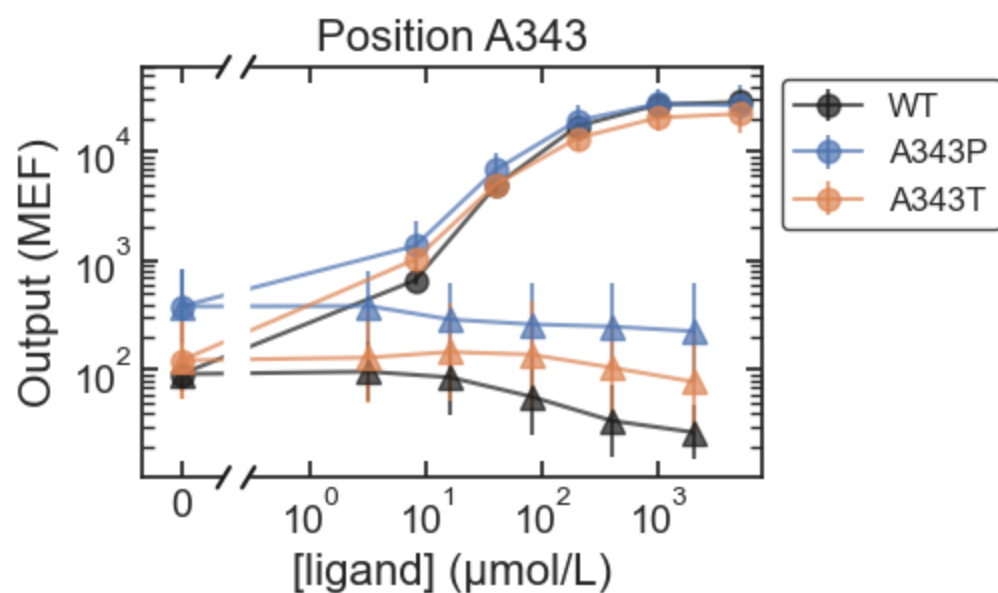
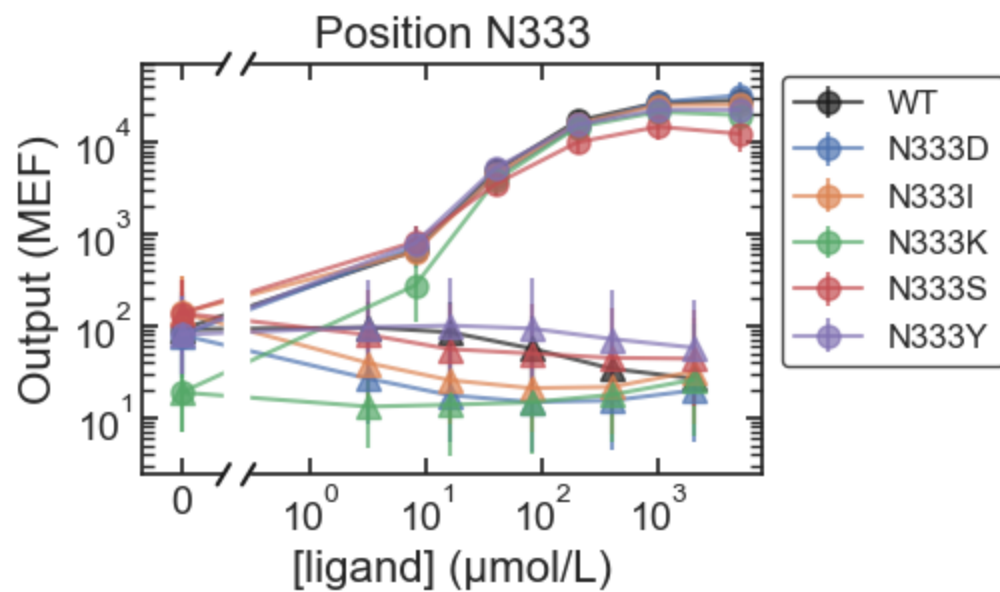
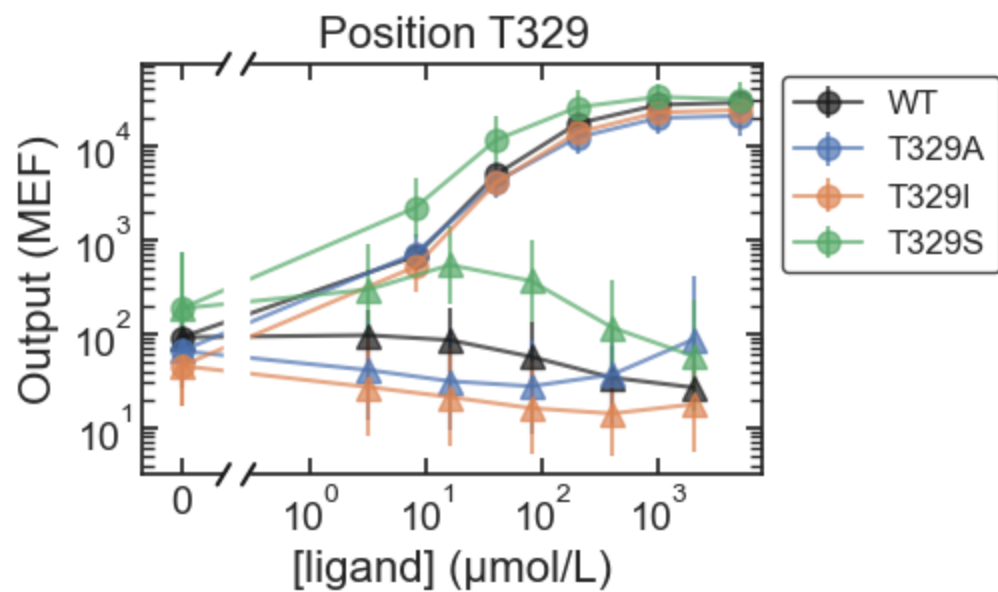


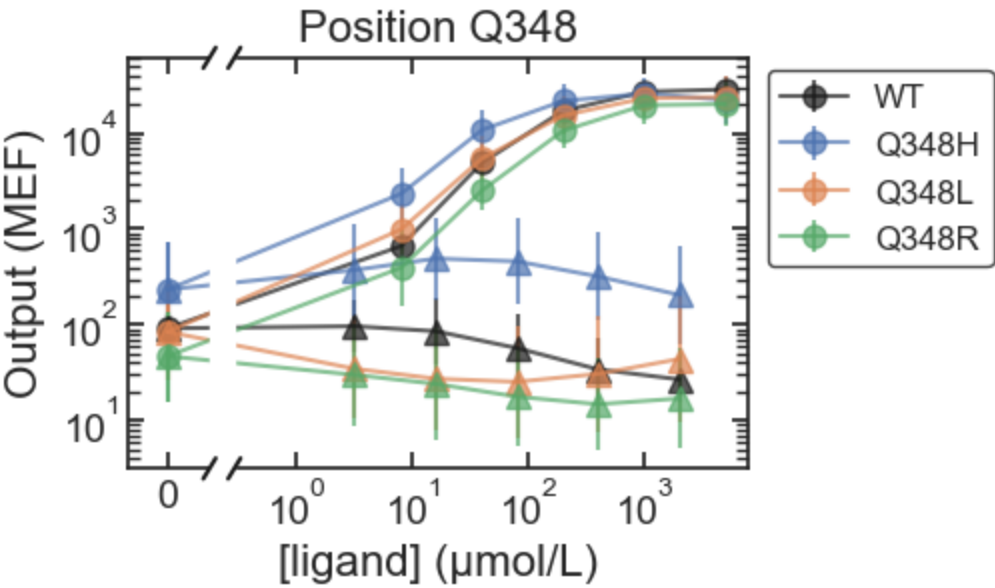




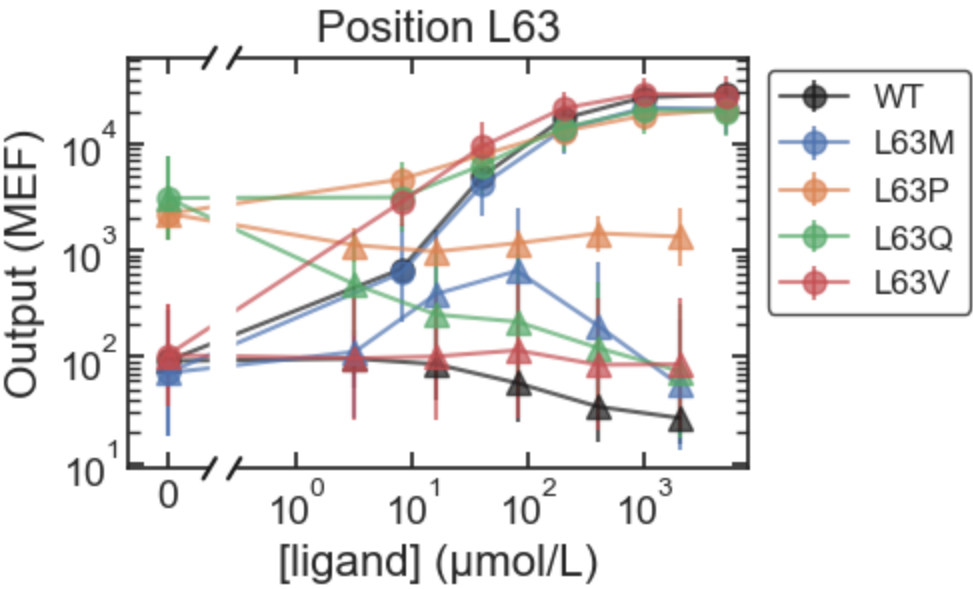


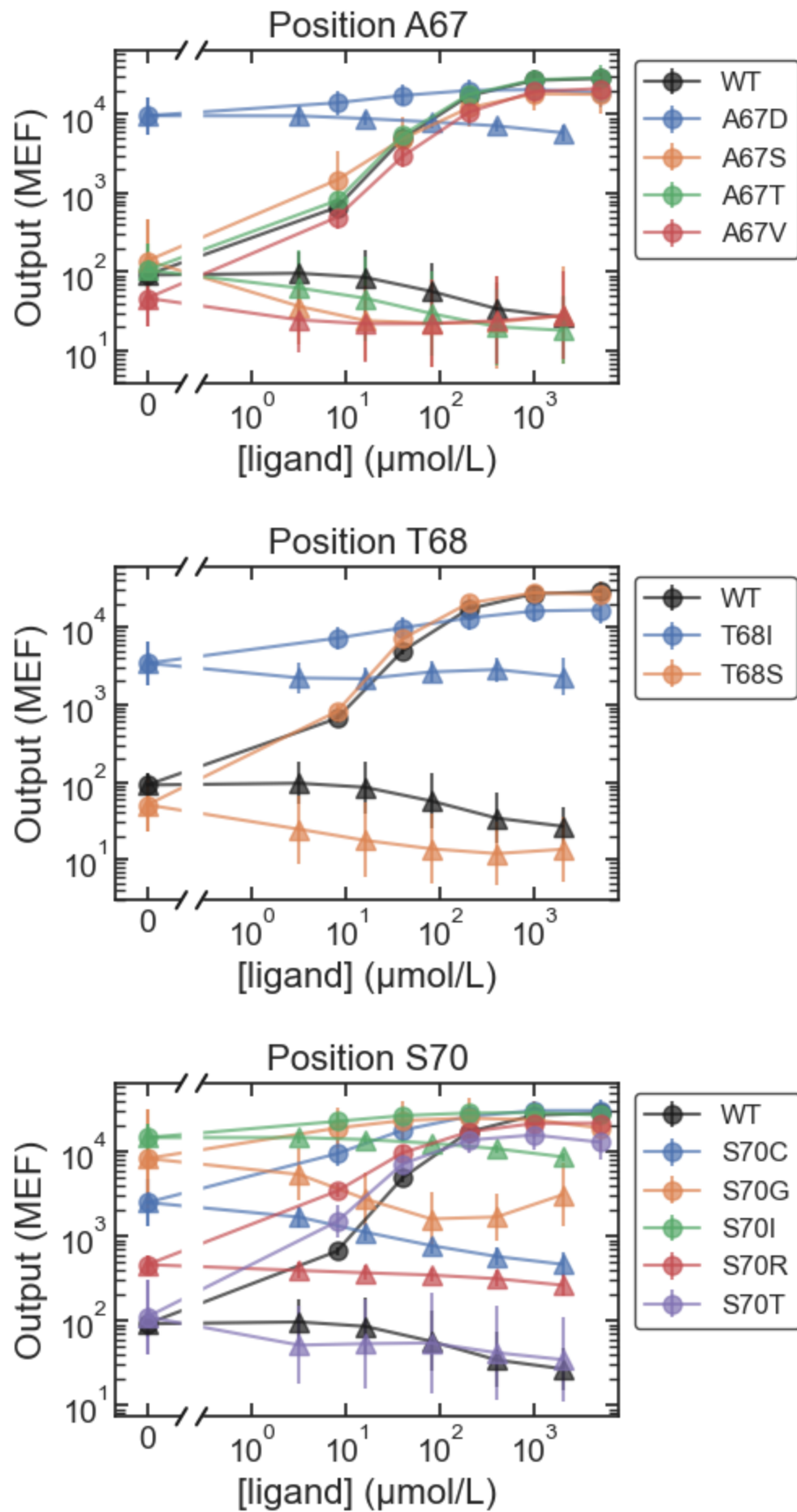


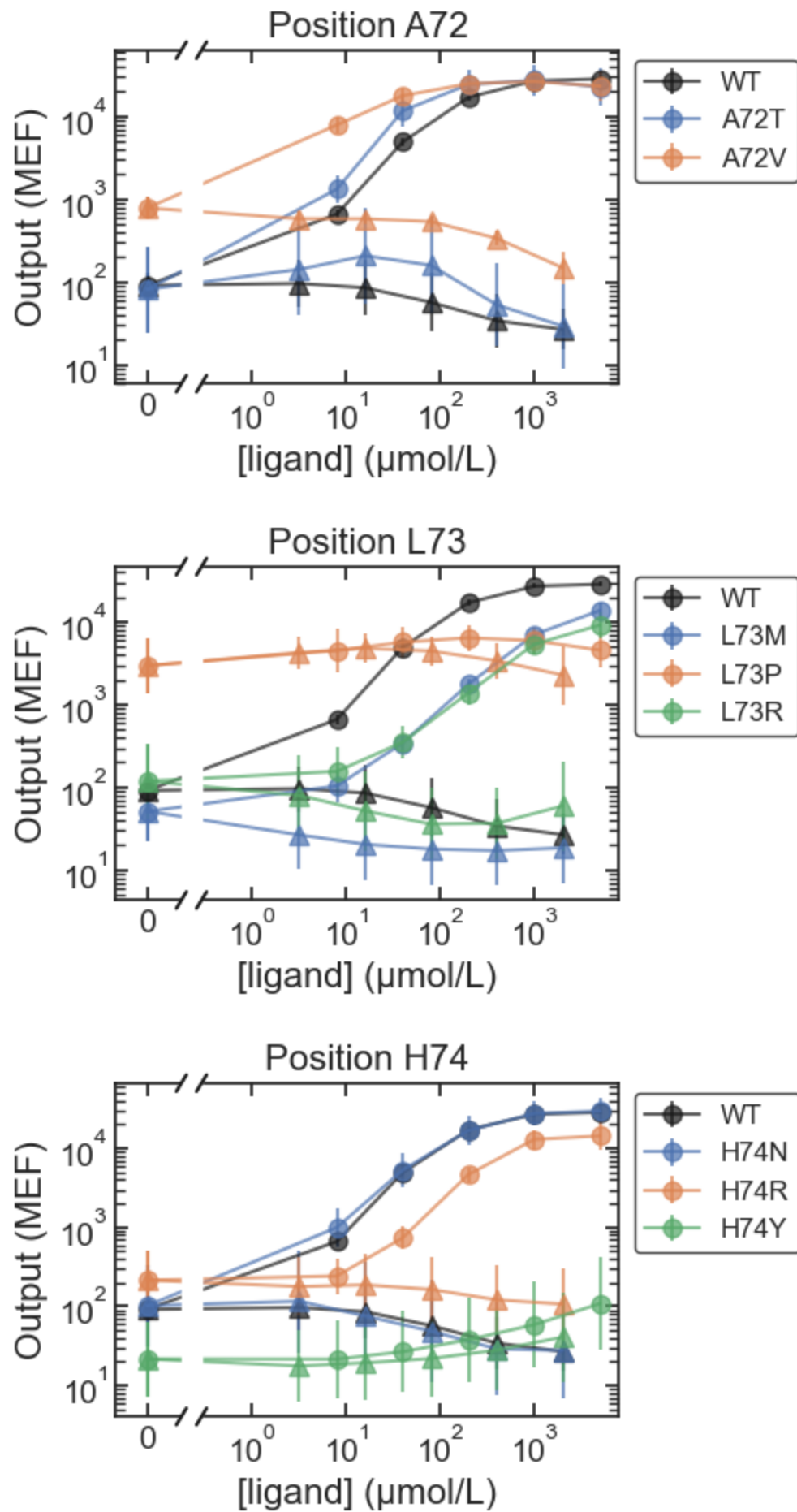


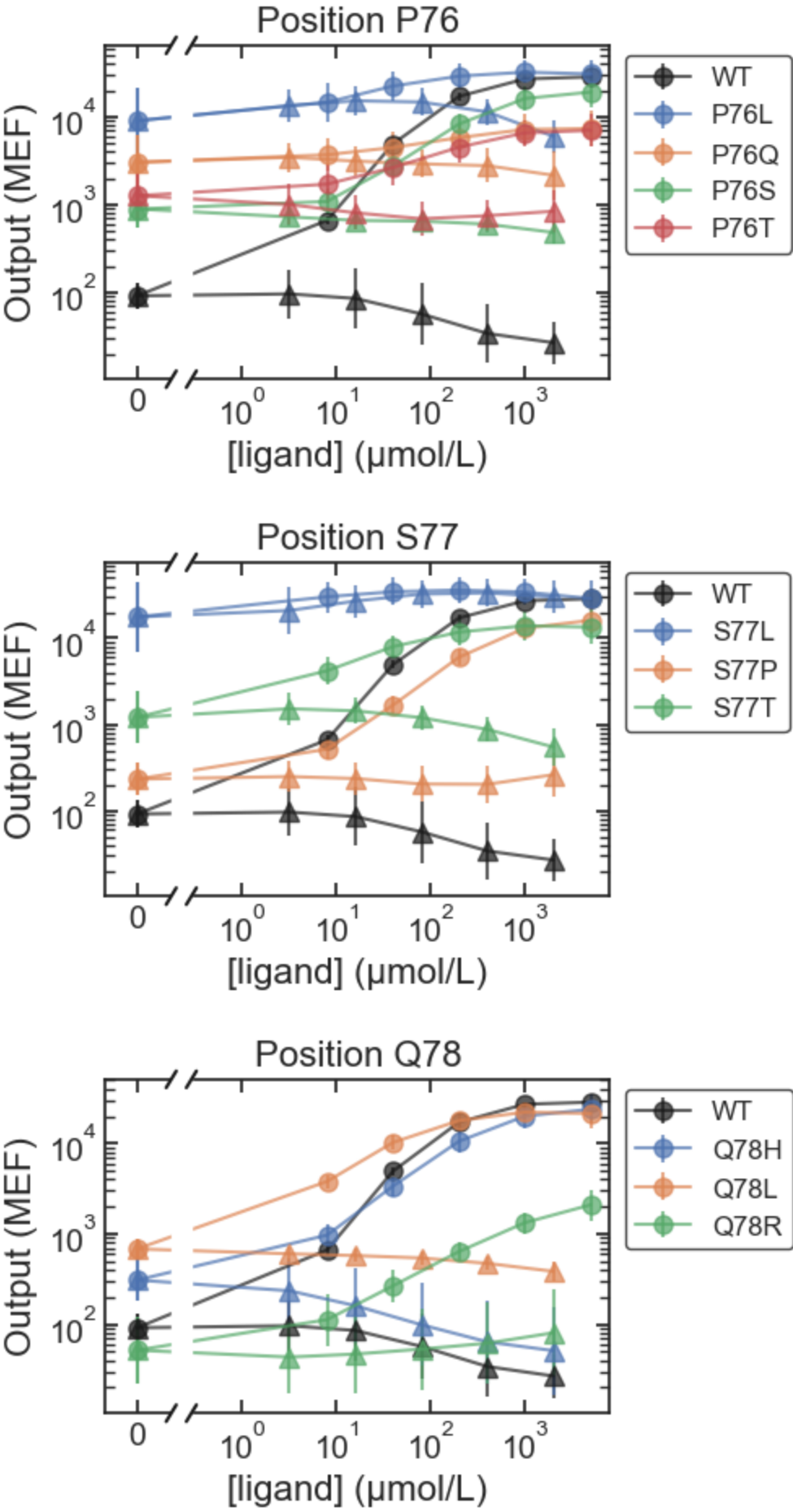


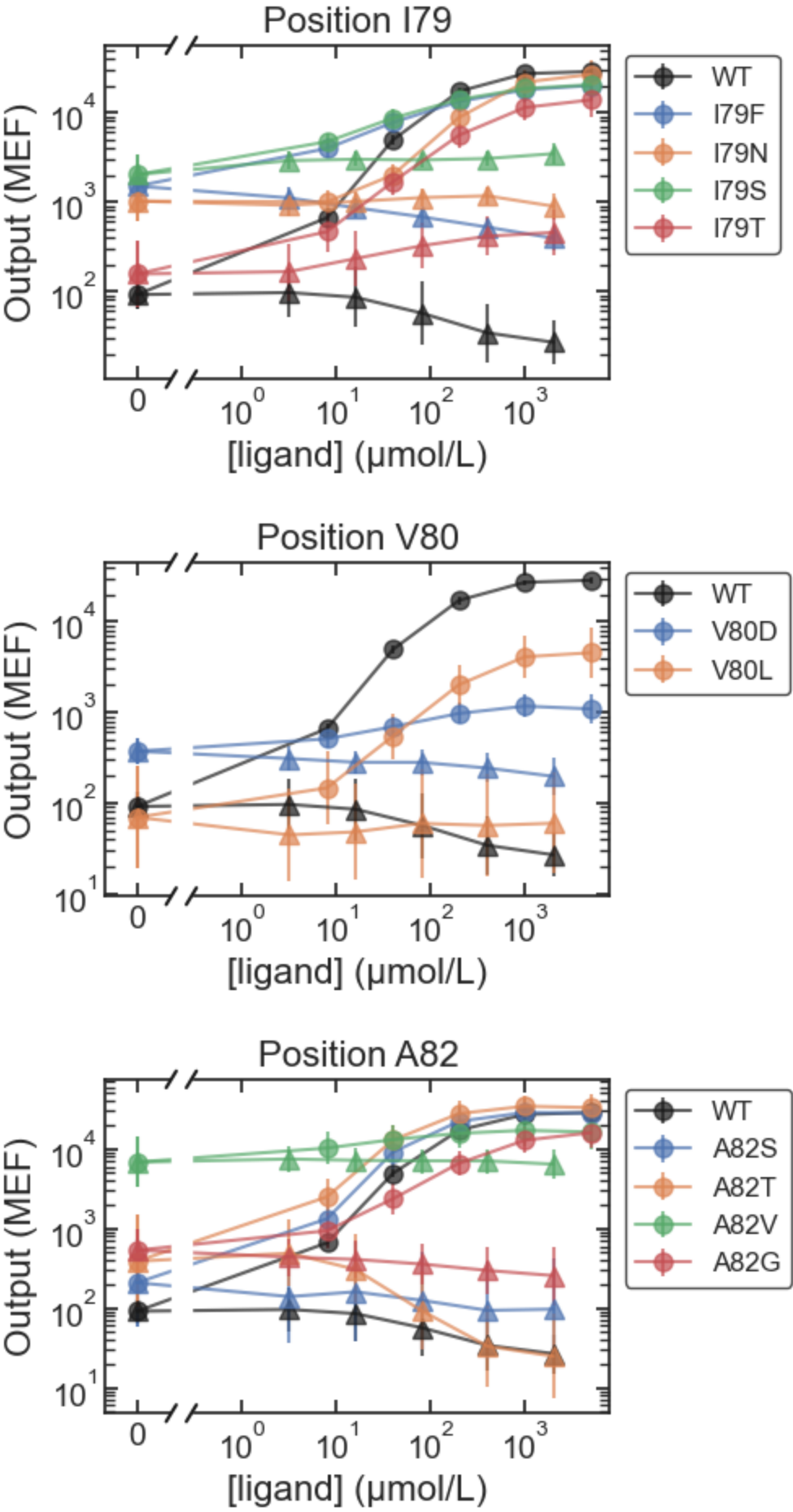
MWC_IPTG

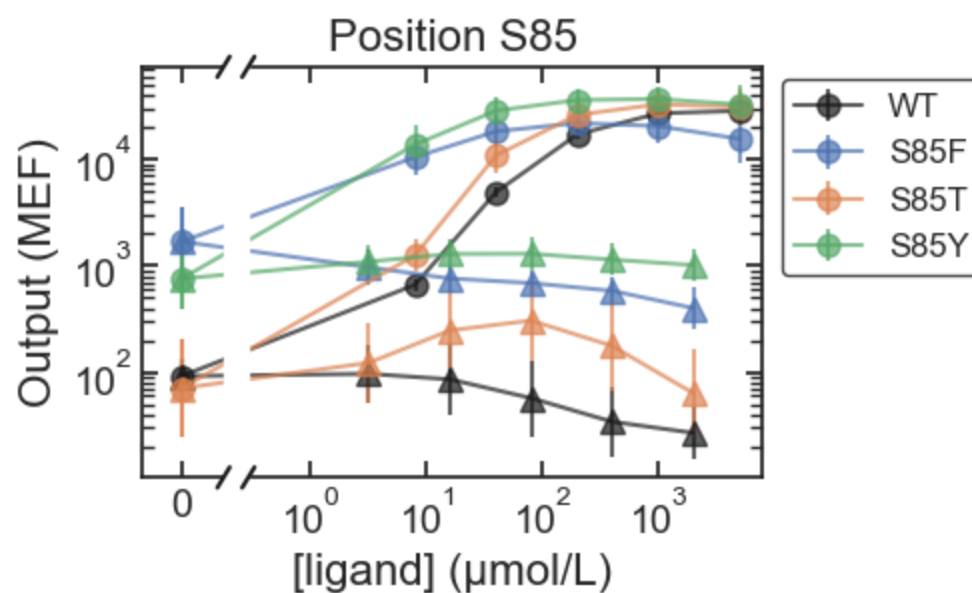
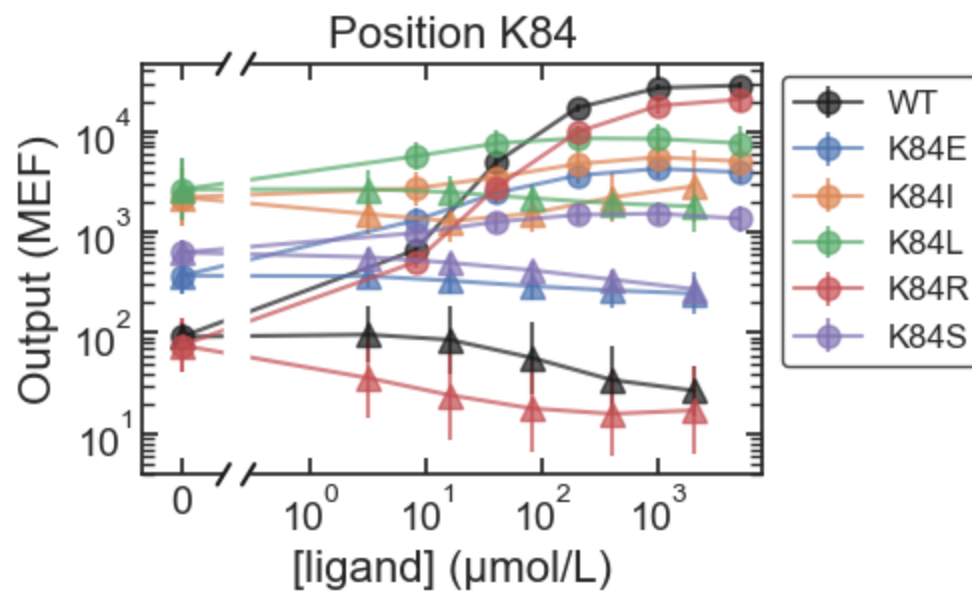
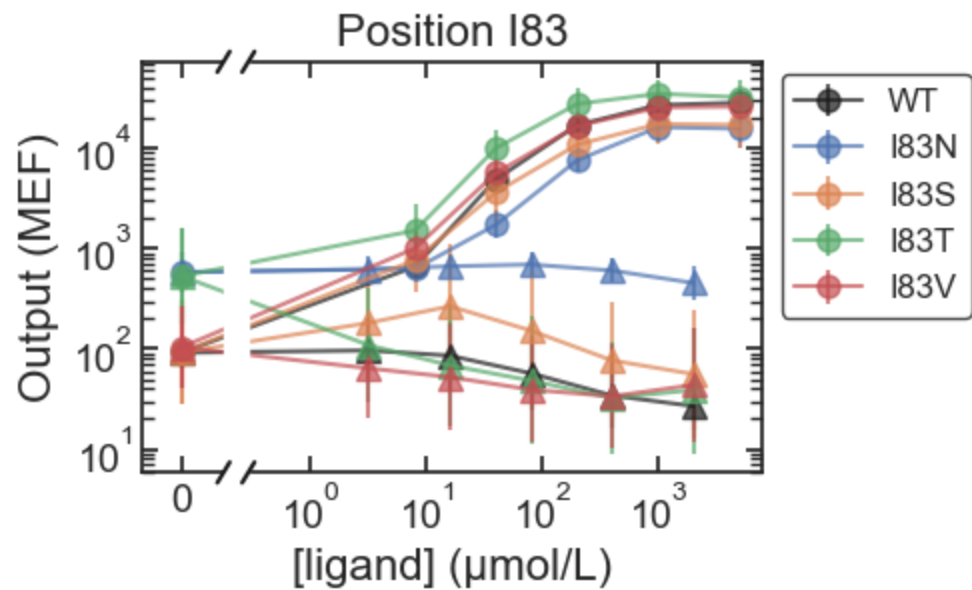


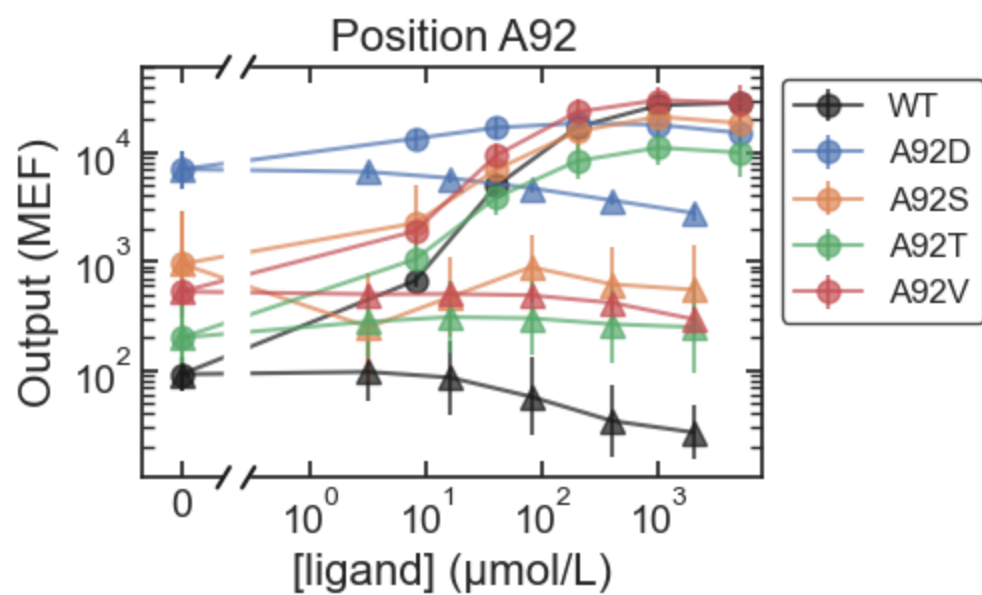
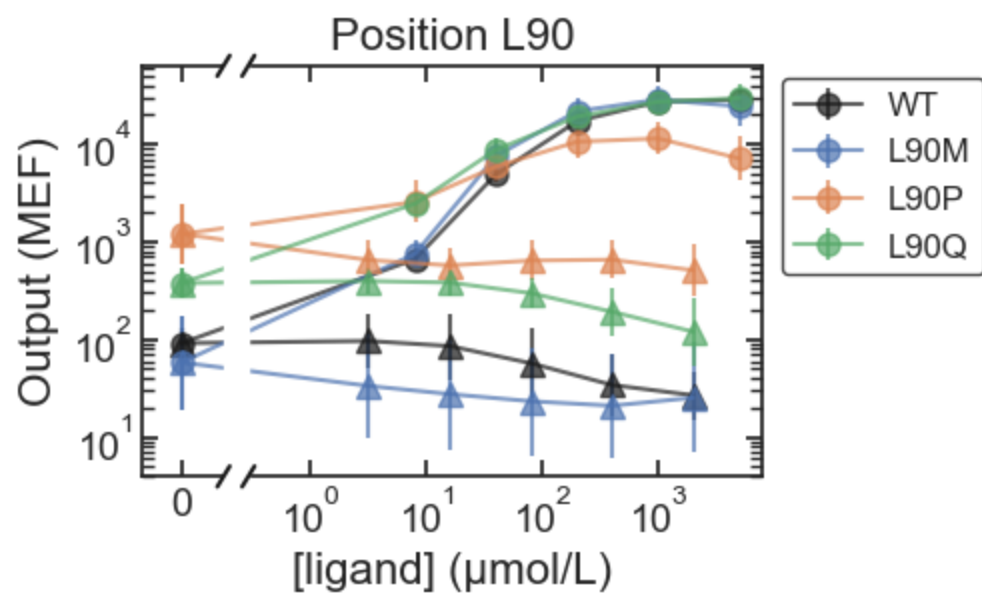
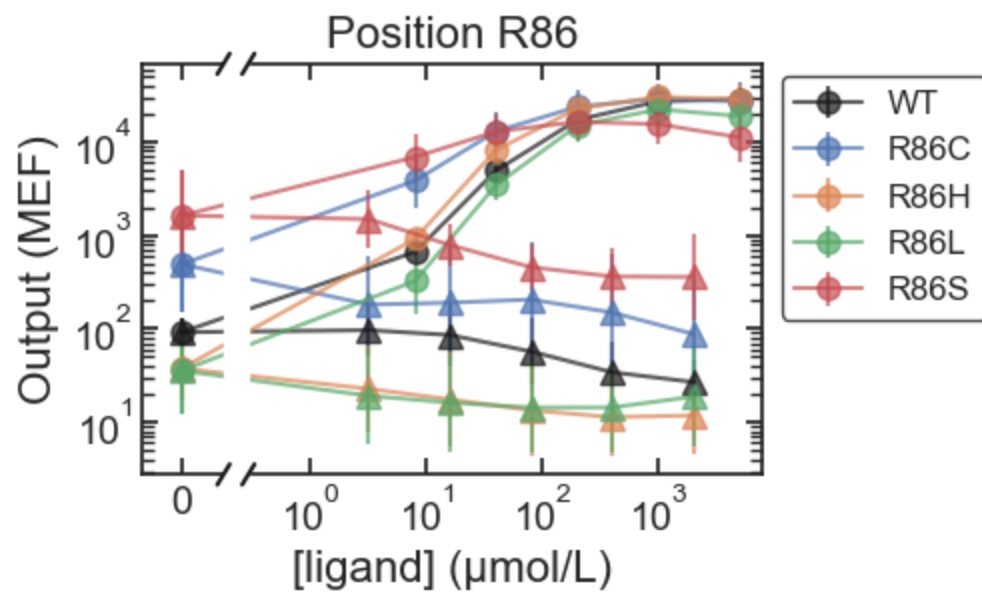


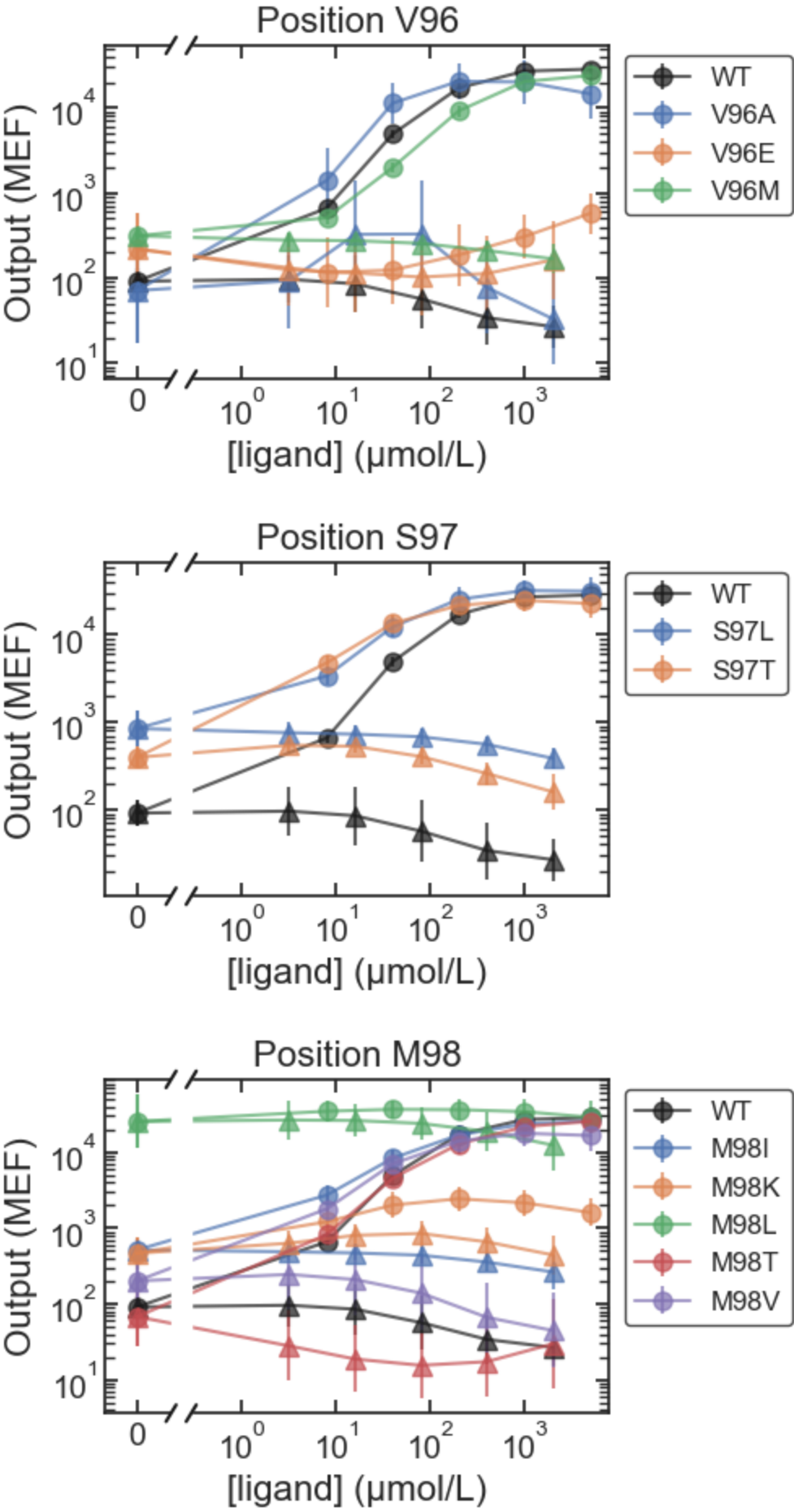


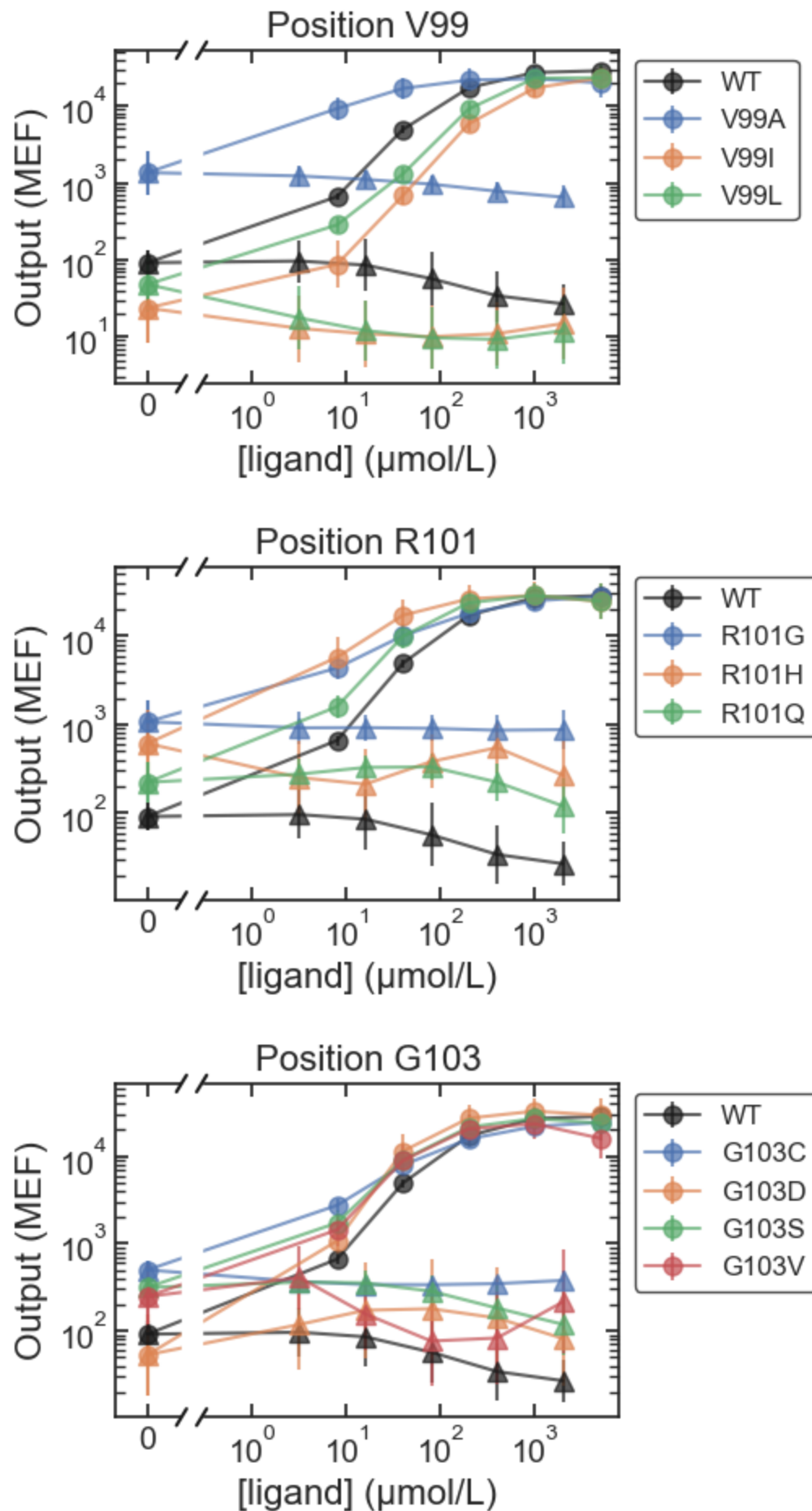


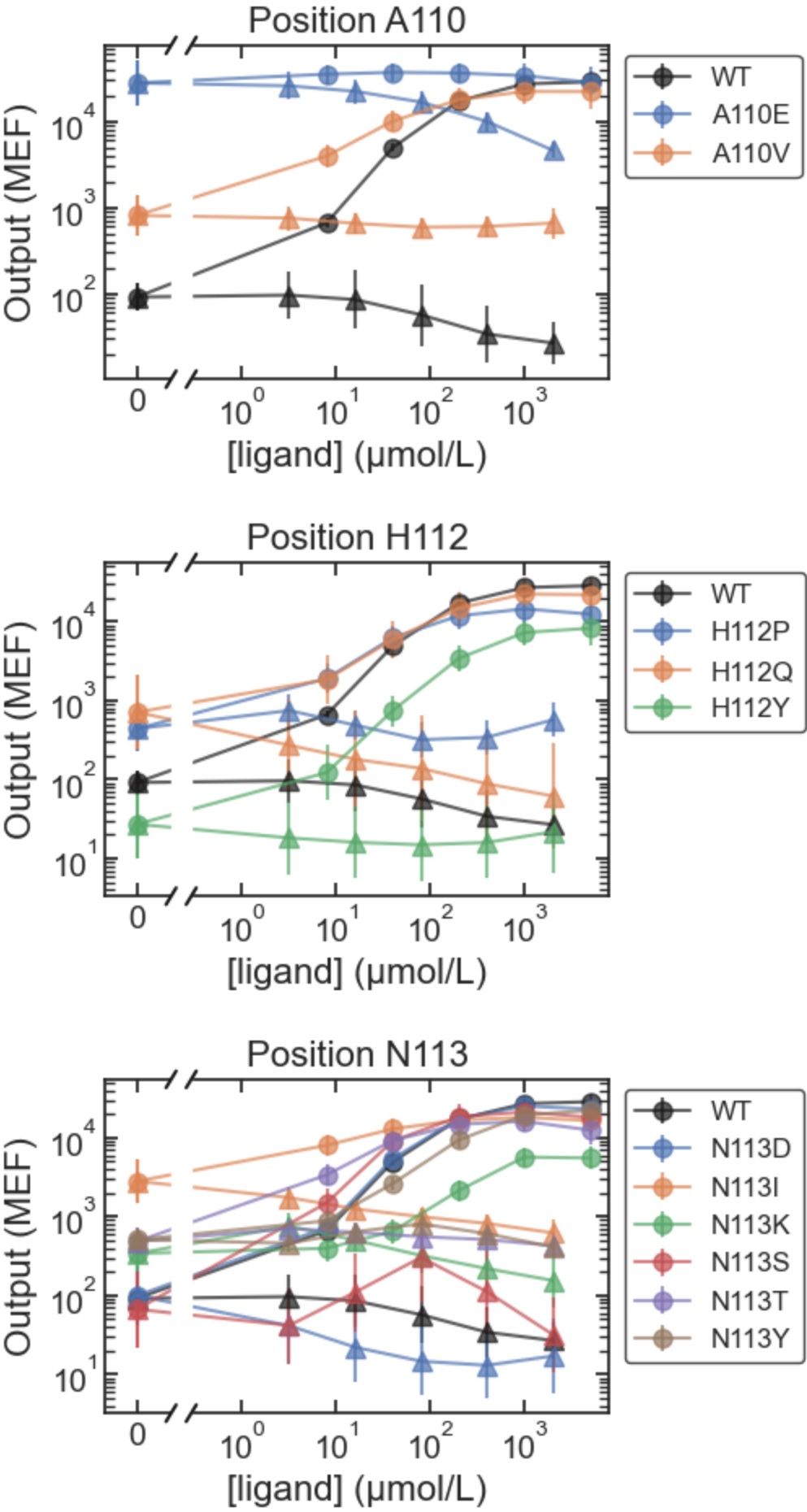


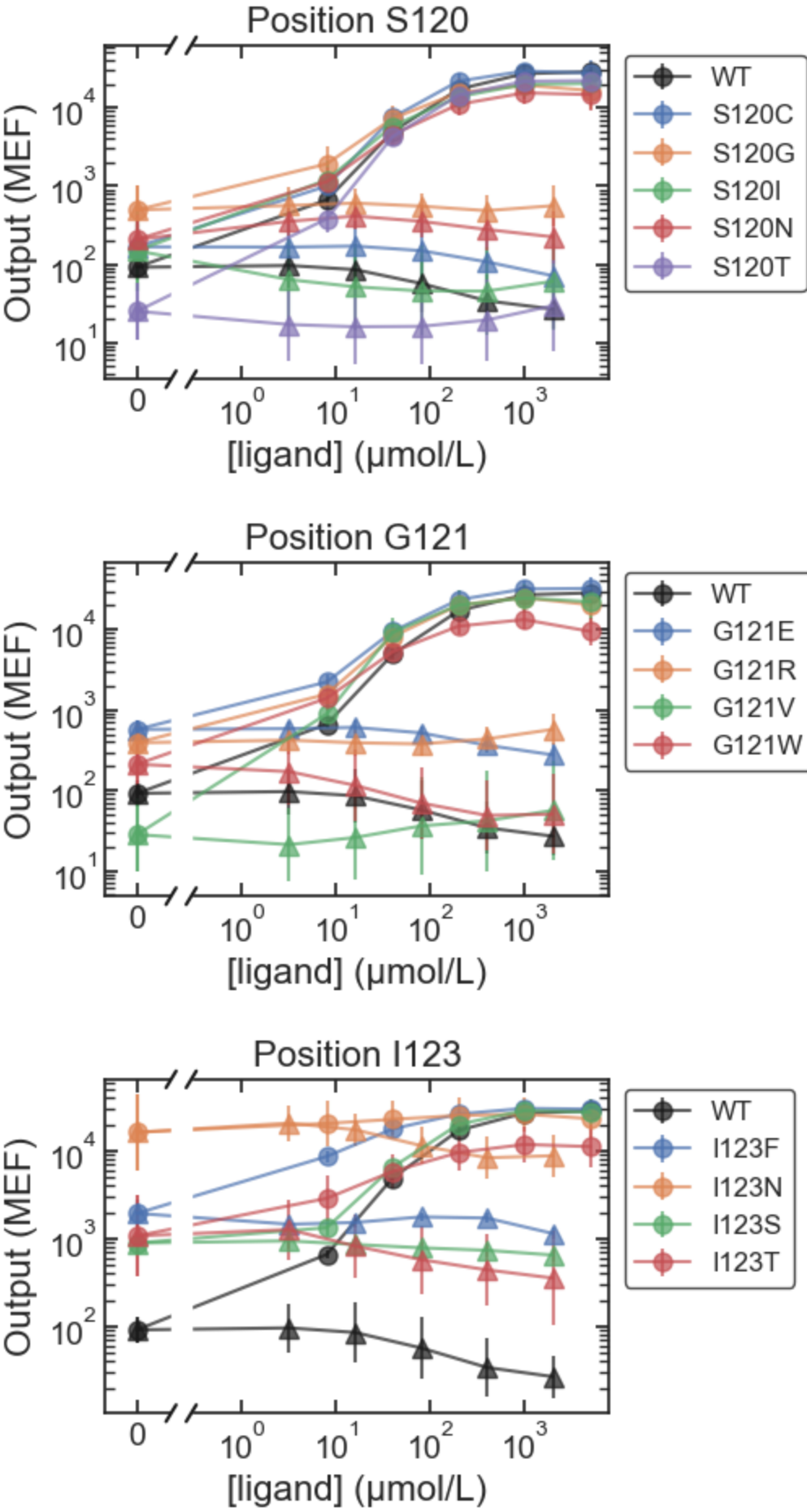


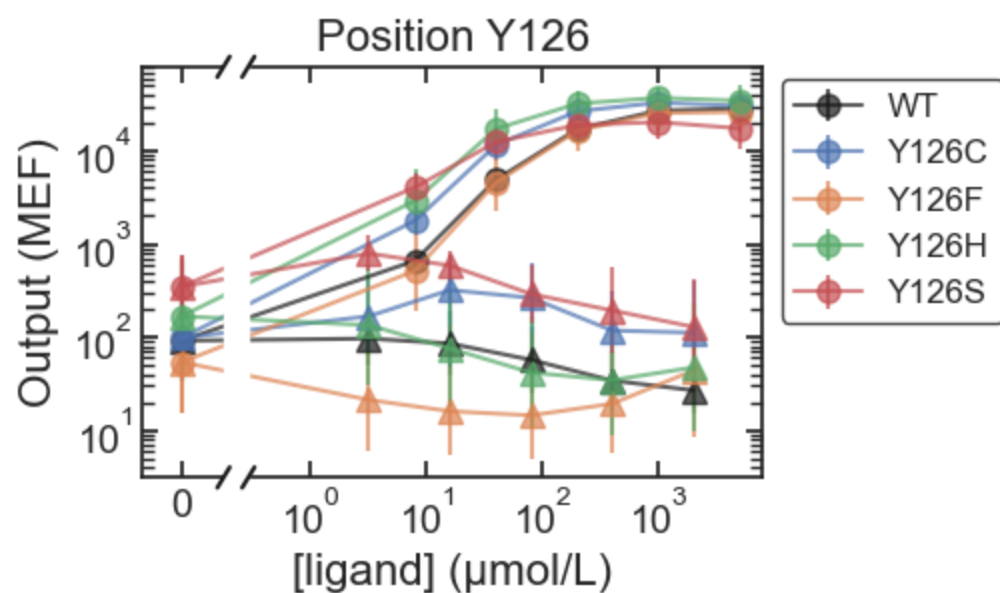
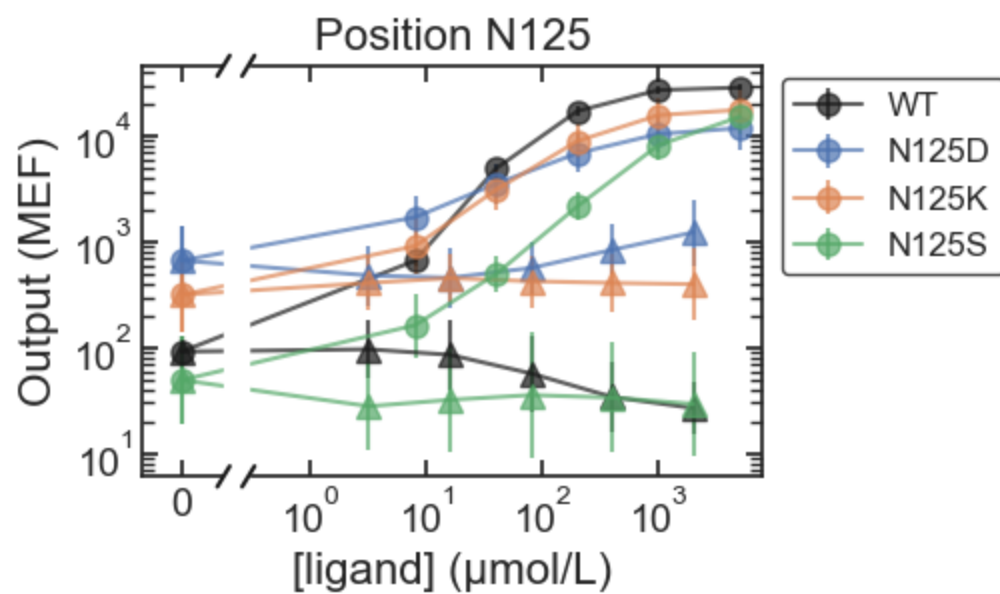
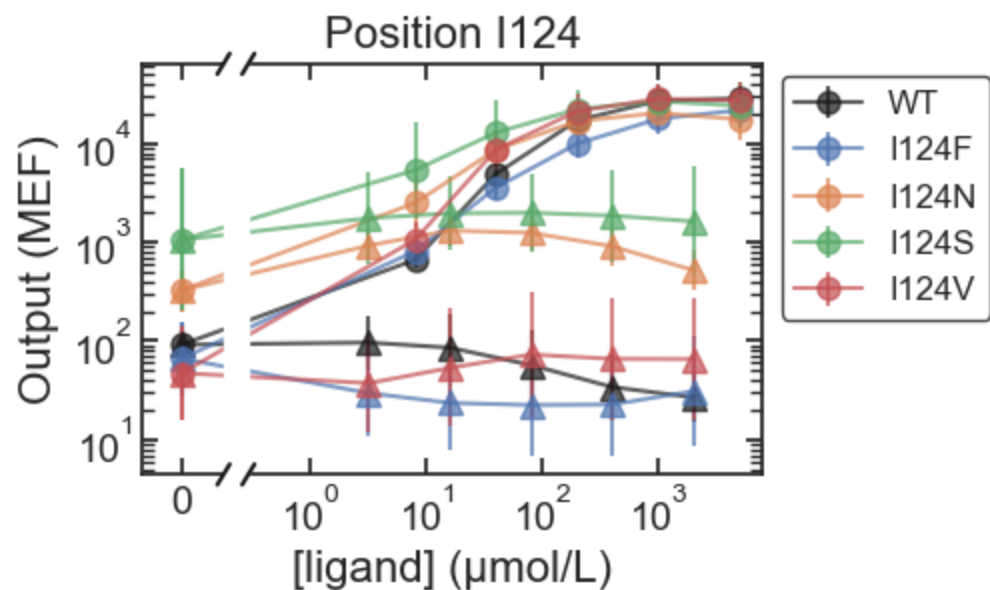


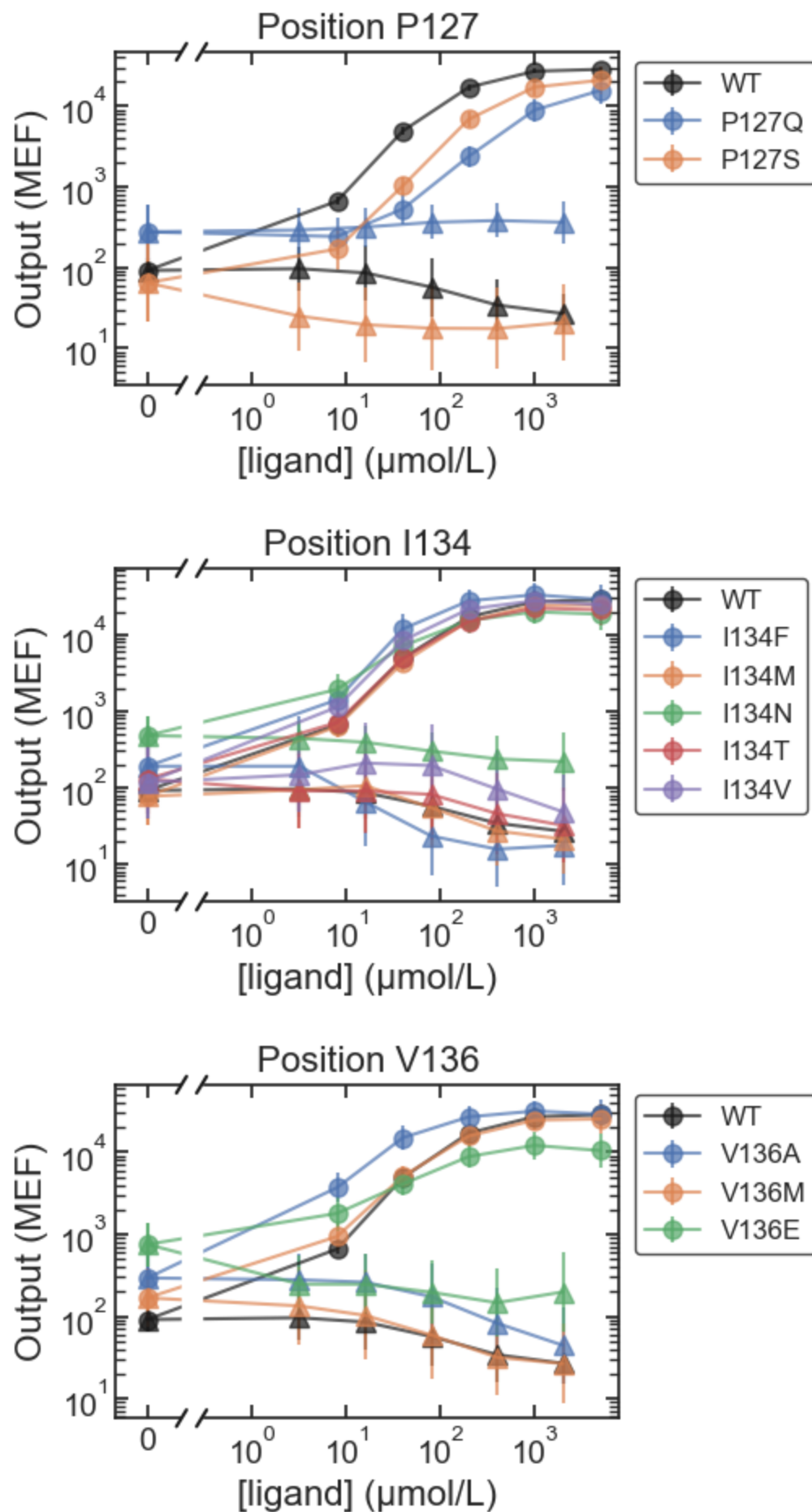


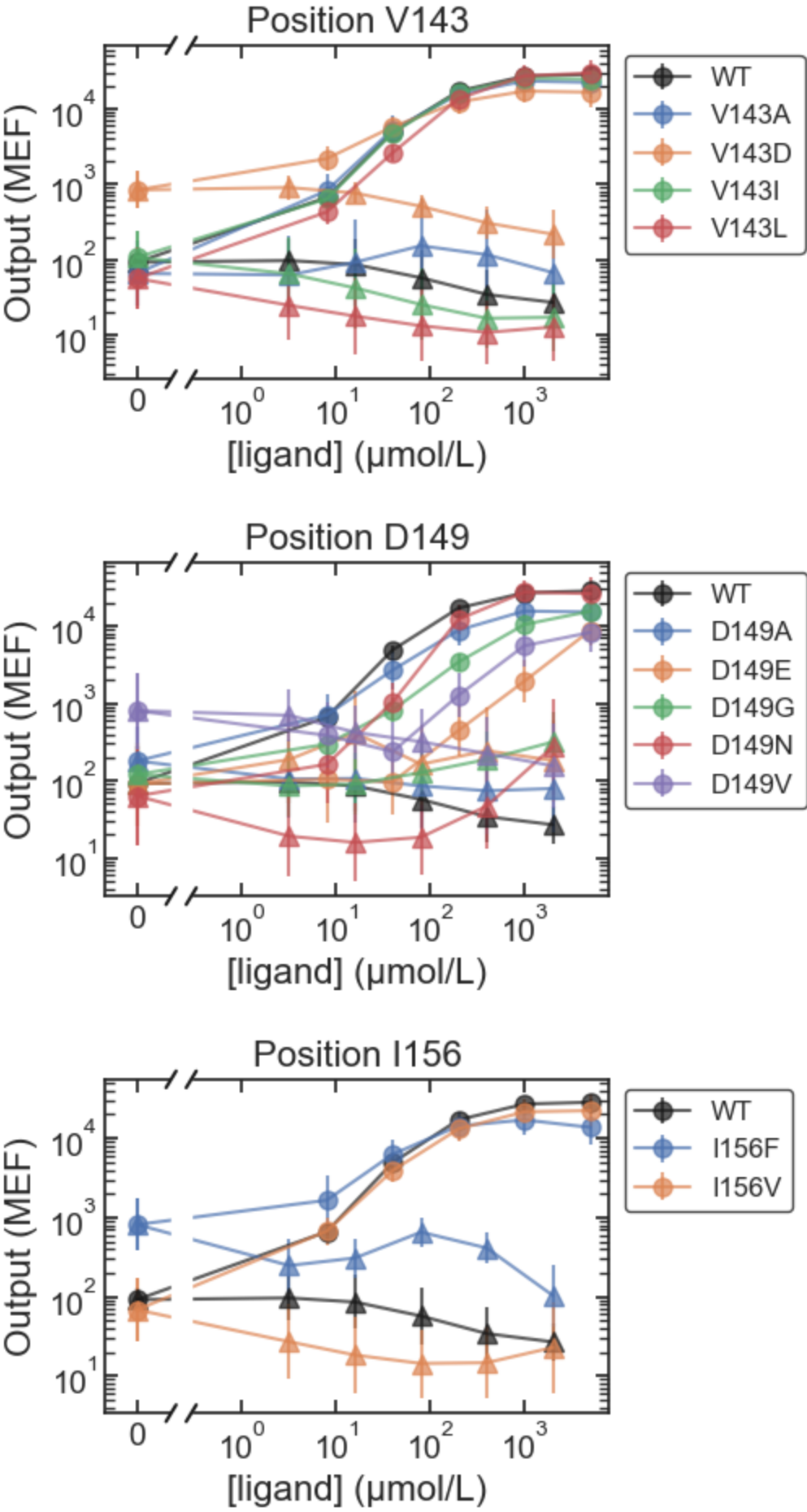


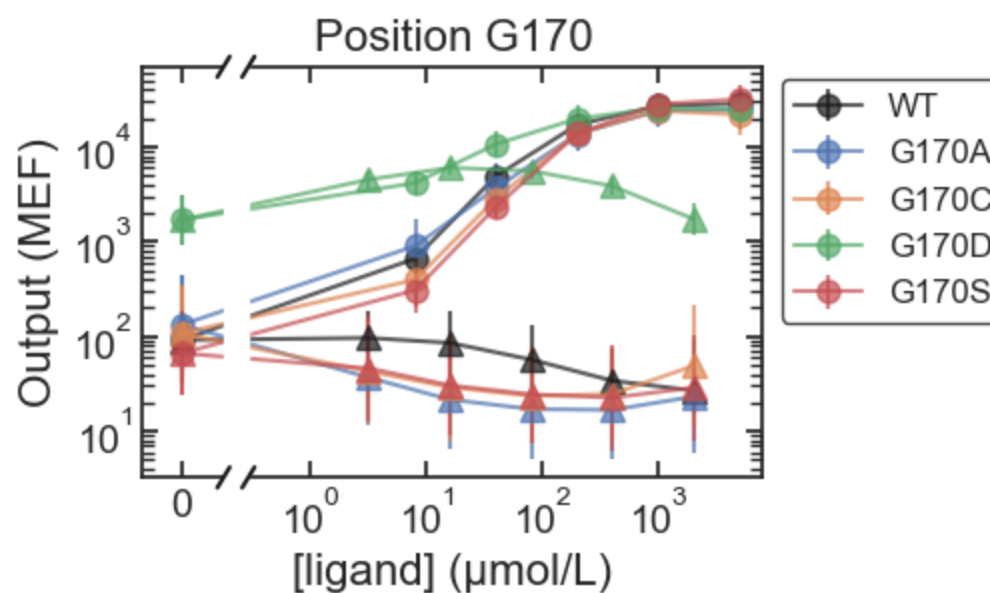
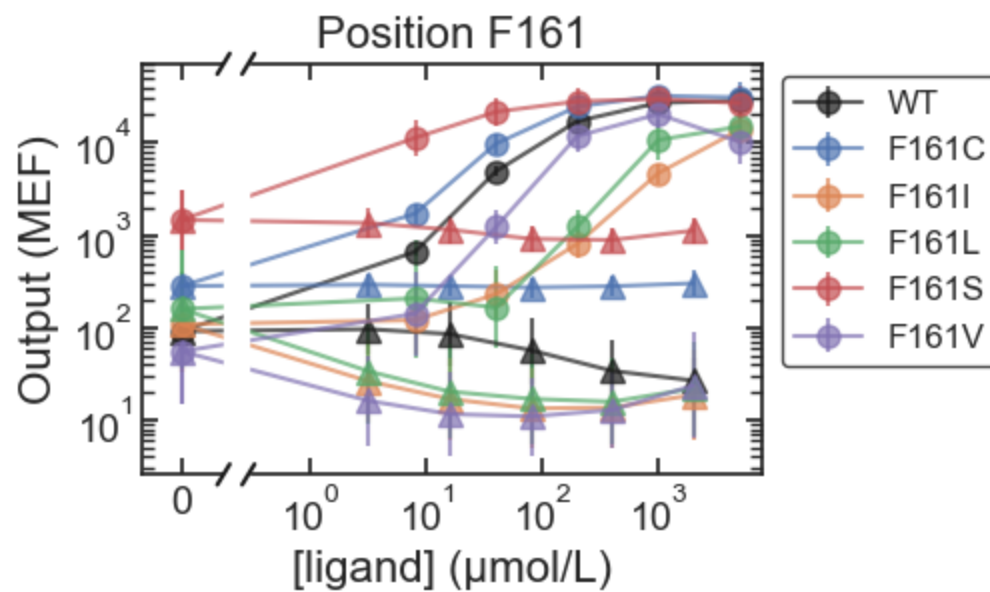
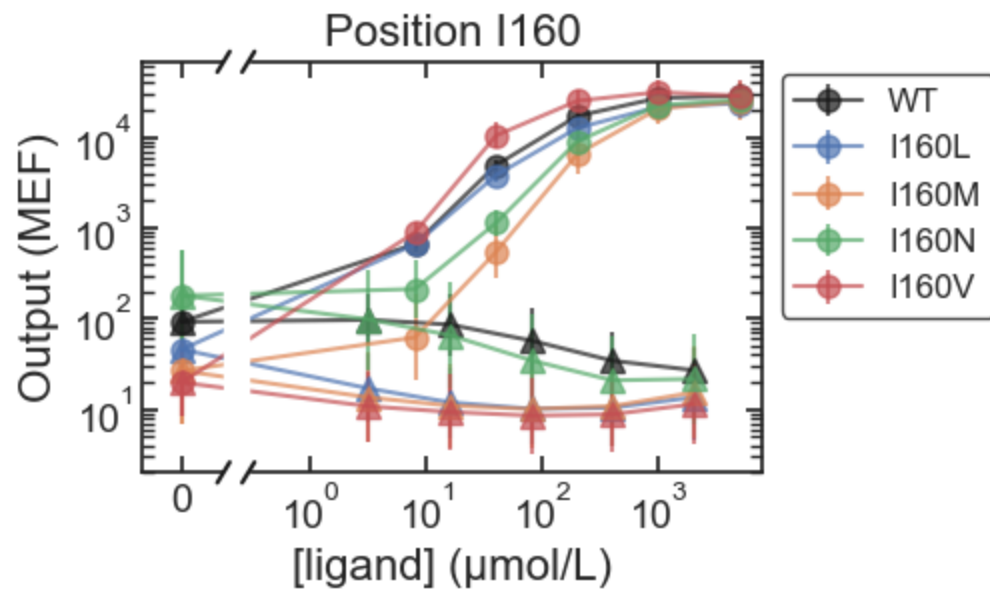


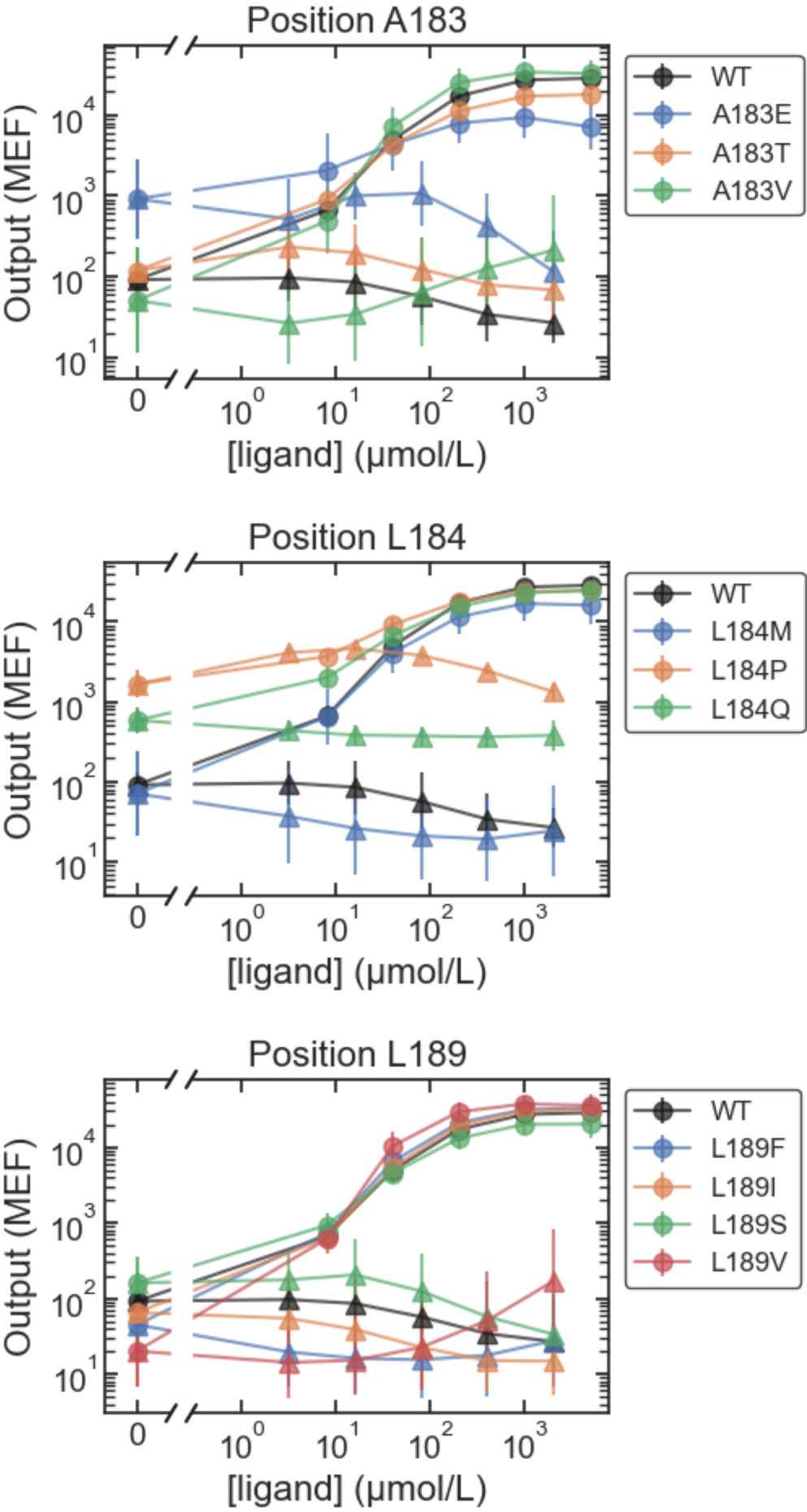


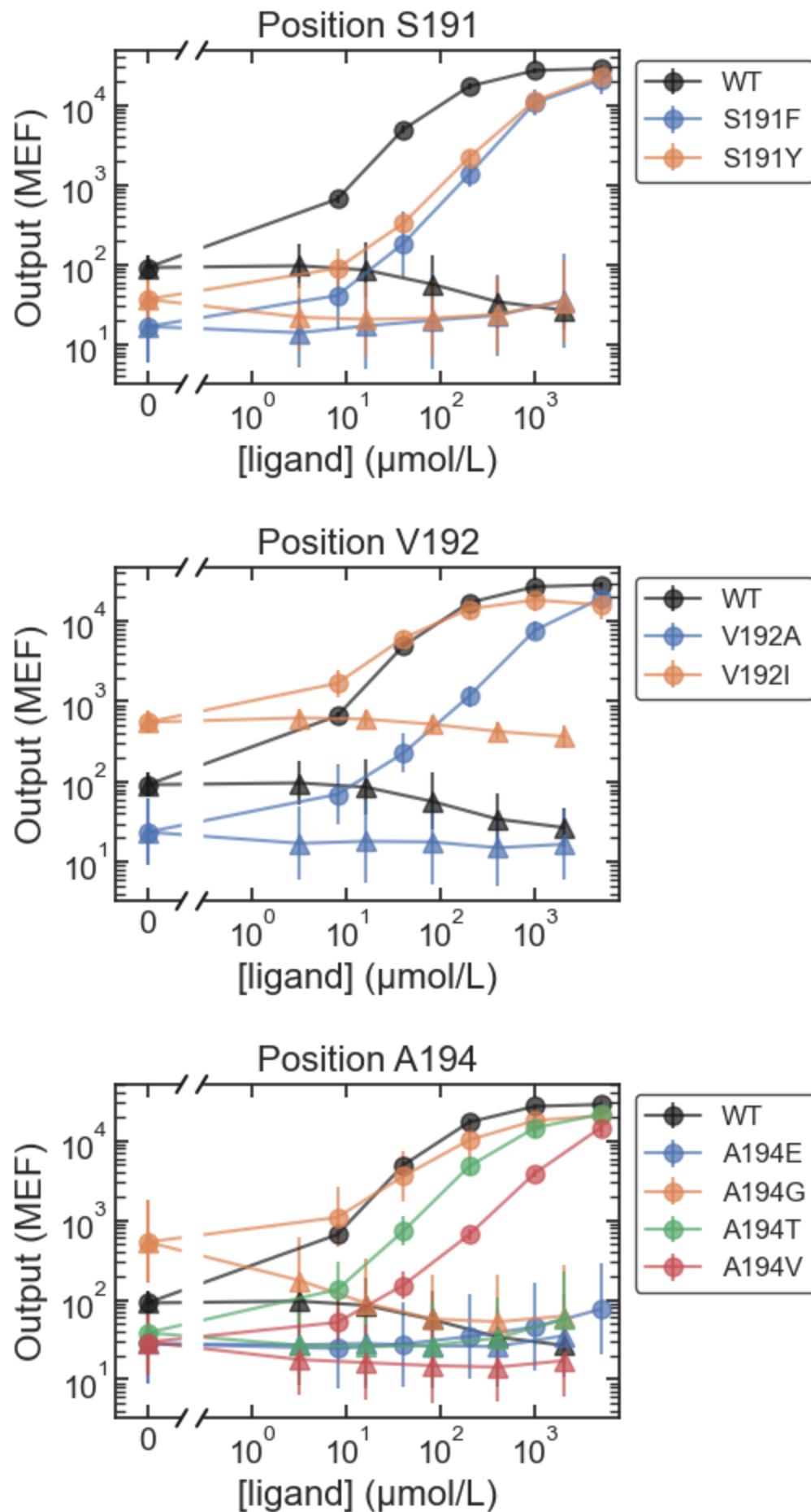


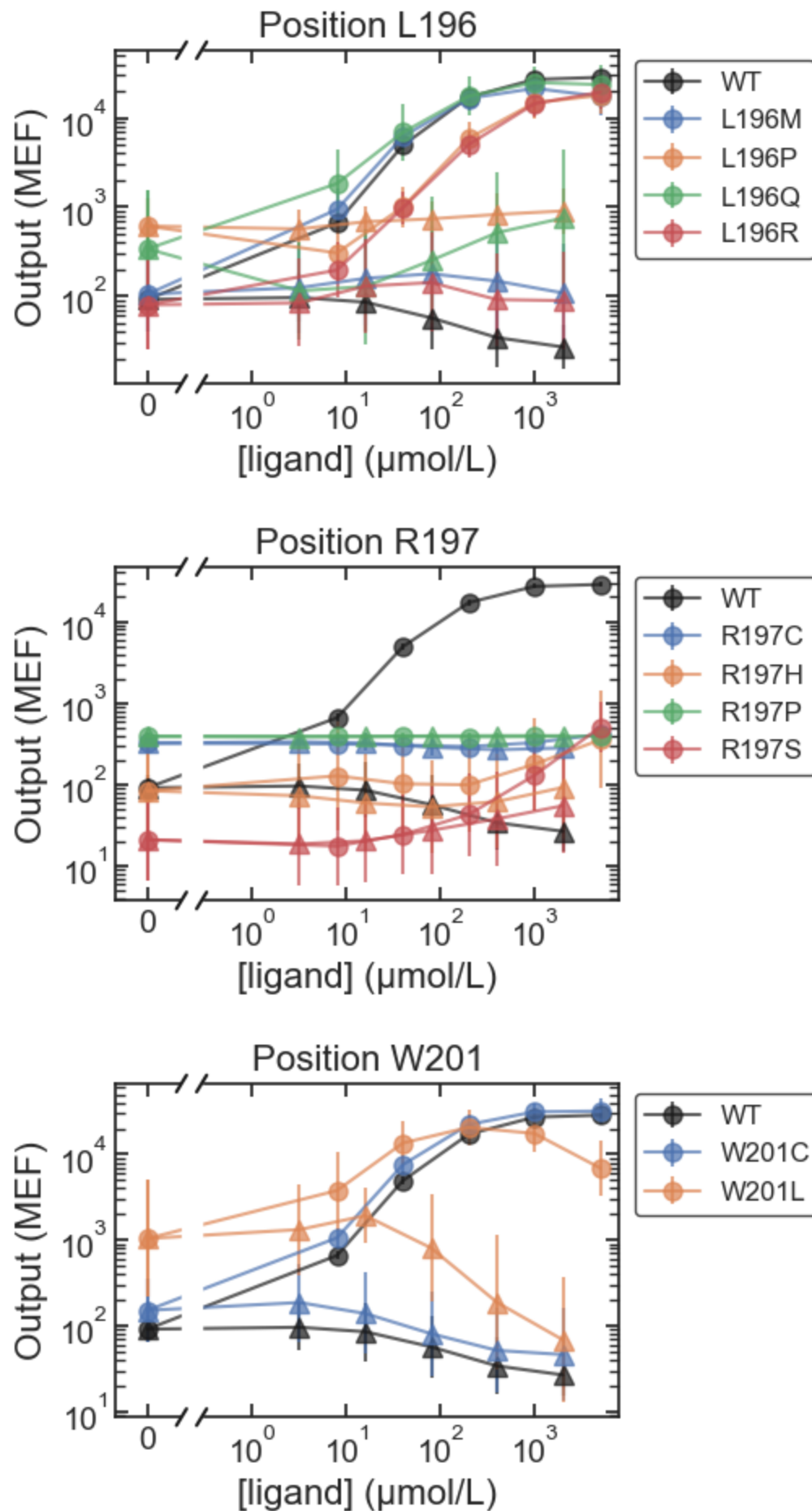


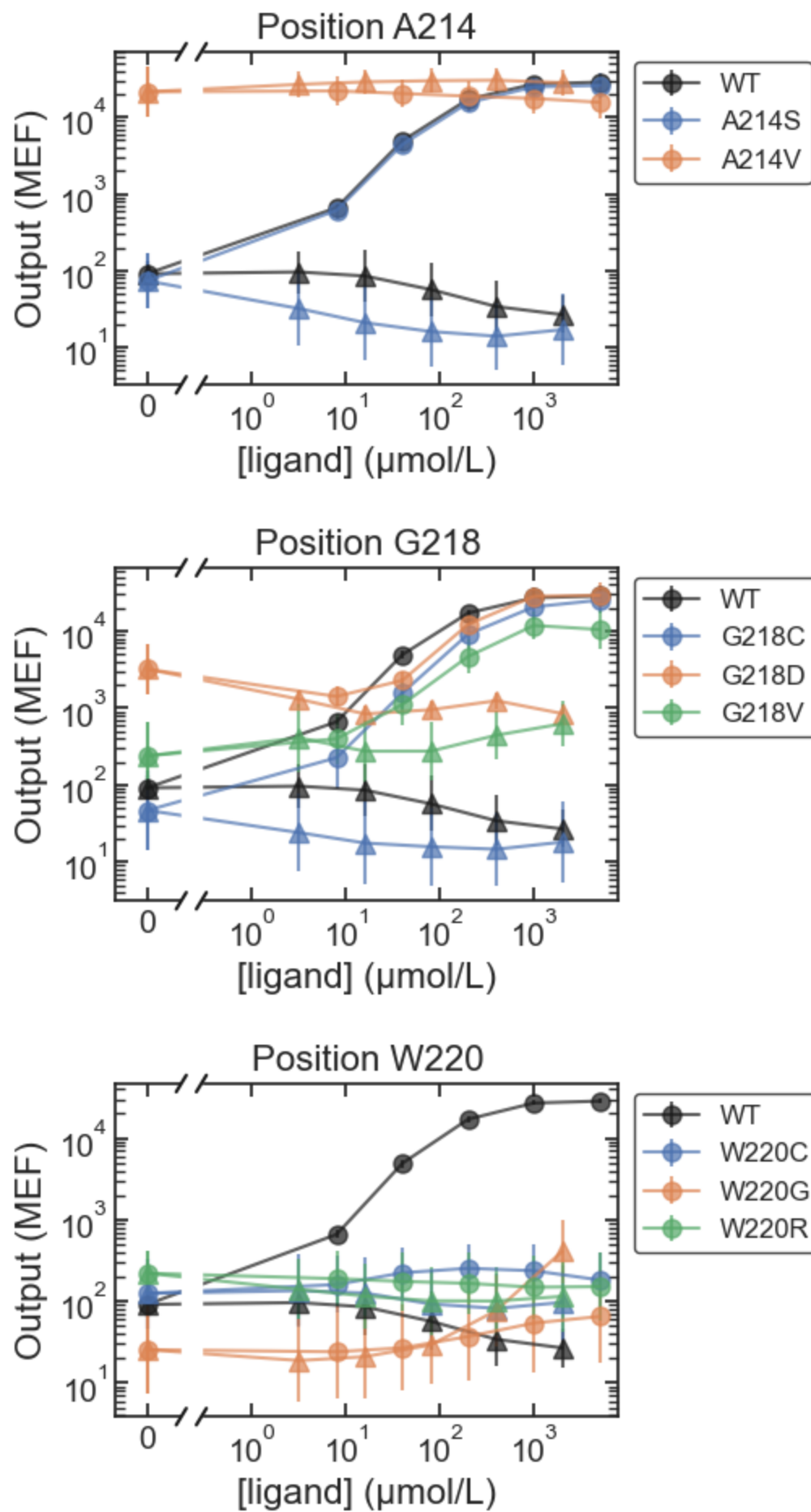


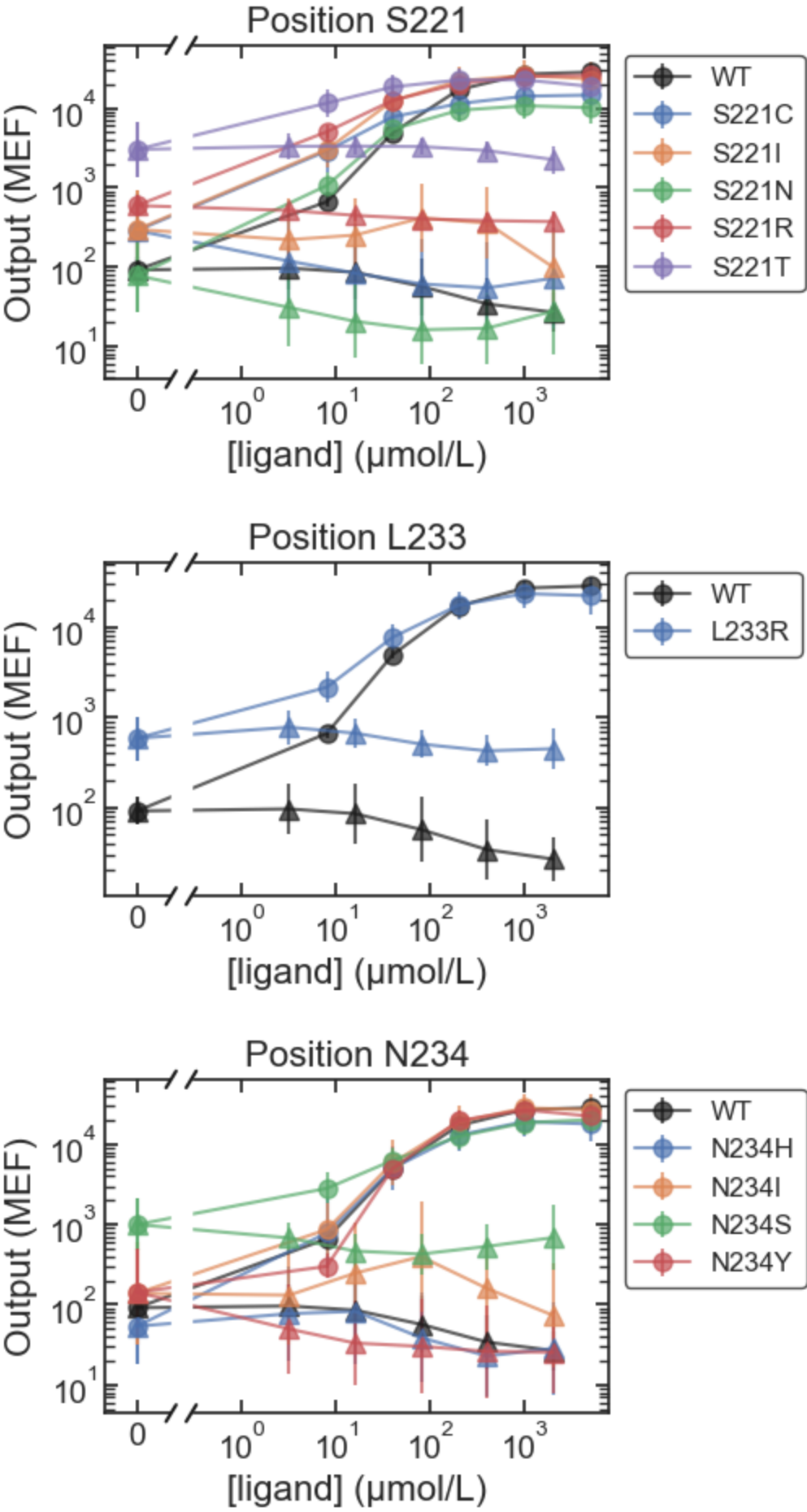


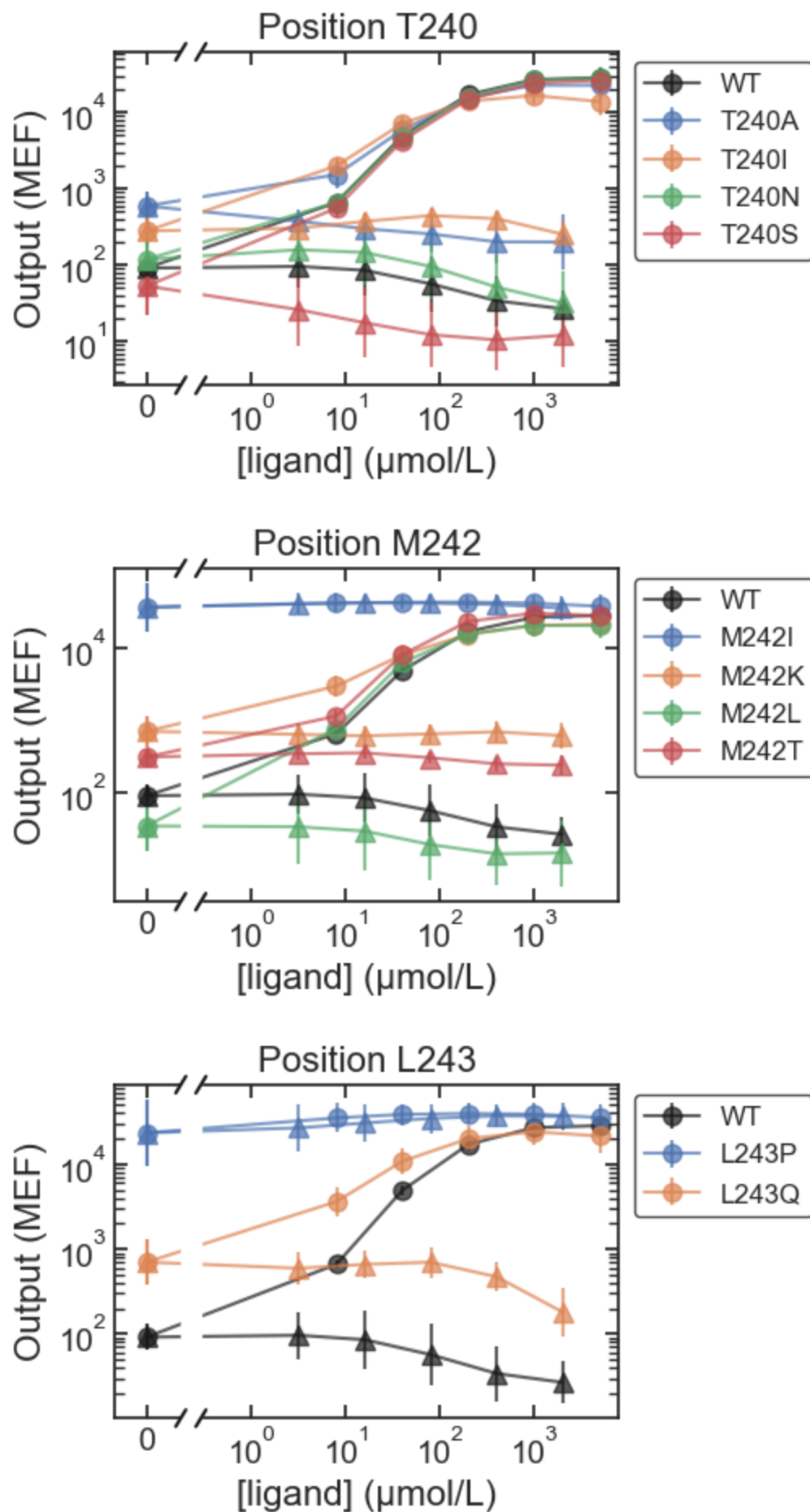


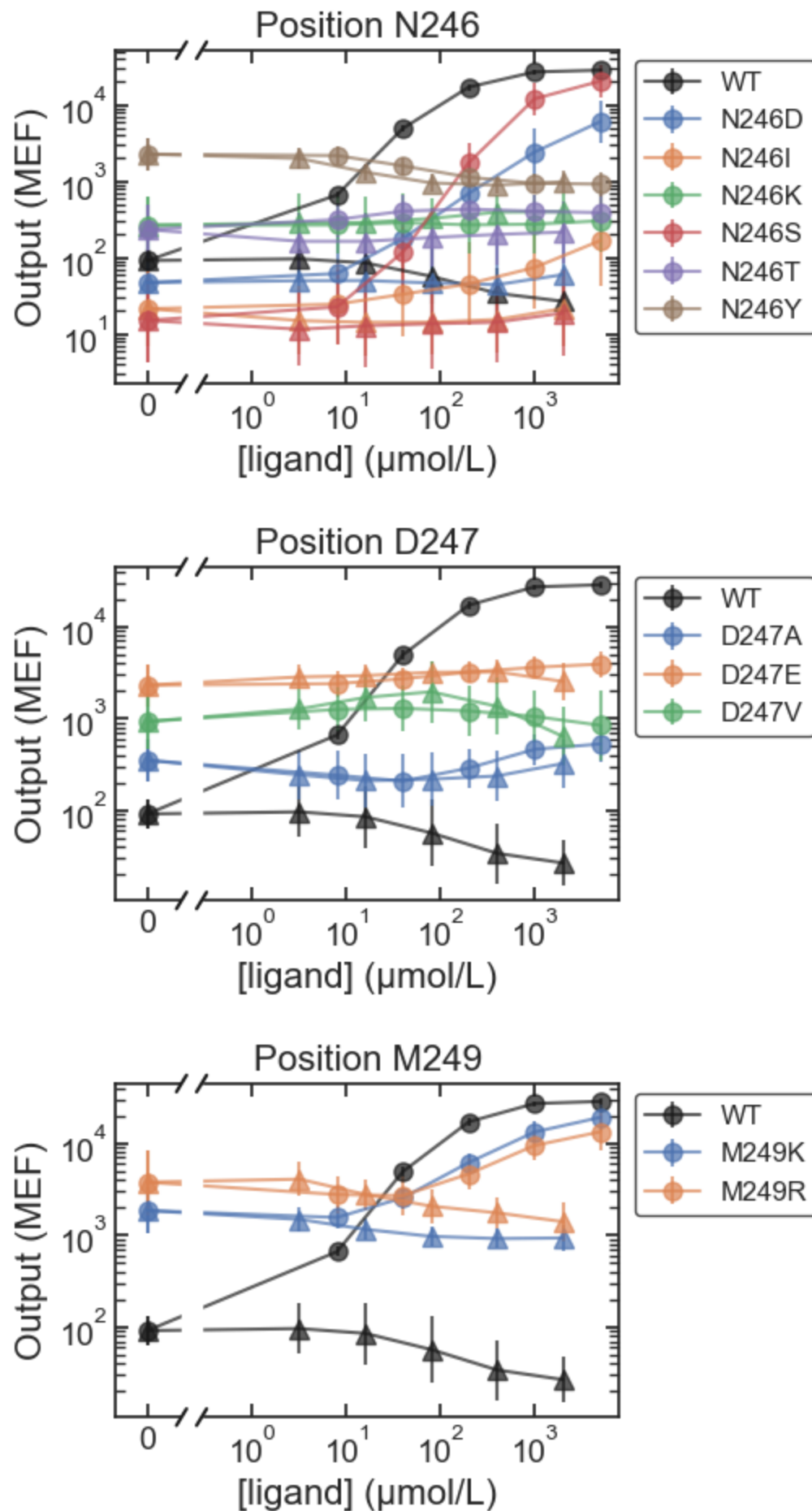


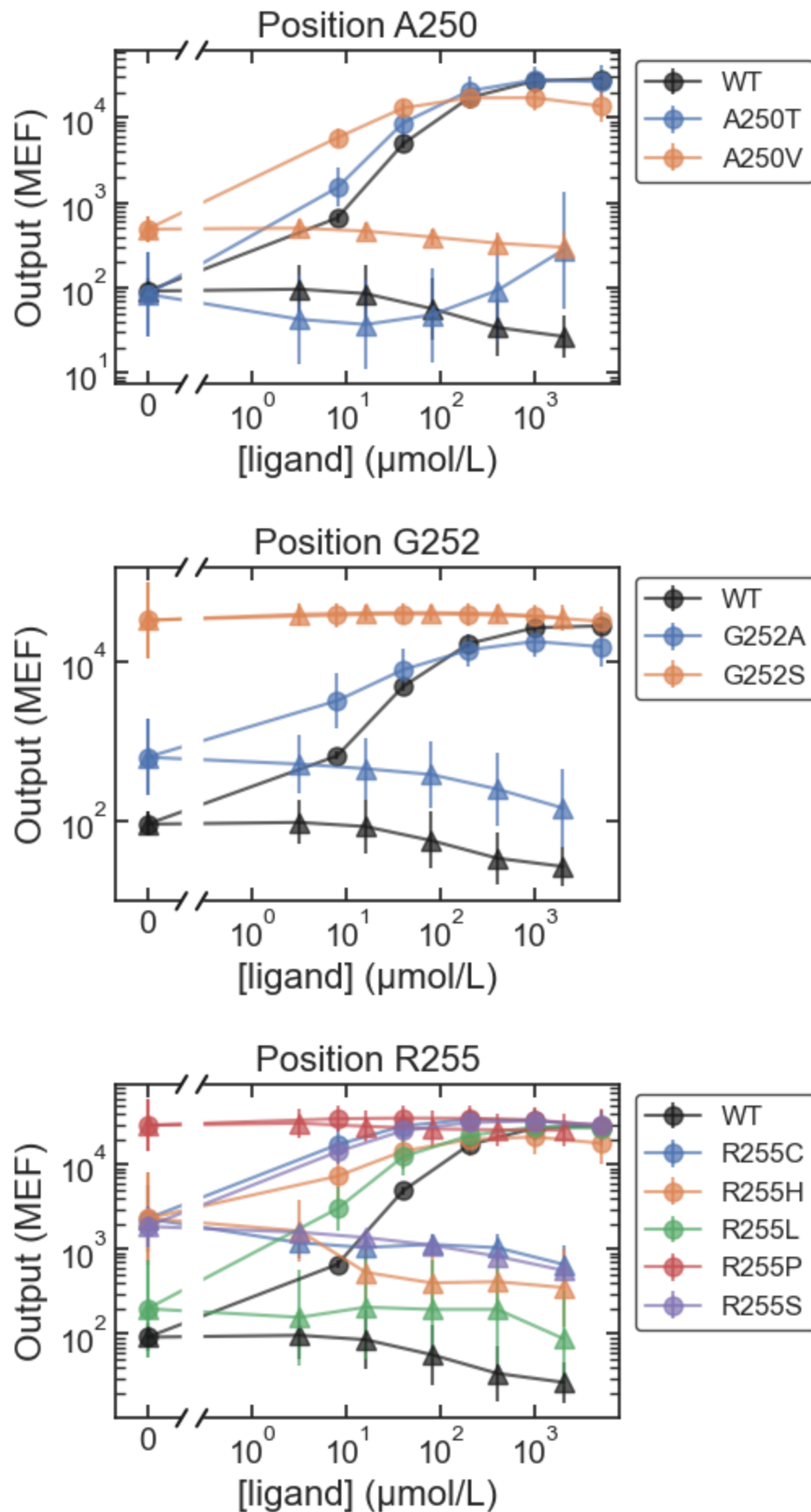


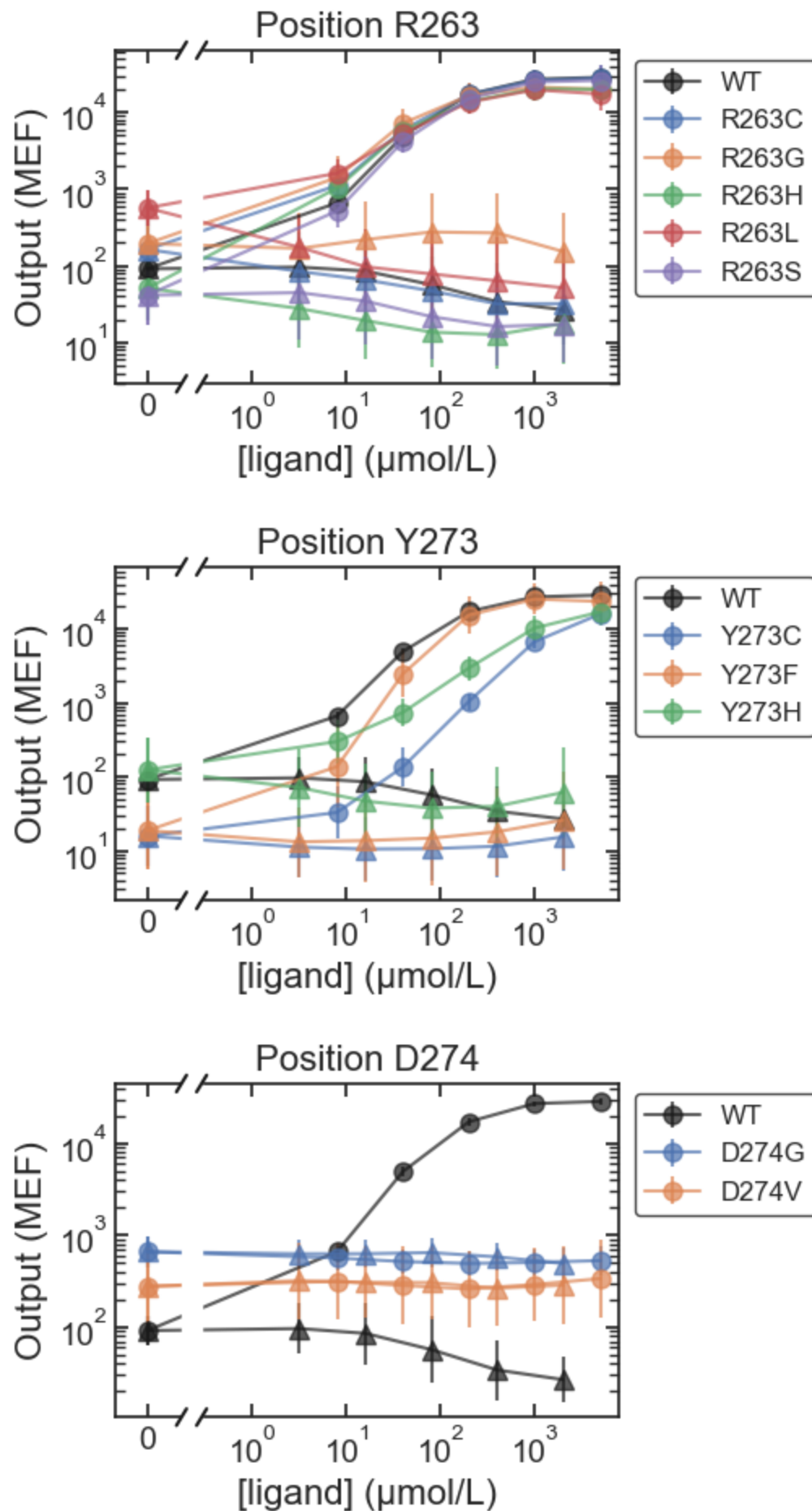


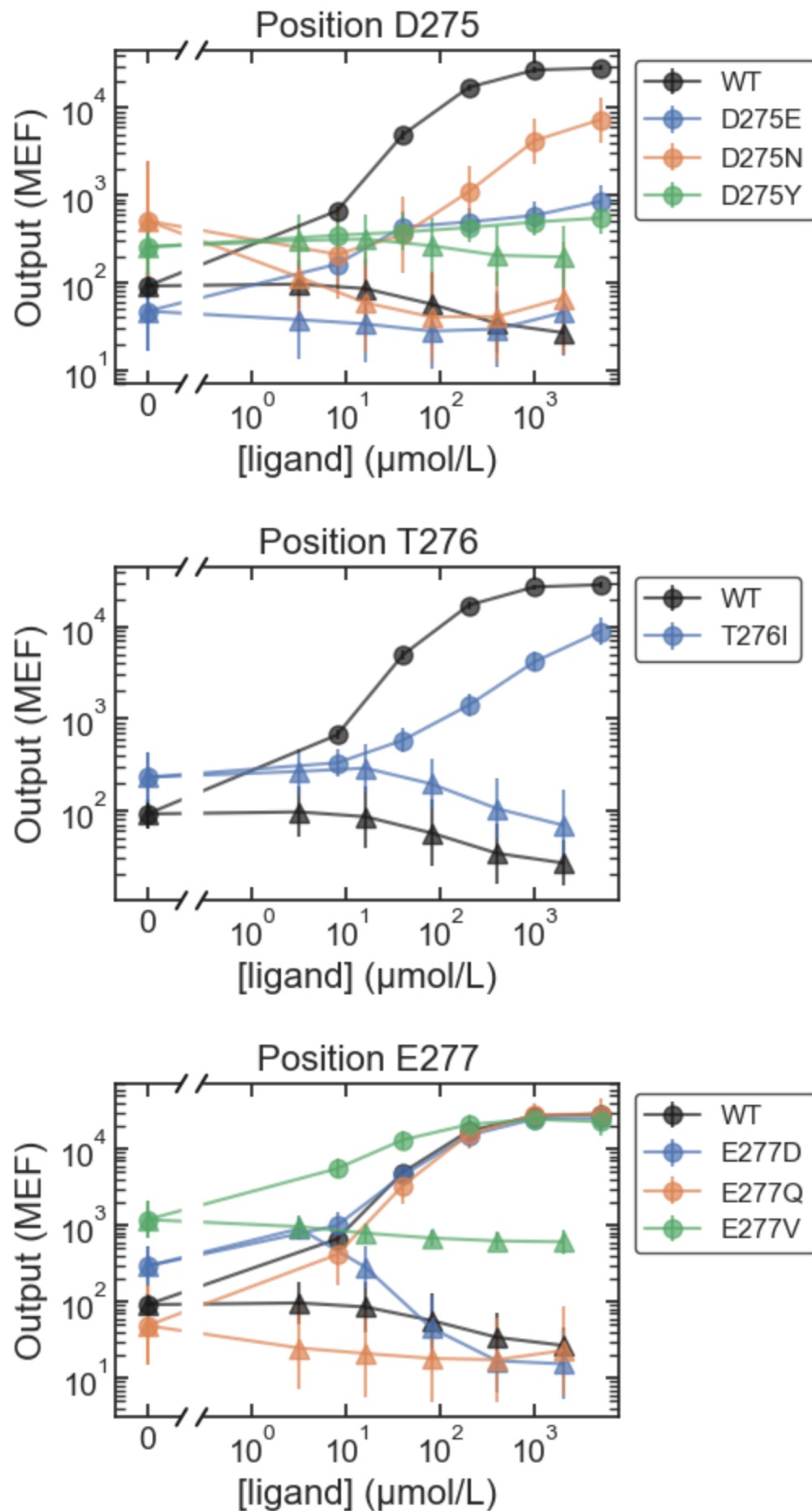


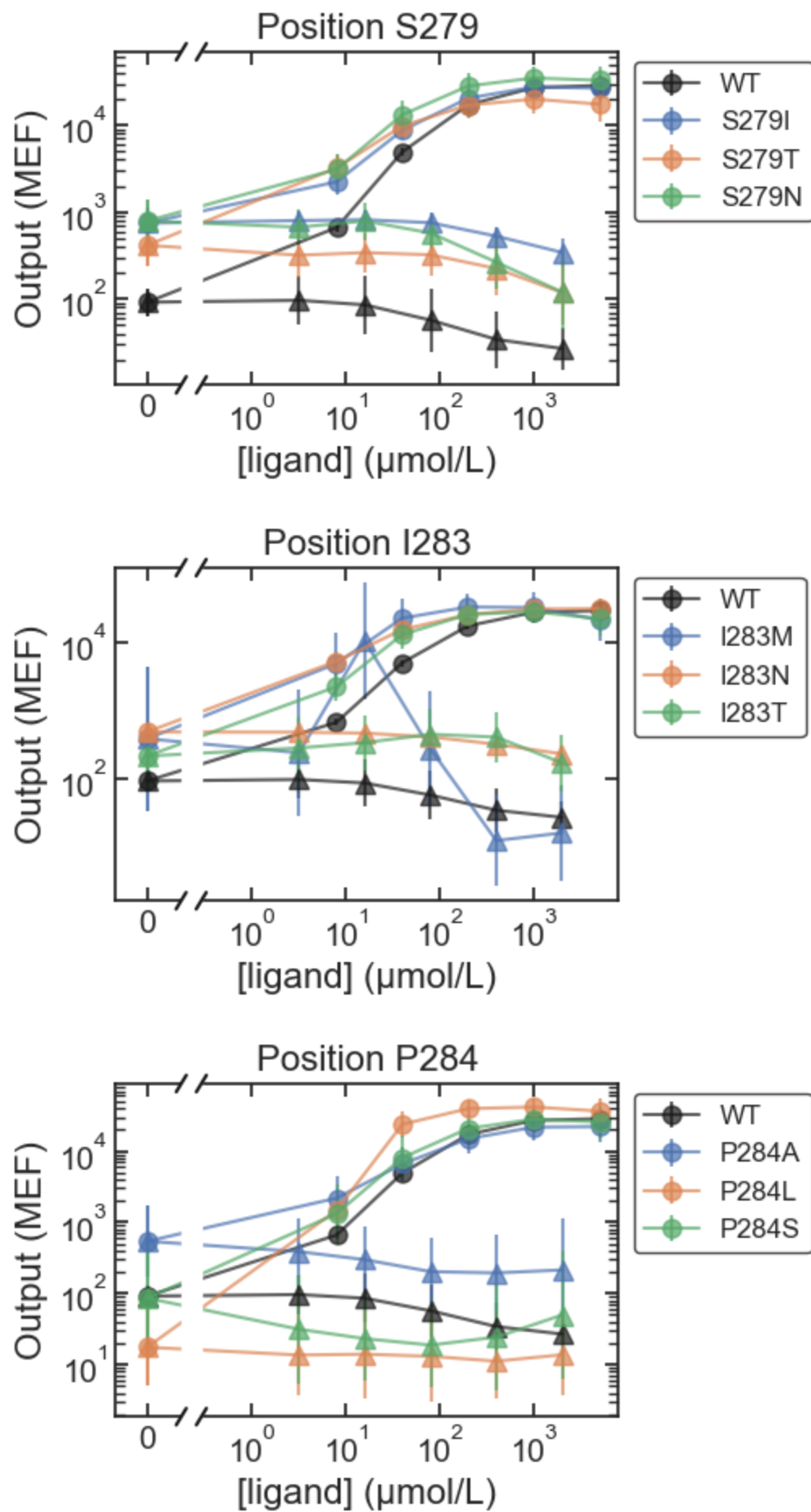


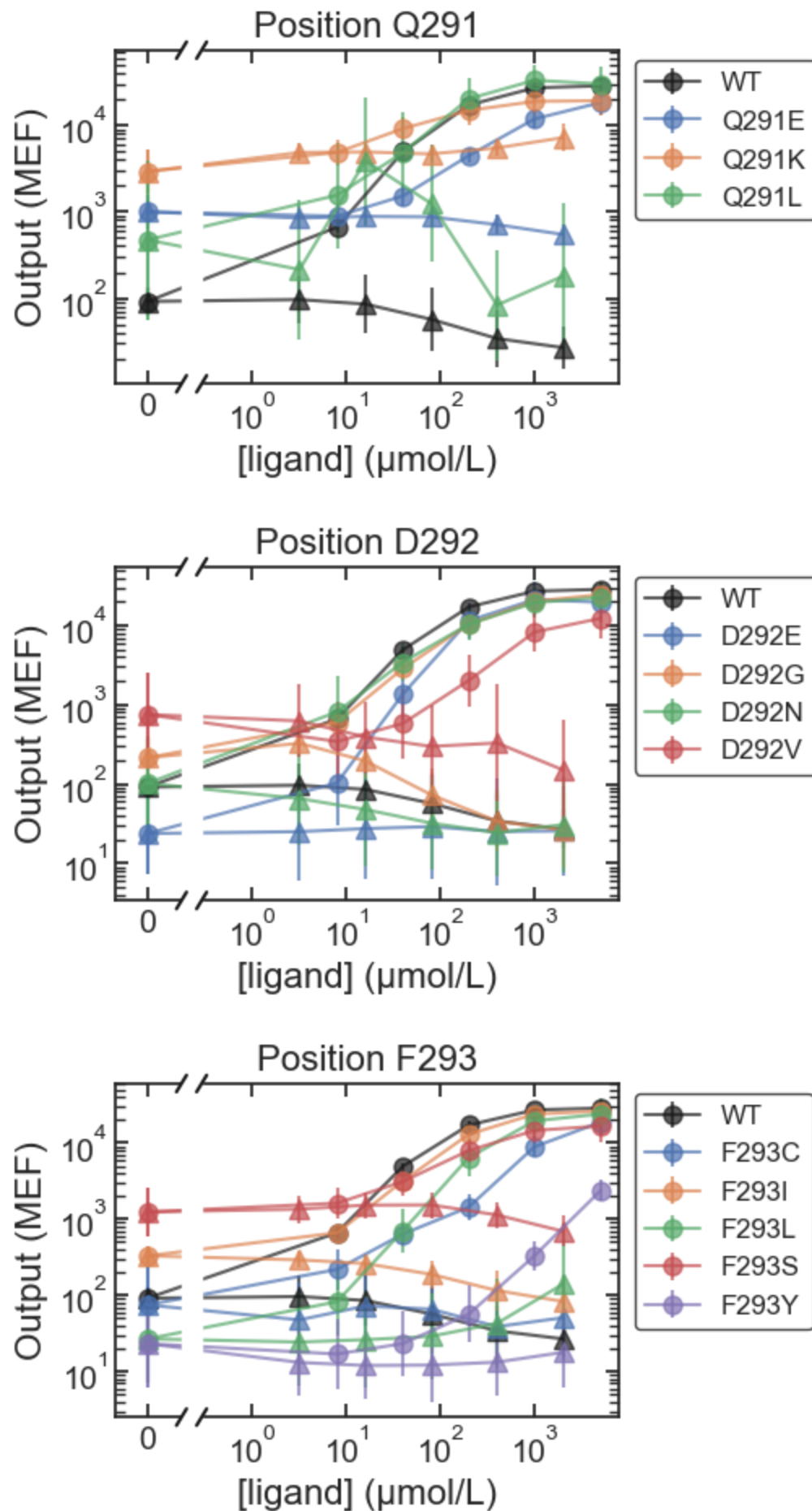


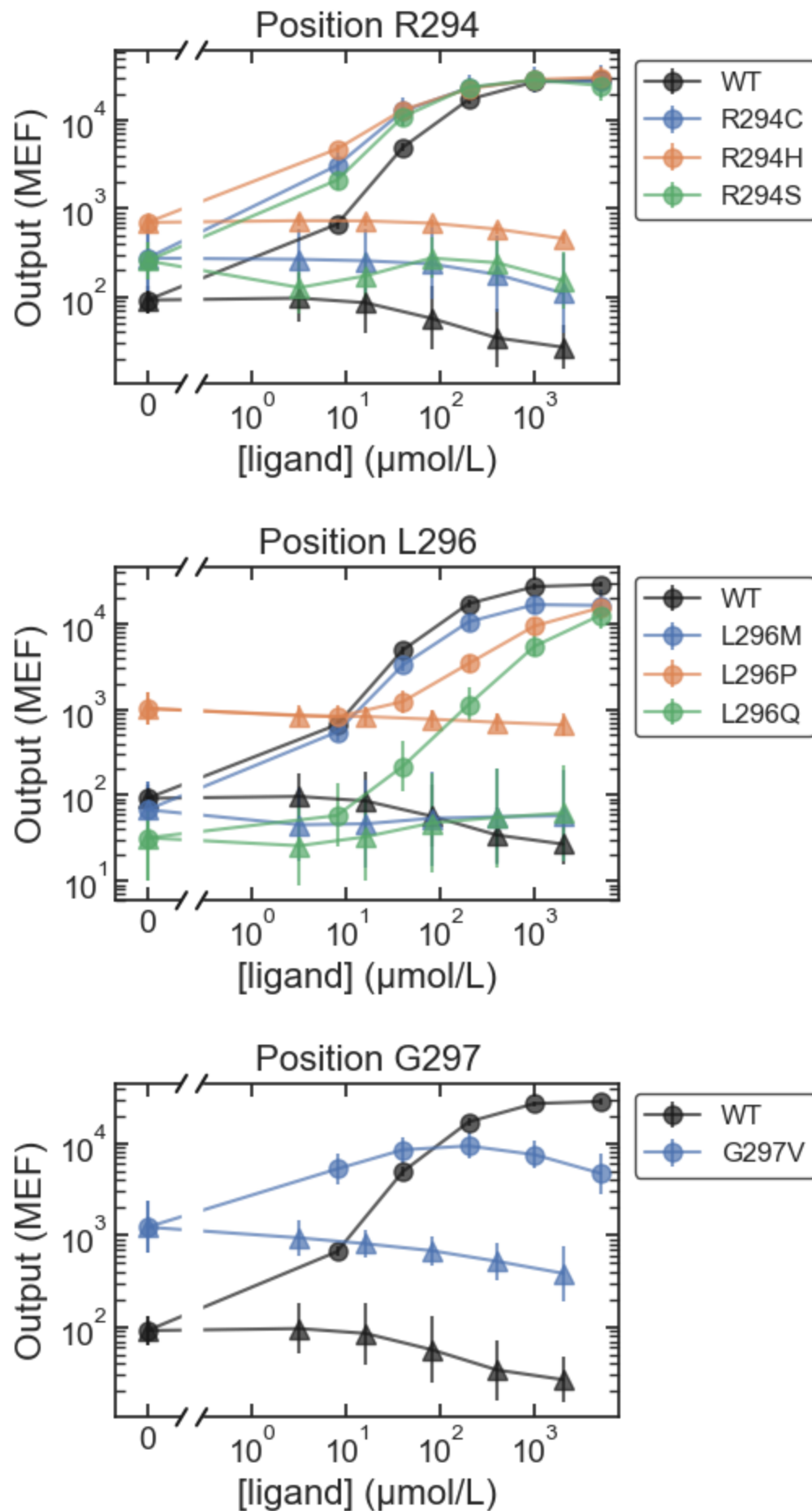


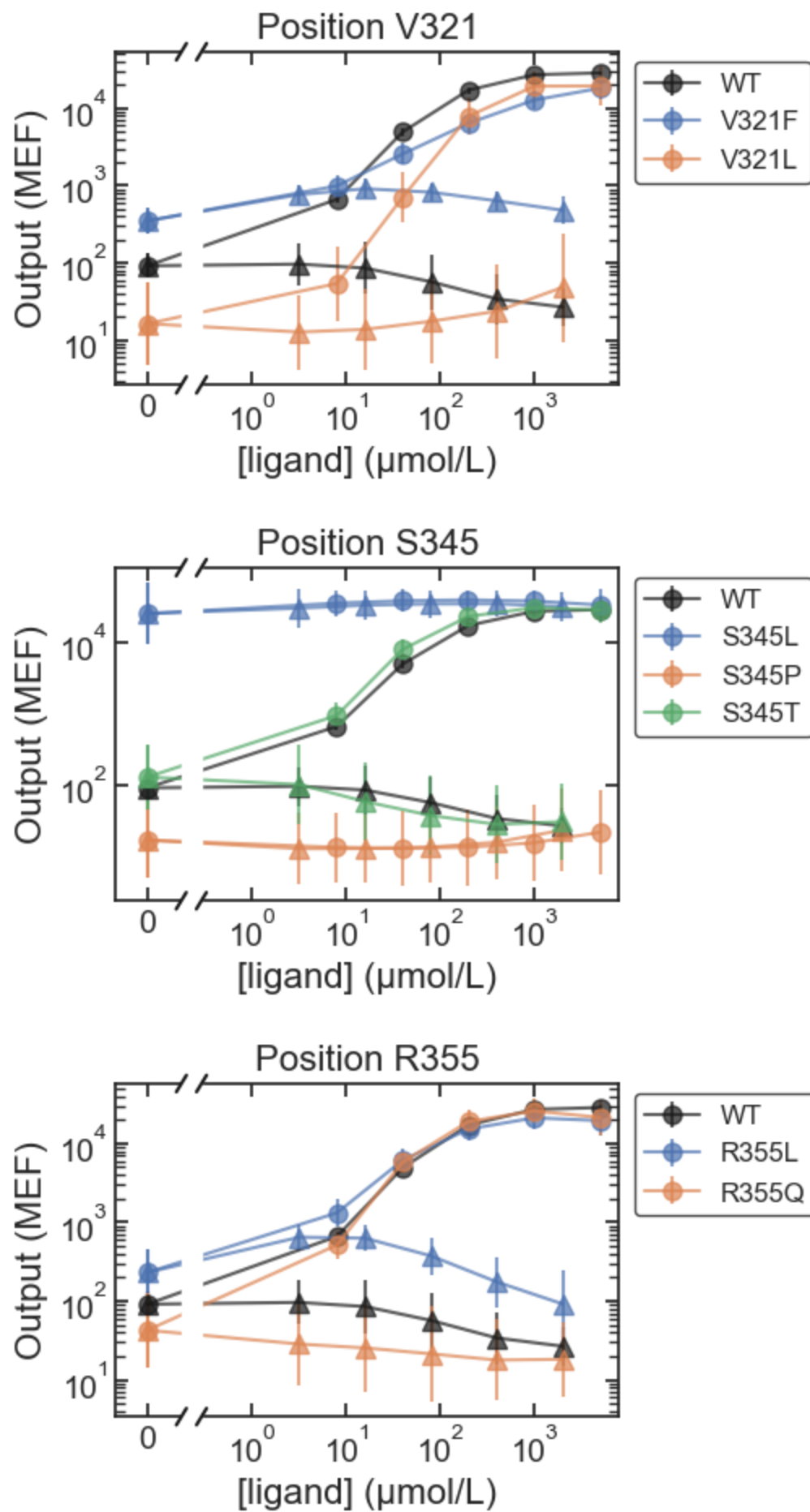


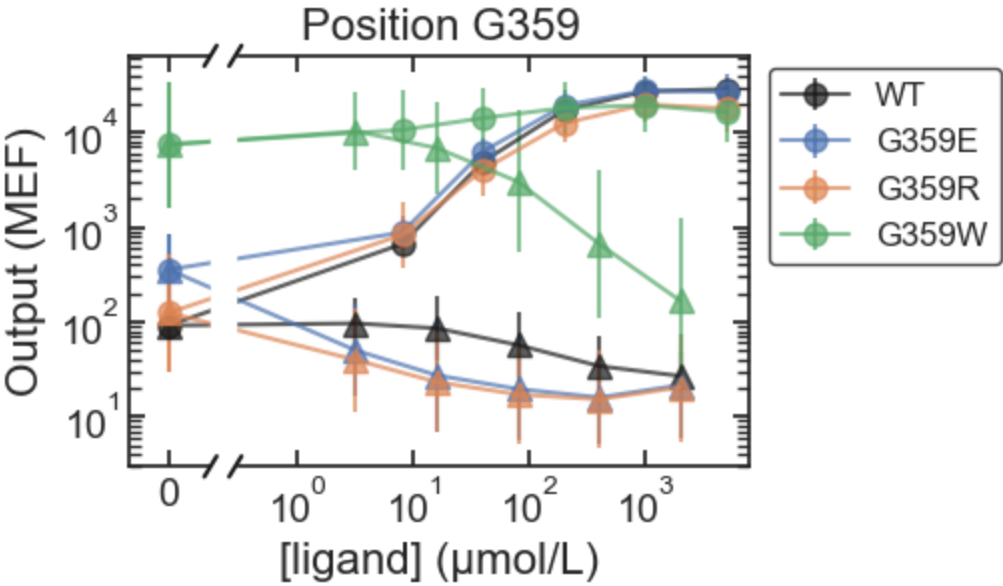




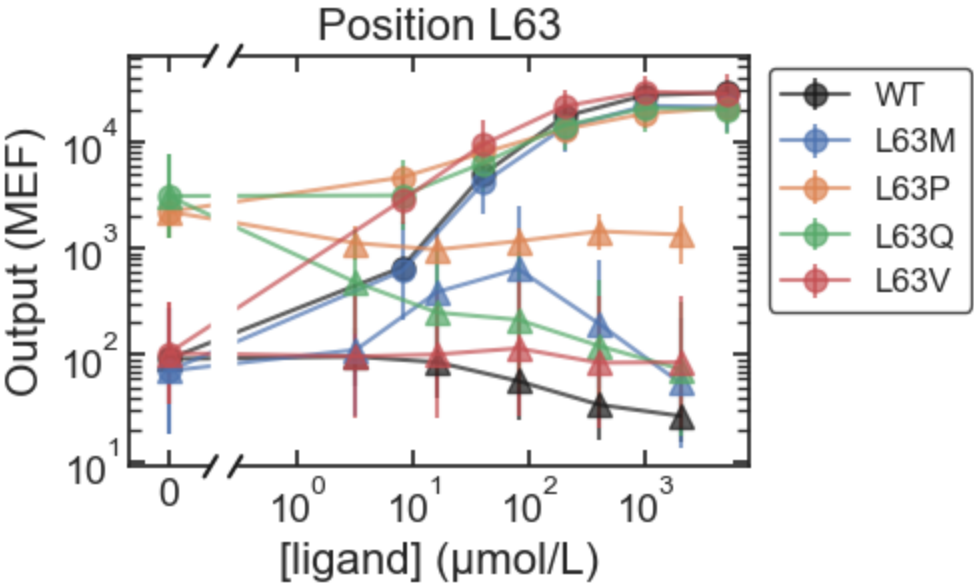


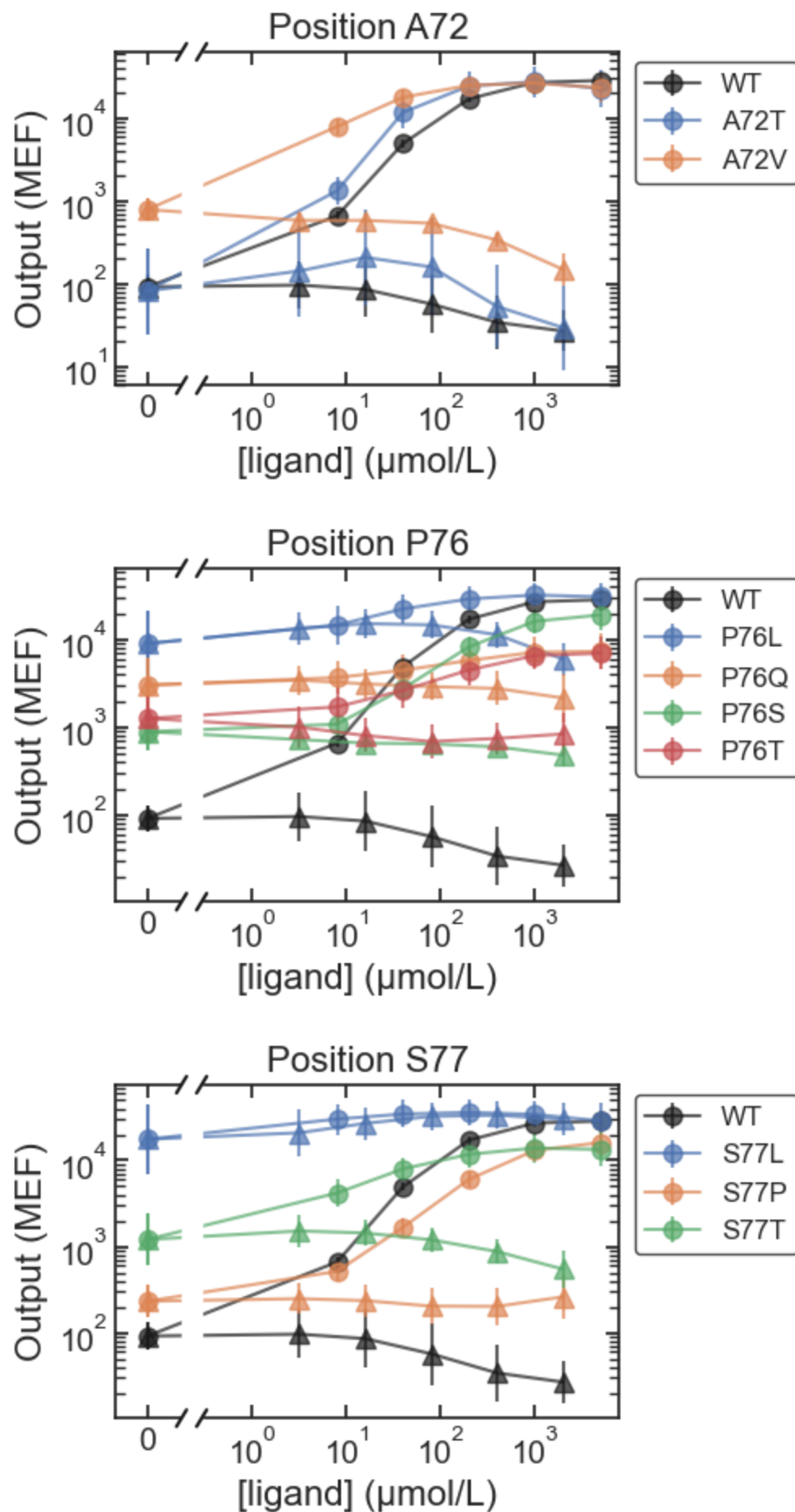


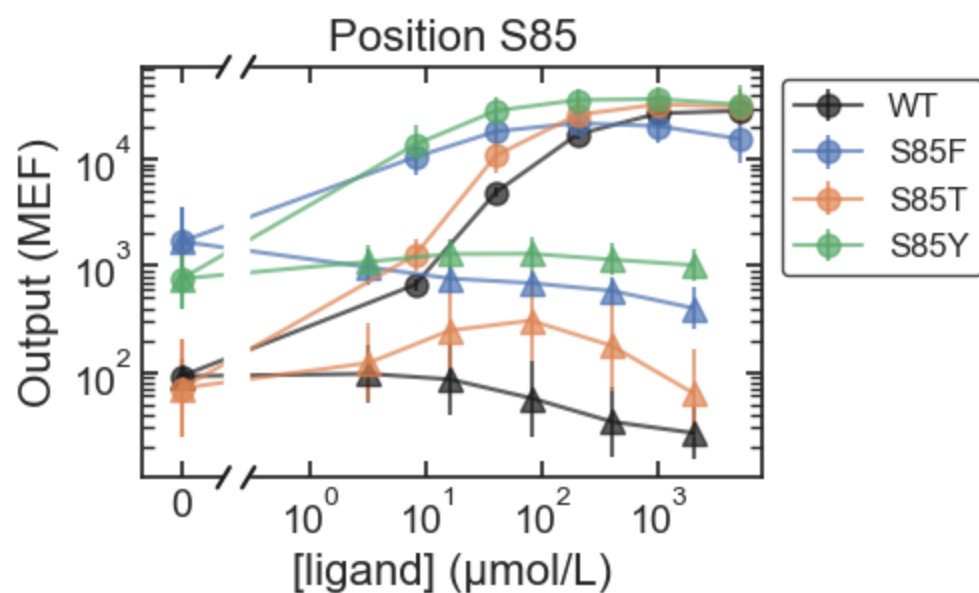
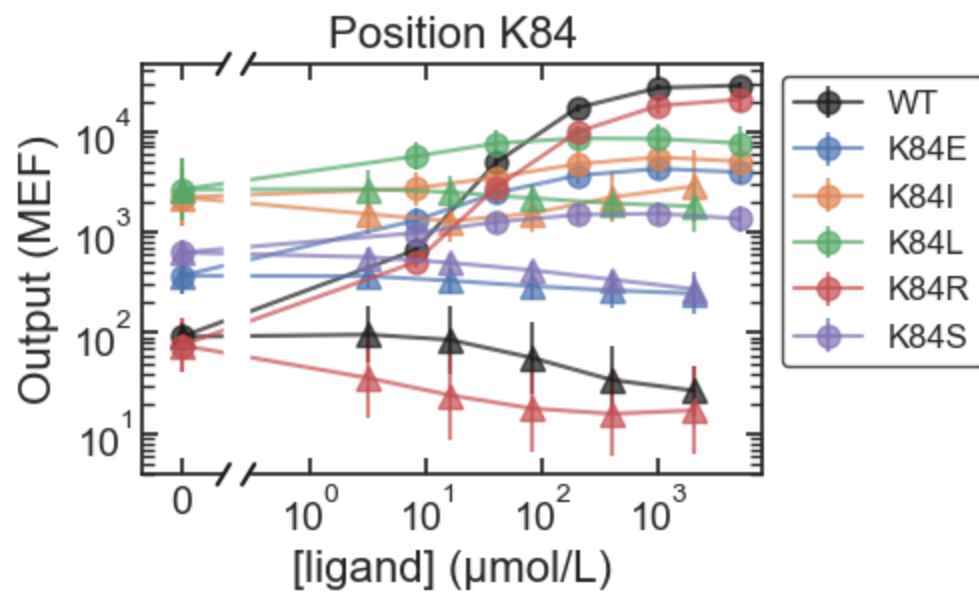
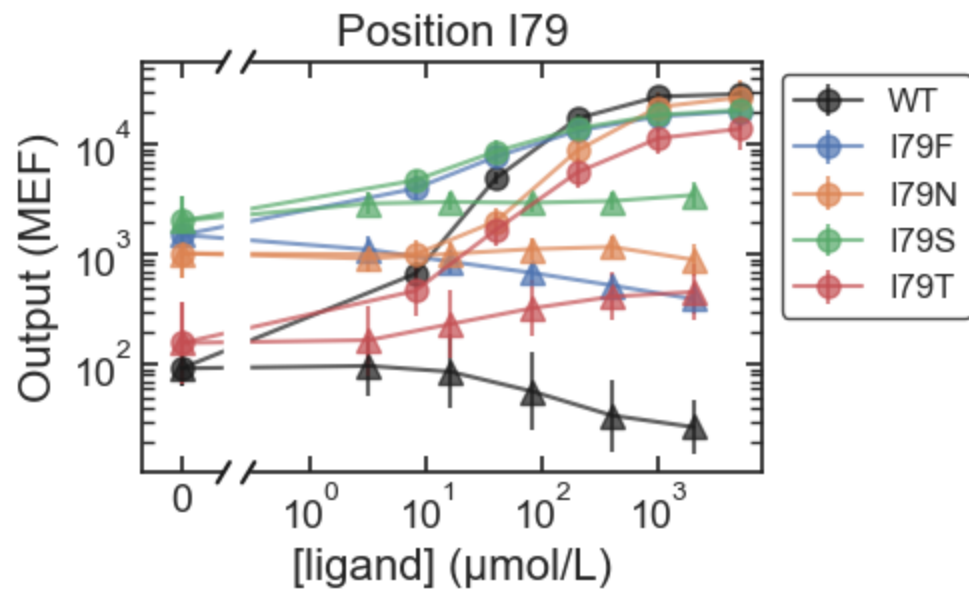


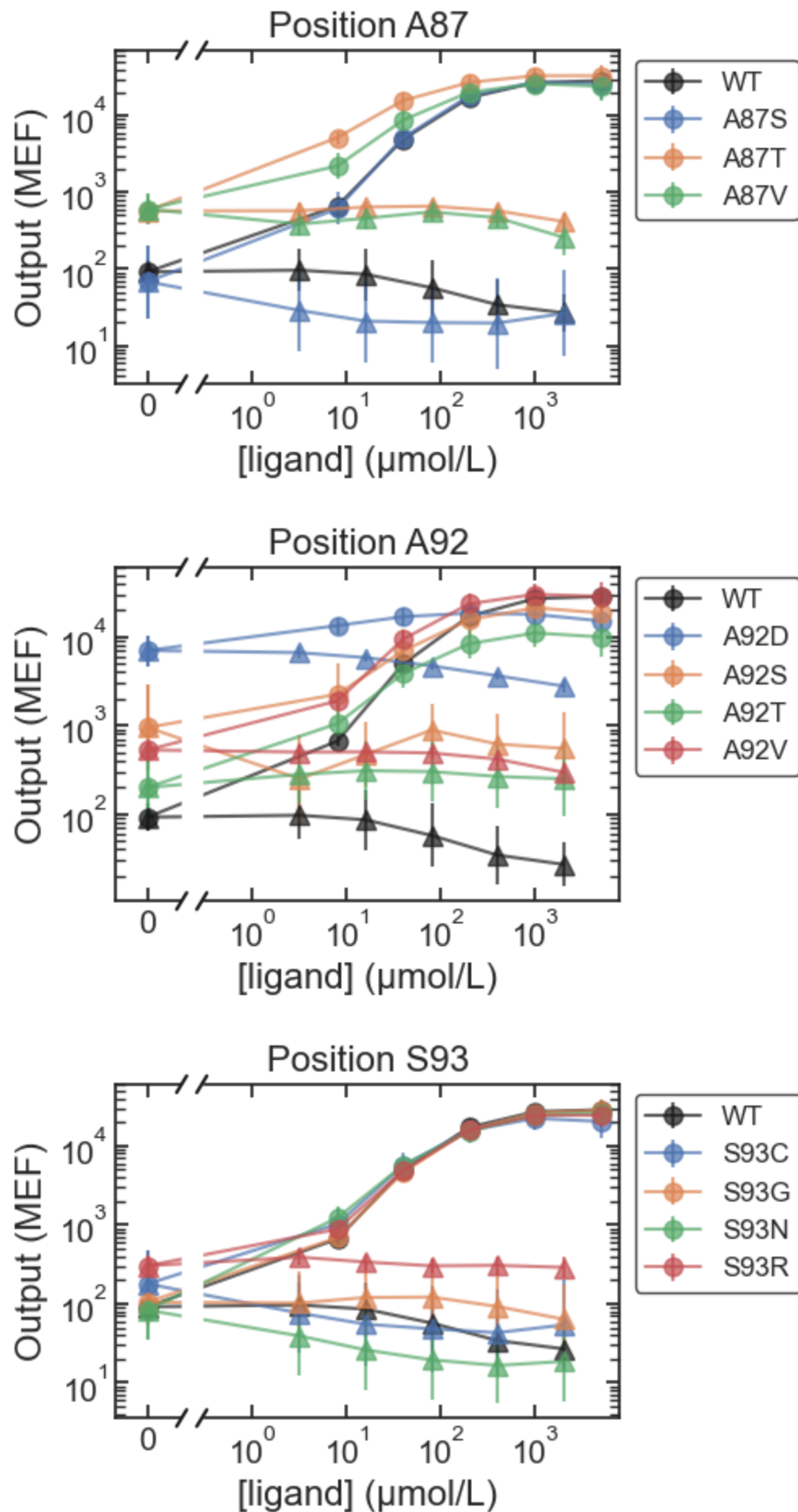


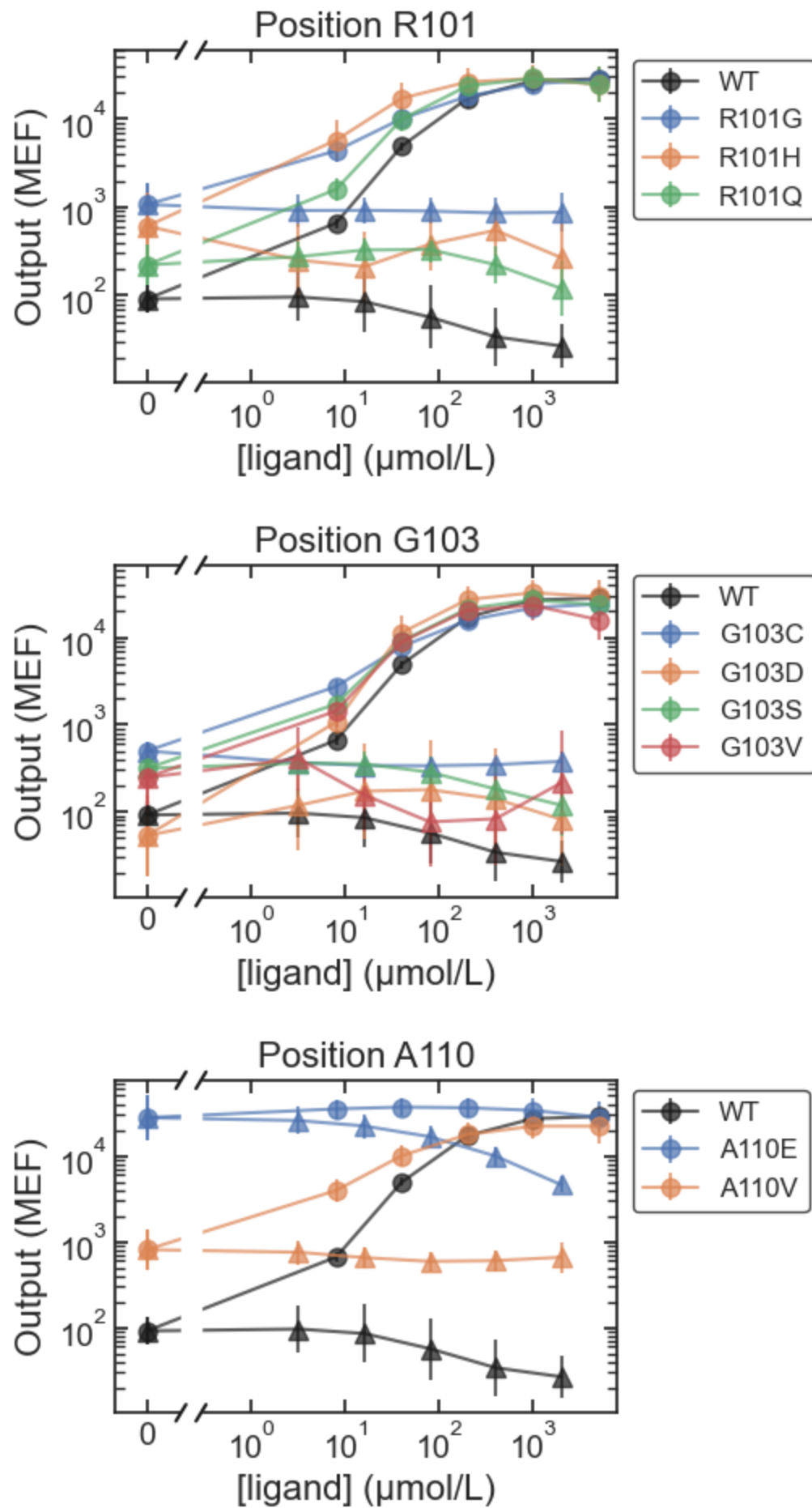
MWC_ONPF

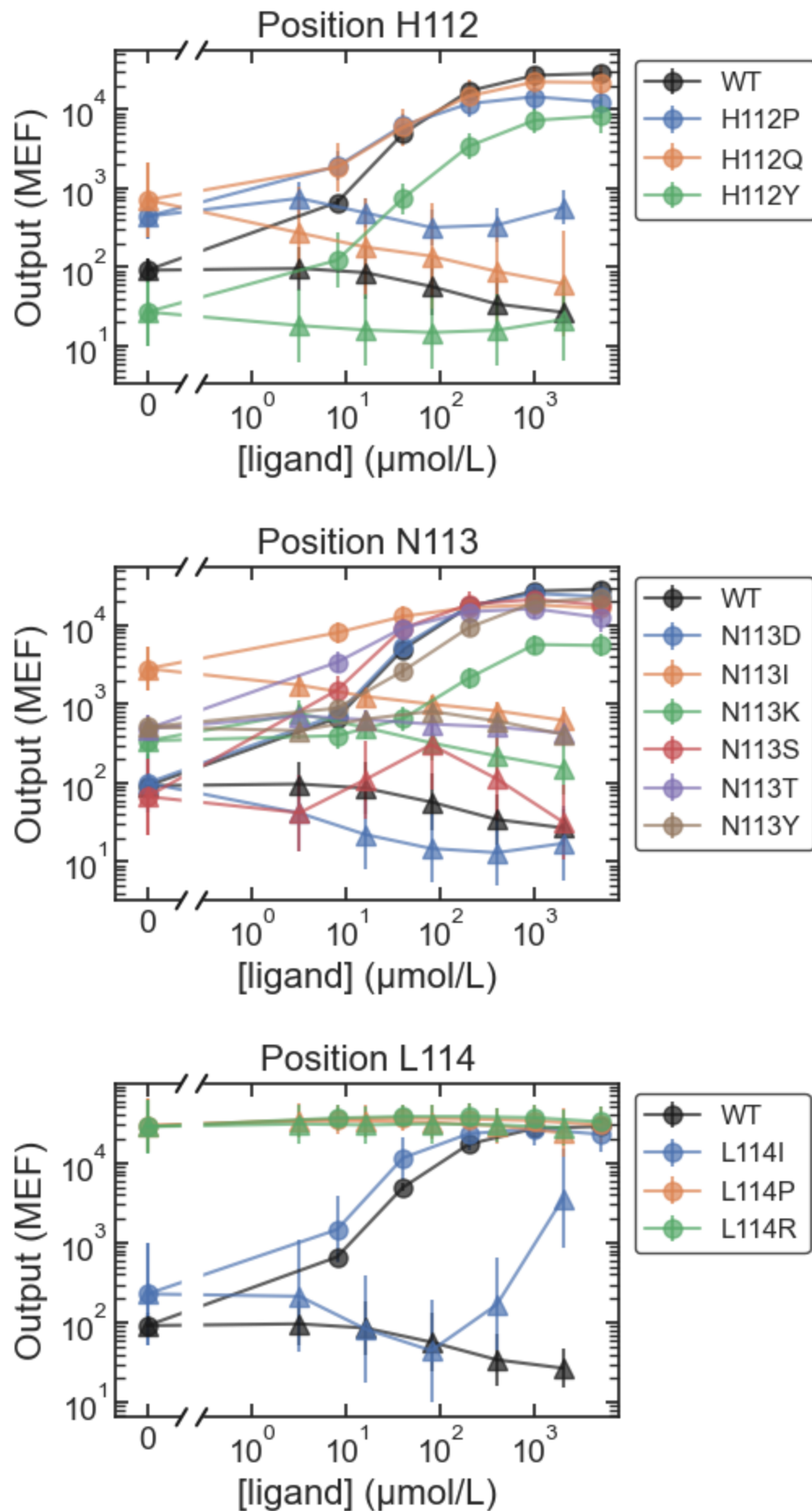


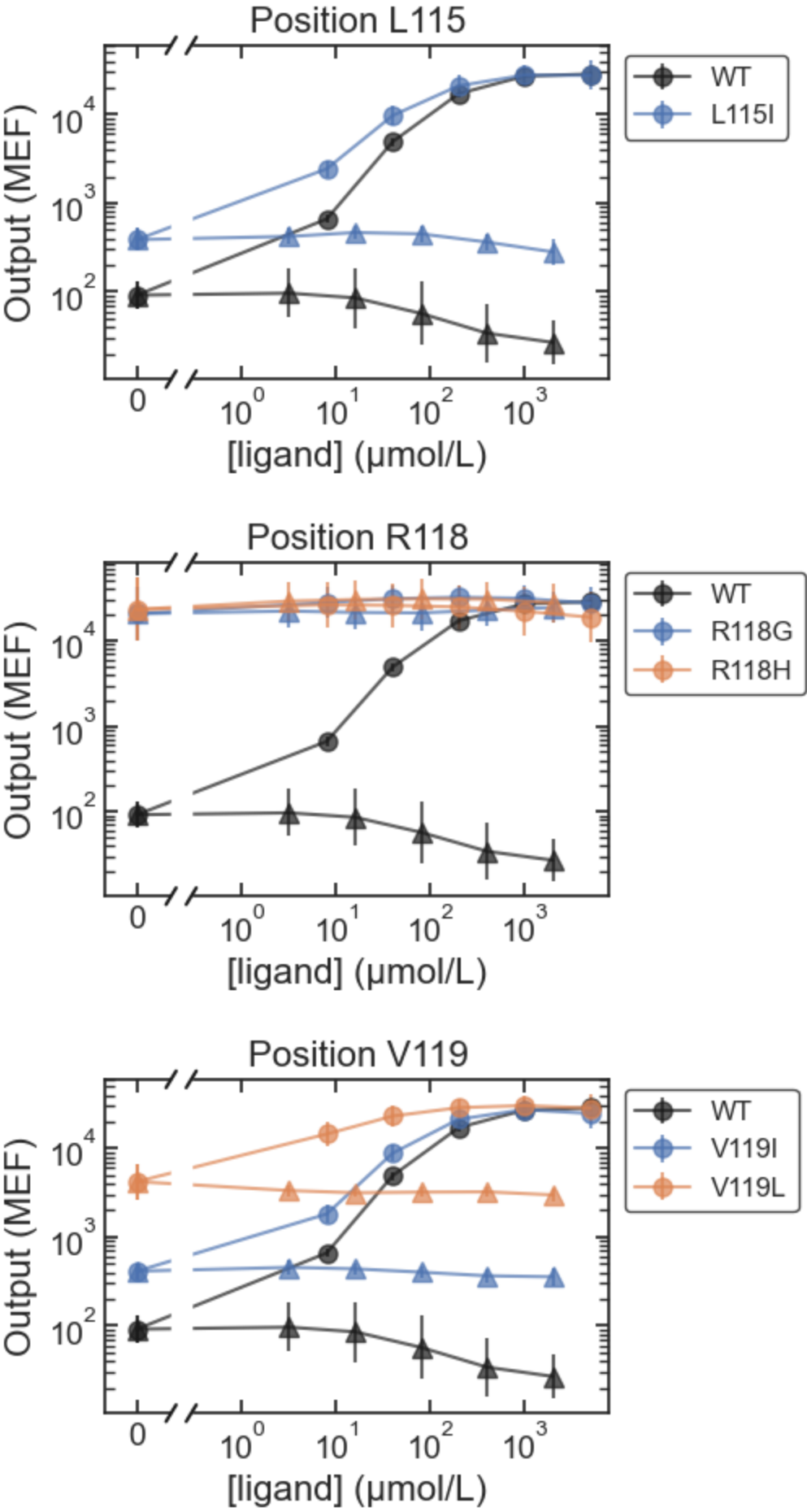


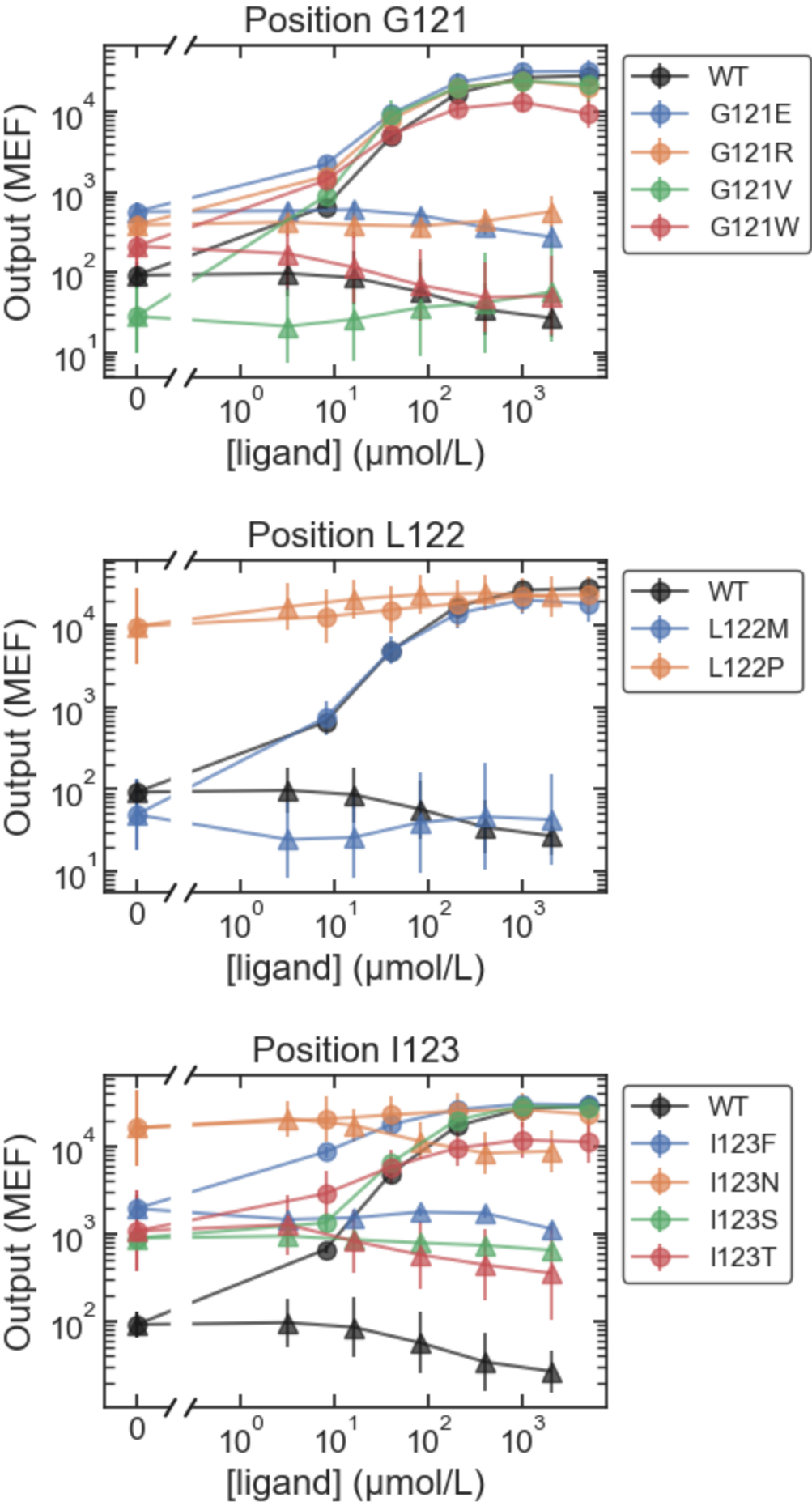


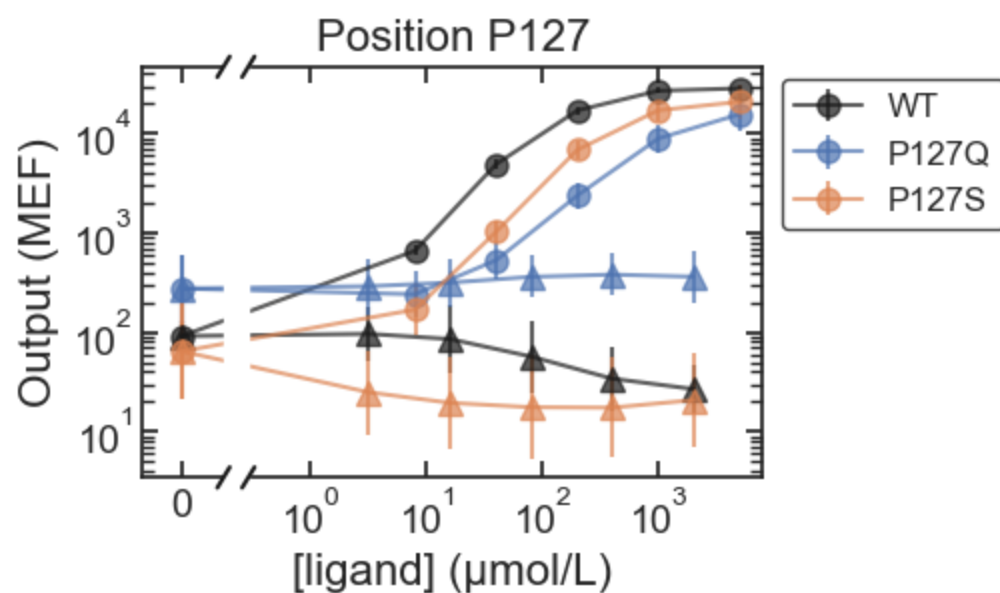
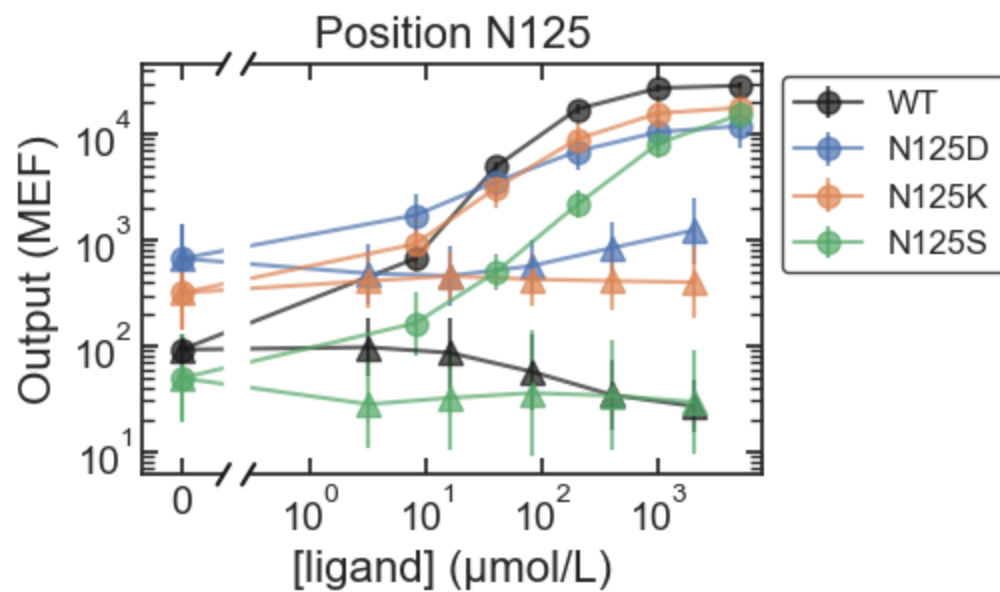
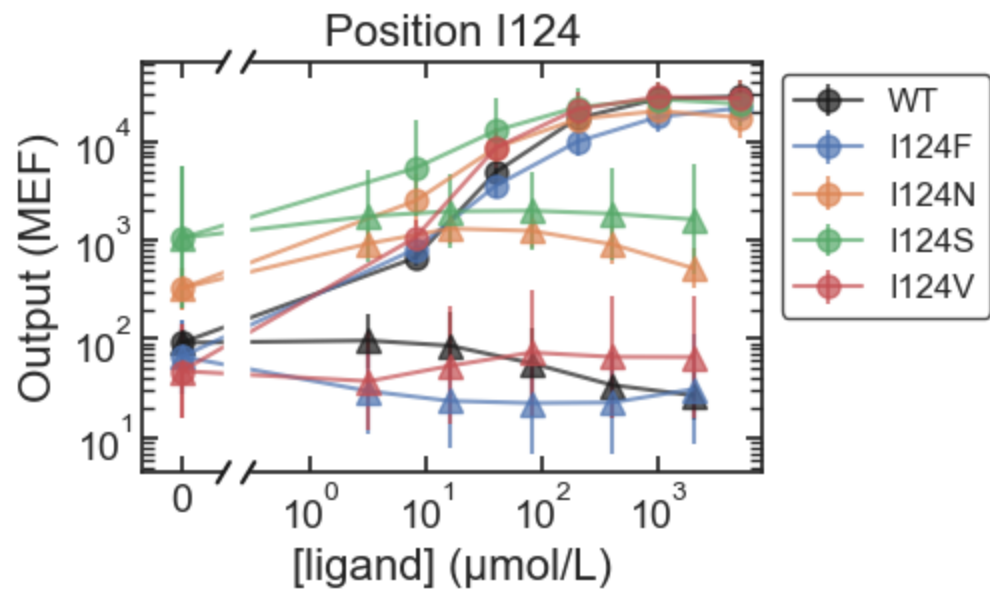


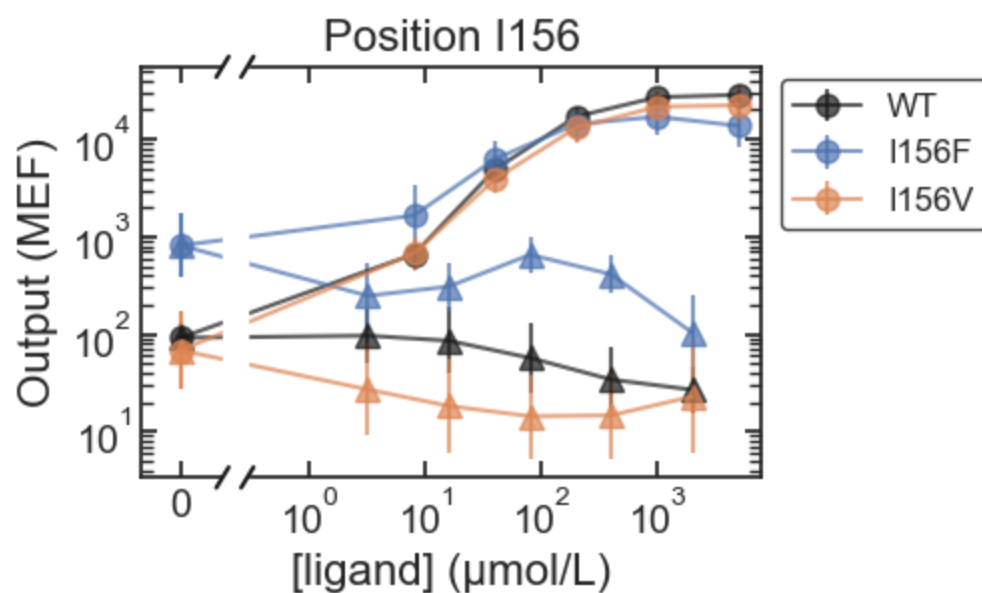
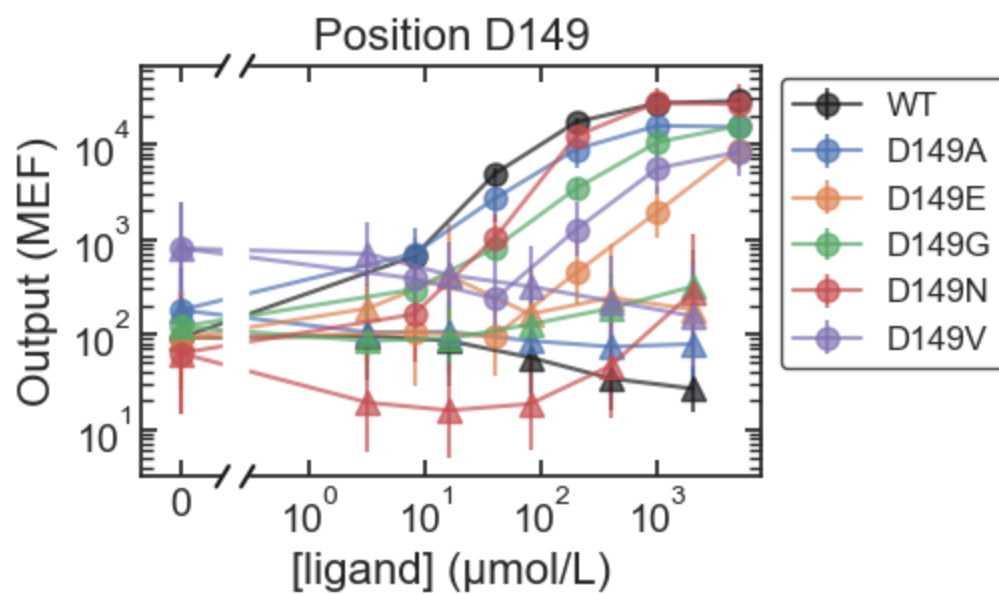
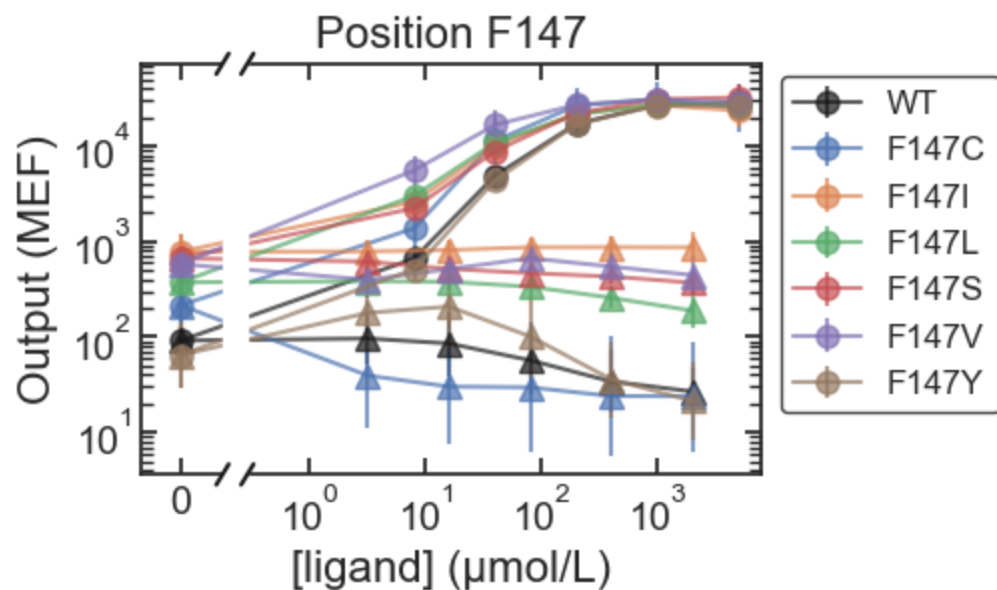


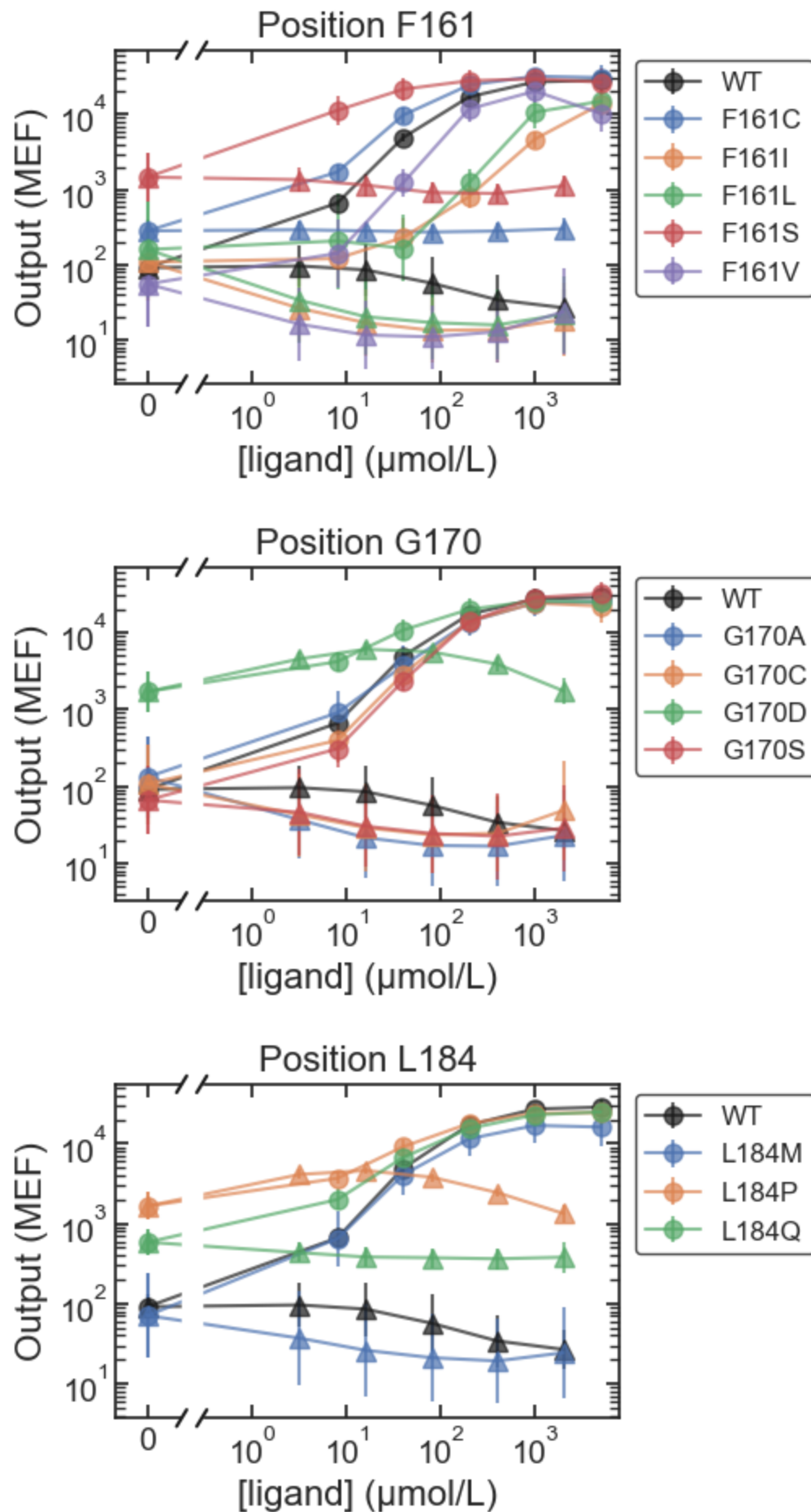


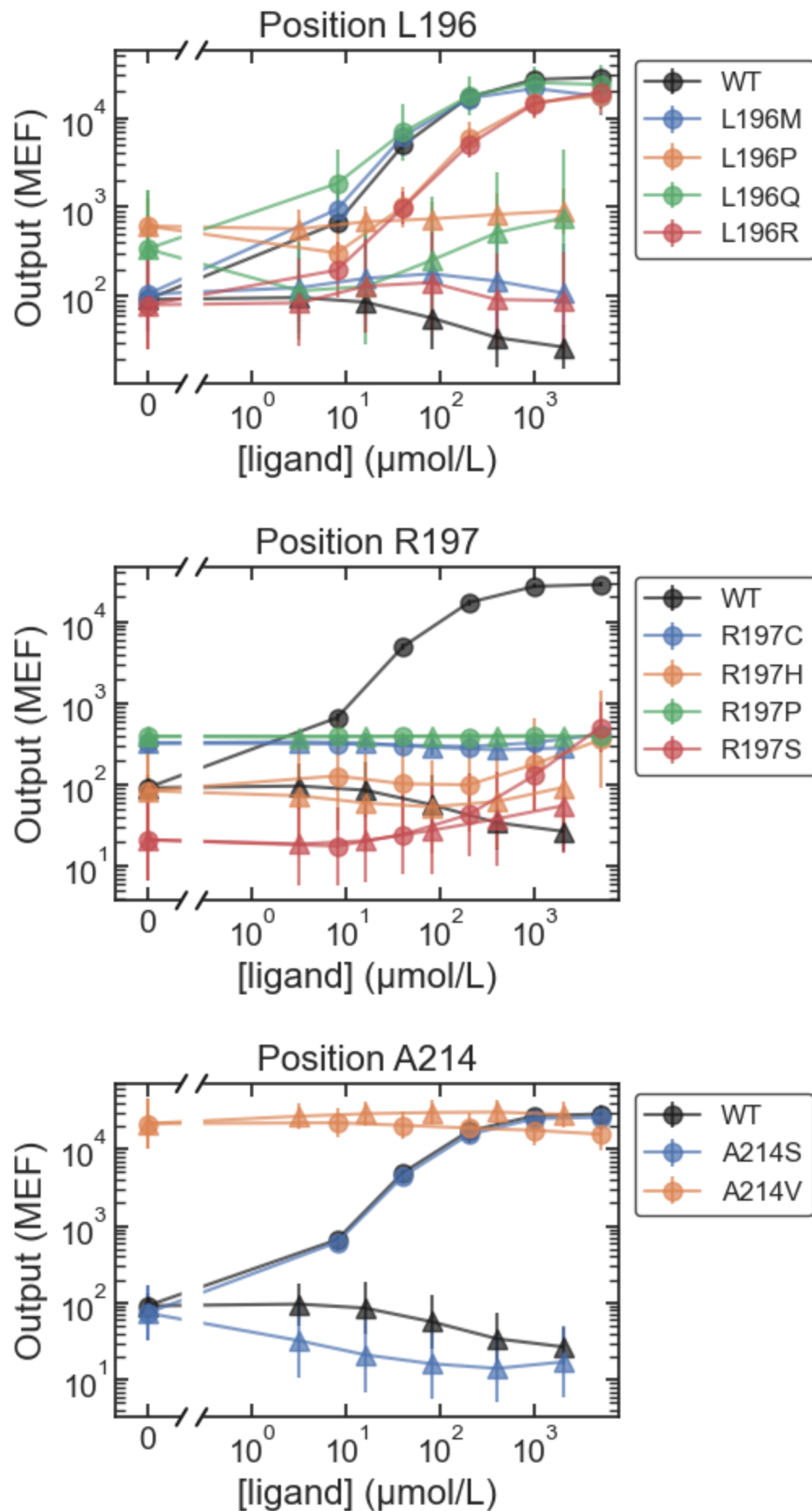


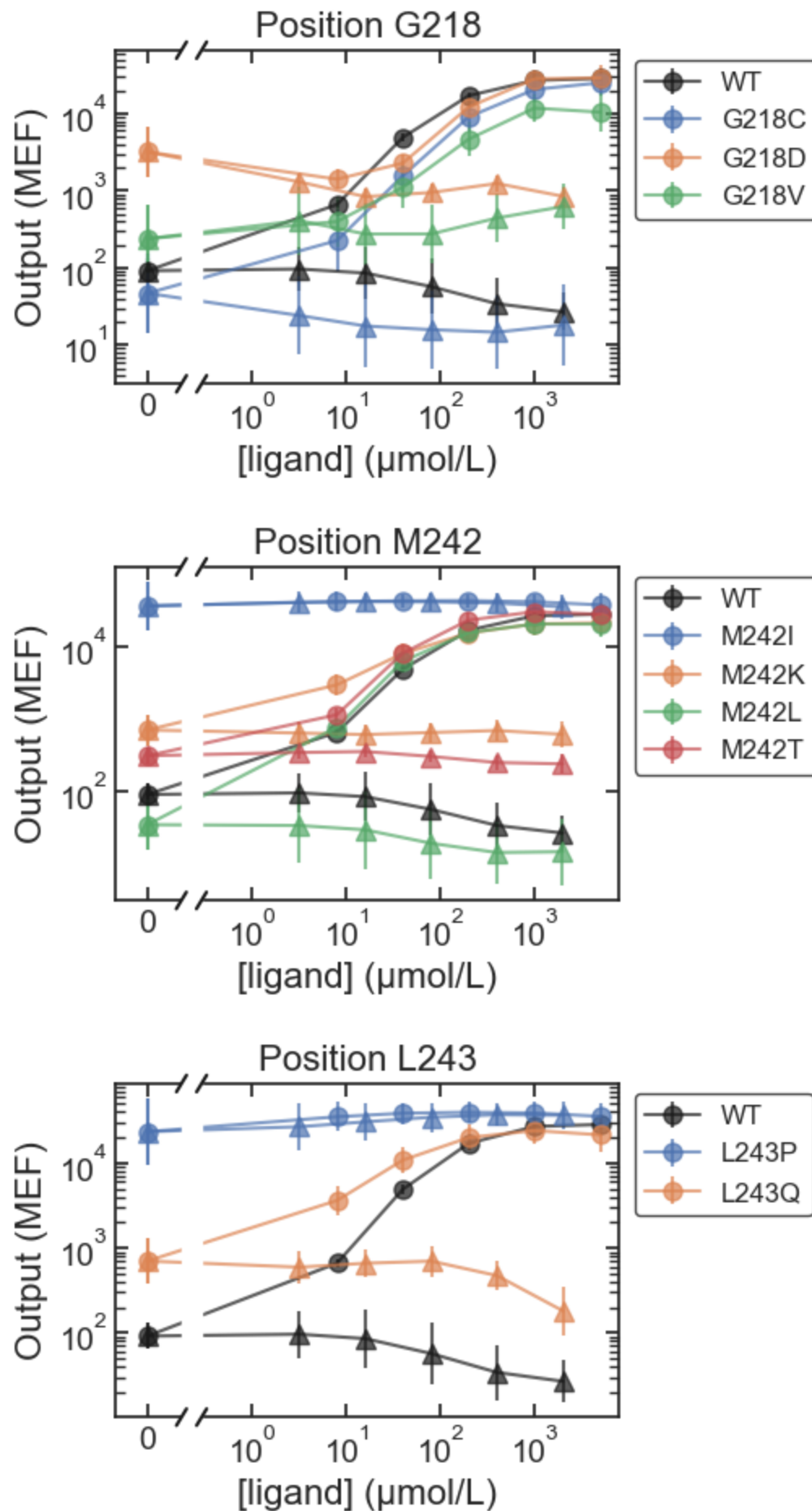


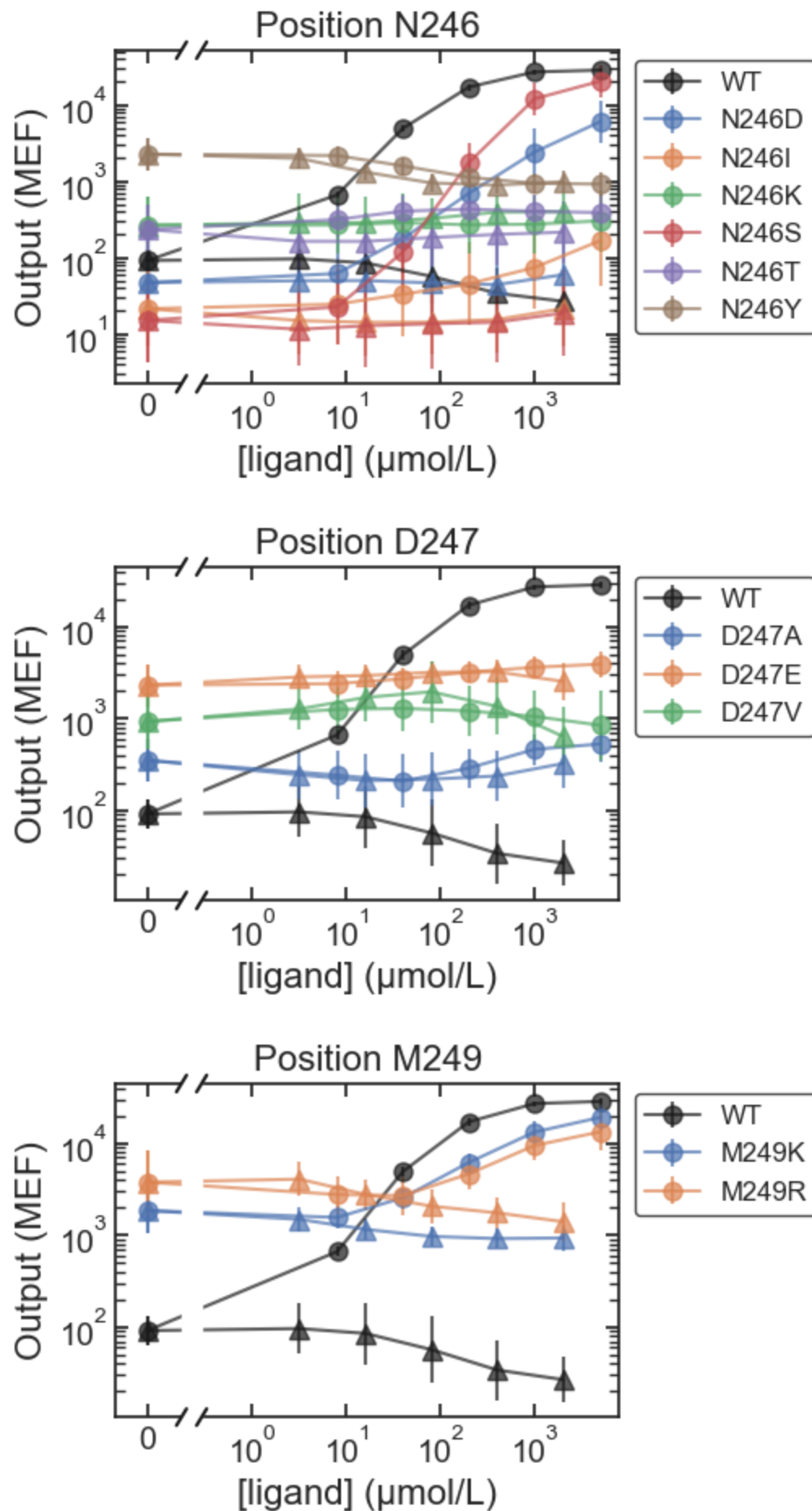


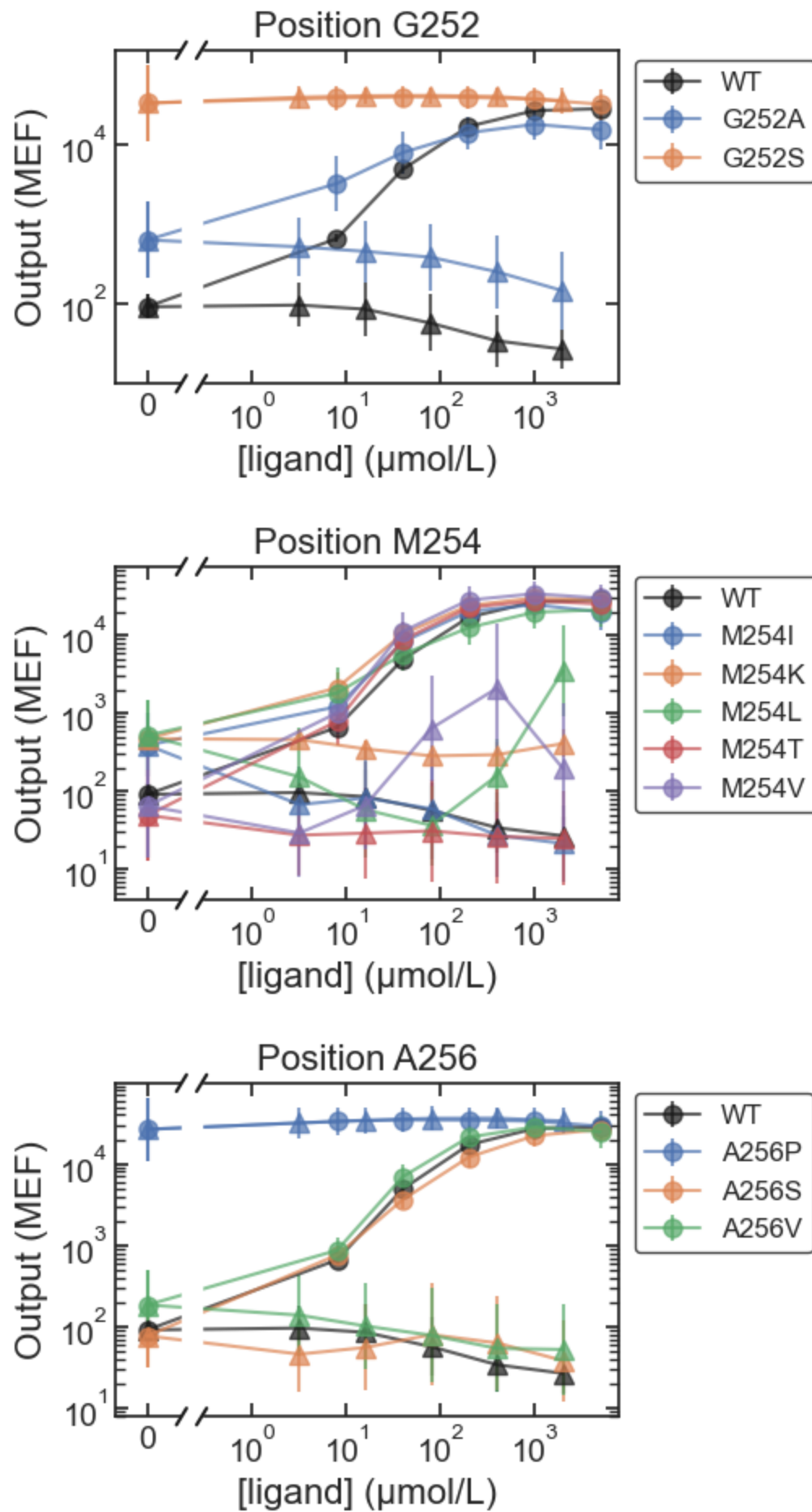


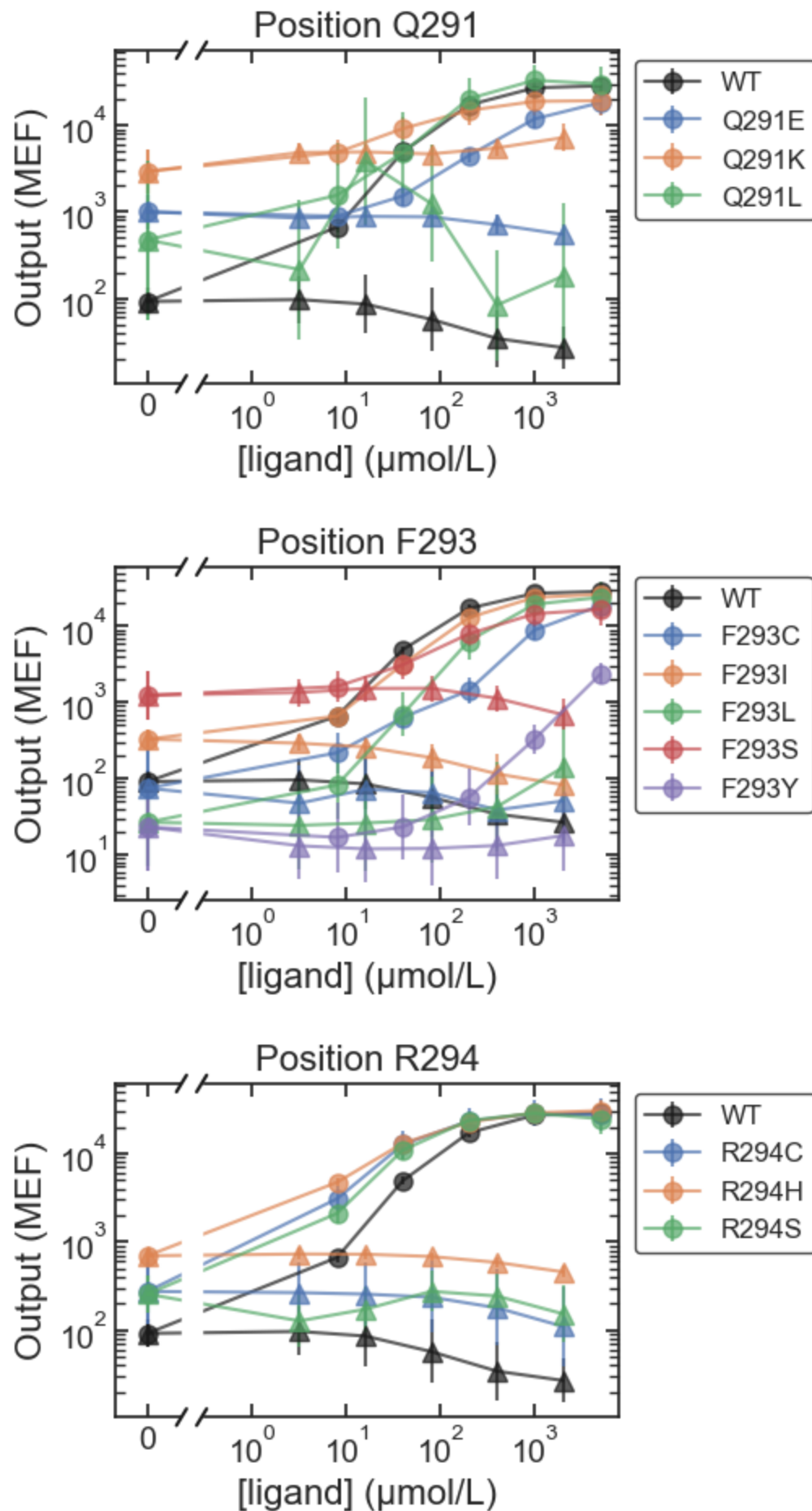


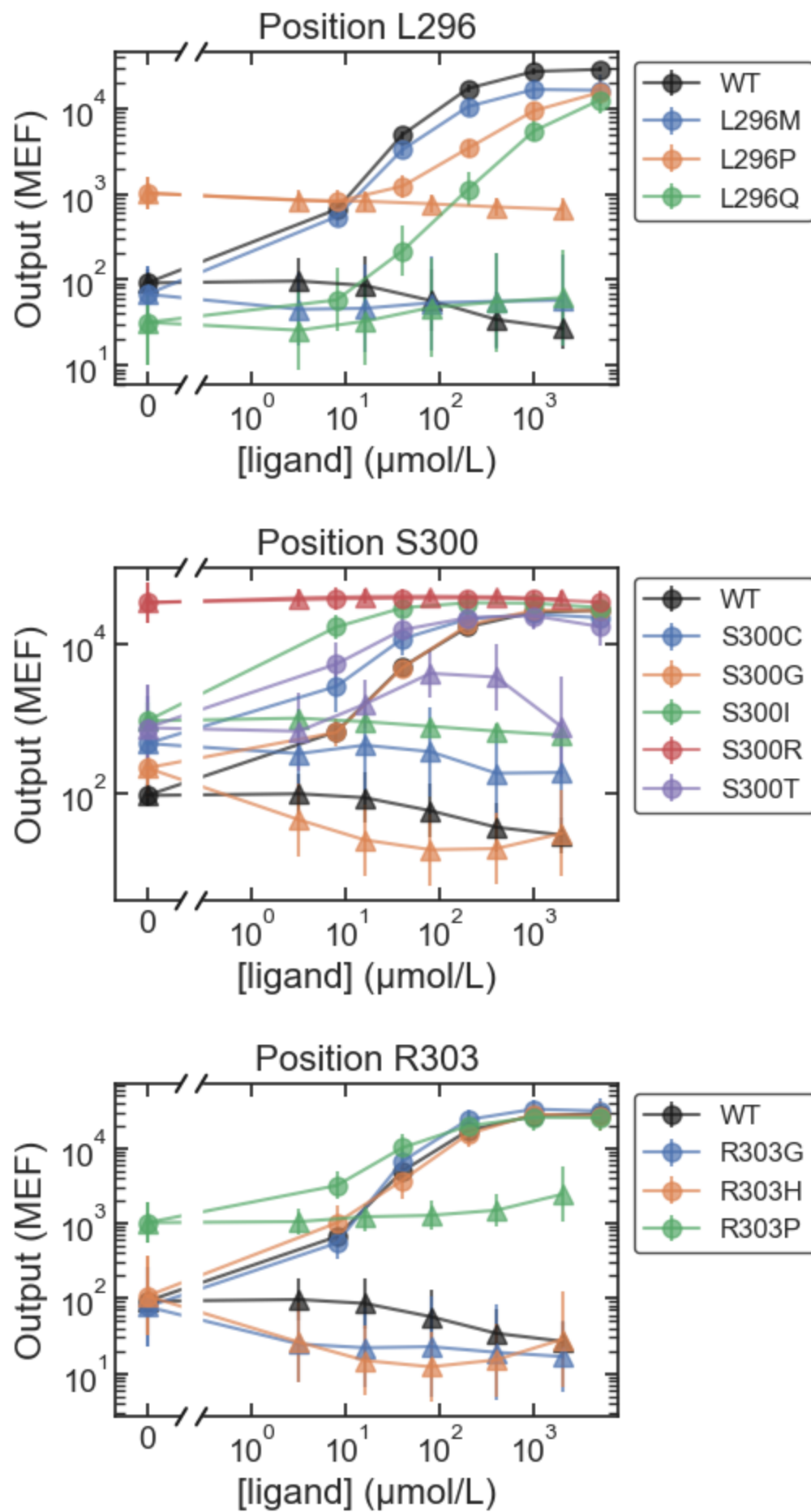


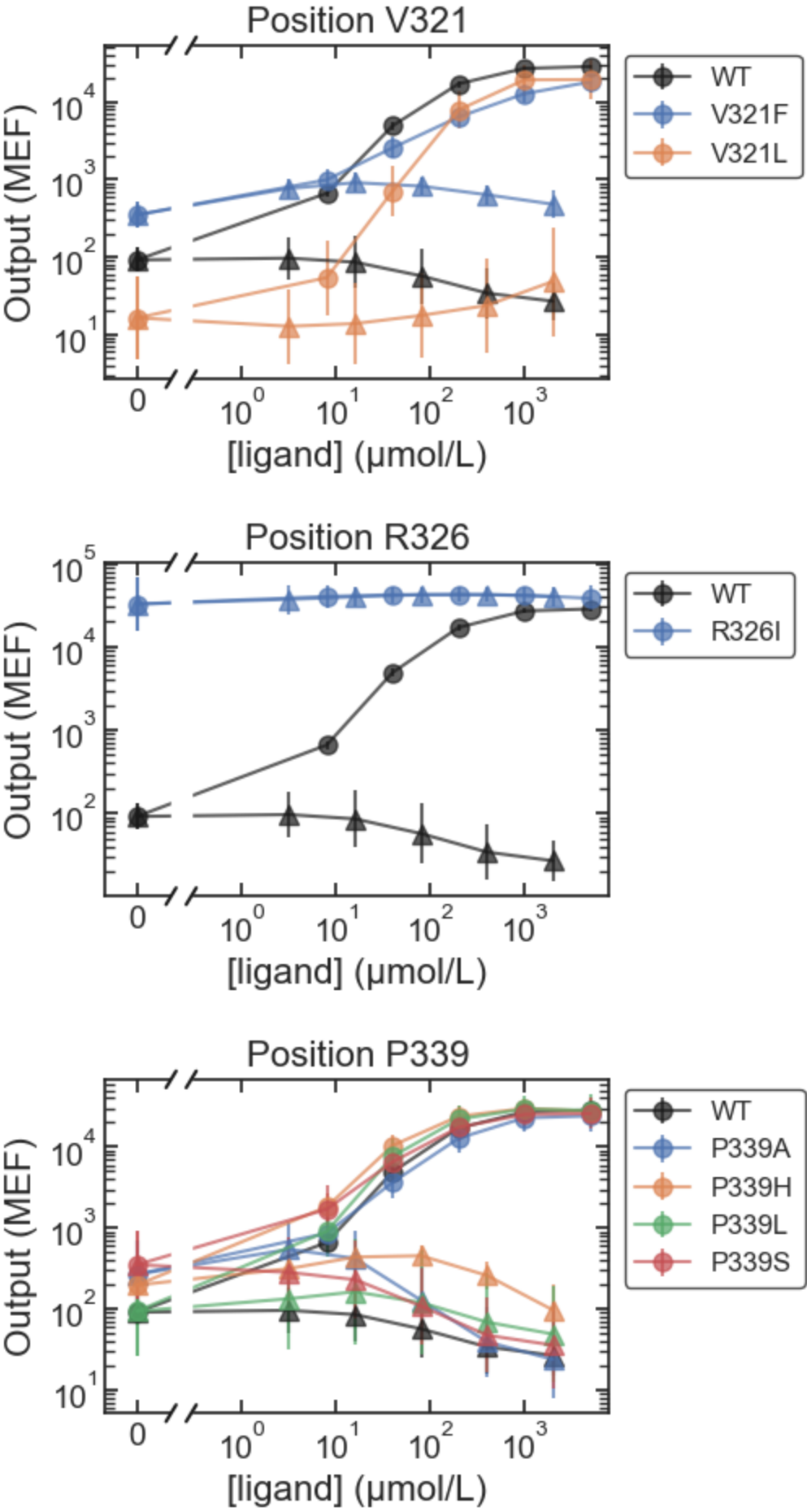


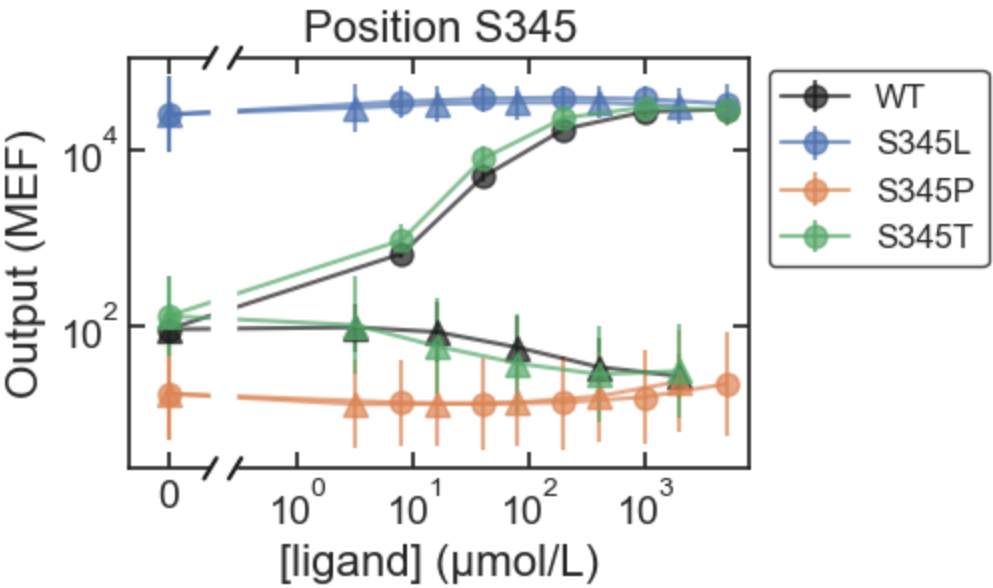
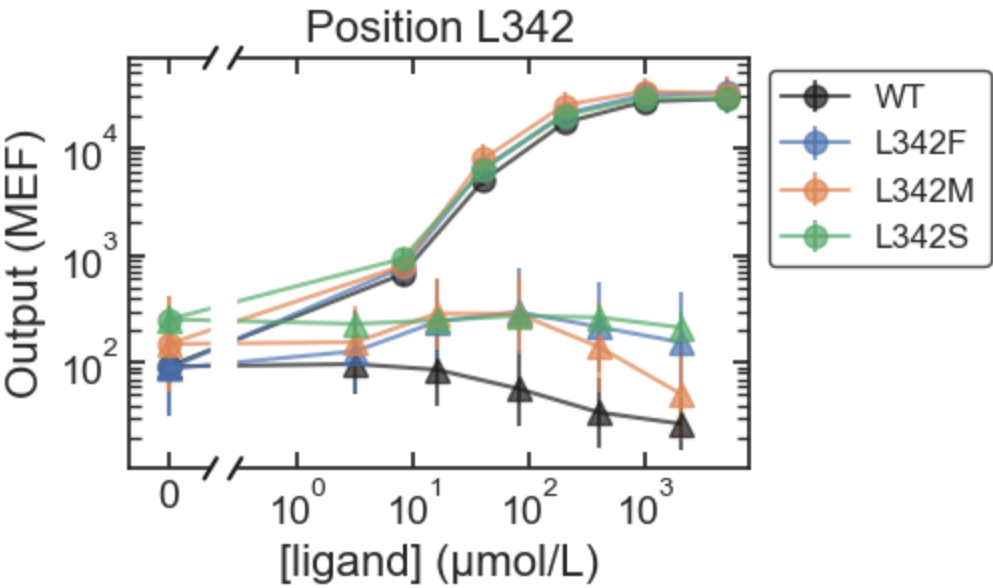




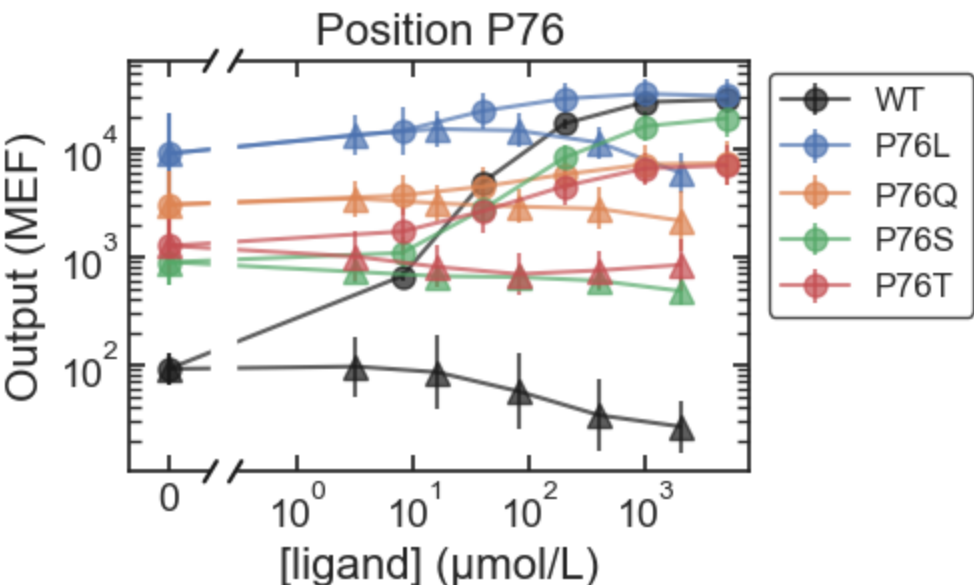
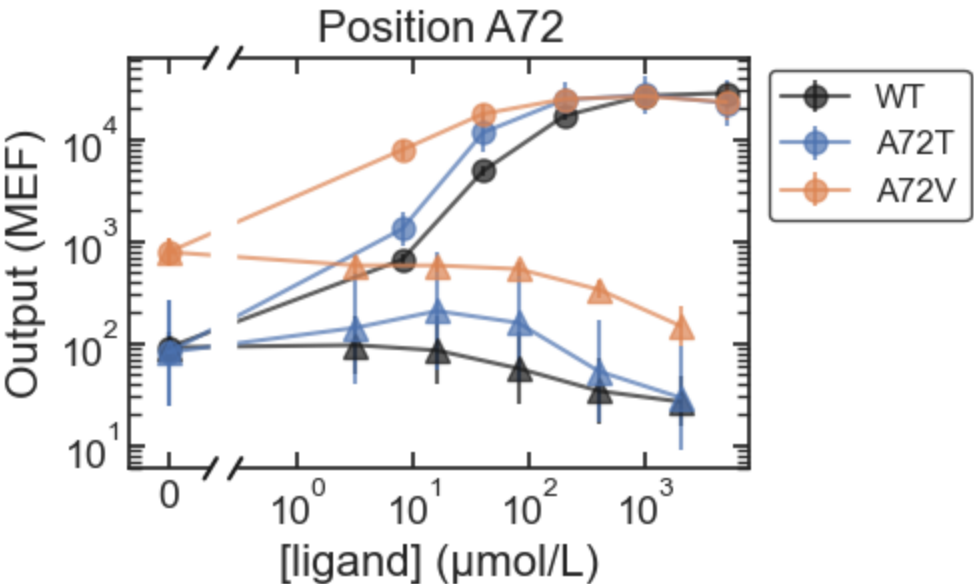
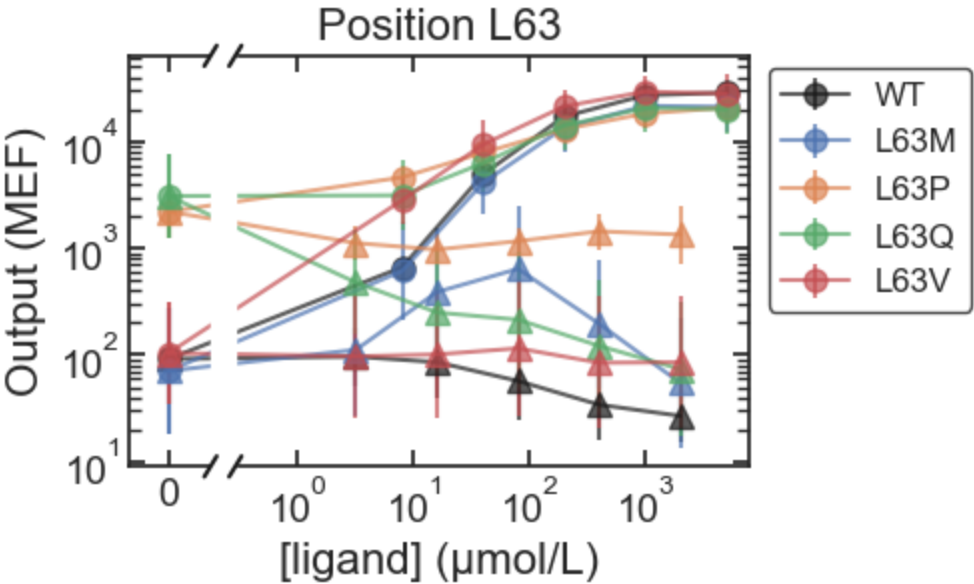


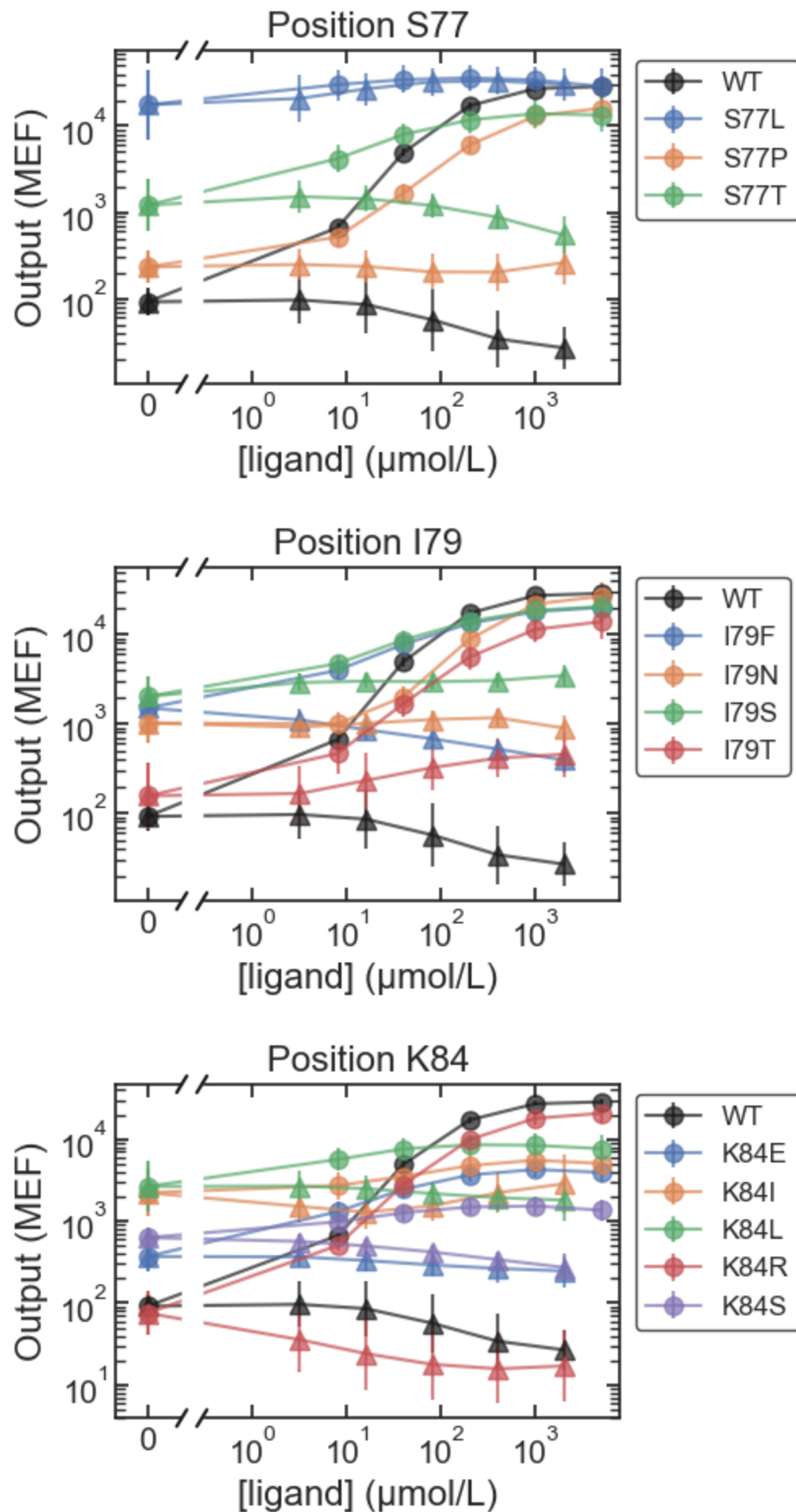


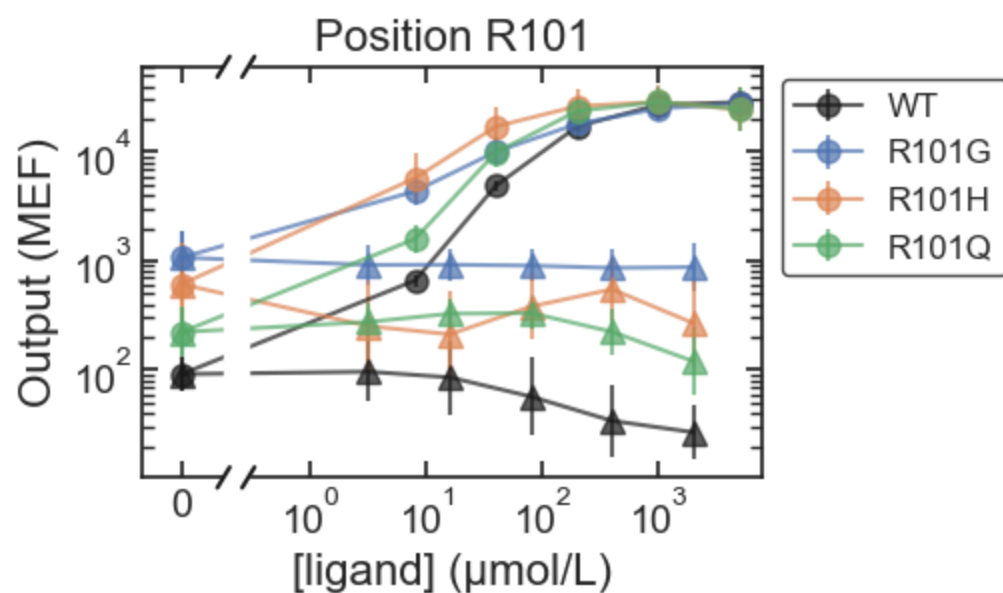
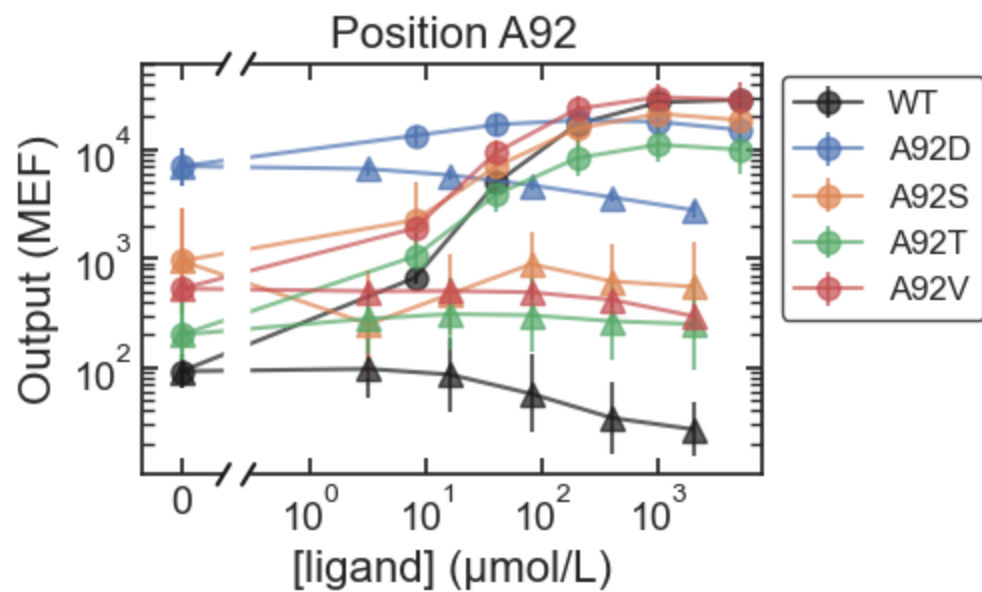
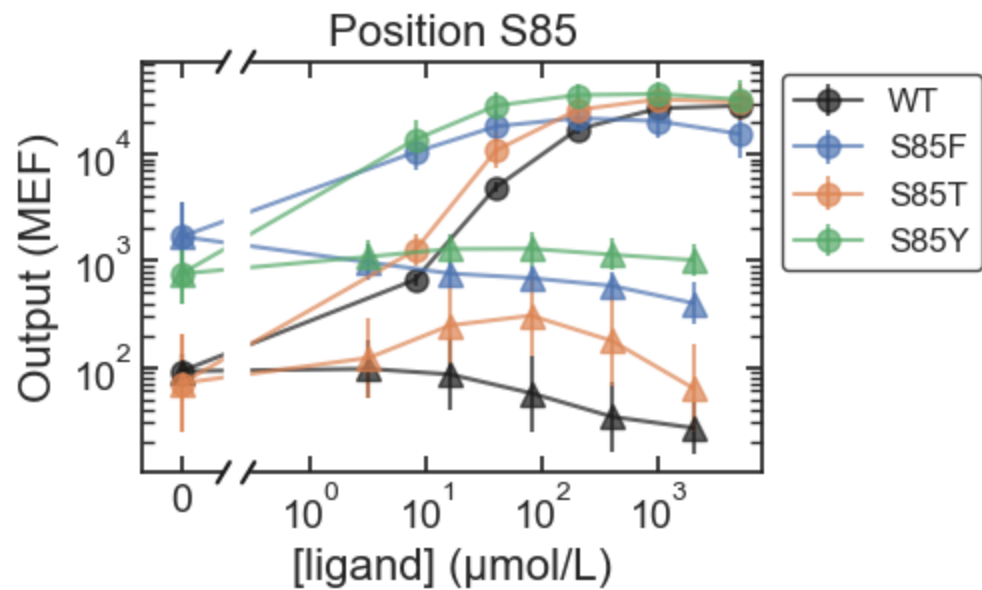


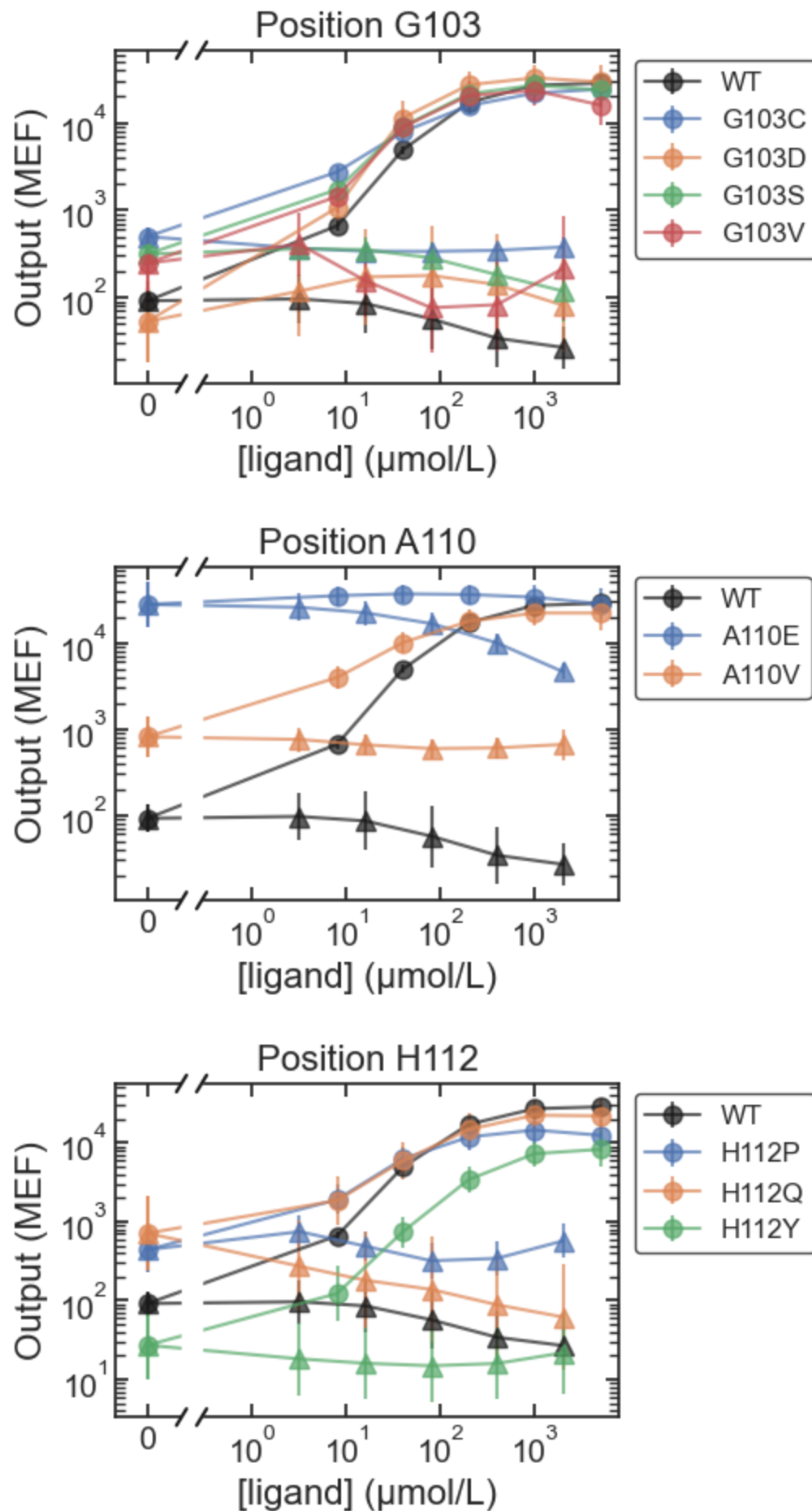


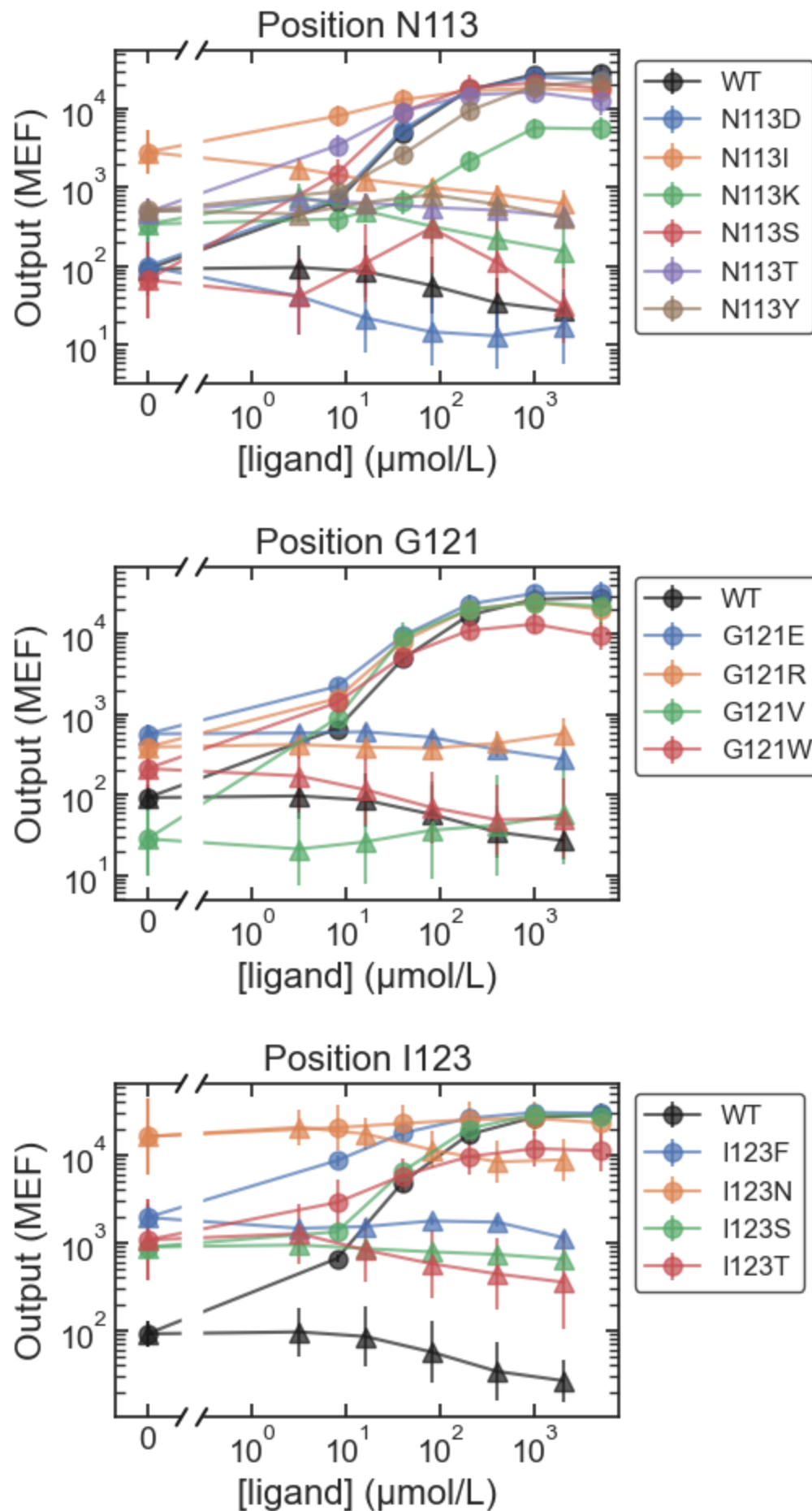
MWC_IPTG_ONPF

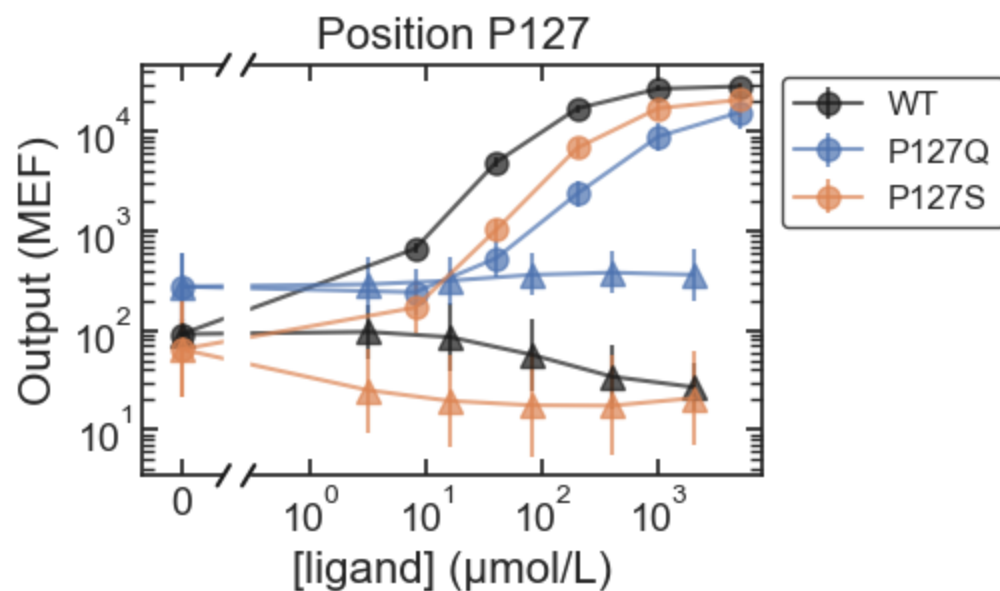
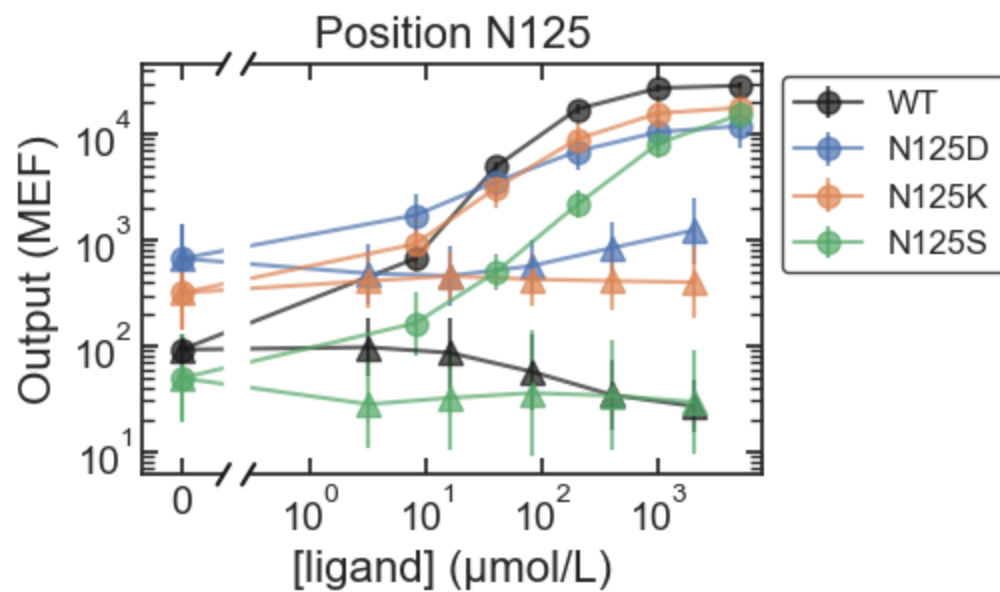
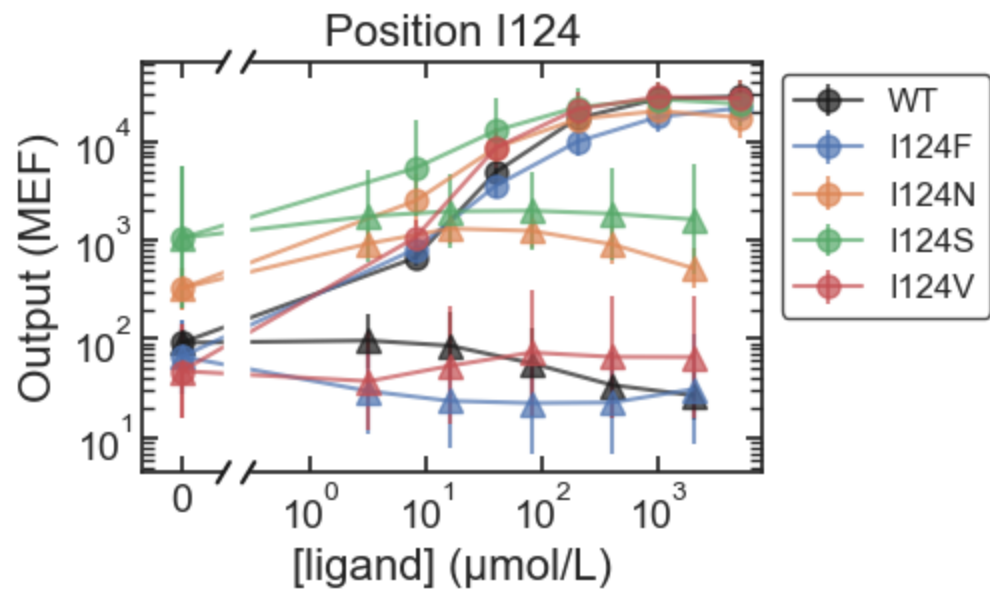


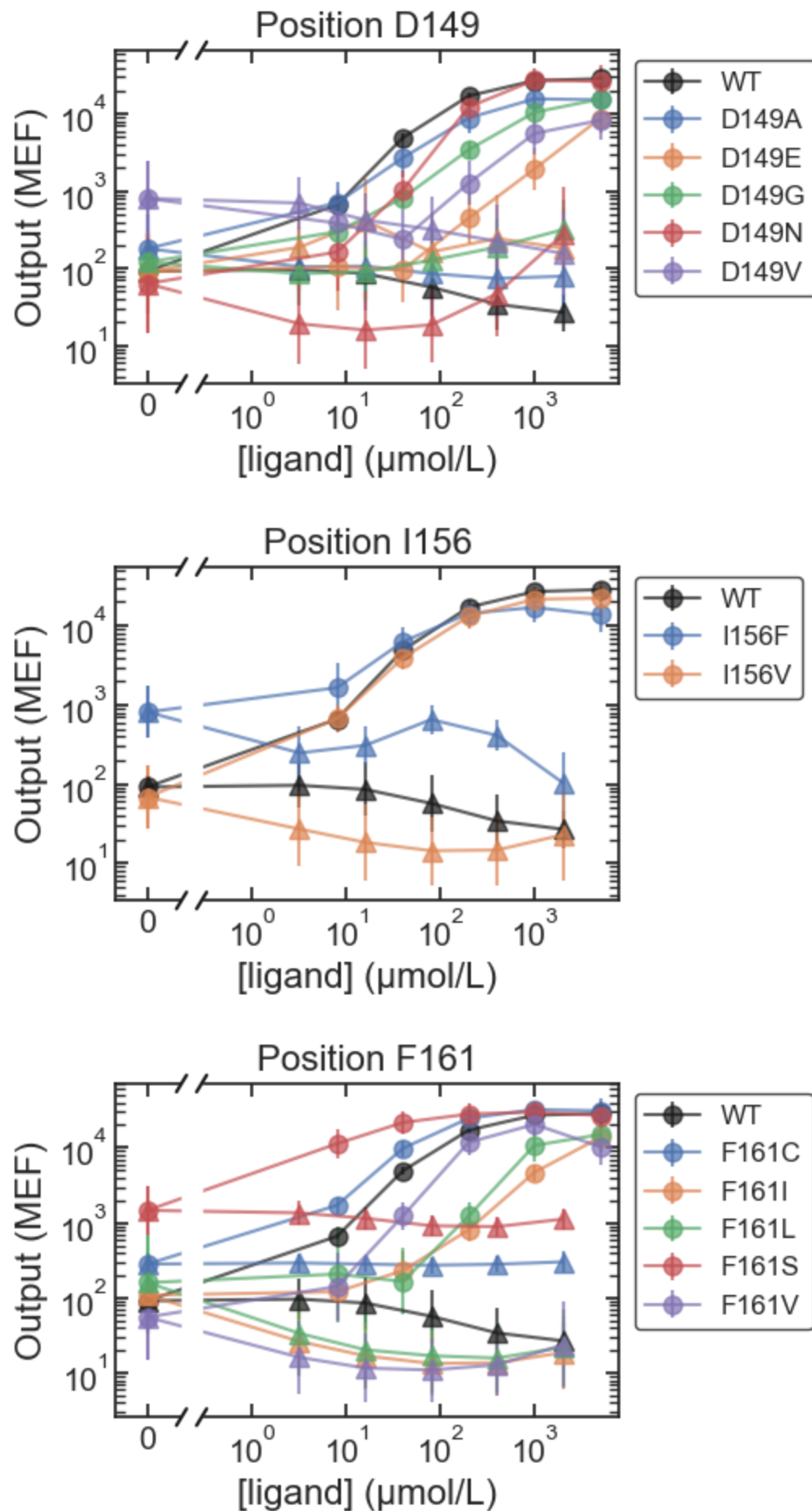


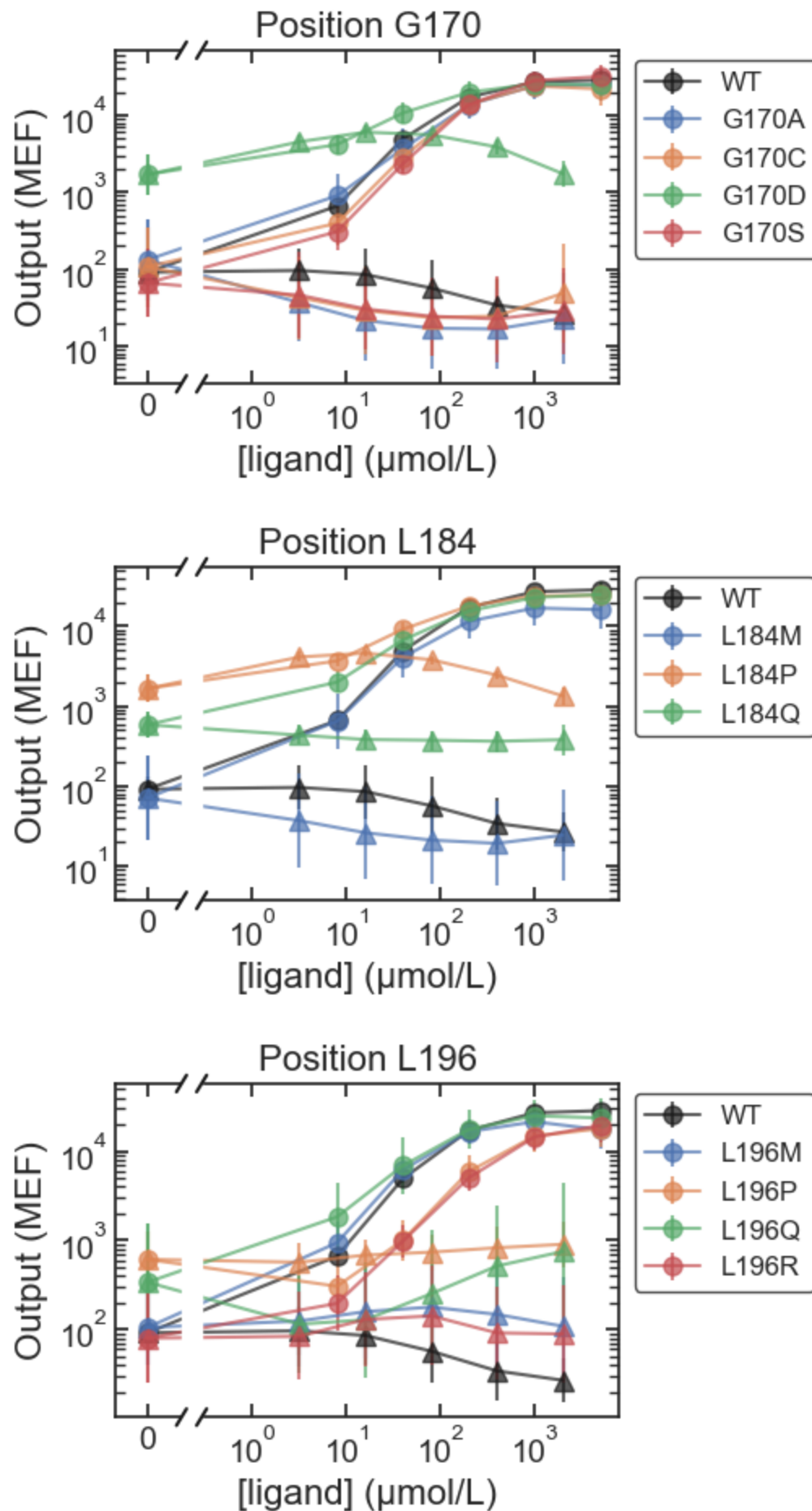


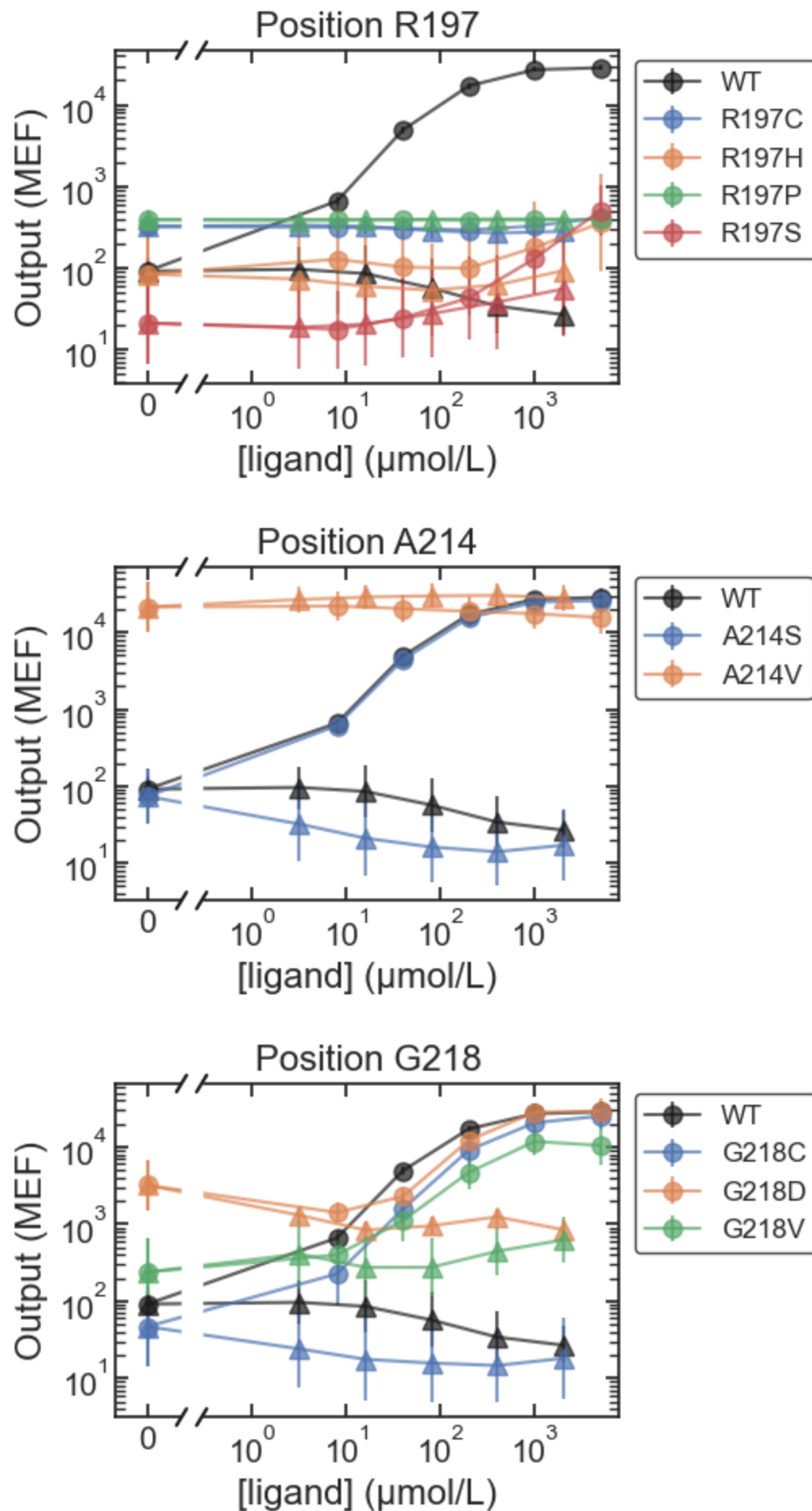


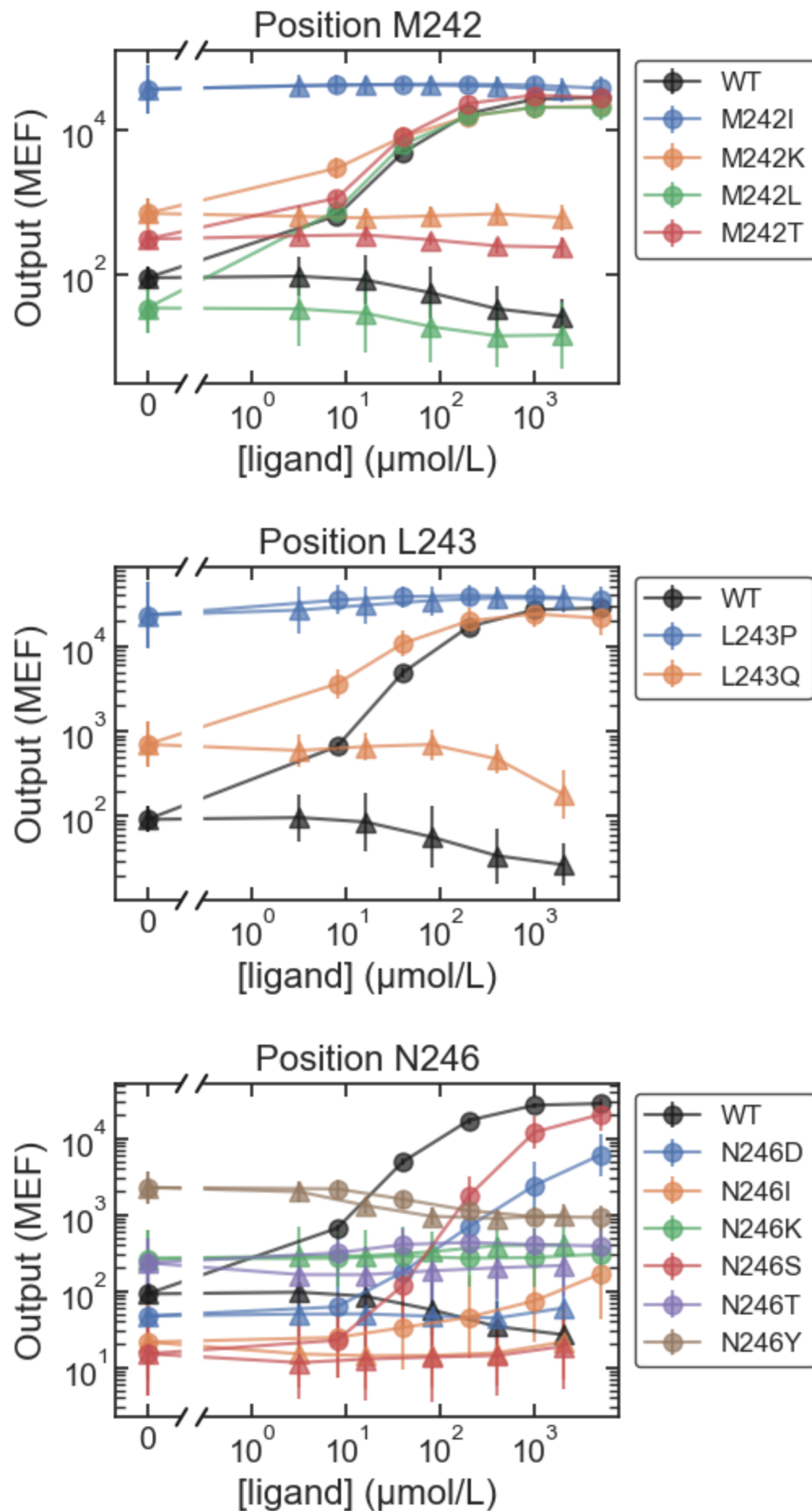


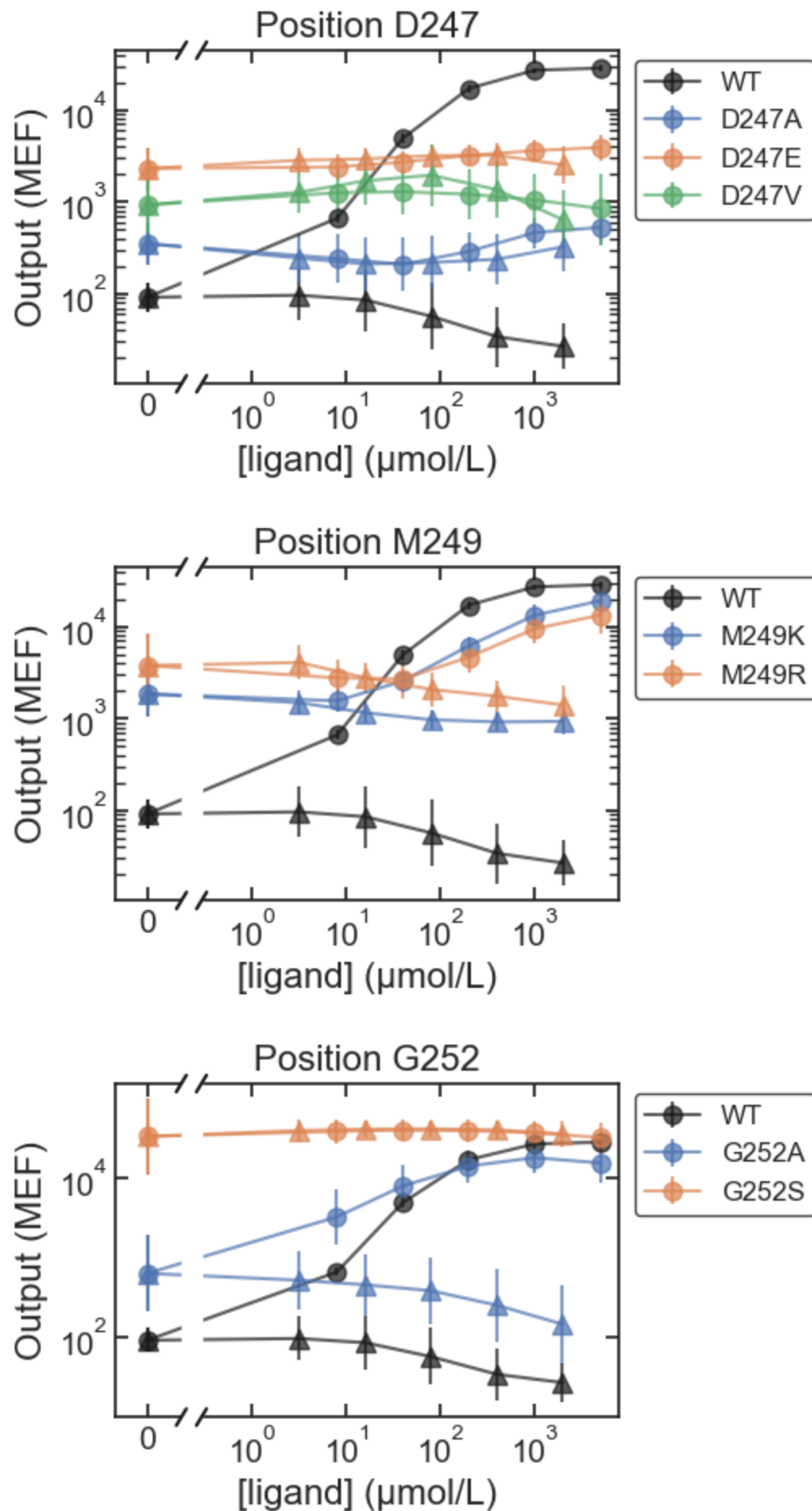


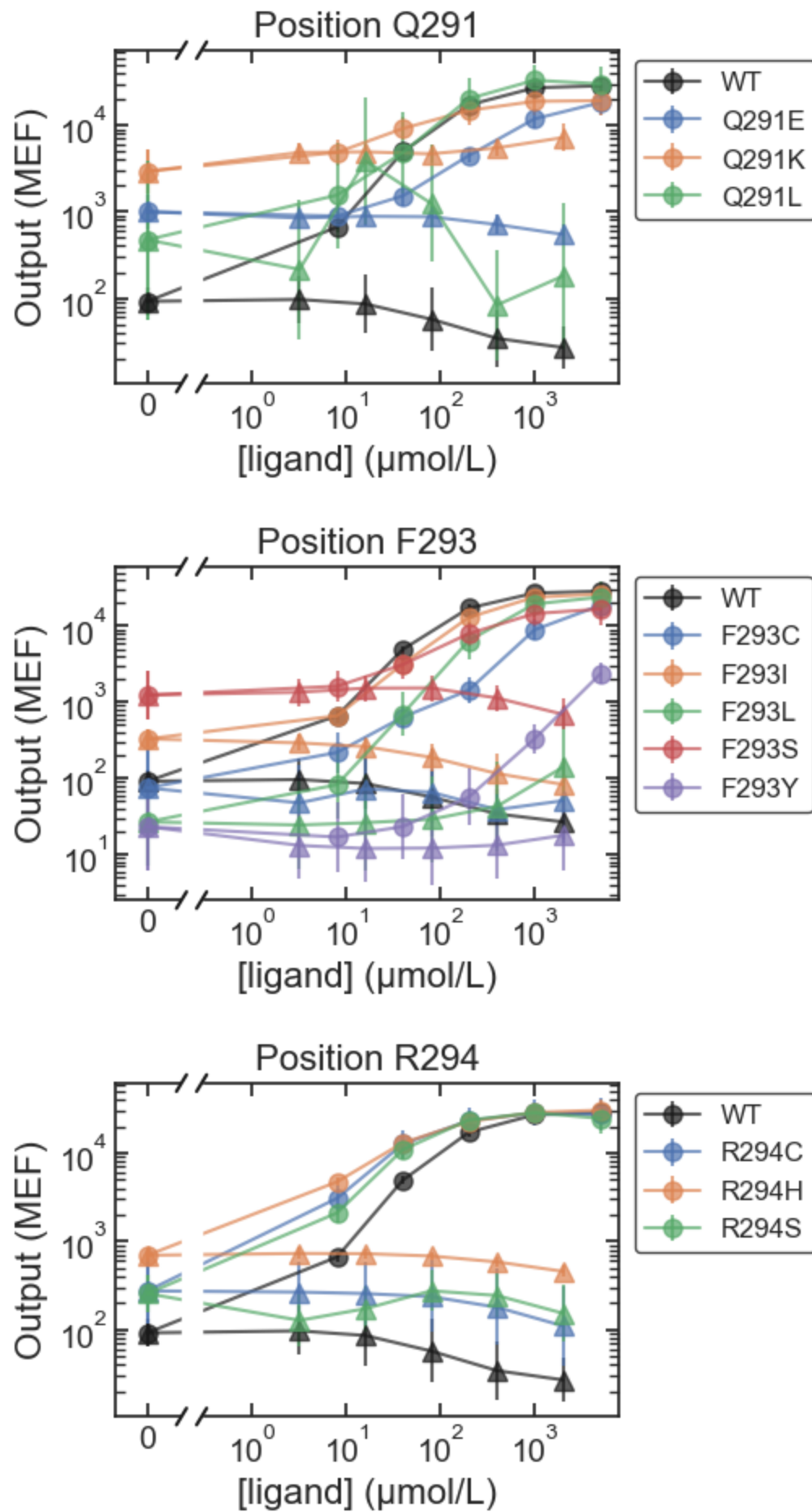


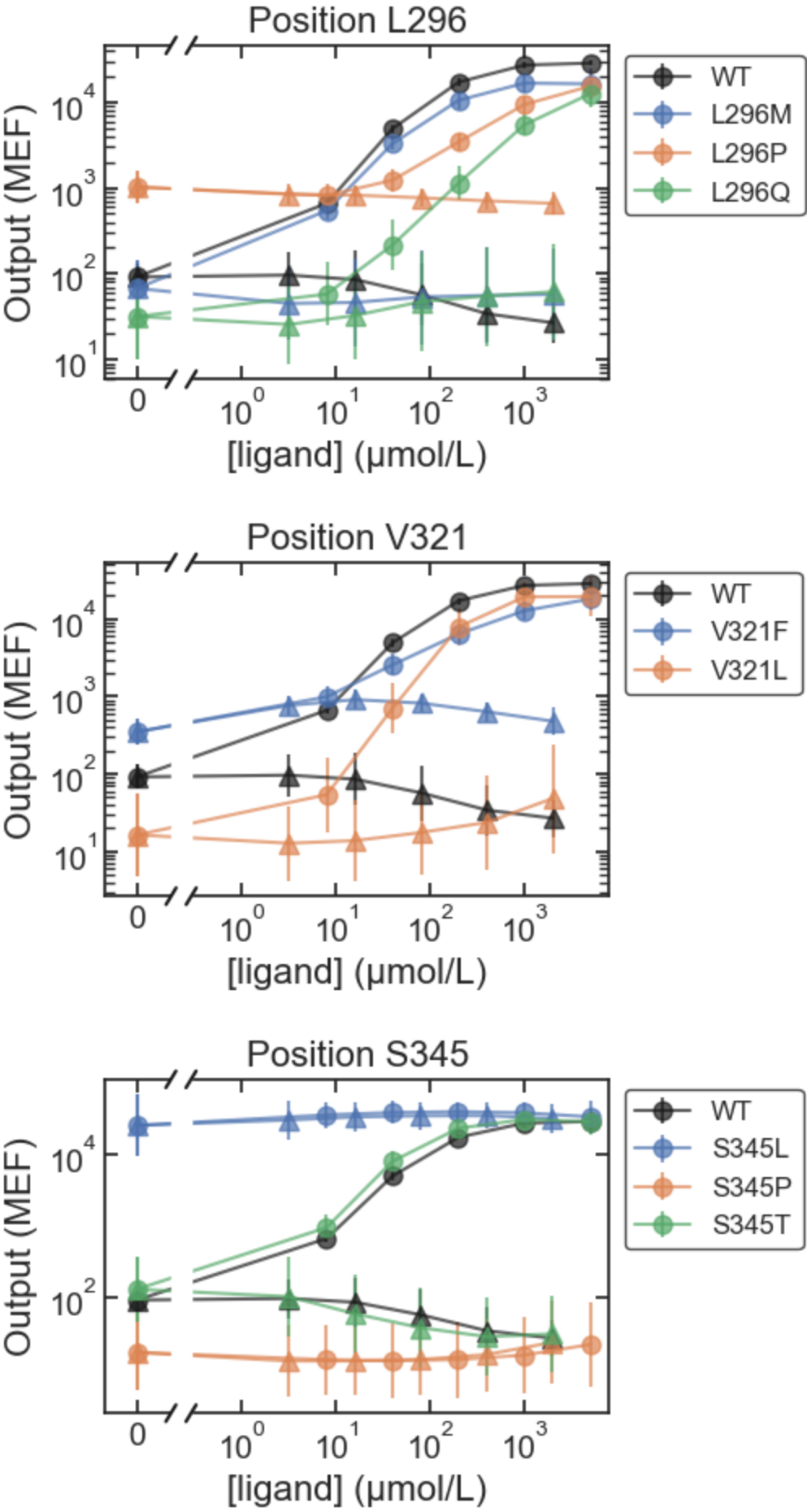




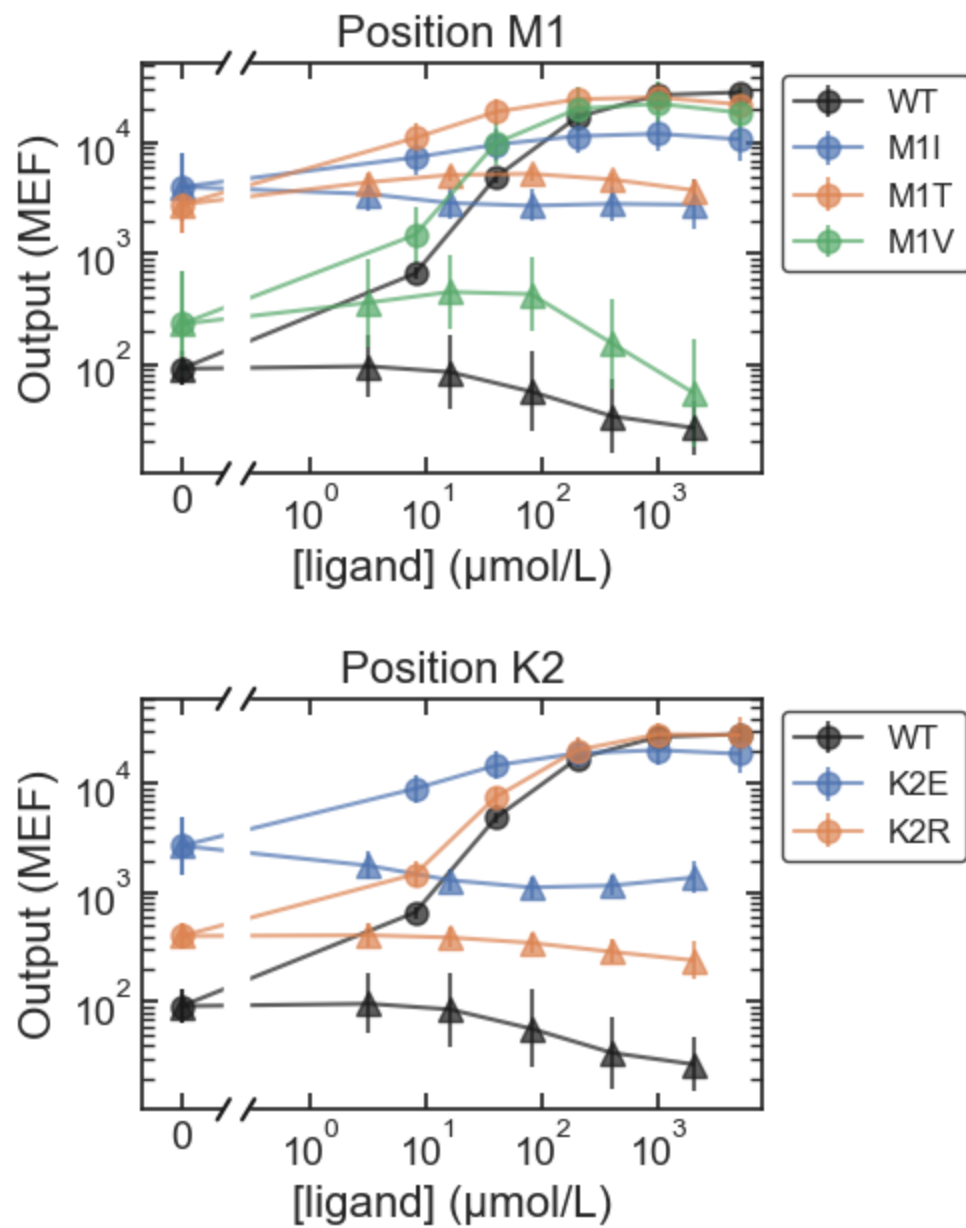


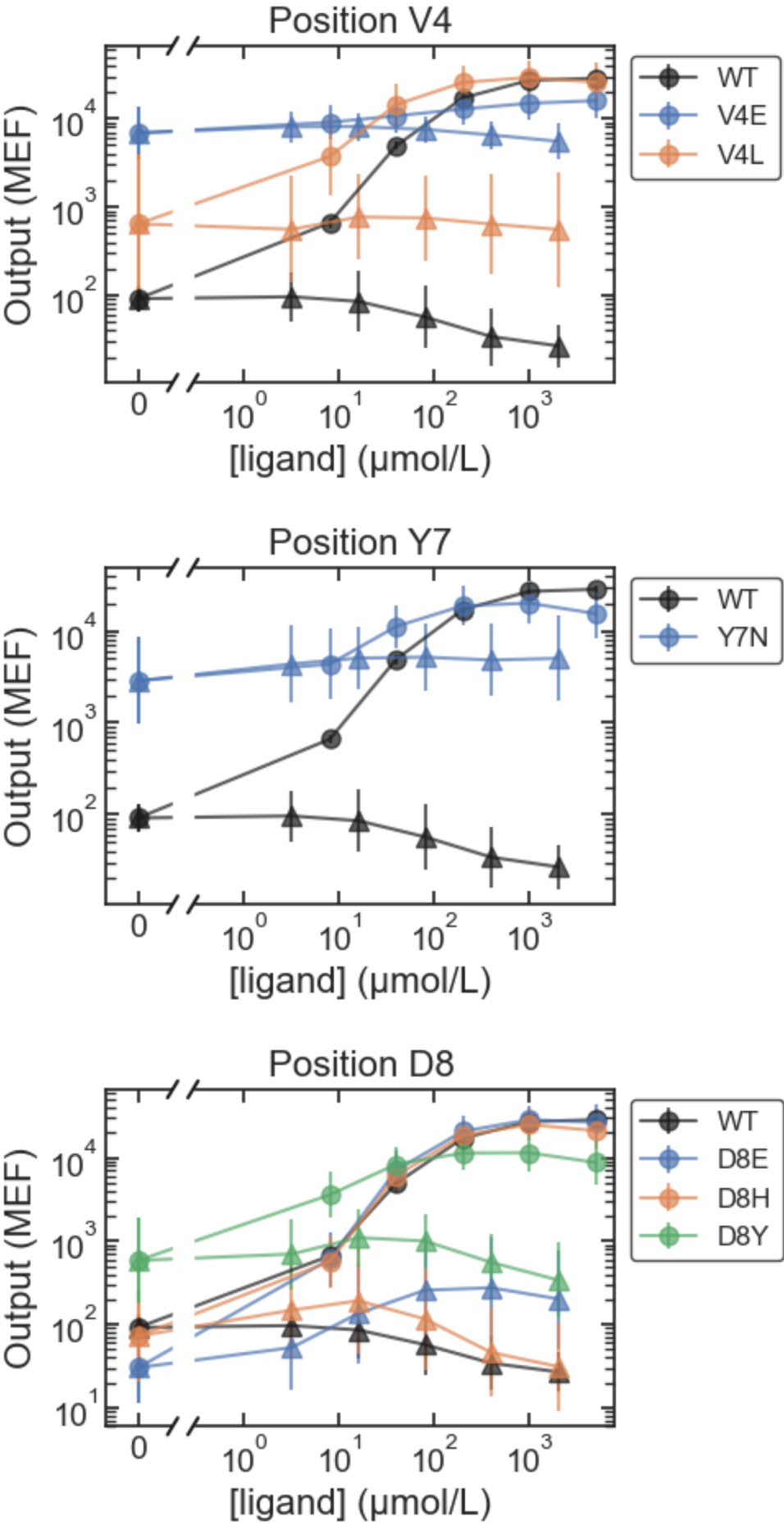


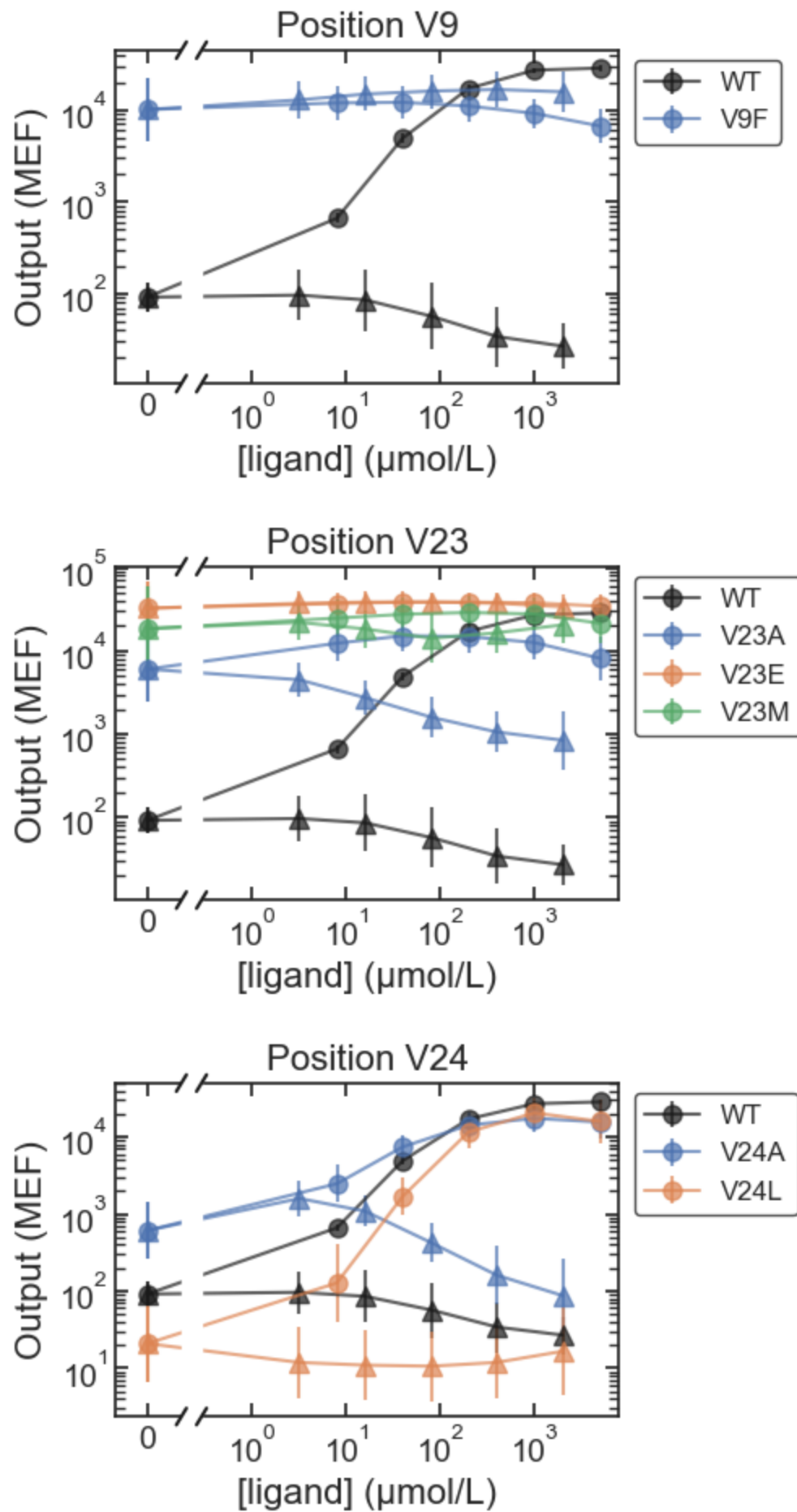


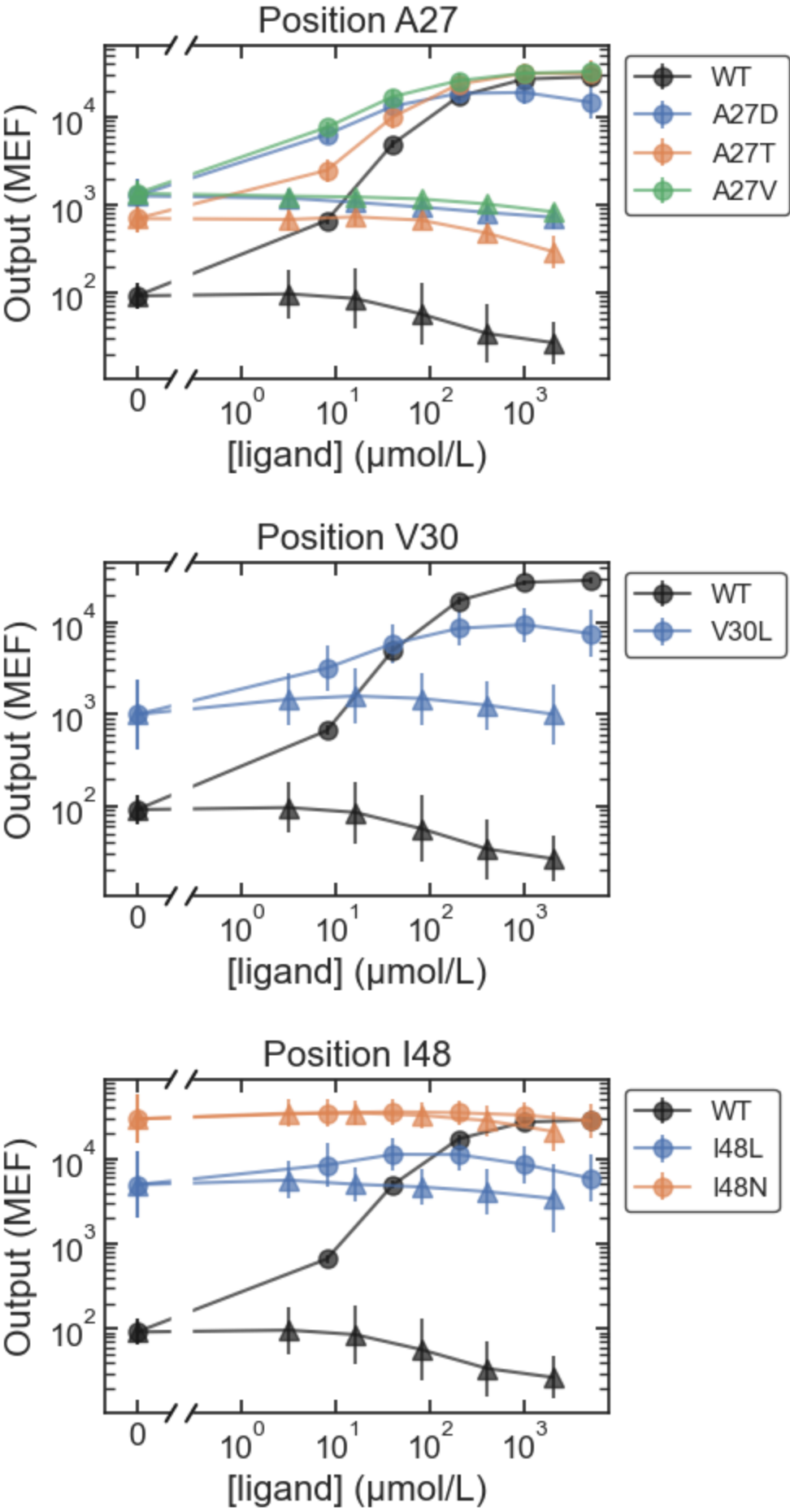


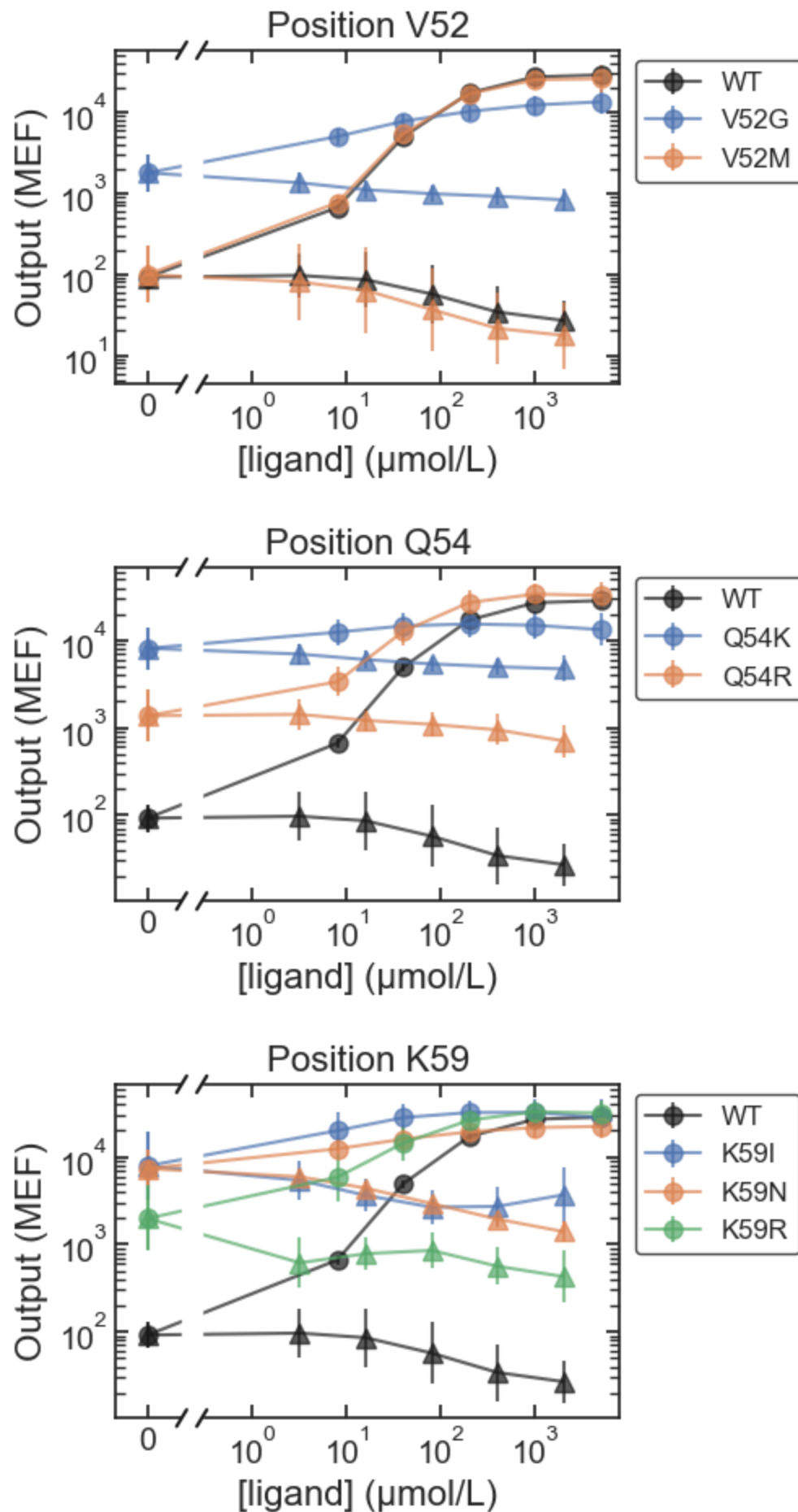
IPTG_only

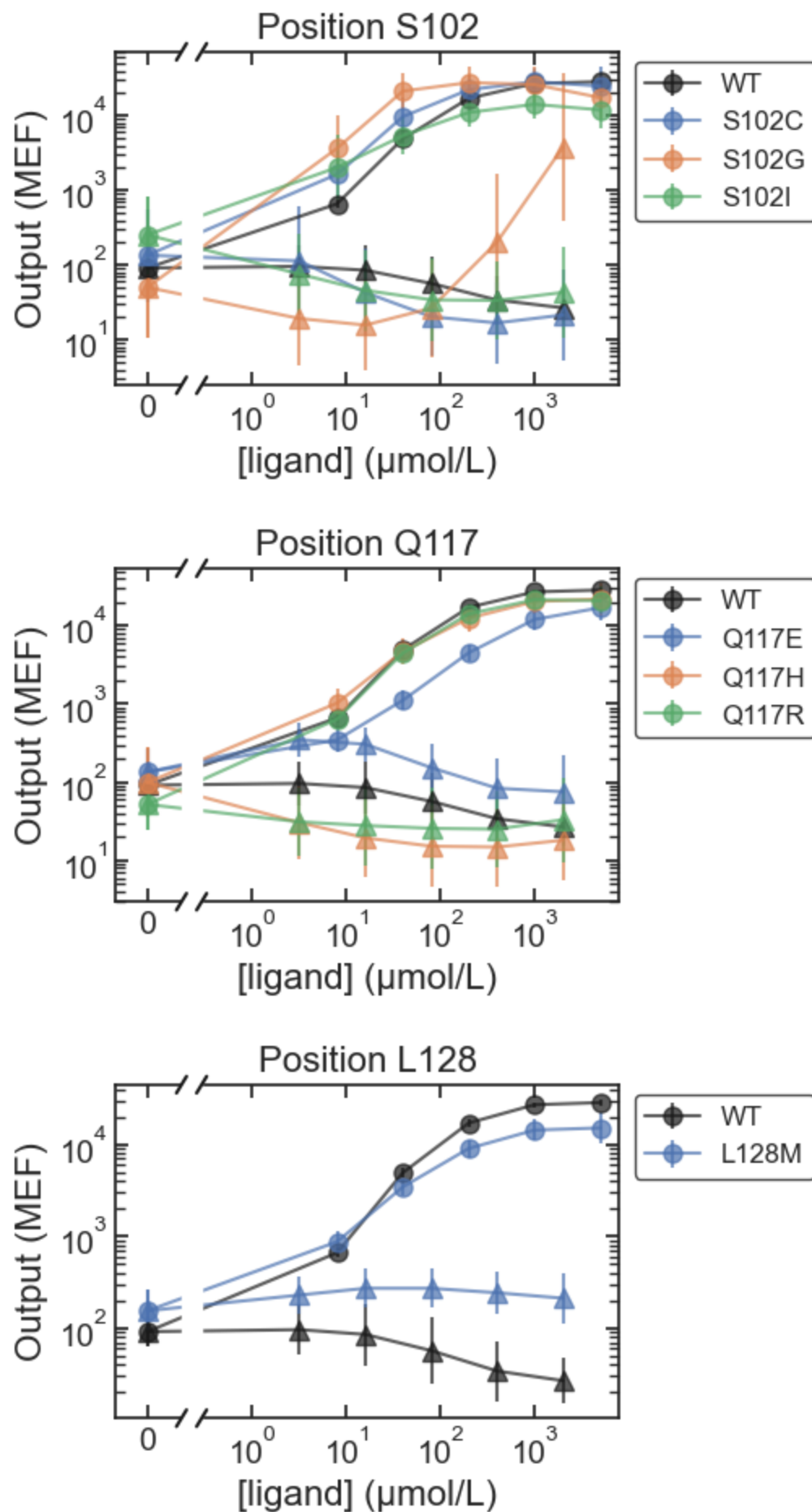


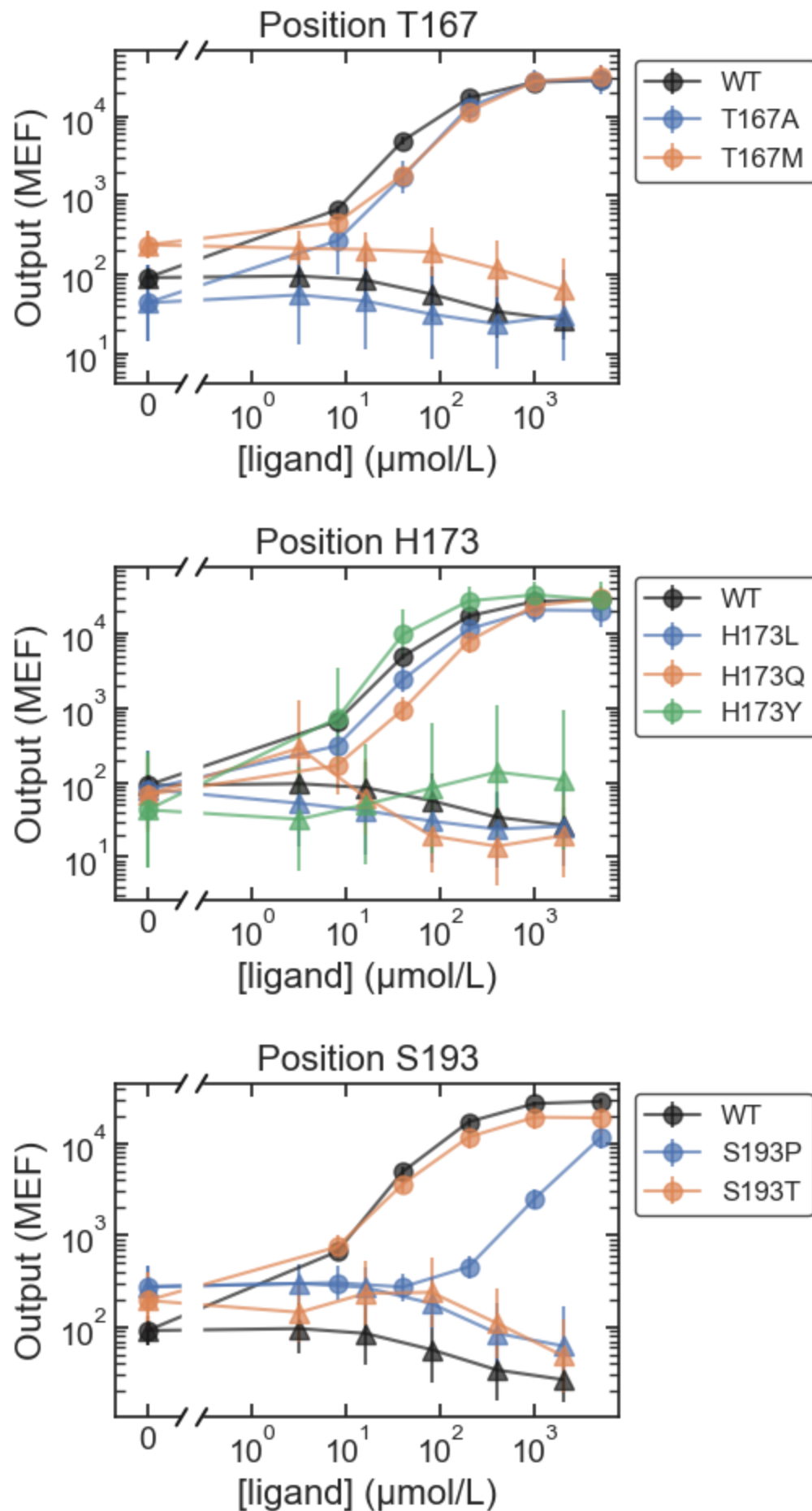


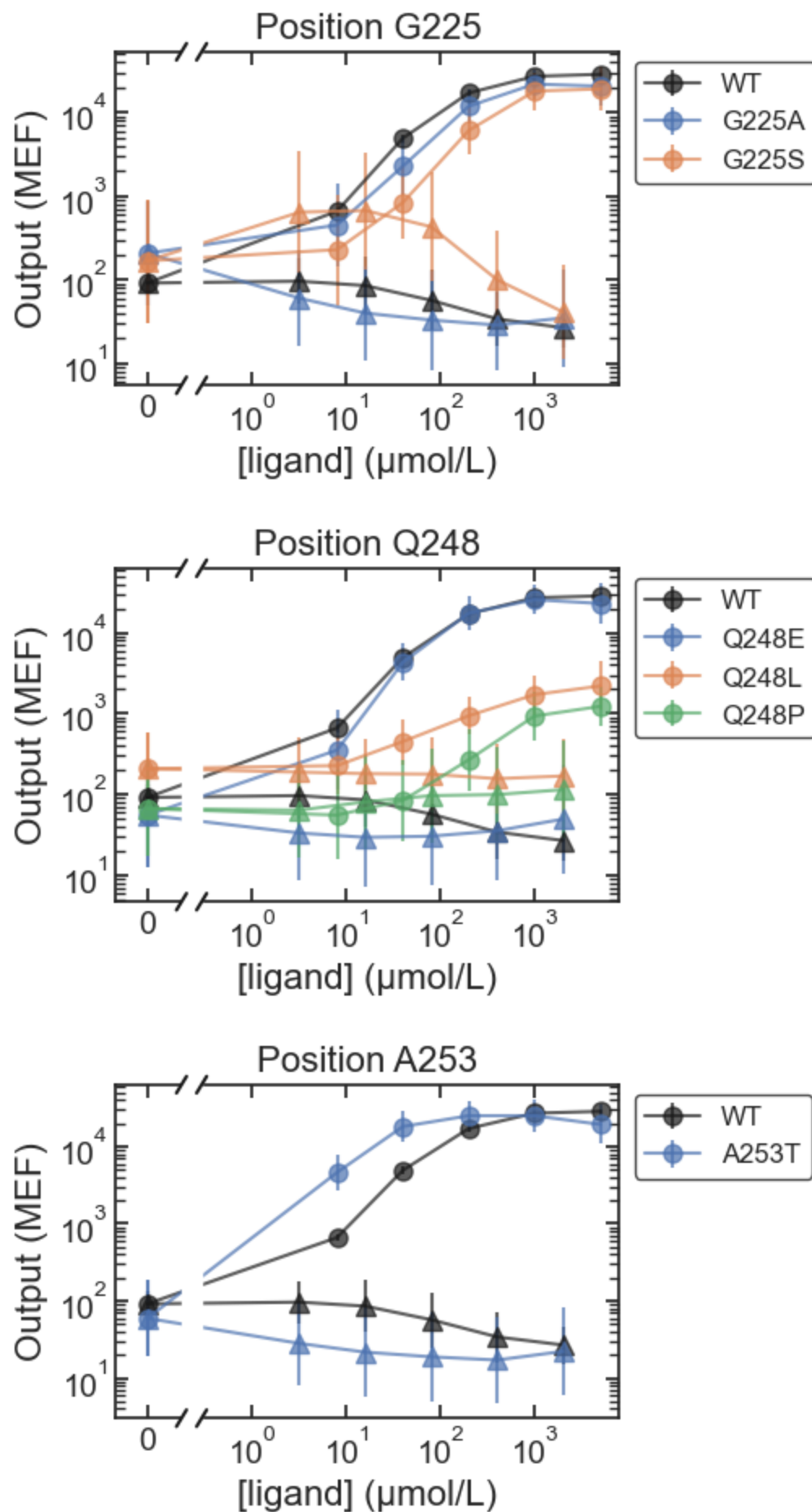


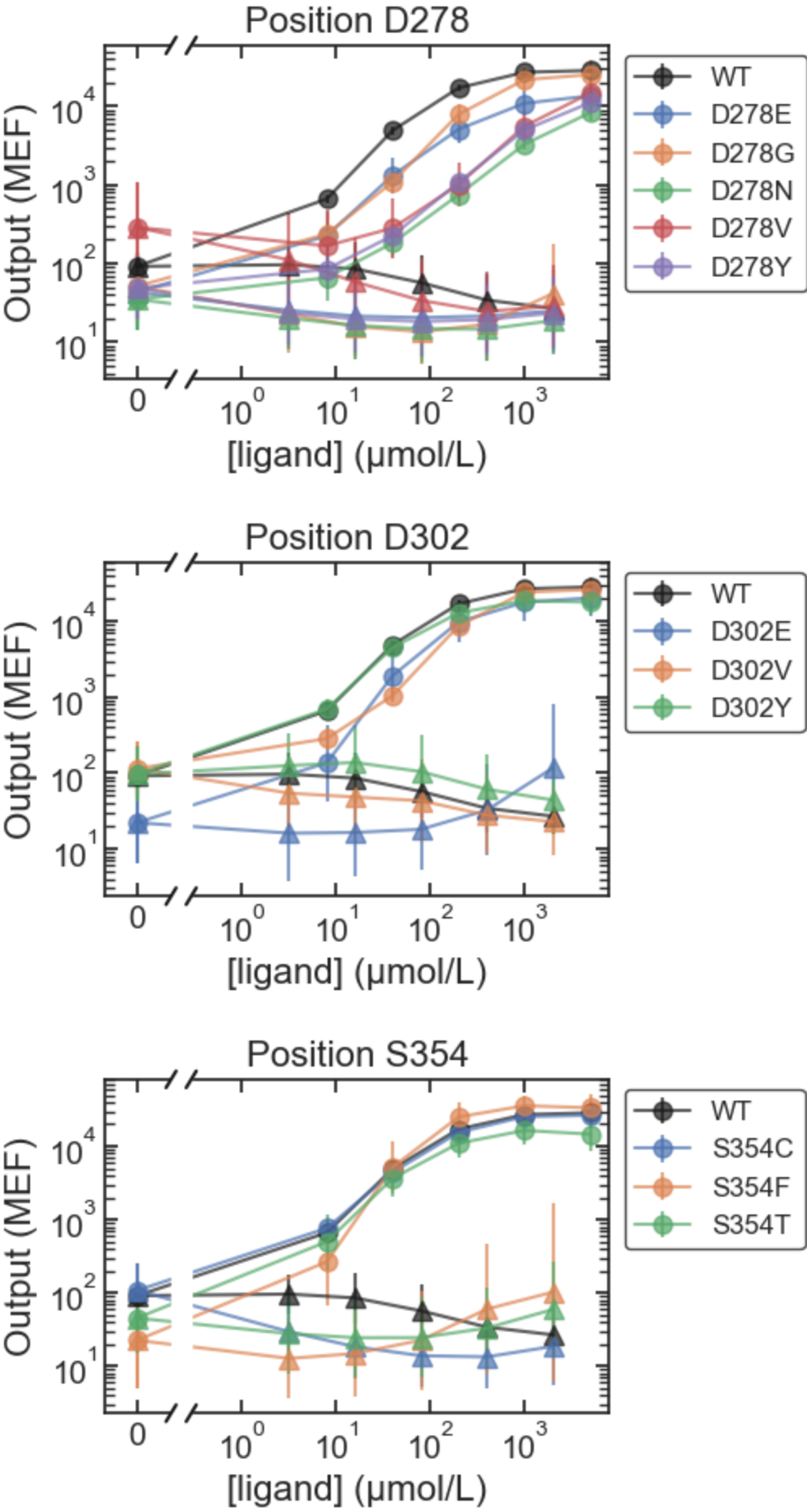




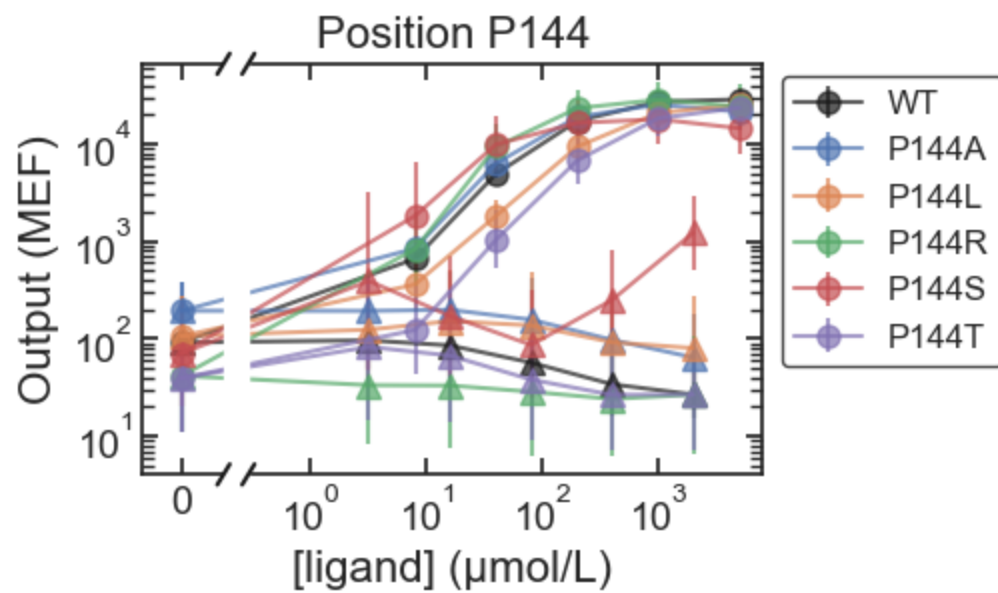
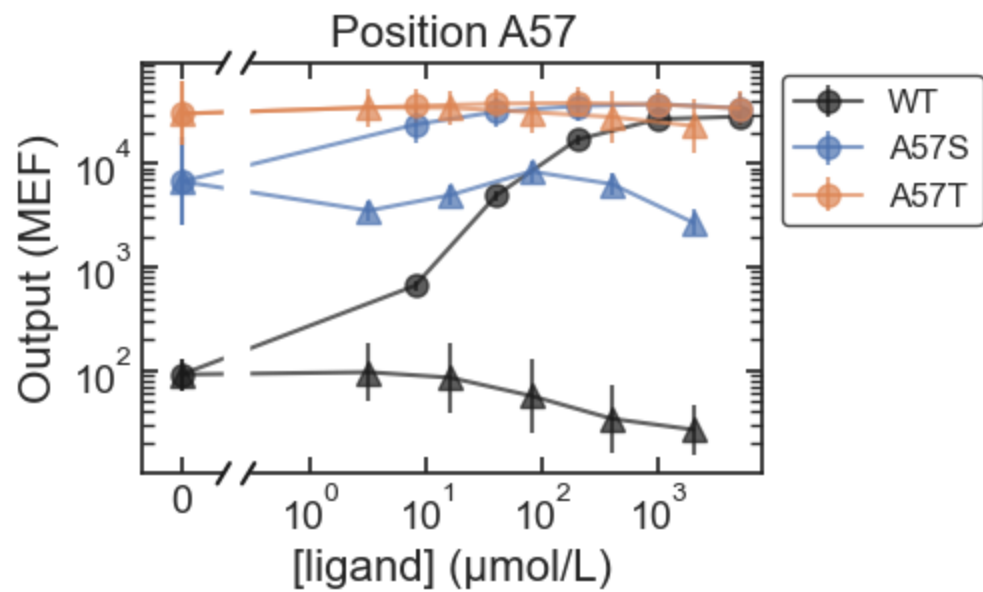


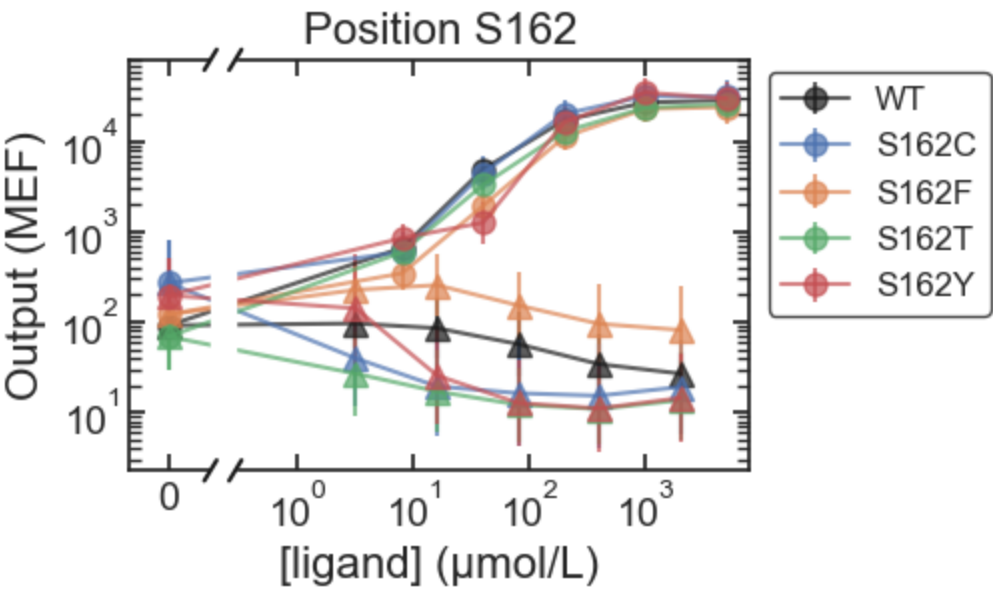




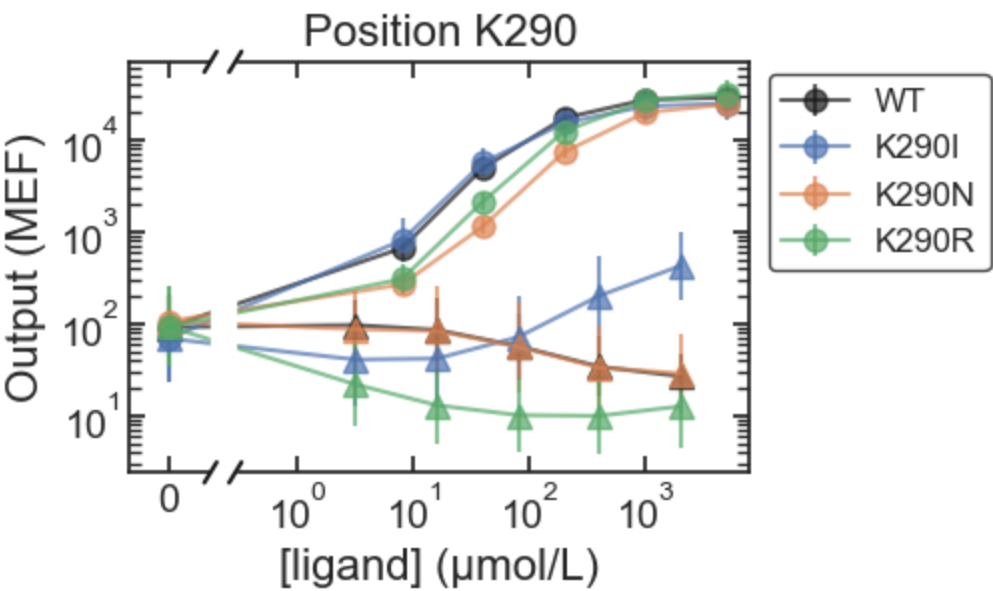


ONPF_only





Both_noMWC



In []:

In []:

In []: